The Production-Ready Engineer

Volume 6: Senior Altitude and the Summit

THE PRINT SET

The Production-Ready Engineer is published in six volumes, cut at Part boundaries so that each one stands on its own and binds cleanly. Page numbers run continuously through the set, and the index in Volume 6 covers all of it.

1	**Foundations** Base Camp and Part I: DevOps, Linux, Networking, Git	pp. 1–734
2	**Build, Ship and Cloud** Parts II and III: Docker, CI/CD, AWS, Terraform	pp. 735–1544
3	**Orchestrate, See and Secure** Parts IV and V: Kubernetes, Helm, Observability, DevSecOps	pp. 1545–2318
4	**Under the Hood** Part VI: Advanced Networking, Databases and Distributed Systems	pp. 2319–2814
5	**Reliability and Platforms at Scale** Parts VII and VIII: SRE, Disaster Recovery, Advanced Kubernetes, GitOps, Platform Engineering	pp. 2815–3548
6	**Senior Altitude and the Summit** **Parts IX and X, Appendices and Index: Architecture, Cost, the Nine Projects**	**pp. 3549–3972**

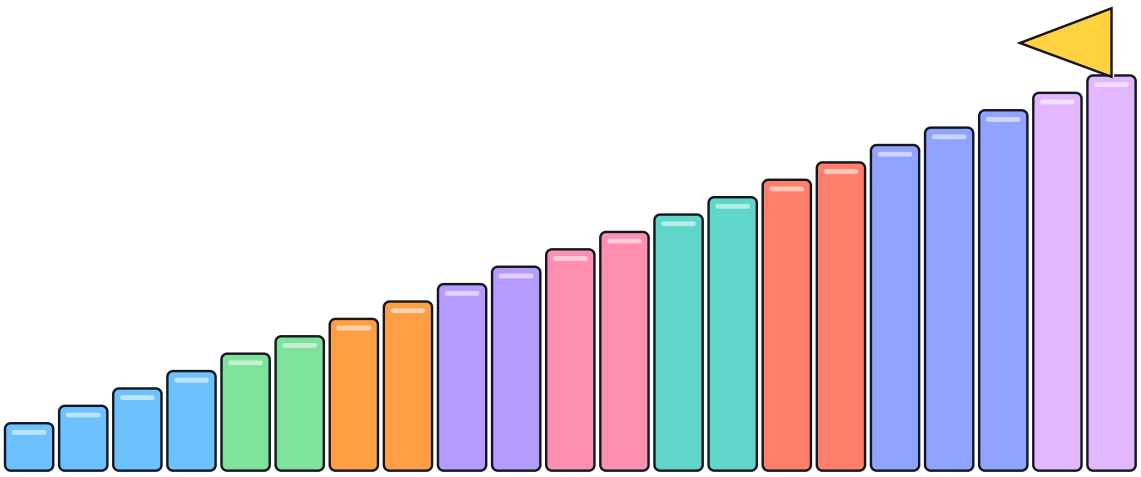
THE COMPLETE DEVOPS ROADMAP

The Production-Ready Engineer

The complete DevOps roadmap, from your first Linux command to senior architect

Volume 6 of 6: Senior Altitude and the Summit

Ubong Eduok



github.com/mascot200/devops-roadmap

The Production-Ready Engineer
The complete DevOps roadmap, from your first Linux command to senior architect
Volume 6 of 6: Senior Altitude and the Summit

Copyright © 2026 Ubong Eduok.

This book is released under the MIT License; the full text is printed in Appendix D, in Volume 6. You may use, copy, modify and share this material, including in your own courses and projects, as long as the copyright and license notice are kept.

Everything here is provided "as is", without warranty of any kind. Run the labs in sandboxes. Read the cost warnings before using a real cloud account, and finish each lab with its teardown steps. Commands that change systems can cause damage or charges; you are responsible for where you run them.

Product and company names are trademarks of their respective owners and are used here only to identify and teach their technology. Their use implies no affiliation or endorsement. Tool icons in the author portrait are from Devicon (MIT licence).

Tools change quickly. Command output in this book is illustrative, and version numbers will age. When in doubt, check the official documentation named in each chapter's further reading.

Source: github.com/mascot200/devops-roadmap, typeset from commit faf55fa.
First edition, 2026.

Do not aim to become the person who knows 50 DevOps tools.

Aim to become the engineer who can take a software application and independently make it production-ready.

Secure, observable, scalable, highly available, automatically deployable, recoverable, and cost-efficient.

How to Read This Book

The anatomy of a chapter

All 248 topic chapters follow the same eleven sections, which fall into three movements. Appendix A, in Volume 6, prints the template that every chapter was written against.

UNDERSTAND	1	**What it is**	A plain-English explanation, with every acronym defined on first use.
	2	**Why it matters**	What breaks, costs money, or wakes someone at 3 a.m. when this is misunderstood.
	3	**Real-world system**	How the topic shows up in a named, publicly documented system.
	4	**Diagram**	The real mechanism: a flow, a sequence, an architecture or a state machine.
	5	**How it works**	Key concepts, comparison tables and the commands that matter.
DO	6	**Step-by-step guide**	A lab you can type in a sandbox, with what to look for at each step.
	7	**Real-world scenario**	A short story under pressure: symptom, investigation, fix, lesson.
	8	**Mistakes and practices**	Two short lists: what not to do, and what to do.
PROVE	9	**Tasks**	Hands-on work, easiest first, each with a "done when" check. The last says Prove it.
	10	**Self-check**	Questions a senior engineer would expect you to answer without notes.
	11	**Further reading**	Official documentation and well-known books.

The eleven sections of every topic chapter

Each level opens with an overview of its goal, recommended time and topics, and closes with a **capstone project** marked with a star, which pulls all of that level's topics together.

Conventions used in print

- **Cross-references.** Where the repository has a hyperlink, this book prints a small tag after the words: Docker Compose §4.8 means Level 4, topic 8. Running heads carry the same numbers, so you can flip straight to it. In a PDF the tags are clickable when the target is in the same volume.
- **Check-boxes.** Every task has a box to tick with a pencil. Where the text mentions changing `- [ ]` to `- [x]`, that is the same idea for readers who keep their progress in a fork of the repository.

- **Answers.** On GitHub the self-check answers are folded away. In print they sit in a shaded box straight after the questions, so cover it with a card and answer first.
- **Code.** Long lines wrap, and the continuation is indented. Type a wrapped line as one line. Output is illustrative and yours may differ in the details.
- **Diagrams.** A few wide diagrams are printed sideways. Turn the book clockwise to read them.
- **Lab notes.** Chapters start on a right-hand page, and any left-hand page this leaves empty is ruled for your notes. Use them. For each task, write what you ran and what you saw. Explaining output is the skill.

Stay safe, and keep it cheap

Use disposable sandboxes: local virtual machines, Docker, kind or minikube, LocalStack, or a small VPS. Anything that can cost money says so and ends with teardown steps. Never practise on a system you cannot afford to lose.

The Whole Climb on One Page



PART I: THE GROUND BENEATH EVERYTHING				
0	What DevOps Actually Is	1 week (reading + mapping)	13 topics	35
1	Linux Fundamentals	2-4 weeks	13 topics	213
2	Networking	2-4 weeks	11 topics	397
3	Git & GitHub	2-3 weeks	13 topics	563
PART II: BUILD IT, SHIP IT				
4	Docker	2-3 weeks	14 topics	737
5	CI/CD	2-3 weeks	12 topics	931
PART III: CLOUD YOU CAN REBUILD				
6	Cloud (AWS)	4-6 weeks	18 topics	1137
7	Terraform / Infrastructure as Code	2-3 weeks	9 topics	1403
PART IV: ORCHESTRATION				
8	Kubernetes	6-10 weeks	19 topics	1547
9	Helm	1-2 weeks	6 topics	1823
PART V: SEE IT, SECURE IT				
10	Observability	3-4 weeks	11 topics	1919
11	Security / DevSecOps	3-4 weeks	13 topics	2123
PART VI: UNDER THE HOOD				
12	Advanced Networking	3-4 weeks	14 topics	2321
13	Databases & Distributed Systems	4-6 weeks	15 topics	2577
PART VII: KEEP IT RUNNING				
14	SRE	3-4 weeks	10 topics	2817
15	Disaster Recovery	2-3 weeks	10 topics	2983
PART VIII: PLATFORMS AT SCALE				
16	Advanced Kubernetes	4-6 weeks	10 topics	3137
17	GitOps	2-3 weeks	7 topics	3303
18	Platform Engineering	4-6 weeks	10 topics	3403
PART IX: SENIOR ALTITUDE				
19	Architecture	Ongoing	11 topics	3551
20	Cost Optimization	2-3 weeks	9 topics	3703

Level numbers follow the roadmap document; the recommended learning order differs slightly (see Base Camp). The nine Summit Projects in Part X then chain the levels into one system.

Contents of Volume 6

PART IX **Senior Altitude** 3549

19	**Architecture**	3551
19.1	Scalability	3553
19.2	Availability	3565
19.3	Security Architecture	3577
19.4	Performance	3589
19.5	Cost	3601
19.6	Operational Complexity	3611
19.7	Deployment Strategy	3623
19.8	Monitoring	3635
19.9	Disaster Recovery	3649
19.10	Reference Production Architecture Walkthrough	3661
19.11	Trade-offs and Architecture Decision Records	3677
★	Capstone: Design the Reference Production Architecture	3689
20	**Cost Optimization**	3703
20.1	FinOps	3705
20.2	Reserved Instances and Savings Plans	3717
20.3	Spot Instances	3729
20.4	Autoscaling for Cost	3743
20.5	Rightsizing	3755
20.6	Storage Lifecycle Policies	3767
20.7	Network Egress Costs	3779
20.8	Database Costs	3791
20.9	Kubernetes Infrastructure Costs	3803
★	Capstone: Audit an AWS and Kubernetes Environment	3819

PART X **The Summit Projects** 3833

	Practical Project Progression	3835
P1	Basic Server	3837
P2	Docker	3849
P3	CI/CD	3863
P4	Terraform	3875
P5	Kubernetes	3889
P6	Production Kubernetes	3903
P7	Observability	3917
P8	Disaster Recovery	3929

P9 Senior Architecture 3943

APPENDICES The Back of the Pack 3957

A Topic Template 3959
B Captions for the DevOps Roadmap posts 3963
C DevOps Roadmap carousel 3967
D MIT License 3971

PART IX

Senior Altitude

Architecture as a series of trade-offs, and cost as an engineering concern rather than a finance surprise.



IN THIS PART

19	Architecture	3551
20	Cost Optimization	3703

LAB NOTES What I ran, what I saw, what it means

LEVEL 19 · PART IX

Architecture

19

Goal: Design complete production systems and evaluate trade-offs like a senior engineer.

RECOMMENDED TIME	TOPICS
Ongoing	11

PRACTICE / CAPSTONE

Design (diagram + written ADRs + cost + DR plan) the reference architecture: CDN → Load Balancer → Kubernetes → API/Worker/Web → Redis → Message Queue → PostgreSQL/S3, with Prometheus, Grafana, Loki, OpenTelemetry and Sentry.

Topics

#	Topic	What you will learn
1	Scalability **p.3553**	scaling axes, stateless services, caching, queues, bottleneck analysis
2	Availability **p.3565**	SPOFs, multi-AZ/region, nines math, graceful degradation
3	Security Architecture **p.3577**	defense in depth, zero trust, threat modeling (STRIDE), blast radius
4	Performance **p.3589**	latency budgets, caching layers, profiling, load testing, tail latency
5	Cost **p.3601**	cost as a design constraint, unit economics, cost-aware design
6	Operational Complexity **p.3611**	boring technology, cognitive load, build vs buy, when not to use Kubernetes
7	Deployment Strategy **p.3623**	environments, release strategies, multi-region rollout, blast-radius control
8	Monitoring **p.3635**	designing observability in, SLO-driven alerting, telemetry pipeline cost
9	Disaster Recovery **p.3649**	choosing a DR tier per component, RPO/RTO driven design
10	Reference Production Architecture Walkthrough **p.3661**	CDN → LB → K8s → API/Worker/Web → Redis → queue → Postgres/S3 + observability stack

#	Topic	What you will learn
11	Trade-offs and Architecture Decision Records **p.3677**	writing ADRs, trade-off matrices, design reviews, estimating
★	Level capstone project **p.3689**	Design (diagram + written ADRs + cost + DR plan) the reference architecture: CDN → Load Balancer → Kubernetes → API/Worker/Web → Redis → Message Queue → PostgreSQL/S3, with Prometheus, Grafana, Loki, OpenTelemetry and Sentry.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



19.1 Scalability

Topic 1 of 11 | ~4 hours | Difficulty: Advanced

1 What it is

Scalability is a system's ability to handle more work (requests, data, users) by adding resources, without a redesign and without cost or latency growing faster than the load. It is not "being fast": a system can be fast for 10 users and collapse at 10,000. Scalability is about the *shape of the curve* when load grows.

There are two **scaling axes**. **Vertical scaling** (scale up) means a bigger machine: more CPU, RAM, faster disks. **Horizontal scaling** (scale out) means more machines behind a load balancer, queue or shard map. Vertical is simple but has a ceiling and a single point of failure; horizontal has almost no ceiling but demands that the application be **stateless** (any instance can serve any request) and that shared state lives somewhere designed for it (database, cache, object storage). Architecture work is mostly finding the *next bottleneck* and deciding which axis removes it.

2 Why it matters in DevOps

- **Launch-day and sale-day outages** are almost always a bottleneck nobody measured: a connection pool, a single database writer, a synchronous call to a third party.
- **Cost:** scaling the wrong tier (adding web pods when the database is the limit) burns money and fixes nothing.
- **Autoscaling is only as good as the design:** Kubernetes can add pods in seconds, but it cannot add database write capacity.
- **Capacity numbers drive everything downstream:** pod counts, node counts, connection limits, queue depth alarms and the cloud bill.

Related reading: High Availability and Scalability §0.8, Horizontal and Vertical Scaling §13.10, Capacity Planning §14.9.

3 Real-world system example

Stateless web tier plus scalable data tier, as documented by large platforms. Instagram's engineering blog has described running Django web servers that are stateless, with PostgreSQL and Memcached/Redis carrying the state, and later sharding PostgreSQL by logical shard ids. Shopify has publicly described its "pods" approach: isolating groups of shops into independent shards of the whole stack so that one huge merchant or one failing database does not take everyone down. Amazon's Builders' Library and the AWS Well-Architected Framework both describe the same recurring pattern: stateless compute that scales horizontally, queues to absorb bursts, caches to shield databases, and partitioned data stores.

The recurring lesson: the compute tier is the *easy* part to scale. The hard parts are the **data tier** and the **shared dependencies**, which is where design effort belongs. (Exact numbers vary by company and year; read the engineering blogs for current figures instead of relying on remembered ones.)

4 Diagram

Where each scaling technique sits in the request path of `shop-api`:

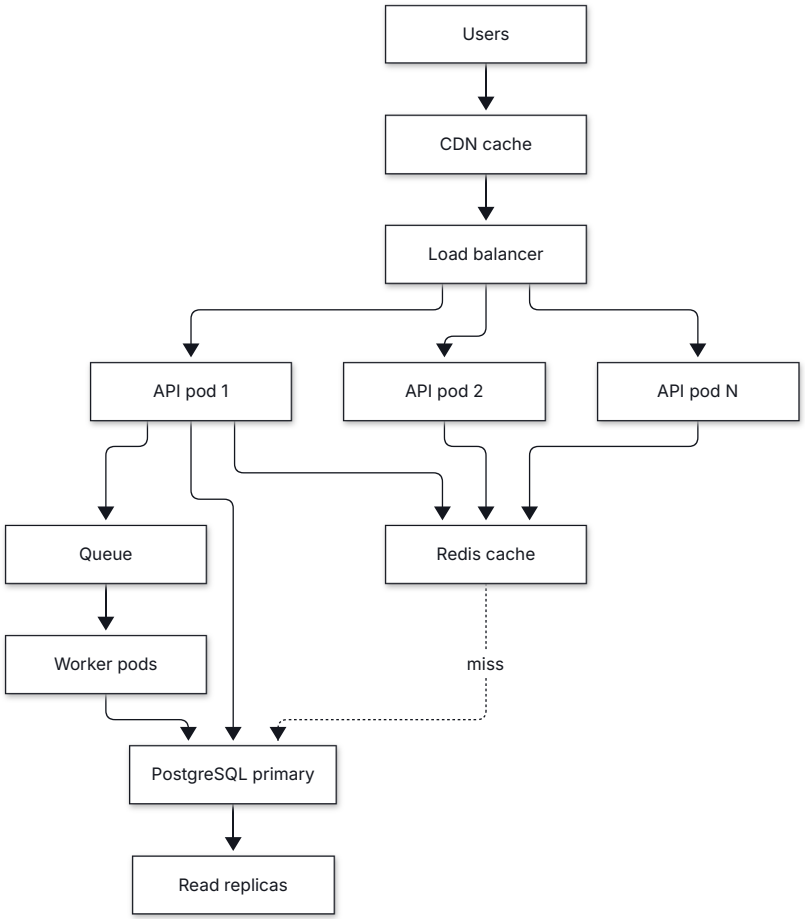


Figure 19.1-1

Finding the bottleneck: utilization of each tier as load grows. The first tier to hit its ceiling sets the system limit.



Figure 19.1-2

5 How it works

Scaling axes

Axis	How	Strengths	Limits
Vertical (scale up)	Bigger instance or DB	No code change, simple ops	Ceiling, downtime to resize, single point of failure, price rises faster than capacity at the top end
Horizontal (scale out)	More identical instances	Near-linear for stateless tiers, also adds redundancy	Needs stateless design, shared state becomes the bottleneck
Functional split	Separate services or queues per workload (API vs worker vs reports)	Scale each part independently	More moving parts to operate

Axis	How	Strengths	Limits
Data partitioning	Shard or partition by key (tenant, user id, date)	Removes the single-writer limit	Cross-shard queries and transactions get hard; re-sharding is painful

Stateless services

A service is stateless when a request can be served by any replica. To get there: sessions live in Redis or a signed token (not process memory), uploads go straight to object storage (S3) via pre-signed URLs, configuration comes from the environment, and any local disk is treated as scratch. Statelessness is what lets Kubernetes kill and reschedule pods freely and lets the autoscaler work. See Horizontal Pod Autoscaling §8.16.

Caching

Caches trade freshness for speed and database relief. Layers: browser, CDN, application (Redis), database (buffer cache). The key numbers are the **hit ratio** and the **invalidation strategy** (TTL, explicit delete, write-through). A cache with a 90% hit ratio removes 90% of read load from the database; at 99% the remaining 1% is a hundred times smaller than the original. The danger is a **cache stampede** (many misses at once when a hot key expires) and a **cold cache** after restart. See Redis §13.7.

Queues

A queue decouples "accept the work" from "do the work". It absorbs bursts (the queue grows instead of the database falling over), allows independent scaling of workers, and enables retries. Costs: eventual consistency, the need for idempotent consumers, and a new thing to monitor (queue depth and age of oldest message). See Kafka §13.8, RabbitMQ §13.9, Idempotency §13.14.

Bottleneck analysis

Use **Little's Law**: `concurrency = throughput x latency`. If an API pod handles requests that take 200 ms, and you need 150 req/s per pod, you need about 30 requests in flight per pod; a pool of 10 database connections cannot sustain that if each request holds a connection for the whole 200 ms. Then apply the **Universal Scalability Law** intuition: contention (locks, a single hot row) and coherence costs (cross-node chatter) make returns diminish and can even reverse throughput past some point. Always measure; never assume linear scaling.

6 Step-by-step guide

Prerequisites: Python 3.9+ (no packages). Optionally Docker for step 5. No cloud cost.

1. **Write a back-of-envelope capacity model for** `shop-api`**.** Save as `capacity.py`:

```python
import math

dau = 2_000_000              # daily active users (assumption, state it)
reqs_per_user_per_day = 40   # assumption
peak_factor = 5              # peak RPS vs daily average (assumption)
avg_rps = dau * reqs_per_user_per_day / 86_400
peak_rps = avg_rps * peak_factor

rps_per_pod = 150            # MEASURE this in a load test, never guess
target_util = 0.6            # headroom for spikes and a lost availability zone
pods = math.ceil(peak_rps / (rps_per_pod * target_util))

print(f"avg RPS   : {avg_rps:,.0f}")
print(f"peak RPS  : {peak_rps:,.0f}")
print(f"pods      : {pods}")

conns_per_pod = 10
print(f"DB conns  : {pods * conns_per_pod} (PostgreSQL default max_connections is
  100)")

hit = 0.9
db_reads = peak_rps * 0.8 * (1 - hit)   # assume 80% of requests are reads
print(f"DB reads/s with {hit:.0%} cache hit ratio: {db_reads:,.0f}")
```

```bash
python3 capacity.py
```

Expected output:

```
avg RPS   : 926
peak RPS  : 4,630
pods      : 52
DB conns  : 520 (PostgreSQL default max_connections is 100)
DB reads/s with 90% cache hit ratio: 370
```

Read the result: compute scales fine (52 pods), but 520 database connections blows past the default limit of 100. The model just told you the first bottleneck is the *connection count*, not CPU. The fix is a pooler (PgBouncer) or fewer connections per pod. See Database Connection Pooling §13.6.

2. **Apply Little's Law to find required concurrency.**

```bash
python3 -c "rps=150; latency=0.2; print('in-flight per pod:', rps*latency)"
```

Expected: `in-flight per pod: 30.0`. If your pod allows 10 DB connections, requests will queue inside the pod.

3. **Add the cache and re-run.** Change `hit` to `0.99` and re-run. Reads to the database should drop from 370 to about 37 per second. Write down the new bottleneck (probably writes, since caching does not help them).
4. **Model a queue absorbing a burst.**

```python
# burst.py: queue depth when arrivals exceed worker capacity
arrival = 400      # msgs/s during a 60 s flash sale
workers = 20
per_worker = 10    # msgs/s each
capacity = workers * per_worker
backlog = max(0, (arrival - capacity) * 60)
drain_s = backlog / capacity
print(f"backlog after burst: {backlog} msgs, drain time: {drain_s:.0f} s")
```

```bash
python3 burst.py
```

Expected: `backlog after burst: 12000 msgs, drain time: 60 s`. Without the queue those 12,000 extra messages would have hit the database at once. With it, users see a one-minute delay in emails instead of errors.

5. **See statelessness break (optional, Docker).**

```bash
docker run -d --name web1 -p 8081:80 nginx:alpine
docker run -d --name web2 -p 8082:80 nginx:alpine
curl -s -o /dev/null -w "%{http_code}\n" localhost:8081 localhost:8082
docker rm -f web1 web2
```

Two identical replicas answer the same way because neither holds user state. Now imagine each held the user's shopping cart in memory: the load balancer would send the next request to the other replica and the cart would vanish. That is why carts belong in Redis.

6. **Write the scaling decision as a one-page table**: for each tier (CDN, API, worker, Redis, queue, PostgreSQL, S3), write the current limit, how you will detect it (the metric), and the next action. This table is the output of bottleneck analysis.

7 Real-world scenario

(Illustrative, not a documented incident.) A retailer's flash sale starts at 12:00. Autoscaling takes the API from 6 to 60 pods in four minutes. Error rate **rises** instead of falling. Dashboards show API CPU at 25%, but PostgreSQL has `FATAL: sorry, too many clients already` in its logs. Each new pod opened 10 connections; 60 pods wanted 600, and the database allows 100. The autoscaler made things worse by adding more connection hunters.

The on-call engineer deploys PgBouncer in transaction-pooling mode in front of PostgreSQL, caps each pod at 3 app-side connections, and raises the cache TTL for the product catalogue from 5 to 60 seconds. Errors stop within minutes. **Lesson:** scale the system by finding the *first* saturated resource, not the most visible one. Load test with realistic pod counts *before* the sale, and alert on connection saturation, not just CPU.

8 Common mistakes and best practices

Don't

- Scale the stateless tier and assume the whole system scales: the database is almost always the real limit.
- Store sessions or uploads on the pod's local disk.
- Cache without a plan for invalidation, TTLs and stampedes.
- Adopt sharding, microservices or multi-region "for scale" before a measurement says you need it.
- Trust a vendor's "unlimited" claim; every managed service has quotas, so find and monitor them.

Do

- Write assumptions down (users, requests per user, peak factor) and keep the model in the repo.
- Measure per-pod throughput in a load test and size from the measurement.
- Put a queue between "accept" and "process" for anything that can be asynchronous.
- Keep headroom (target 50-70% utilisation at peak) so you survive a lost zone or a spike.
- Re-run the bottleneck analysis after every significant change; the bottleneck moves.

9 Tasks

- [ ] **T1 — Build the capacity model** *(≈40 min, intermediate)*
  - **Goal:** produce a reproducible back-of-envelope estimate for `shop-api`.

- **Steps:**
  1. Create `capacity.py` from step 6.1 and run it.
  2. Add a `STORAGE` section: 2 KB per order, 50,000 orders per day, 3 years retention, plus 30% index overhead.
  3. Print total storage in GB.
- **Done when:** the script prints pods, DB connections and storage, and the storage figure matches a hand calculation of `50000 x 365 x 3 x 2 KB x 1.3` (about 142 GB in decimal units).

☐ **T2 — Find the first bottleneck** *(≈30 min, intermediate)*

- **Goal:** use numbers to name the first thing that breaks.
- **Steps:**
  1. Extend the model with limits: PostgreSQL `max_connections=100`, Redis 50,000 ops/s, queue 5,000 msgs/s, each pod 150 RPS.
  2. For peak RPS of 1,000, 5,000 and 20,000 print the utilisation of each resource.
  3. Mark the first resource above 100% for each load level.
- **Done when:** a table shows which resource fails first at each load, and you can justify the fix for each.

☐ **T3 — Cache what-if analysis** *(≈30 min, intermediate)*

- **Goal:** quantify what caching buys.
- **Steps:**
  1. Compute database read rate at hit ratios 0%, 50%, 90%, 99%, 99.9% for a fixed 4,000 RPS.
  2. Plot or tabulate the results and note the diminishing returns.
  3. Add a stampede scenario: the hit ratio drops to 0% for 10 seconds after a deploy. Compute the peak DB load.
- **Done when:** you can state the DB read rate in each case and name two mitigations for the stampede (for example request coalescing and jittered TTLs).

☐ **T4 — Prove the queue protects the database** *(≈45 min, advanced)*

- **Goal:** break it, then fix it, in simulation.
- **Steps:**
  1. Write a Python simulation: 400 msgs/s arrive for 60 s; the database handles 150 writes/s; without a queue, excess writes are dropped (errors).
  2. Count dropped writes. Add a queue with 20 workers each handling 10 msgs/s and count again.

3. Add a worker autoscale rule that doubles workers after 30 s of backlog and report the new drain time.

- **Done when:** the simulation shows dropped writes without the queue and zero with it, with backlog drain times printed.

☐ **T5 — Make a service stateless** *(≈60 min, advanced)*

- **Goal:** find and remove hidden state.
- **Steps:**
  1. Take any small web app you have (or `shop-api`) and list every place it keeps state: memory, local files, sticky sessions, in-process caches, cron on one host.
  2. For each, write the replacement (Redis, S3, database, a Kubernetes CronJob).
  3. Run two replicas locally (Docker Compose) and prove a user session survives a request landing on either.
- **Done when:** with two replicas behind Nginx, repeated `curl` requests with the same cookie succeed regardless of which replica answers.

☐ **T6 — Prove it: scaling review** *(≈45 min, advanced)*

- **Goal:** defend a scaling plan.
- **Steps:**
  1. Without the guide, write a one-page scaling plan for 10x growth of `shop-api`: assumptions, capacity numbers, first three bottlenecks, and what you will measure.
  2. Explain it aloud (or write it) as if to a skeptical staff engineer: why not just buy a bigger database?
- **Done when:** the plan names each bottleneck with a metric to detect it, and states one thing you deliberately will *not* build yet and why.

10 Self-check questions

1. What makes a service "stateless", and what three kinds of state must move out of the pod?
2. Why can adding more API pods make an outage worse?
3. State Little's Law and use it to size a connection pool.
4. What are the costs of adding a queue between the API and the database?
5. Why is the data tier harder to scale horizontally than the compute tier?

ANSWERS

1. Any replica can serve any request. Sessions, uploaded files and in-process caches/scheduled jobs must move to Redis or tokens, object storage, and shared or external schedulers.

2. Each pod adds database connections and load on shared dependencies. If the database or pooler is the bottleneck, more pods increase contention and failures.
3. Concurrency = throughput x latency. At 150 req/s with 200 ms per request holding a connection, you need about 30 connections of concurrency per pod (or a pooler that multiplexes).
4. Eventual consistency, duplicate delivery (so idempotent consumers), an extra system to run and monitor (depth and age), and harder debugging end to end.
5. State must be kept consistent: writes need a single owner or coordination, and partitioning forces cross-shard queries, transactions and rebalancing.

11 Further reading

- *Designing Data-Intensive Applications* by Martin Kleppmann — chapters on replication, partitioning and batch/stream processing.
- *Site Reliability Engineering* (Google, free online) — chapters on load balancing and handling overload.
- Amazon Builders' Library (aws.amazon.com/builders-library) — articles on load shedding and queue-based designs.
- AWS Well-Architected Framework — Performance Efficiency and Reliability pillars.
- *The Art of Scalability* by Abbott and Fisher — the scale cube (x, y, z axes).

LAB NOTES What I ran, what I saw, what it means

19.2 Availability

Topic 2 of 11 | ~4 hours | Difficulty: Advanced

1 What it is

Availability is the fraction of time (or fraction of requests) for which a system does what users need it to do. It is usually written as "nines": 99.9% is "three nines". Designing for availability means finding every **single point of failure (SPOF)** (a component whose failure takes the whole system down), adding redundancy across independent failure domains (instance, rack, **availability zone (AZ)**, region), and deciding in advance how the system behaves when something it depends on is broken (**graceful degradation**).

Availability is a *number you choose and then pay for*. Going from 99.9% to 99.99% typically means more redundancy, more automation, more testing, and a more expensive on-call practice. The architect's job is to pick the right target per component, not to maximise it everywhere.

2 Why it matters in DevOps

- **Customer trust and contracts:** an SLA (Service Level Agreement) with credits turns downtime into money. See SLOs §14.2 and SLAs §14.3.
- **Dependencies multiply:** a chain of five 99.9% components is not 99.9%; it is about 99.5%. Most "mystery" availability gaps are arithmetic nobody did.
- **Redundancy that was never tested is not redundancy.** Failover that has never been exercised usually fails when needed.
- **It shapes the bill:** multi-AZ roughly doubles some costs, multi-region more so. You need to know what you are buying.

3 Real-world system example

AWS regions and Availability Zones. AWS documents that each Region consists of multiple AZs, which are physically separate groups of data centres with independent power, cooling and networking, connected by low-latency links. Its Well-Architected guidance recommends

spreading workloads across at least two AZs, and services such as RDS Multi-AZ, ALB, and S3 are built around that model (S3 stores objects redundantly across multiple facilities in a Region for most storage classes). A well-known lesson from public AWS post-event summaries is that regional control-plane issues can still affect customers who are "multi-AZ" inside one Region, which is why teams with strict targets also design for *static stability* (data plane keeps working if the control plane is impaired) and sometimes multi-region. Google's SRE book makes the complementary point: you should not target more availability than your users can perceive or your dependencies (for example the network and phones) can deliver.

4 Diagram

A multi-AZ layout for `shop-api` with no single point of failure in the request path:

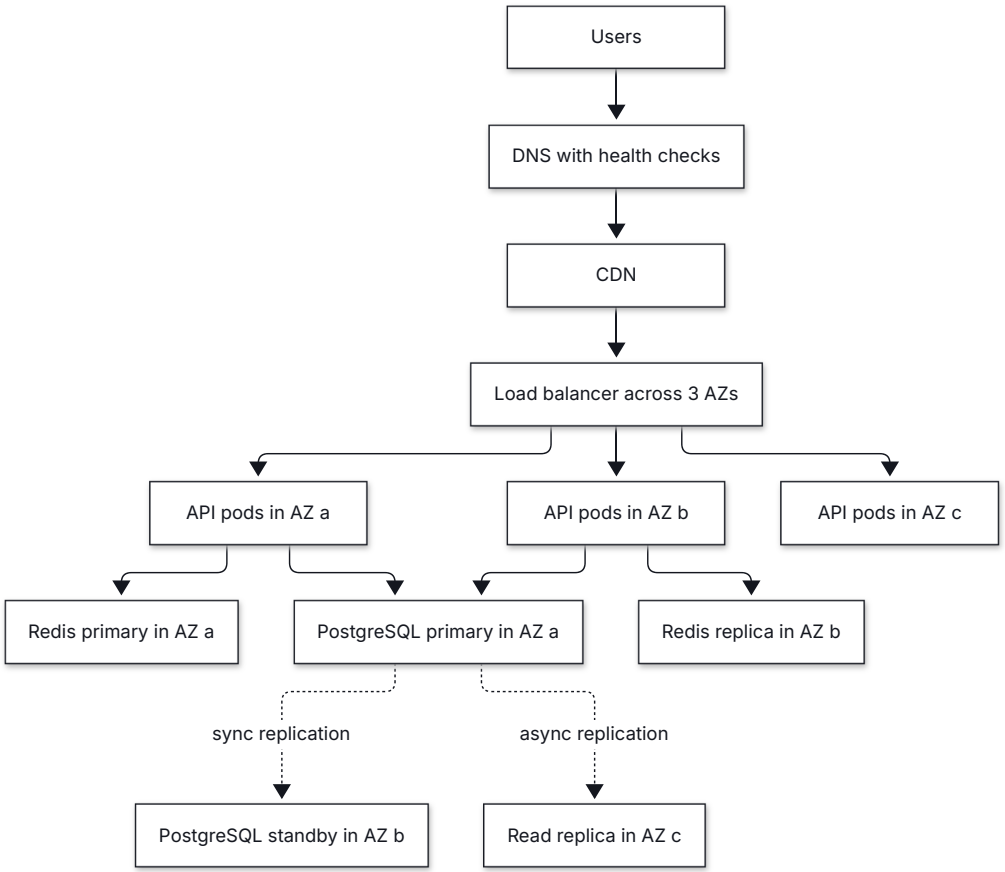


Figure 19.2-1

Graceful degradation as a state machine:

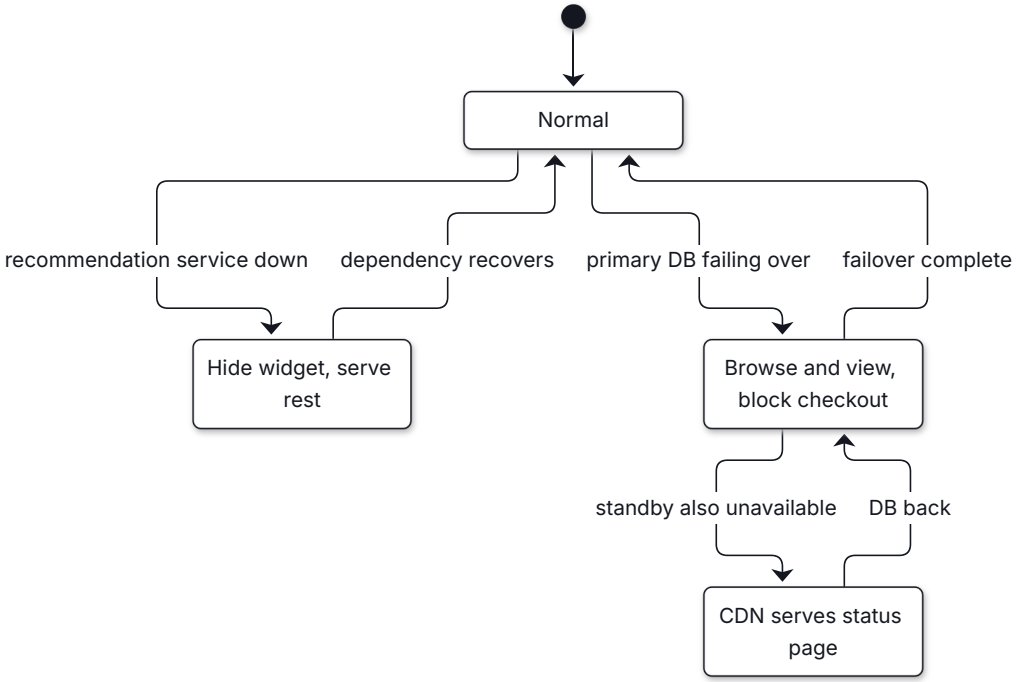


Figure 19.2-2

5 How it works

The nines table

Target	Downtime per 30 days	Downtime per year
99%	7.2 hours	3.65 days
99.9%	43.2 minutes	8.76 hours
99.95%	21.6 minutes	4.38 hours
99.99%	4.32 minutes	52.6 minutes
99.999%	26 seconds	5.26 minutes

At four nines you have ~4 minutes a month. A human cannot be paged, log in and diagnose in that time, so the recovery must be **automatic** (health checks, failover, rollbacks).

Series and parallel math

- **In series** (all must work): availability multiplies. `A = A1 x A2 x ... x An`. More hard dependencies always lowers availability.
- **In parallel** (any one works): unavailability multiplies. `A = 1 - (1-A1) x (1-A2)`. Two independent 99.9% replicas give 99.9999% *in theory*.
- The theory breaks when failures are **correlated**: same bad deploy, same config, same cloud region, same shared database. Real redundancy needs independent failure domains and independent change (do not roll out to all replicas at once).

Failure domains

Domain	Example failure	Mitigation
Process / pod	OOM kill, crash	Replicas, restart policy, probes
Node / instance	Hardware, kernel panic	Spread pods across nodes (anti-affinity, topology spread)
AZ	Power, network, cooling	Multi-AZ subnets, nodes in 3 AZs, replicated data
Region	Regional service event, major disaster	Multi-region active-passive or active-active, DR plan
Provider	Account suspension, global control plane issue	Backups outside the account, exit plan
Change	Bad deploy, bad config	Canary, staged rollout, fast rollback
Dependency	Payment API down	Timeouts, retries with backoff, circuit breakers, fallbacks

Most real outages come from **change** and **dependencies**, not hardware. See Circuit Breakers §12.14 and Failover §15.5.

Graceful degradation patterns

- **Shed non-essential features** (recommendations, avatars) before core flows (checkout).
- **Serve stale data** from cache when the origin is down (`stale-if-error` at the CDN).
- **Read-only mode** during database failover; queue writes where possible.
- **Load shedding:** reject excess traffic early with `503` rather than let everything time out.
- **Static fallback:** the CDN serves a status or cached catalogue page.

6 Step-by-step guide

Prerequisites: Python 3.9+, no cloud cost.

1. **Compute the nines table and a dependency chain.** Save as `nines.py`:

```python
def downtime_min(avail, days=30):
    return (1 - avail) * days * 24 * 60

for n in ["99", "99.9", "99.95", "99.99", "99.999"]:
    a = float(n) / 100
    print(f"{n:>7}% → {downtime_min(a):8.2f} min/30d  {downtime_min(a, 365)/60:7.2f}
  h/yr")

def series(*a):
    p = 1.0
    for x in a:
        p *= x
    return p

def parallel(*a):
    p = 1.0
    for x in a:
        p *= (1 - x)
    return 1 - p

# CDN, load balancer, app, database - all in the critical path
chain = series(0.9999, 0.999, 0.9995, 0.999)
print("series chain availability:", round(chain * 100, 4), "%")
print("two independent 99.9 app replicas:", round(parallel(0.999, 0.999) * 100, 6),
  "%")
```

```bash
python3 nines.py
```

Expected output:

```
      99% →   432.00 min/30d    87.60 h/yr
    99.9% →    43.20 min/30d     8.76 h/yr
   99.95% →    21.60 min/30d     4.38 h/yr
   99.99% →     4.32 min/30d     0.88 h/yr
 99.999% →     0.43 min/30d     0.09 h/yr
series chain availability: 99.7402 %
two independent 99.9 app replicas: 99.9999 %
```

Notice: four components that are each "three nines or better" combine to about 99.74%, which is roughly 112 minutes of downtime per 30 days. Redundancy at the database layer is what moves the chain, not more replicas of the app.

2. **Find the SPOFs.** Redraw the architecture from section 4 on paper and for every box ask: "If this disappears, what breaks and what is the recovery time?" Record the answers in a table with columns *component, failure mode, redundancy, detection, recovery time, tested on (date)*.
3. **Spread pods across zones in Kubernetes** (the manifest pattern; apply in a kind or cloud cluster if you have one).

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: shop-api }
spec:
  replicas: 6
  selector: { matchLabels: { app: shop-api } }
  template:
    metadata: { labels: { app: shop-api } }
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: DoNotSchedule
          labelSelector: { matchLabels: { app: shop-api } }
      containers:
        - name: api
          image: example.com/shop-api:1.0.0
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: shop-api }
spec:
  minAvailable: 4
  selector: { matchLabels: { app: shop-api } }
```

   With three zones and six replicas you get two pods per zone; losing a zone leaves four, which equals the PDB `minAvailable`. See Pods §8.2 and Scheduling §8.13.
4. **Compute the capacity you need to survive an AZ loss ("N+1 across zones").** If peak needs 52 pods at 60% target utilisation, losing 1 of 3 zones leaves 2/3 of capacity:

```bash
python3 -c "import math; need=52; zones=3; print('pods total:',
  math.ceil(need*zones/(zones-1)))"
```

   Expected: `pods total: 78`. You are paying for 50% extra capacity as the price of zone-failure tolerance (or you accept degraded performance and rely on headroom and autoscaling).

5. **Decide the degradation ladder.** Write a table of what the user sees when each dependency is down:

Dependency down	User impact	Mechanism
Recommendations	Widget hidden	Timeout 200 ms, fallback to empty list
Redis	Slower pages	Bypass cache, rate-limit expensive queries
Payment provider	Checkout shows "try again"	Circuit breaker, queue order for retry
PostgreSQL primary	Read-only browsing	Failover to standby, API returns 503 on writes with Retry-After

6. **Test the redundancy.** In a sandbox cluster, delete one pod (`kubectl delete pod`), then drain a node (`kubectl drain <node> --ignore-daemonsets`) while running `curl` in a loop against the service. Count failed requests. Record the result: this is evidence, not an assumption.

7 Real-world scenario

(Illustrative, not a documented incident.) A team advertises 99.95% and runs two API replicas, a managed PostgreSQL in one AZ, and a single NAT gateway. One evening the AZ hosting the database and NAT has a networking event. The API pods in the other AZ are healthy but cannot reach the database or the internet. The page goes out; recovery is a manual restore of the database into a second AZ and takes 70 minutes. The month's budget for 99.95% is 21.6 minutes.

The review finds: the "two replicas" were redundancy for the *easy* tier, while the data tier and the egress path were SPOFs. Fixes: Multi-AZ database with automatic failover, one NAT gateway per AZ, a read-only degraded mode, a game day twice a year, and the SLO is restated at 99.9% until those are proven. **Lesson:** availability is the weakest component in the critical path, and untested redundancy is a belief, not a control.

8 Common mistakes and best practices

Don't

- Multiply nines in your head: compute them, including every hard dependency (DNS, auth, payments, the CI that gates rollbacks).
- Count replicas in the same AZ, same deploy, or sharing the same database as independent.
- Promise 99.99% with a manual failover runbook.

- Retry without backoff and jitter: retries from a thousand clients become a self-inflicted outage.
- Make a non-critical feature a hard dependency of the critical path.

Do

- Draw the critical path and write a target per component and per user journey.
- Use three AZs for stateful quorum systems and spread stateless pods with topology constraints.
- Set timeouts everywhere, use circuit breakers, and define the fallback behaviour in code and in the design doc.
- Roll changes out progressively; most outages follow a change.
- Run game days: kill a zone, a node, a dependency, and write down what actually happened.

9 Tasks

- [ ] **T1 — Nines calculator** *(≈30 min, intermediate)*
  - **Goal:** convert between availability targets and downtime budgets.
  - **Steps:**
    1. Create `nines.py` from step 6.1 and run it.
    2. Add a function `budget(avail, window_days)` returning minutes, and print 30-day and 90-day budgets for 99.9% and 99.95%.
    3. Compute how many 20-minute incidents per quarter a 99.9% target allows.
  - **Done when:** the output shows 129.6 minutes for 99.9% over 90 days, and you state that this allows 6 such incidents per quarter.

- [ ] **T2 — Critical-path availability** *(≈40 min, intermediate)*
  - **Goal:** compute a realistic composite availability.
  - **Steps:**
    1. List the hard dependencies of one `shop-api` checkout request with assumed availabilities (use obviously labelled example numbers: DNS 99.99%, CDN 99.99%, LB 99.99%, API 99.95%, Redis 99.9%, PostgreSQL 99.95%, payment provider 99.9%).
    2. Compute the series result.
    3. Show the effect of making Redis optional (cache bypass) and the payment call asynchronous (queued).

- **Done when:** you have two numbers (before and after) and the written reason each change helps.

- [ ] **T3 — SPOF hunt** *(≈45 min, advanced)*
  - **Goal:** find hidden single points of failure.
  - **Steps:**
    1. Take the diagram from section 4 and extend it with DNS, secrets, CI/CD, the container registry, the IdP (login), and NAT.
    2. For each, state if it is in the runtime critical path or only in the deploy path.
    3. Mark five SPOFs and write a mitigation plus its cost class (low, medium, high) for each.
  - **Done when:** the table has five SPOFs with detection, mitigation and a statement of what you accept.

- [ ] **T4 — Zone-failure drill (kind or cloud)** *(≈60 min, advanced)*
  - **Goal:** observe real behaviour when capacity disappears.
  - **Steps:**
    1. Create a kind cluster with 3 worker nodes labelled with `topology.kubernetes.io/zone=a|b|c`.
    2. Apply the Deployment and PDB from step 6.3 with `nginx` as the image.
    3. Run `while true; do curl -s -o /dev/null -w "%{http_code}\n" <service-url>; sleep 0.2; done | sort | uniq -c` in one terminal, then cordon and drain one node in another.
  - **Done when:** you can show the status-code counts, the remaining pod distribution (`kubectl get pods -o wide`), and explain whether the PDB blocked or allowed the drain.

- [ ] **T5 — Design a degradation ladder** *(≈45 min, advanced)*
  - **Goal:** decide failure behaviour before the failure.
  - **Steps:**
    1. List 6 dependencies of `shop-api` and classify each as critical or optional.
    2. For each optional one specify timeout, fallback response and user-visible effect.
    3. Implement one fallback in code (for example return cached or empty recommendations if the call takes longer than 200 ms).
  - **Done when:** a test that makes the dependency sleep 2 s still returns a `200` from your endpoint within about 250 ms.

- [ ] **T6 — Prove it: defend a target** *(≈45 min, advanced)*
  - **Goal:** choose and justify an availability target.
  - **Steps:**
    1. Without the guide, choose SLO targets for three components (CDN-served pages, checkout API, nightly report job) and justify each in a sentence.
    2. Show the composite availability and the monthly downtime budget.
    3. State what you would need to add to move the checkout API one nine higher and what it would cost.
  - **Done when:** each target has a number, a budget in minutes, a justification and a list of required controls.

10 Self-check questions

1. What is the availability of three 99.9% components in series, and of two in parallel (assuming independence)?
2. Why does the independence assumption often fail for replicas?
3. What are the typical most common causes of outages, and which design practice addresses each?
4. Why is a 99.99% target incompatible with a manual runbook-only recovery?
5. What does "static stability" mean?

ANSWERS

1. Series: 0.999^3, about 99.7%. Parallel: 1 - 0.001^2 = 99.9999%.
2. Replicas share deployment, configuration, region, a database or a dependency, so one bad change or event kills them together.
3. Changes (progressive rollout, fast rollback) and dependencies (timeouts, breakers, fallbacks), plus capacity exhaustion (load shedding, headroom) and infrastructure failure (multi-AZ).
4. The monthly budget is about 4 minutes; paging, diagnosis and action by a human cannot reliably fit. Detection and recovery must be automatic.
5. The system keeps operating in its current state when a control plane or dependency is impaired, because it does not need to make changes (scale, fail over) to keep serving; capacity is pre-provisioned.

11 Further reading

- *Site Reliability Engineering* and *The Site Reliability Workbook* (Google, free online) — embracing risk, service level objectives.
- AWS Well-Architected Framework — Reliability pillar.
- Amazon Builders' Library — articles on static stability and avoiding fallback in distributed systems.
- *Release It!* by Michael Nygard — stability patterns (timeouts, bulkheads, circuit breakers).
- Kubernetes documentation — Pod Topology Spread Constraints and Disruptions (PodDisruptionBudget).

LAB NOTES What I ran, what I saw, what it means

19.3 Security Architecture

Topic 3 of 11 | ~4 hours | Difficulty: Advanced

1 What it is

Security architecture is designing a system so that a single mistake, bug or stolen credential does not become a breach. Instead of one strong wall, you build several independent layers (**defense in depth**), you refuse to trust a request just because of where it came from (**zero trust**), you systematically ask "what can go wrong?" before building (**threat modeling**, commonly with **STRIDE**), and you shrink the damage any one compromise can do (**blast radius**).

STRIDE is a checklist of six threat categories: **S**poofing (pretending to be someone else), **T**ampering (modifying data or code), **R**epudiation (denying an action with no proof), **I**nformation disclosure (leaking data), **D**enial of service (making the system unavailable), and **E**levation of privilege (gaining rights you should not have).

2 Why it matters in DevOps

- **Most breaches chain small weaknesses**: an exposed dashboard, an over-broad IAM role, a secret in Git. Architecture breaks the chain.
- **You build the paths attackers use**: CI/CD pipelines, Kubernetes service accounts, cloud roles and registries are the new perimeter.
- **Retrofitting is expensive**: network segmentation, identity-based auth and encryption are cheap at design time and painful after launch.
- **Compliance and customer reviews** ask for threat models, data-flow diagrams, and evidence of least privilege.

Related reading: DevSecOps §0.12, Least-privilege IAM §11.2, mTLS §12.12, NetworkPolicies §8.19.

3 Real-world system example

Google BeyondCorp and the zero-trust model. Google has publicly described BeyondCorp, its move from a privileged corporate network to a model where access to applications depends on the identity of the user and the state of the device, not on being "inside the net-

work". NIST published SP 800-207 (Zero Trust Architecture), which formalises the same ideas. **Capital One (2019)** is a widely documented cautionary case: public reporting and the US government complaint described a misconfigured web application firewall that let an attacker obtain credentials for an over-privileged cloud role and read data from storage; the lessons usually drawn are least-privilege roles, metadata-service hardening and detection, not "the cloud is insecure". **Kubernetes** itself bakes in layering: authentication, RBAC authorization, admission control, Pod Security Standards, NetworkPolicies and audit logs are separate layers. (Treat details of any incident as a pointer to read the primary sources, not as a complete account.)

4 Diagram

Defense in depth for a request to `shop-api`, with a control at every layer:

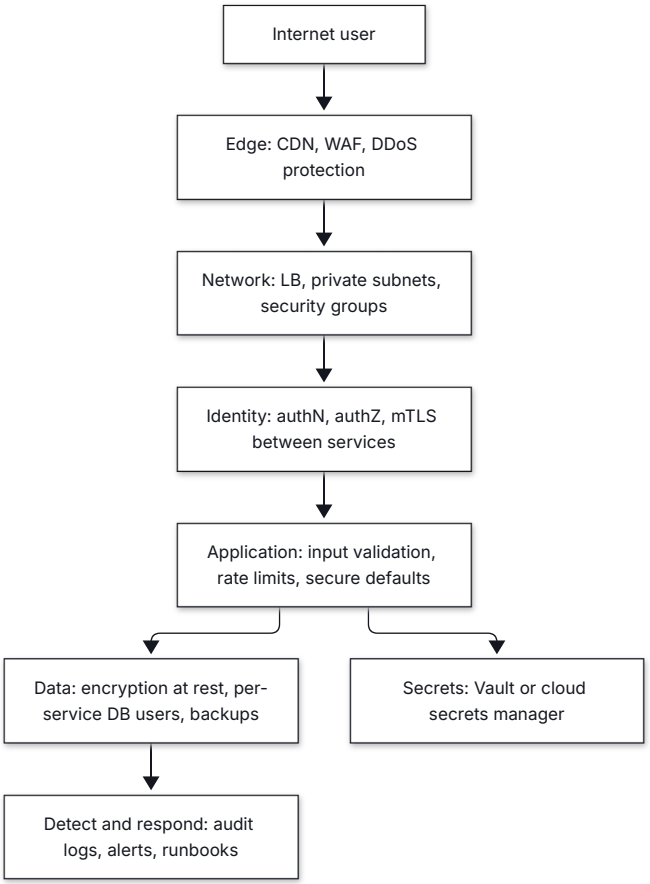


Figure 19.3-1

A data flow diagram with trust boundaries, the starting point for STRIDE:

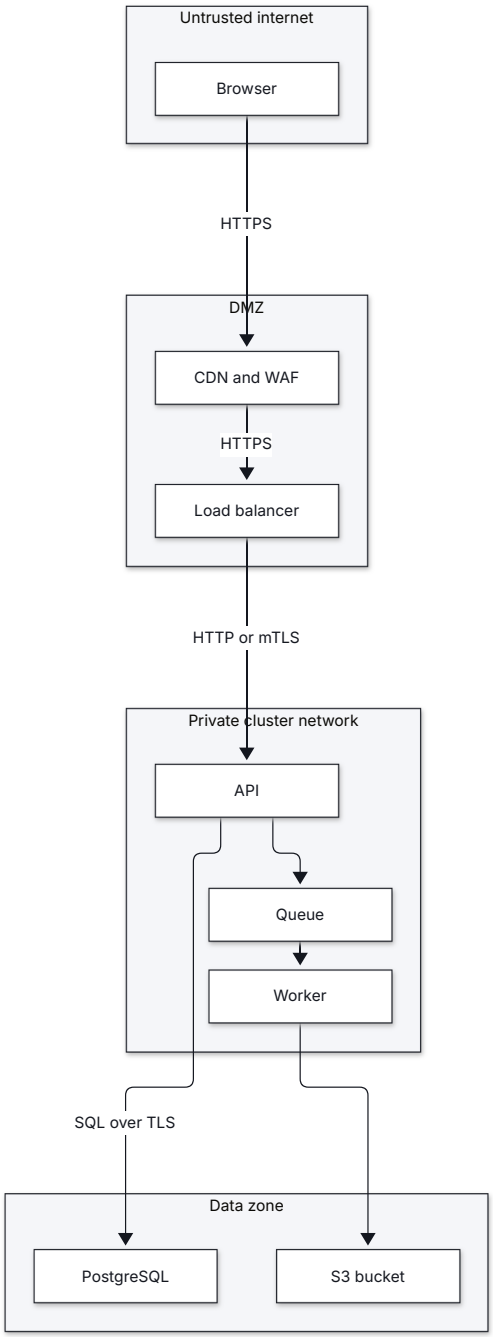


Figure 19.3-2

5 How it works

Defense in depth

Assume any single control will fail and make sure another still stands. Layers for a typical cloud-native app:

Layer	Example controls
Edge	CDN, WAF (Web Application Firewall), DDoS protection, bot rules
Network	Private subnets, security groups, Kubernetes NetworkPolicies default-deny
Identity	OIDC login, short-lived tokens, mTLS between services, workload identity (no static cloud keys)
Compute	Non-root minimal images, Pod Security Standards, read-only root filesystem, patched nodes
Application	Input validation, parameterised queries, authorization checks per object
Data	Encryption in transit and at rest (KMS), column-level protection for secrets/PII, backups in another account
Supply chain	Image scanning, pinned dependencies, signed images, protected branches, CI with least privilege
Detect and respond	Audit logs (CloudTrail, Kubernetes audit), alerts on anomalies, practised incident response

Zero trust in practice

Core principles (from NIST SP 800-207): never trust based on network location; authenticate and authorize *every* request; use least privilege; assume breach; log everything. In a cluster this translates to mTLS between workloads (service mesh or SPIFFE-style identities), authorization policies per service, per-workload cloud identity instead of node-wide roles, and short-lived credentials. See Service Mesh §12.9.

Threat modeling with STRIDE

Process: (1) draw a data flow diagram with **trust boundaries**; (2) for every element and flow, ask each STRIDE question; (3) record a threat, a mitigation and an owner; (4) decide: mitigate, accept, transfer, or avoid; (5) revisit when the architecture changes.

Category	Violates	Typical question	Typical mitigation
Spoofing	Authentication	Could someone pretend to be this service or user?	mTLS, OIDC, signed tokens, MFA
Tampering	Integrity	Could data be changed in flight or at rest?	TLS, signatures, immutable logs, DB permissions

Category	Violates	Typical question	Typical mitigation
Repudiation	Non-repudiation	Could someone deny an action?	Audit logs with identity, append-only storage
Information disclosure	Confidentiality	Could data leak?	Encryption, least privilege, no secrets in logs
Denial of service	Availability	Could this be overwhelmed?	Rate limits, autoscaling, quotas, WAF
Elevation of privilege	Authorization	Could a low-privilege actor gain more?	RBAC, no root, validated input, segmentation

Blast radius

Blast radius is how much is lost if one thing is compromised or breaks. Reduce it with separate cloud accounts per environment, separate IAM roles per workload, namespaces with quotas and default-deny network policies, per-service database users, and **cells** (independent copies of the stack serving subsets of customers). Smaller blast radius also helps availability: the same design serves both goals. See RBAC §8.18.

6 Step-by-step guide

Prerequisites: Python 3.9+. Optional: Docker and `kind` for step 5. No cloud cost.

1. **Draw the data flow diagram.** Reuse the second diagram in section 4 and list every arrow as a numbered flow. Mark each trust boundary crossing (internet to DMZ, DMZ to cluster, cluster to data zone). Threats cluster at crossings.
2. **Fill in a STRIDE worksheet.** Create `stride.py`; it records mitigations per flow and reports gaps:

```python
STRIDE = ["Spoofing", "Tampering", "Repudiation", "Information disclosure",
          "Denial of service", "Elevation of privilege"]

model = {
    "Browser → CDN": {
        "Spoofing": "TLS 1.2+, HSTS",
        "Tampering": "TLS",
        "Denial of service": "CDN rate limiting and WAF",
    },
    "CDN → Load balancer": {
        "Spoofing": "Origin only accepts CDN header and IP ranges",
        "Tampering": "TLS to origin",
    },
    "API → PostgreSQL": {
        "Spoofing": "mTLS or IAM auth, per-service DB user",
```

```
        "Tampering": "Least-privilege grants, parameterised queries",
        "Information disclosure": "TLS, encryption at rest",
        "Elevation of privilege": "No superuser from the app",
    },
    "Worker → Queue": {"Tampering": "Signed or validated messages"},
}

total = covered = 0
for element, threats in model.items():
    missing = [s for s in STRIDE if s not in threats]
    total += len(STRIDE)
    covered += len(STRIDE) - len(missing)
    print(f"{element}: {len(threats)}/6 covered; open: {', '.join(missing) or
  'none'}")
print(f"overall coverage: {covered}/{total} = {covered/total:.0%}")
```

BASH

```
python3 stride.py
```

Expected output:

```
Browser → CDN: 3/6 covered; open: Repudiation, Information disclosure, Elevation of
  privilege
CDN → Load balancer: 2/6 covered; open: Repudiation, Information disclosure, Denial
  of service, Elevation of privilege
API → PostgreSQL: 4/6 covered; open: Repudiation, Denial of service
Worker → Queue: 1/6 covered; open: Spoofing, Repudiation, Information disclosure,
  Denial of service, Elevation of privilege
overall coverage: 10/24 = 42%
```

Not every cell must be filled: some are "not applicable" or "accepted". The point is that each gap is a *decision*, written down, not an oversight.

3. **Quantify blast radius for different designs.**

PYTHON

```
customers = 120_000   # illustrative
for design, cells in [("single shared stack", 1), ("4 cells", 4), ("20 cells", 20)]:
    print(f"{design:20s} → one cell holds {customers/cells:>9,.0f} customers
  ({100/cells:5.1f}%)")
```

Expected:

```
single shared stack  → one cell holds   120,000 customers (100.0%)
4 cells              → one cell holds    30,000 customers ( 25.0%)
20 cells             → one cell holds     6,000 customers (  5.0%)
```

4. **Write the mitigation backlog.** For each open STRIDE gap decide: mitigate / accept / transfer / avoid, an owner and a ticket. Example: *Worker -> Queue, Spoofing: mitigate, use per-worker credentials with publish/consume permissions limited to one queue.*
5. **Implement one layer: Kubernetes default-deny and an explicit allow** (needs a CNI that enforces NetworkPolicy, such as Calico or Cilium; kind's default CNI does not enforce it).

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny, namespace: shop }
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: api-to-postgres, namespace: shop }
spec:
  podSelector: { matchLabels: { app: postgres } }
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector: { matchLabels: { app: shop-api } }
      ports:
        - { protocol: TCP, port: 5432 }
```

   Remember that default-deny egress also blocks DNS; you must add an allow rule for CoreDNS. See NetworkPolicies in Depth §16.2 and Pod Security §16.3.
6. **Review identities, not just networks.** List every credential in the system (DB passwords, cloud roles, CI tokens, registry pull secrets, Sentry DSN) with: who holds it, scope, lifetime, rotation. Anything long-lived and broad is the first target for replacement with short-lived workload identity. See Secrets Management and Encryption §11.4.

7 Real-world scenario

(Illustrative, not a documented incident.) A developer accidentally exposes the `shop-api` debug endpoint `/debug/env` on the public ingress. An attacker finds it and reads environment variables including a database URL and a cloud access key. What happens next is decided by architecture, not luck.

Because the database sits in a private subnet and its security group only allows the API pods, the database URL is useless from the internet. Because the cloud key was replaced with workload identity (the pod gets a role limited to one S3 prefix, via short-lived tokens) the stolen key expires within an hour and can read only that prefix. CloudTrail alerts on the un-

usual role usage from a foreign IP and the team rotates within 20 minutes. The incident is a bug plus a contained exposure instead of a breach. **Lesson:** each layer assumed the previous one would fail; blast radius, not perfection, was the design goal.

8 Common mistakes and best practices

Don't

- Rely on a network perimeter ("it is inside the VPC, so it is trusted").
- Share one credential or one IAM role across services and environments.
- Do threat modeling once, as a document nobody updates, or only after launch.
- Put long-lived cloud keys in CI variables or container images.
- Treat the build pipeline as outside the threat model; it can push to production.

Do

- Model trust boundaries explicitly and run STRIDE on each crossing.
- Default-deny at the network and IAM layers, then add narrow allows.
- Use per-workload identities and short-lived credentials; rotate what cannot be short-lived.
- Keep separate accounts or projects for production, staging and security tooling.
- Log security-relevant events to storage that attackers in the app cannot modify, and alert on them.
- Prefer secure defaults in the platform (Pod Security Standards, admission policies) over code review alone.

9 Tasks

- [ ] **T1 — Draw the data flow diagram with trust boundaries** *(≈40 min, intermediate)*
  - **Goal:** produce the artefact STRIDE needs.
  - **Steps:**
    1. Create a Mermaid flowchart of `shop-api` with at least 4 trust boundaries (use `subgraph`).
    2. Number every data flow and note the protocol and authentication used.
    3. Commit it to a repo folder `docs/security/dfd.md`.
  - **Done when:** the diagram renders on GitHub and each arrow has a number, a protocol and an auth method.

- [ ] **T2 — STRIDE worksheet** *(≈60 min, advanced)*
  - **Goal:** find at least 10 distinct threats and handle them.
  - **Steps:**
    1. Using your DFD, create a table with columns: flow, STRIDE category, threat, likelihood (L/M/H), impact (L/M/H), mitigation, decision (mitigate, accept, transfer, avoid), owner.
    2. Cover every flow crossing a trust boundary against all six categories.
    3. Extend `stride.py` to read your table from a dict and print uncovered cells.
  - **Done when:** at least 10 threats are recorded, each with a decision, and the script lists every unaddressed STRIDE cell.

- [ ] **T3 — Blast radius budget** *(≈40 min, advanced)*
  - **Goal:** compare designs by damage if one component is compromised.
  - **Steps:**
    1. For each of: one shared database user, one DB user per service, one cloud role for the cluster nodes, one role per workload, one cloud account vs three accounts, write what an attacker with the app's credentials can read or change.
    2. Express the exposure as a percentage of data or customers (use clearly labelled assumptions).
    3. Pick the cheapest change that cuts the exposure most.
  - **Done when:** you have a before/after table and a single recommended change with a justification.

- [ ] **T4 — Default-deny network in a real cluster** *(≈60 min, advanced)*
  - **Goal:** implement and test a network segmentation layer.
  - **Steps:**
    1. Create a kind cluster with Calico or Cilium (see their quickstarts) so NetworkPolicy is enforced.
    2. Deploy `nginx` labelled `app=shop-api` and `postgres` (or `nginx` pretending) labelled `app=postgres` in namespace `shop`, plus a `test` pod labelled `app=other`.
    3. Apply the default-deny and allow policies from step 6.5, plus a DNS allow rule.
    4. Test with `kubectl exec` and `curl`/`nc` from both pods.
  - **Done when:** `shop-api` reaches `postgres:5432` while `app=other` times out, and you show both outputs.

- [ ] **T5 — Credential inventory and rotation plan** *(≈45 min, intermediate)*
  - **Goal:** find where long-lived secrets hide.

- **Steps:**
  1. List every secret in your sample project (repo files, CI variables, Kubernetes Secrets, env vars).
  2. Run a secret scanner on the repository (for example `gitleaks detect` if installed) and record results.
  3. For each credential, write scope, lifetime and a replacement (workload identity, short-lived token, secrets manager reference).
- **Done when:** the inventory table exists, the scan result is attached, and each long-lived credential has a replacement plan.

☐ **T6 — Prove it: attack walk-through** *(≈60 min, advanced)*

- **Goal:** explain an attack path and where your architecture stops it.
- **Steps:**
  1. Without the guide, write an attack path for "stolen CI token to production data" as a numbered chain.
  2. For each step name the layer and control that should stop or detect it.
  3. Identify the weakest link and propose a change.
- **Done when:** the chain has at least five steps, each mapped to a control, and the weakest link has a concrete fix with an owner.

10 Self-check questions

1. What does STRIDE stand for and which security property does each letter violate?
2. What is the difference between defense in depth and zero trust?
3. Why are trust boundary crossings the most important places to look for threats?
4. Name three ways to reduce blast radius in Kubernetes and the cloud.
5. Why does default-deny egress usually break DNS first?

ANSWERS

1. Spoofing (authentication), Tampering (integrity), Repudiation (non-repudiation), Information disclosure (confidentiality), Denial of service (availability), Elevation of privilege (authorization).
2. Defense in depth is multiple independent layers of control. Zero trust is the principle of verifying every request by identity and context rather than network location. Zero trust is one way to implement the identity layer of defense in depth.
3. Data crosses between different levels of trust and control there, so authentication, validation and encryption must be enforced and attackers have the most leverage.

4. Namespaces with RBAC and quotas, default-deny NetworkPolicies, per-workload service accounts and cloud roles, separate clusters or accounts per environment, cells for customers.
5. Pods need to resolve names via CoreDNS (UDP/TCP 53). If egress is denied by default and no rule allows DNS, name lookups fail before any application traffic.

11 Further reading

- NIST SP 800-207, *Zero Trust Architecture* (free).
- *Threat Modeling: Designing for Security* by Adam Shostack.
- OWASP Threat Modeling Cheat Sheet and OWASP Top 10 (owasp.org).
- Kubernetes documentation — Security concepts, Pod Security Standards, Network Policies.
- AWS Well-Architected Framework — Security pillar. Google's BeyondCorp papers (research.google).

LAB NOTES What I ran, what I saw, what it means

19.4 Performance

Topic 4 of 11 | ~4 hours | Difficulty: Advanced

1 What it is

Performance in architecture means how quickly and efficiently a system responds under a given load. Two numbers matter: **latency** (how long one request takes) and **throughput** (how many requests per second the system sustains). They are linked: past a certain load, queues form and latency climbs steeply.

Averages hide the truth. If 1 request in 100 takes 2 seconds, the average looks fine while 1% of users (and, as you will see, many more of your *page views*) have a bad experience. So we talk in **percentiles**: p50 (median), p95, p99. The slow end is **tail latency**. A **latency budget** divides the total time you allow (for example 300 ms) between the components on the path, so each team knows what it may spend.

2 Why it matters in DevOps

- **Latency is a product feature and a revenue lever**: slow checkouts convert less, and slow APIs time out their callers, which retry, which overloads you further.
- **SLOs are written in percentiles**: "99% of requests under 300 ms" is a performance target you must design, measure and alert on. See SLOs §14.2.
- **Performance is cost**: a 2x faster service needs half the servers.
- **Regressions ship silently** unless a load test or benchmark runs in the pipeline.

3 Real-world system example

Google's "The Tail at Scale". Jeff Dean and Luiz Barroso's paper (Communications of the ACM, 2013) describes how large fan-out services see tail latency dominate: when a request touches many backends, the chance that at least one is slow grows quickly, so the response is slow even when each backend is usually fast. The paper describes mitigation techniques such as hedged requests (send a duplicate after a short delay and take the first reply) and tied requests. **Content delivery networks** (Cloudflare, Fastly, CloudFront, Akamai) exist because

physics sets a floor: light in fibre travels roughly 200,000 km per second, so a round trip across an ocean costs tens of milliseconds before any code runs; caching near the user removes the trip. **PostgreSQL** itself documents how the planner uses indexes and how `EXPLAIN ANALYZE` reveals where time goes, which is the daily tool for database performance work.

4 Diagram

Where time goes in one `shop-api` checkout request (a latency budget made visible):

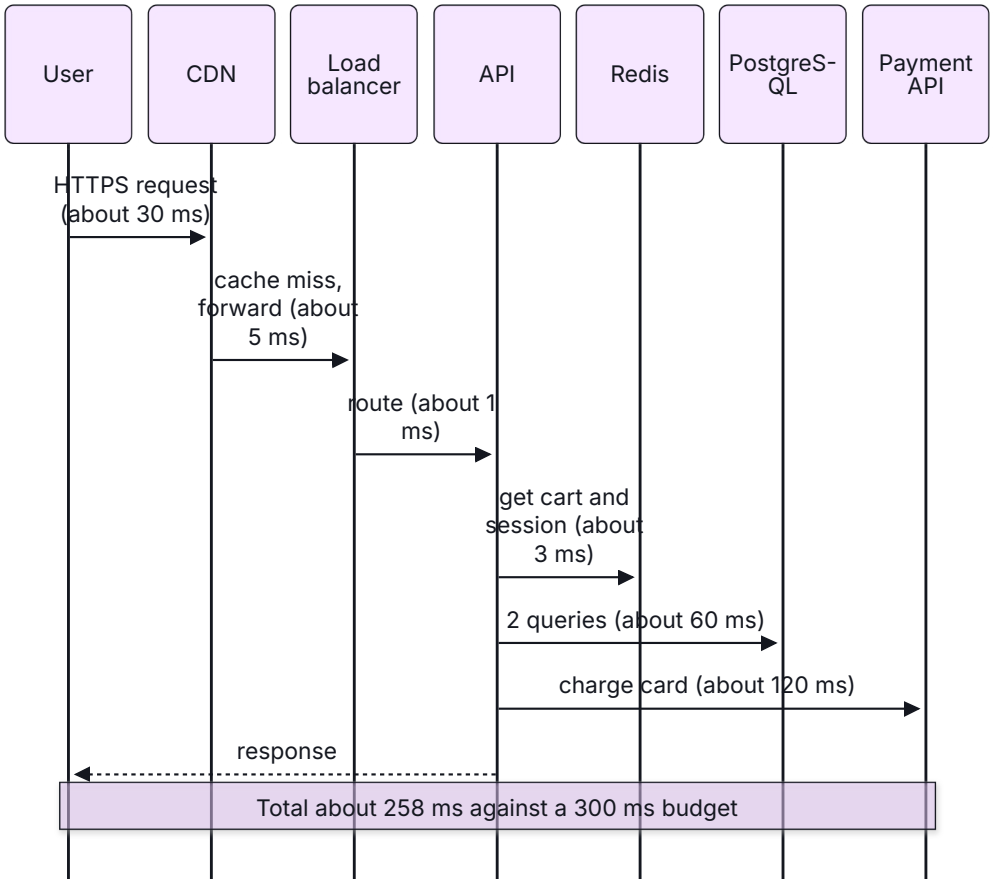


Figure 19.4-1

Caching layers, from closest to the user to closest to the data:

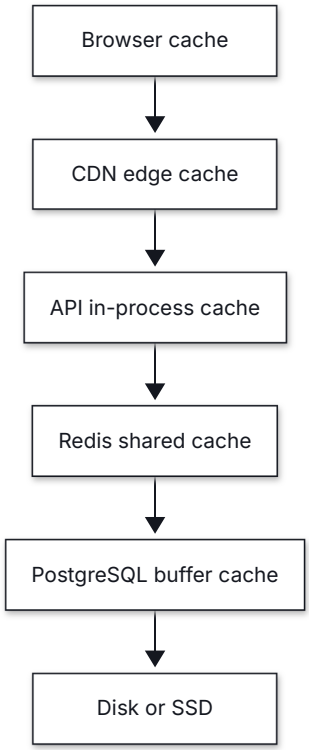


Figure 19.4-2

5 How it works

Latency budgets

Pick the user-facing target (for example p99 under 300 ms for checkout), subtract network and TLS time, then allocate the remainder to components. When a component exceeds its allocation you know where to work. Budgets also set **timeouts**: a downstream timeout must be shorter than the caller's own deadline or you will waste work on requests nobody is waiting for.

Caching layers

Layer	Typical TTL	Good for	Watch out for
Browser	minutes to a year (versioned assets)	static assets	stale assets without file-name hashing

Layer	Typical TTL	Good for	Watch out for
CDN	seconds to hours	public pages, images, API GETs without user data	caching private data by mistake (`Vary`, `Cache-Control: private`)
Application / Redis	seconds to minutes	computed results, sessions, lookups	invalidation, stampedes, cold start
Database buffer cache	automatic	hot rows and indexes	working set larger than RAM

See CDNs §12.7 and Redis §13.7.

Profiling: find the real cost first

Never optimise by guessing. Use the right tool for the question:

Question	Tool
Which code path burns CPU?	CPU profiler / flame graph (for Node.js `node --prof`, `clinic`, or `perf`; for Go `pprof`)
Which request is slow and why?	Distributed tracing (OpenTelemetry) shows spans per hop
Which SQL is slow?	`pg_stat_statements`, `EXPLAIN (ANALYZE, BUFFERS)`, slow query log
Is the machine saturated?	USE method: utilisation, saturation, errors for CPU, memory, disk, network
Is the app waiting?	Connection pool metrics, event-loop lag, queue depth

See Distributed Tracing §10.8 and PostgreSQL Indexes §13.1.

Load testing

A load test applies realistic traffic to find capacity and regressions. Types: **smoke** (does it work), **load** (expected peak), **stress** (beyond peak until it breaks), **soak** (hours, finds leaks), **spike** (sudden burst). Use realistic data and a **closed-model vs open-model** awareness: a closed model (fixed number of virtual users who wait for responses) hides overload because slow responses reduce the offered load ("coordinated omission"); an open model (fixed arrival rate) shows real queueing. Tools: k6, Locust, wrk, vegeta, Gatling.

Tail latency

Causes: garbage collection pauses, noisy neighbours, retries, lock contention, cold caches, connection setup, queueing. The fan-out effect: if one request calls `n` backends and each exceeds the target with probability `p`, the chance at least one does is `1 - (1-p)^n`.
Mitigations: timeouts and deadlines, hedged requests for idempotent reads, load shedding, bulkheads (separate pools), smaller fan-out, and caching.

6 Step-by-step guide

Prerequisites: Python 3.9+; optional Docker for step 7. Everything runs locally on loopback; no cost.

1. **Budget the request.** Save as `budget.py`:

```python
budget_ms = 300
parts = {"CDN+TLS": 30, "LB": 5, "API handler": 40, "Redis": 3,
         "PostgreSQL (2 queries)": 60, "Payment API": 120}
total = sum(parts.values())
for k, v in parts.items():
    print(f"{k:24s}{v:5d} ms {v/budget_ms:6.1%}")
print(f"{'TOTAL':24s}{total:5d} ms  budget {budget_ms} ms  headroom {budget_ms-total}
  ms")
```

```bash
python3 budget.py
```

Expected output:

```
CDN+TLS                     30 ms  10.0%
LB                           5 ms   1.7%
API handler                 40 ms  13.3%
Redis                        3 ms   1.0%
PostgreSQL (2 queries)      60 ms  20.0%
Payment API                120 ms  40.0%
TOTAL                      258 ms  budget 300 ms  headroom 42 ms
```

The payment provider is 40% of the budget and outside your control, so you protect the rest with a timeout and an async fallback.

2. **Compute fan-out tail risk.** Save as `fanout.py`:

```python
p_slow = 0.01   # 1% of calls to a backend exceed the latency target
for n in (1, 5, 10, 50, 100):
    print(f"fan-out {n:>3}: P(at least one slow call) = {1-(1-p_slow)**n:6.1%}")
```

```bash
python3 fanout.py
```

Expected output:

```
fan-out   1: P(at least one slow call) =   1.0%
fan-out   5: P(at least one slow call) =   4.9%
fan-out  10: P(at least one slow call) =   9.6%
fan-out  50: P(at least one slow call) =  39.5%
fan-out 100: P(at least one slow call) =  63.4%
```

A page that calls 50 services with a "99% fast" dependency is slow on four views in ten.

3. **Create a server with a realistic long tail.** Save as `server.py` (98% of responses take about 20 ms, 2% take about 400 ms):

```python
import random, time
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        time.sleep(0.4 if random.random() < 0.02 else 0.02)
        self.send_response(200)
        self.send_header("Content-Length", "2")
        self.end_headers()
        self.wfile.write(b"ok")
    def log_message(self, *a):
        pass

class S(ThreadingHTTPServer):
    request_queue_size = 128   # avoid SYN backlog effects distorting the test

S(("127.0.0.1", 8099), H).serve_forever()
```

```bash
python3 server.py &
```

4. **Write a small load generator that reports percentiles.** Save as `loadtest.py`:

```python
import statistics, time, urllib.request
from concurrent.futures import ThreadPoolExecutor

URL, N, CONC = "http://127.0.0.1:8099/", 2000, 50

def one(_):
    t = time.perf_counter()
    urllib.request.urlopen(URL).read()
    return (time.perf_counter() - t) * 1000
```

```
start = time.perf_counter()
with ThreadPoolExecutor(CONC) as ex:
    lat = sorted(ex.map(one, range(N)))
elapsed = time.perf_counter() - start

def pct(p):
    return lat[min(len(lat) - 1, int(len(lat) * p / 100))]

print(f"requests={N} concurrency={CONC} throughput={N/elapsed:.0f} req/s")
print(f"mean={statistics.mean(lat):.0f} ms  p50={pct(50):.0f} ms  "
      f"p95={pct(95):.0f} ms  p99={pct(99):.0f} ms  max={lat[-1]:.0f} ms")
```

BASH

```
python3 loadtest.py
```

Output varies by machine; expect something **similar to**:

```
requests=2000 concurrency=50 throughput=800 req/s
mean=50 ms  p50=28 ms  p95=120 ms  p99=600 ms  max=1270 ms
```

The point: the median is near 20-30 ms, the mean is about twice that, and p99 is more than ten times the median. (The Python client and its GIL add some noise of their own; production tests use k6 or wrk.) Stop the server with `kill %1`.

5. **Repeat with k6 for a realistic open-model test** (install from the official k6 docs). Save as `test.js`:

JAVASCRIPT

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  scenarios: {
    steady: { executor: 'constant-arrival-rate', rate: 200, timeUnit: '1s',
              duration: '1m', preAllocatedVUs: 50, maxVUs: 200 },
  },
  thresholds: {
    http_req_failed: ['rate<0.01'],
    http_req_duration: ['p(95)<250', 'p(99)<500'],
  },
};

export default function () {
  const res = http.get('http://127.0.0.1:8099/');
  check(res, { 'status 200': (r) => r.status === 200 });
}
```

```bash
k6 run test.js
```

The `thresholds` make k6 exit non-zero when the SLO is violated, so the same file can gate a CI pipeline. With the 2% slow tail against this server, `p(99)<500` is borderline: observe it and explain why.

6. **Profile a database query.** In any PostgreSQL (for example `docker run -d --name pg -e POSTGRES_PASSWORD=pw -p 5432:5432 postgres:16`):

```sql
CREATE TABLE orders AS SELECT g AS id, (g % 1000) AS customer_id, now() AS created
  FROM generate_series(1, 1000000) g;
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE customer_id = 42;
CREATE INDEX ON orders (customer_id);
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE customer_id = 42;
```

   Before the index you will see a `Seq Scan`; after, an `Index Scan` or `Bitmap Heap Scan` with a much lower execution time (exact times depend on the machine). Clean up with `docker rm -f pg`.

7. **Write the performance review table** for your own system: for each hop, the budget, the measured p50/p99 (from a trace or load test), and the action if over budget.

7 Real-world scenario

(Illustrative, not a documented incident.) After a Monday deploy, `shop-api` dashboards show average latency flat at 85 ms, so nobody reacts, but support receives complaints about "random slow pages". The p99 panel shows 1.2 s, up from 400 ms. Traces reveal that the new product page makes 12 sequential calls to an inventory service, and one in a hundred inventory calls waits on a lock for 400 ms; 12 sequential calls make "at least one slow" happen on about 11% of page views. The team batches the 12 calls into one, adds a 150 ms timeout with a cached fallback, and adds a p99 threshold to the load test in CI. **Lesson:** alert on percentiles, not averages, and keep fan-out small.

8 Common mistakes and best practices

Don't

- Report or alert on average latency.
- Optimise before measuring; the bottleneck is rarely where you think.

- Load-test an empty database, a warm cache only, or from the same machine as the server and call it capacity.
- Let timeouts at inner layers be longer than the caller's deadline.
- Cache user-specific data at a shared layer without a correct cache key.

Do

- Define SLOs in percentiles and track p50/p95/p99 per route.
- Keep a latency budget per request path and review it when architecture changes.
- Use traces to locate slow hops and a profiler for CPU-bound code.
- Run load tests in CI against a production-like environment and fail the build on threshold violations.
- Use caching layers deliberately, with TTLs, versioned keys and stampede protection.

9 Tasks

- [ ] **T1 — Measure the tail** *(≈30 min, intermediate)*
  - **Goal:** see that mean and p99 tell different stories.
  - **Steps:**
    1. Run `server.py` and `loadtest.py` from steps 6.3 and 6.4.
    2. Record mean, p50, p95, p99 for three runs.
    3. Change the slow probability to 5% and re-run.
  - **Done when:** you have a table of 6 runs and a written explanation of why the mean moved less than p99.
- [ ] **T2 — Fan-out arithmetic** *(≈30 min, intermediate)*
  - **Goal:** quantify tail amplification.
  - **Steps:**
    1. Extend `fanout.py` to take `p_slow` and `n` from command-line arguments.
    2. Find the largest `n` such that the probability of a slow page stays under 5% for `p_slow` of 1%, 0.1% and 0.01%.
    3. Write one sentence on what that implies about dependency quality.
  - **Done when:** the script prints three `n` values (5, 51, 512 with integer rounding down) and you can explain them.
- [ ] **T3 — Hedged requests experiment** *(≈60 min, advanced)*
  - **Goal:** reduce tail latency by duplicating slow requests.

- **Steps:**
  1. Modify `loadtest.py` so that after waiting 60 ms with no reply it sends a second identical request and uses whichever returns first.
  2. Compare p99 with and without hedging, and count the extra requests sent (percentage).
  3. Explain why this is only safe for idempotent reads.
- **Done when:** p99 drops noticeably, the extra load is under 10%, and your explanation mentions idempotency.

☐ **T4 — Index and EXPLAIN** *(≈45 min, intermediate)*

- **Goal:** use query plans to prove an improvement.
- **Steps:**
  1. Follow step 6.6 to create a table with 1,000,000 rows.
  2. Capture `EXPLAIN (ANALYZE, BUFFERS)` before and after the index.
  3. Record execution time and buffers read in both cases.
- **Done when:** you can show the plan changed from `Seq Scan` to an index-based scan and that time and buffers fell.

☐ **T5 — Build the latency budget and threshold** *(≈60 min, advanced)*

- **Goal:** turn a budget into an automated gate.
- **Steps:**
  1. Write a budget for one endpoint of a sample app, with p99 targets per hop.
  2. Translate the end-to-end target into a k6 threshold (`p(99)<...`) as in step 6.5.
  3. Run the test, then deliberately add a 300 ms sleep to the handler and run again.
- **Done when:** the first run exits 0, the second exits non-zero, and both outputs are saved.

☐ **T6 — Prove it: performance review** *(≈45 min, advanced)*

- **Goal:** explain a regression end to end.
- **Steps:**
  1. Without the guide, write the investigation plan for "p99 doubled after a deploy": which three data sources do you check first, in what order and why.
  2. Name the metric that would have caught it before release and where in the pipeline it would sit.
- **Done when:** the plan names traces, a profile or `EXPLAIN`, and a CI threshold, and a peer can follow it.

10 Self-check questions

1. Why are percentiles preferred over averages for latency?
2. If each of 20 backend calls is slow 1% of the time, how often is the overall request slow (assuming independence)?
3. What is coordinated omission and how does an open-model load test avoid it?
4. Why must inner timeouts be shorter than outer deadlines?
5. When is a hedged request safe?

ANSWERS

1. The average hides rare, very slow requests that real users experience; percentiles show the distribution and match SLO wording.
2. 1 - 0.99^20, about 18%.
3. A closed-model test waits for each response before sending the next, so when the system slows down the test offers less load and under-reports delays. An open model sends at a fixed arrival rate regardless, exposing queueing.
4. Otherwise the inner call keeps working after the caller has given up, wasting capacity and causing retries and cascading overload.
5. When the operation is idempotent (reads or safely repeatable writes), and the extra load from duplicates is bounded.

11 Further reading

- Dean and Barroso, *The Tail at Scale* (Communications of the ACM, 2013).
- *Systems Performance* by Brendan Gregg — USE method, flame graphs.
- *Designing Data-Intensive Applications* by Martin Kleppmann — chapter 1, describing load and performance (percentiles).
- k6 documentation (grafana.com/docs/k6) and PostgreSQL documentation — "Using EXPLAIN".
- web.dev (Google) — performance and Core Web Vitals for the front-end side.

LAB NOTES What I ran, what I saw, what it means

19.5 Cost

Topic 5 of 11 | ~3 hours | Difficulty: Advanced

1 What it is

Cost architecture treats money as a design constraint on the same level as latency, availability and security. Every architectural choice has a price tag: a Multi-AZ database, a NAT gateway per zone, a log pipeline that keeps every debug line, a cross-region replica. Cost-aware design means estimating these before building, choosing consciously what to pay for, and measuring **unit economics**: the cost to serve one unit of business value (one order, one active user, one 1,000 requests).

Total cloud spend is a poor signal on its own: a bill that doubled while traffic tripled is *good news*; a flat bill with falling traffic is bad. **Unit cost** (cost per order, cost per tenant) tells you whether the architecture is getting more or less efficient as it grows.

> **Important:** all prices in this topic are **illustrative example prices** in USD, chosen to be in the same range as public on-demand list prices. They change often and vary by region. Always get real numbers from your provider's pricing page or calculator before making a decision.

2 Why it matters in DevOps

- **Surprise bills are an architecture smell**: the usual culprits (NAT gateway data processing, cross-AZ and internet egress, unbounded logs and metrics, idle environments) are design decisions.
- **Trade-offs are explicit**: availability, security and cost pull against each other; a senior engineer can price each option.
- **Startups die of burn and enterprises lose budgets** when cost has no owner. FinOps makes engineering accountable.
- **Cost is a reliability concern**: a team that cannot afford redundancy skips it, and a team that over-provisions everything cannot afford to test.

Related reading: FinOps §20.1, Network Egress Costs §20.7, Kubernetes Infrastructure Costs §20.9.

3 Real-world system example

Amazon Prime Video's video-quality monitoring service. In 2023 the Prime Video engineering team published a blog post describing how they moved a monitoring service from a distributed, serverless-orchestrated design (AWS Step Functions and Lambda, with intermediate data in S3) to a single-process design on containers, and reported a large reduction in infrastructure cost (they stated over 90% for that component). The post is a good reminder that the "right" architecture depends on scale and workload: the cost structure (state transitions and data transfer between components) mattered more than fashion. **Dropbox** has publicly described moving most of its storage off a public cloud onto its own infrastructure (the "Magic Pocket" project), again driven by unit economics at very large scale. Both are examples of changing architecture because the numbers changed, not because a pattern was or was not popular. (Read the primary sources for precise figures and context; they apply to specific workloads, not to every system.)

4 Diagram

The cost drivers in the reference architecture, with the usual biggest line items at the top:

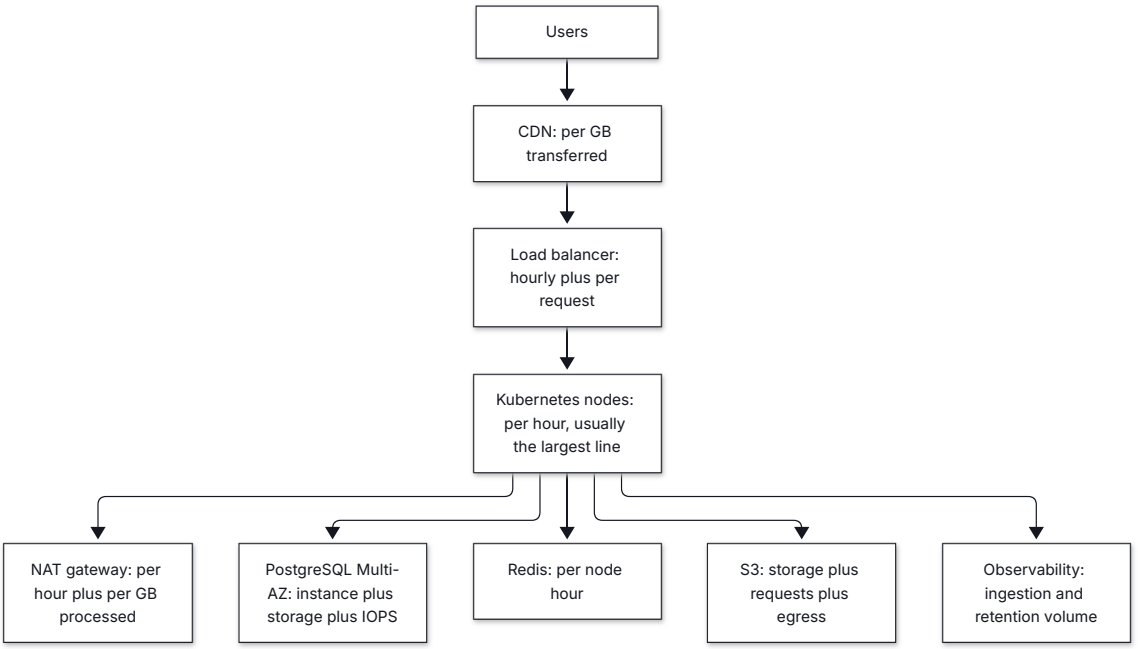


Figure 19.5-1

A cost-aware decision loop:

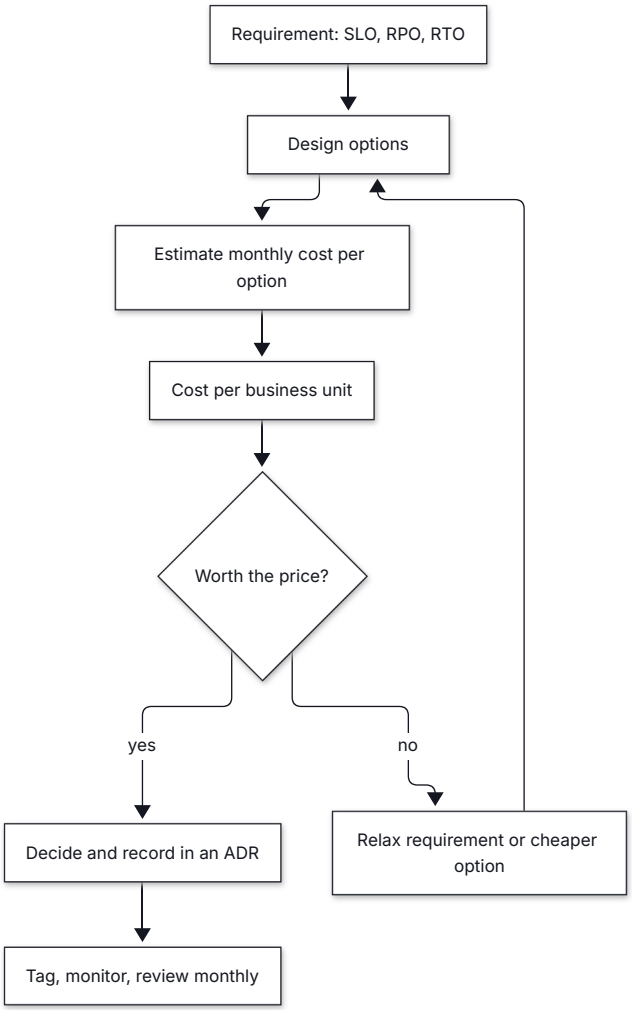


Figure 19.5-2

5 How it works

Where the money goes

Category	Typical drivers	Design levers
Compute	Node hours, over-provisioned requests, idle environments	Right-size, autoscale, Spot for stateless/worker tiers, commitments for the baseline

Category	Typical drivers	Design levers
Data stores	Instance size, Multi-AZ, provisioned IOPS, storage growth, backups	Right-size, read replicas only when needed, retention policies
Network	Cross-AZ traffic, internet egress, NAT processing, load balancer data	CDN, VPC endpoints for S3, keep chatty services in one AZ where acceptable, compress
Storage	Hot data kept forever, versioning without lifecycle	Lifecycle rules, tiering, TTLs
Observability	Metrics cardinality, log volume, trace sampling	Sampling, drop debug logs, retention tiers, recording rules
People	On-call, toil, complexity	Boring technology, managed services (see the next topic)

Unit economics

`cost per unit = total cost of the service / units of value delivered`. Choose a unit that the business understands (orders, active users, API calls, tenants). Track it monthly. If it rises with scale your architecture has negative economies of scale; if it falls you have leverage. Set a **gross-margin-style target** (for example, infrastructure cost per order under 1% of average order value) and design to it.

Cost-aware design patterns

- **Tier your reliability**: only the checkout path needs three AZs; the nightly report job can run on Spot in one AZ.
- **Cache and CDN** reduce both latency and egress. Compare per-GB prices: CDN transfer is often cheaper than origin egress.
- **Avoid chatty cross-AZ calls** on hot paths; they are billed per GB in each direction on most clouds.
- **Use VPC endpoints** for high-volume S3/ECR traffic so it does not pay NAT processing charges.
- **Cap telemetry**: sample traces, drop high-cardinality labels, set log retention, and budget the volume per service.
- **Buy committed capacity for the baseline only** (Savings Plans / Reserved Instances), keep the burst on-demand or Spot. See Reserved Instances and Savings Plans §20.2 and Spot Instances §20.3.
- **Tag everything** (team, service, environment) so cost can be attributed; untagged spend cannot be owned.

6 Step-by-step guide

Prerequisites: Python 3.9+. Optional: the AWS Pricing Calculator (a free web tool). No cloud account needed.

1. **Build a monthly cost model with clearly labelled example prices.** Save as `cost.py`:

```python
# EXAMPLE PRICES ONLY (illustrative USD). Replace with current numbers from your
  provider.
HOURS = 730  # hours per month
PRICE = {
    "node_hr": 0.17,          # 4 vCPU / 16 GB worker node
    "control_plane_hr": 0.10, # managed Kubernetes control plane
    "alb_hr": 0.0225, "alb_lcu_hr": 0.008,
    "db_multiaz_hr": 0.34,    # mid-size PostgreSQL, Multi-AZ
    "db_gb_month": 0.115,
    "redis_node_hr": 0.07,
    "s3_gb_month": 0.023,
    "egress_gb": 0.09,
    "nat_hr": 0.045, "nat_gb": 0.045,
    "cdn_gb": 0.085,
}
nodes, db_gb, s3_gb, egress_gb = 6, 200, 2_000, 3_000
cdn_share, nat_gb, redis_nodes, nats = 0.8, 1_500, 2, 3
orders_per_month = 400_000

lines = {
    "Kubernetes nodes": nodes * PRICE["node_hr"] * HOURS,
    "Managed control plane": PRICE["control_plane_hr"] * HOURS,
    "Load balancer": (PRICE["alb_hr"] + 5 * PRICE["alb_lcu_hr"]) * HOURS,
    "PostgreSQL Multi-AZ": PRICE["db_multiaz_hr"] * HOURS + db_gb *
  PRICE["db_gb_month"] * 2,
    "Redis": redis_nodes * PRICE["redis_node_hr"] * HOURS,
    "S3 storage": s3_gb * PRICE["s3_gb_month"],
    "CDN transfer": egress_gb * cdn_share * PRICE["cdn_gb"],
    "Origin egress": egress_gb * (1 - cdn_share) * PRICE["egress_gb"],
    "NAT gateways": nats * PRICE["nat_hr"] * HOURS + nat_gb * PRICE["nat_gb"],
}
total = sum(lines.values())
for k, v in sorted(lines.items(), key=lambda kv: -kv[1]):
    print(f"{k:24s} ${v:9,.2f} {v/total:6.1%}")
print(f"{'TOTAL / month':24s} ${total:9,.2f}")
print(f"cost per order: ${total/orders_per_month:.4f}")
```

```bash
python3 cost.py
```

Expected output (with these example prices):

```
Kubernetes nodes         $   744.60  43.0%
PostgreSQL Multi-AZ      $   294.20  17.0%
CDN transfer             $   204.00  11.8%
NAT gateways             $   166.05   9.6%
Redis                    $   102.20   5.9%
Managed control plane    $    73.00   4.2%
Origin egress            $    54.00   3.1%
S3 storage               $    46.00   2.7%
Load balancer            $    45.62   2.6%
TOTAL / month            $ 1,729.68
cost per order: $0.0043
```

(The database storage is doubled in the model to approximate Multi-AZ storage; this is a simplification.)

2. **Run what-if scenarios.** Change one variable at a time and re-run:
   - `nats = 1` (one NAT gateway instead of one per AZ): saves about $66 per month in the example (2 x 0.045 x 730 hours), but a zone failure then breaks egress for the other zones. That is an availability-for-cost trade, to record in an ADR.
   - `cdn_share = 0.95`: moves traffic from the $0.09 to the $0.085 line, and also reduces origin load (the larger saving is often fewer nodes).
   - Add Spot for the worker pool: assume 3 of the 6 nodes are Spot at a 65% discount (illustrative) and recompute the node line.
3. **Compute cost per unit and a target.** If the average order value is $40 and your target is infrastructure cost below 1% of order value ($0.40), the example is at $0.0043 per order, a tiny fraction of the target. The exercise is the *method*: when this number trends up, investigate before the bill surprises you.
4. **Price the redundancy choice explicitly.** Write a three-row table: single-AZ database (cost X, RTO hours), Multi-AZ (cost 2X, RTO minutes), cross-region replica (cost 3X plus transfer, RTO tens of minutes). Add the expected loss: `downtime minutes x revenue per minute`. If an hour of outage costs $5,000 and Multi-AZ costs $150 more per month, it pays for itself if it avoids one such hour every 33 months.
5. **Enforce cost guardrails.** Create a budget alert (AWS Budgets or equivalent) at 50%, 80% and 100% of the monthly budget, require tags (`team`, `service`, `env`) in Terraform with a policy check, and set log and metric retention in code. See Policy as Code §18.7.
6. **Schedule a monthly cost review**: top five line items, unit cost trend, biggest increase and its cause, one action. Keep the notes in the repository next to the ADRs.

7 Real-world scenario

(Illustrative, not a documented incident.) A team's bill grows from $4,000 to $9,500 in a month with only 10% more traffic. Cost Explorer filtered by usage type shows "NatGateway-Bytes" tripled. A new nightly job in a private subnet pulls 4 TB of container images and dataset files from S3 and public registries through the NAT gateway, which charges per GB processed.

The fix is a free S3 gateway VPC endpoint (so S3 traffic bypasses the NAT), a pull-through cache for the registry, and an alert on NAT bytes. The bill returns to normal in the next cycle. **Lesson:** unit cost and a breakdown by usage type reveal the cause in minutes; the root issue was an architecture default (all egress through NAT), not "the cloud being expensive".

8 Common mistakes and best practices

Don't

- Optimise cost by deleting redundancy that the SLO requires, without recording the decision.
- Compare prices without including operations time: the "cheap" self-hosted option has a salary behind it.
- Commit (Savings Plans) before you understand the baseline; you may pay for unused capacity.
- Leave non-production environments running 24/7 at production size.
- Ship unbounded telemetry: log everything at debug, forever, with user ids as metric labels.

Do

- Estimate the monthly cost of every option in the ADR and record the assumptions and price dates.
- Track unit cost (per order or tenant) on a dashboard next to latency and error rate.
- Tag resources and use account/project boundaries so every dollar has an owner.
- Review the top-five line items monthly and attack the largest first.
- Match reliability spending to business value per component.

9 Tasks

- [ ] **T1 — Run and extend the cost model** *(≈40 min, intermediate)*
  - **Goal:** reproduce the estimate and make it yours.
  - **Steps:**

1. Create `cost.py` from step 6.1 and run it; confirm the total is about $1,729.68.
2. Replace the example prices with current prices for your region from the provider's pricing page; write the date and URL in comments.
3. Add an observability line: Loki/Grafana/Prometheus storage (S3 GB) plus a managed Sentry plan (use the vendor's published price).

- **Done when:** the script prints a total, a per-order cost, a pricing date comment for every line, and the observability line.

☐ **T2 — Three what-if scenarios** *(≈45 min, intermediate)*

- **Goal:** quantify design trade-offs.
- **Steps:**
  1. Scenario A: one NAT gateway instead of three. Scenario B: 50% of worker nodes on Spot. Scenario C: 95% CDN offload.
  2. For each, print the new total and the percentage change.
  3. Write the availability or risk cost of each saving in one line.
- **Done when:** a table lists A, B, C with monthly savings and the risk accepted.

☐ **T3 — Redundancy vs expected loss** *(≈40 min, advanced)*

- **Goal:** justify (or reject) spending on redundancy with arithmetic.
- **Steps:**
  1. Assume revenue per hour (state it) and an expected frequency of zone-level incidents (state it; do not invent facts about a provider).
  2. Compute expected annual loss for single-AZ vs Multi-AZ using the downtime assumptions of each.
  3. Compare against the annual extra cost of Multi-AZ.
- **Done when:** the output shows a break-even number and a recommendation with its assumptions listed.

☐ **T4 — Find the hidden network cost** *(≈45 min, advanced)*

- **Goal:** model egress and NAT charges for a data-heavy workload.
- **Steps:**
  1. Take a job pulling 2 TB per day from S3 to a pod through a NAT gateway; compute monthly NAT processing cost using the example `nat_gb` price.
  2. Recompute with an S3 gateway endpoint.
  3. List two more services in the reference architecture where a VPC endpoint or caching avoids per-GB charges.

- **Done when:** the before/after numbers are printed and the savings percentage is stated.

☐ **T5 — Cost guardrails in code** *(≈60 min, advanced)*

- **Goal:** make overspending hard.
- **Steps:**
  1. Write a Terraform or Kubernetes policy (or a simple script) that fails when a resource lacks `team` and `env` tags/labels.
  2. Add an S3 lifecycle rule (transition to infrequent access after 30 days, expire after 365) to a sample bucket definition.
  3. Add a budget definition (AWS Budgets via Terraform) with an email alert at 80%. Do not apply it to a real account unless you want to; `terraform plan` is enough.
- **Done when:** the policy check fails on an untagged example and passes on a tagged one, and `terraform validate` succeeds on the bucket and budget code.

☐ **T6 — Prove it: cost review** *(≈45 min, advanced)*

- **Goal:** defend an architecture's cost.
- **Steps:**
  1. Without the guide, produce a one-page cost estimate for the reference architecture with monthly total, top three line items and the unit cost.
  2. State three changes that would cut 20% without violating a 99.9% SLO, and one that would not be worth doing.
- **Done when:** the estimate lists assumptions and price dates, and each proposed saving names its risk.

10 Self-check questions

1. Why is unit cost a better health signal than total spend?
2. Name four non-obvious sources of cloud cost that come from architecture rather than from instance size.
3. When does paying for Multi-AZ make economic sense?
4. Why should commitments cover only the baseline load?
5. What is the risk of using a single NAT gateway to save money?

ANSWERS

1. It normalises for growth: spend rising with proportionally rising business value is healthy; unit cost rising signals inefficiency.

2. NAT gateway data processing, cross-AZ transfer, internet egress, log/metric/trace volume, idle environments, snapshot and versioned-object accumulation.
3. When the expected cost of downtime (downtime minutes x revenue or reputational loss per minute x probability) exceeds the extra monthly cost, or when the SLO/RTO cannot be met otherwise.
4. Commitments are paid whether used or not; the variable portion should stay flexible (on-demand or Spot) to avoid paying for idle capacity.
5. A failure of that gateway's zone, or the gateway itself, cuts egress for the other zones too: a new SPOF in the critical path.

11 Further reading

- *Cloud FinOps* by J.R. Storment and Mike Fuller; the FinOps Foundation framework (finops.org).
- AWS Well-Architected Framework — Cost Optimization pillar; the AWS Pricing Calculator.
- Amazon Prime Video Tech blog post on scaling up the audio/video monitoring service (2023) and Dropbox engineering posts on Magic Pocket.
- Kubecost / OpenCost documentation (opencost.io) for Kubernetes cost allocation.

19.6 Operational Complexity

Topic 6 of 11 | ~3 hours | Difficulty: Advanced

1 What it is

Operational complexity is the ongoing human effort needed to keep a system running: the number of moving parts, the number of things an on-call engineer must understand at 3 a.m., the upgrades, the failure modes and the runbooks. It is a cost that never appears on the cloud bill but is usually larger than it.

Three ideas anchor this topic. **Boring technology**: prefer well-understood, widely used tools (PostgreSQL, Linux, Nginx) and spend your limited appetite for novelty only where it earns a real advantage. **Cognitive load**: the amount of mental effort a team can sustain is finite, so architecture must fit the team that runs it. **Build vs buy**: for each component decide whether to run it yourself or pay someone else to, based on total cost of ownership and whether it differentiates your business.

2 Why it matters in DevOps

- **Every component is a future incident.** Each new system adds upgrade work, security patching, monitoring, backups, runbooks and on-call knowledge.
- **Complexity compounds**: the interactions between ten components outnumber the components.
- **Teams burn out and leave** when on-call requires understanding too much. A system only one person understands is a bus-factor risk.
- **The most common architectural mistake** is copying a design built for a company 1,000 times larger. Kubernetes, service meshes and microservices solve real problems, at a real price.

Related reading: Platform Engineering §0.13, Internal Developer Platforms §18.1, Cluster, Nodes and Control Plane §8.1.

3 Real-world system example

"Choose Boring Technology" and Team Topologies. Dan McKinley (formerly of Etsy) wrote the essay *Choose Boring Technology*, which proposes that a team has a small budget of "innovation tokens" to spend on unproven technology; everything else should be proven and boring. The book *Team Topologies* (Skelton and Pais) makes the same case about people: team cognitive load is a design constraint, and platform teams exist to reduce the load on stream-aligned teams. **Basecamp (37signals)** has publicly written about moving off the cloud to run its own hardware to cut costs, and has also argued for the "majestic monolith"; **Stack Overflow** has described serving very large traffic from a small number of servers using a monolith and SQL Server. **Shopify** has described a "modular monolith" for a very large Rails application. These are public examples that simple architectures can scale a long way; they are not proof that your system should look like theirs. Meanwhile, **Kubernetes** itself came out of Google's experience with Borg, running thousands of services across many teams, which is exactly the scale at which its complexity pays for itself.

4 Diagram

How complexity grows with the number of components (every pair is a possible interaction or failure path):

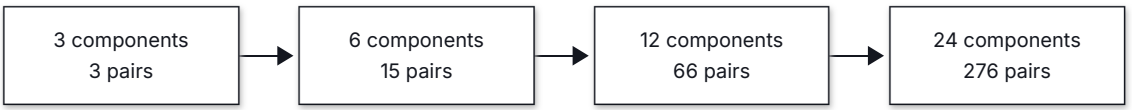


Figure 19.6-1

A decision flow for adding a new technology:

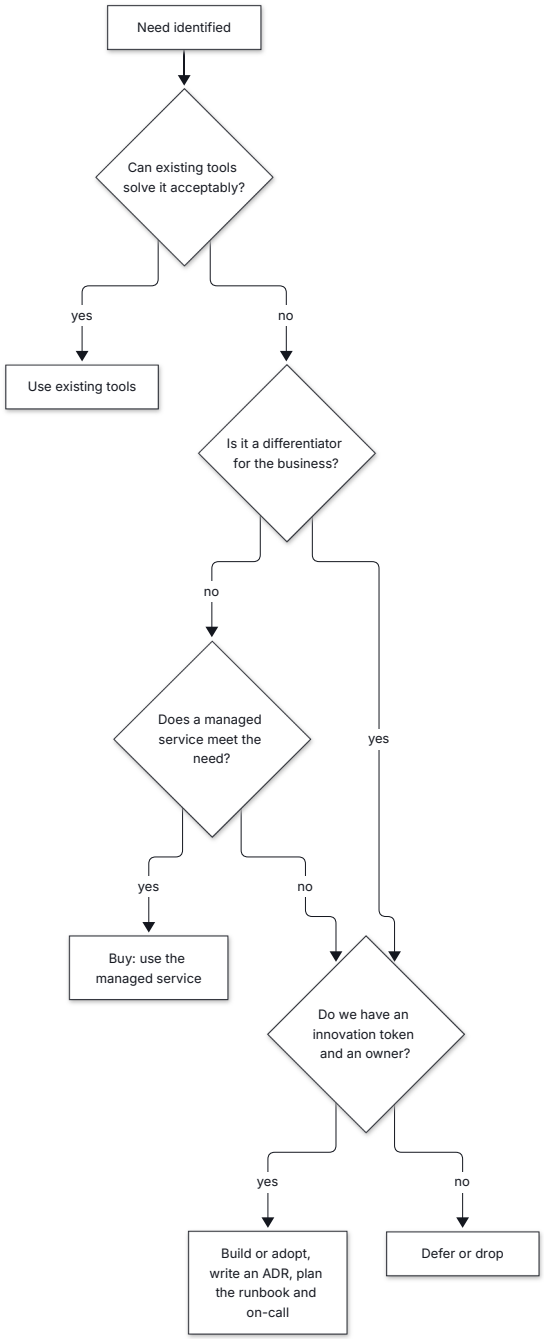


Figure 19.6-2

5 How it works

Boring technology

A technology is "boring" when its failure modes are well known, hiring is easy, documentation is deep and your team has operated it before. Boring does not mean old or bad: PostgreSQL is boring and excellent. The rule is to **spend innovation tokens deliberately**: a team might afford two or three unfamiliar technologies in total. If you adopt Kubernetes, a service mesh, event sourcing and Kafka in the same quarter, you have overspent, and each one will be operated badly.

Cognitive load

Three kinds: **intrinsic** (the domain itself), **extraneous** (fighting tooling, YAML, yak-shaving) and **germane** (valuable learning). Architecture should minimise extraneous load: golden paths, templates, sane defaults, one way to deploy, one way to observe. See Golden Paths §18.4. Practical limits: can a new engineer deploy to production in their first week? Can the on-call engineer explain the request path in five minutes? If not, the system is more complex than the team can carry.

Build vs buy

Factor	Lean towards buying (managed/SaaS)	Lean towards building/self-hosting
Differentiation	Commodity (email, auth, monitoring, databases)	Core to the product
Team size and skill	Small team, no specialist	Specialist team exists and wants to own it
Scale/economics	Moderate scale, where the vendor premium is smaller than staff time	Very large scale where unit costs dominate
Control/compliance	Vendor meets requirements	Data residency or customisation requires control
Lock-in	Acceptable, exit plan exists	Lock-in is unacceptable
Time to market	Need it now	Can invest months

Always compare **total cost of ownership (TCO)**: licence or infrastructure plus engineering build time plus ongoing operations (patching, upgrades, on-call, incident time).

When not to use Kubernetes

Kubernetes is a platform for building platforms; it is excellent when you run many services, need portable scheduling, rich autoscaling, and have people to operate it (or a managed offering plus platform engineers). Consider alternatives when:

Situation	Often a better fit
One to a few services, small team	A PaaS (Heroku-style, Render, Fly.io, App Runner), or managed container services (ECS/Fargate, Cloud Run)
A monolith with a database	Two or three VMs plus a load balancer, with Docker Compose or systemd
Event-driven, spiky and small	Serverless functions
Batch only	Managed batch services or scheduled jobs
No one wants to be on call for the cluster	Managed services all the way

Kubernetes is the right answer when the pain it removes (bin-packing, self-healing, uniform deployment for dozens of services, GitOps, ecosystem) outweighs the cost it adds (upgrades, networking, RBAC, security, a steep learning curve). See Stateful Workloads §16.8 for what running databases there involves.

6 Step-by-step guide

Prerequisites: Python 3.9+. No cost.

1. **Count the interaction paths of your design.** Pairs = n(n-1)/2.

```bash
python3 -c "
for n in (3, 6, 12, 24):
    print(f'{n:>2} components → {n*(n-1)//2:>3} possible pairwise interactions')"
```

Expected:

```
 3 components →   3 possible pairwise interactions
 6 components →  15 possible pairwise interactions
12 components →  66 possible pairwise interactions
24 components → 276 possible pairwise interactions
```

Not every pair interacts, but every pair *could*, and each real dependency is an incident waiting to happen.

2. **Run an innovation token audit.** Save as `tokens.py`:

```python
BUDGET = 3
proposal = {            # name: is it new to this team?
    "Kubernetes": True, "Service mesh": True, "PostgreSQL": False,
    "Redis": False, "Event sourcing": True, "Kafka": True, "Node.js": False,
}
spent = [k for k, new in proposal.items() if new]
print(f"tokens spent: {len(spent)} of {BUDGET} → {', '.join(spent)}")
if len(spent) > BUDGET:
    print(f"OVER BUDGET by {len(spent)-BUDGET}: drop or defer {len(spent)-BUDGET}
  item(s)")
```

```bash
python3 tokens.py
```

Expected:

```
tokens spent: 4 of 3 → Kubernetes, Service mesh, Event sourcing, Kafka
OVER BUDGET by 1: drop or defer 1 item(s)
```

Decide what to defer. A defensible answer: defer the service mesh (use plain TLS and NetworkPolicies first) and event sourcing (plain tables and a queue first).

3. **Do a build-vs-buy TCO.** Save as `tco.py` (example numbers: replace with yours):

```python
ENGINEER_MONTHLY = 12_000      # fully loaded monthly cost of one engineer
  (illustrative)
options = {
    # name: (monthly infrastructure or licence, build effort in engineer-months, ops
  effort in engineer-months per month)
    "Managed PostgreSQL (buy)":      (900, 0.5, 0.10),
    "Self-hosted PostgreSQL on VMs": (450, 2.0, 0.40),
}
months = 36
for name, (run, build, ops) in options.items():
    people = (build + ops * months) * ENGINEER_MONTHLY
    total = run * months + people
    print(f"{name:32s} infra ${run*months:>8,.0f}  people ${people:>9,.0f}  3-year TCO
  ${total:>9,.0f}")
```

```bash
python3 tco.py
```

Expected:

```
Managed PostgreSQL (buy)         infra $  32,400  people $   49,200  3-year TCO $
  81,600
Self-hosted PostgreSQL on VMs    infra $  16,200  people $  196,800  3-year TCO $
  213,000
```

The cheaper-looking infrastructure line produces the more expensive option once staff time is counted. Change the ops effort to 0.05 and see where the answer flips; that sensitivity is the real insight. (The effort numbers are assumptions, not benchmarks.)

4. **Fill in a Kubernetes fit assessment** for your own situation. Score each 0-2 (0 = no, 2 = yes):
   - More than 10 services deployed independently?
   - A team (or managed provider) responsible for cluster upgrades and security?
   - A need for autoscaling, multi-tenant isolation or portable deployment?
   - Engineers who already know Kubernetes?
   - Workloads that do not fit a PaaS or a few VMs?

   A total under 5 suggests a simpler platform first. Write the result and the reasoning down; it becomes an ADR.
5. **Measure cognitive load.** Time two things: how long a new engineer needs to get a change from laptop to production, and how long it takes someone to explain the request path of the system from memory. Anything over a week or over five minutes respectively is a signal to simplify or add a golden path.
6. **Write the runbook inventory.** For every component list: owner, upgrade cadence, backup method, the dashboard, the three most likely failures, and the runbook link. Components without all six are operational debt; each new technology should add a row before it ships.

7 Real-world scenario

(Illustrative, not a documented incident.) A five-person startup adopts Kubernetes, Istio, Kafka and a microservice per feature for an app serving a few hundred requests per second. Within six months one engineer spends most of their time on cluster upgrades and mesh certificate issues, deploys take 25 minutes, and an outage is traced to an Envoy sidecar memory limit that nobody understood. The product roadmap slips.

A review applies the token audit and TCO exercise. They merge four services into the main application (a modular monolith), remove the mesh (keeping TLS via the ingress and NetworkPolicies), replace Kafka with a managed queue and move the app to a managed container service with a managed PostgreSQL. Deploys take 4 minutes and on-call becomes tolerable. **Lesson:** the architecture was fine for a larger organisation; it was wrong for this team. Complexity must be matched to the size of the problem and of the team.

8 Common mistakes and best practices

Don't

- Choose technology for the résumé or because a large company uses it.
- Count only infrastructure when comparing build and buy.
- Introduce several unfamiliar systems at once.
- Split into microservices before you have clear boundaries and a platform to support them.
- Run stateful systems on Kubernetes without the skills to operate them, when a managed service exists.

Do

- Keep a short, explicit list of approved technologies and a process (an ADR) to add to it.
- Prefer managed services for undifferentiated heavy lifting.
- Start with a modular monolith and extract services when there is a team, scale or release-cadence reason.
- Design for the on-call engineer: fewer moving parts, clear dashboards, tested runbooks.
- Revisit decisions: when scale or team changes, the right answer changes.

9 Tasks

- [ ] **T1 — Complexity inventory** *(≈40 min, intermediate)*
  - **Goal:** list every moving part of the reference architecture and its ongoing cost.
  - **Steps:**
    1. List all components (CDN, LB, Kubernetes, ingress, API, worker, web, Redis, queue, PostgreSQL, S3, Prometheus, Grafana, Loki, OpenTelemetry Collector, Sentry).
    2. For each, write who upgrades it, how often, and the failure you fear most.
    3. Compute the pairwise interaction count with the formula from step 6.1.

- **Done when:** you have a table of 16 components with owner/upgrade/failure, and the interaction count.

- [ ] **T2 — Innovation token audit** *(≈30 min, intermediate)*
  - **Goal:** make newness explicit.
  - **Steps:**
    1. Run `tokens.py` with a proposal for a team you know (say honestly what is new to *them*).
    2. If over budget, decide what to defer and write one sentence on how you will meet the need with boring tools.
  - **Done when:** the audit prints a count within budget after your changes, with reasons.

- [ ] **T3 — Build vs buy TCO with sensitivity** *(≈45 min, advanced)*
  - **Goal:** find the break-even.
  - **Steps:**
    1. Extend `tco.py` to compare managed Redis against self-hosted Redis on Kubernetes (choose and state example numbers).
    2. Loop over ops effort values from 0.02 to 0.5 engineer-months per month and print the TCO of each option.
    3. Identify the value at which self-hosting becomes cheaper (if it does).
  - **Done when:** the output shows a break-even point or a statement that none exists, with your assumptions.

- [ ] **T4 — Kubernetes: yes or no** *(≈45 min, advanced)*
  - **Goal:** make and defend a platform choice.
  - **Steps:**
    1. Choose a scenario: (a) a 3-person team with one API, (b) a 40-engineer company with 60 services. Answer the step 6.4 scorecard for each.
    2. For (a), name the specific alternative platform and what you give up.
    3. For (b), name the platform capabilities you would build around Kubernetes.
  - **Done when:** each scenario has a score, a recommendation and the main trade-off in one sentence.

- [ ] **T5 — Reduce a design** *(≈60 min, advanced)*
  - **Goal:** remove complexity without losing requirements.
  - **Steps:**
    1. Take a design with at least 10 components (the reference architecture will do).

2. Remove or merge three components and show which requirements still hold and what is given up (for example: replace the queue with a Postgres-backed job table using `SELECT ... FOR UPDATE SKIP LOCKED`).
3. Recount the pairwise interactions.

- **Done when:** the simplified diagram renders, a requirement-by-requirement table shows what still holds, and the interaction count dropped.

☐ **T6 — Prove it: defend simplicity** *(≈45 min, advanced)*

- **Goal:** argue against unnecessary complexity.
- **Steps:**
  1. Without the guide, role-play: a teammate proposes adding a service mesh to a 6-service app. Write your response as a short memo covering the problem it solves, the alternatives, the tokens and runbooks it costs, and the conditions under which you would say yes.
- **Done when:** the memo contains a clear recommendation, at least two alternatives, and measurable conditions for revisiting.

10 Self-check questions

1. What is an innovation token and why is the budget small?
2. Why can self-hosting be more expensive even when infrastructure is cheaper?
3. Name three situations where Kubernetes is probably not the right choice.
4. What are the three kinds of cognitive load and which should architecture reduce first?
5. How does a modular monolith differ from microservices?

ANSWERS

1. A unit of "allowed novelty": each unproven technology costs learning, debugging and operating effort; teams can only absorb a few before quality drops.
2. People time (build, patching, upgrades, on-call, incidents) is usually larger than infrastructure savings at moderate scale.
3. A single service or monolith with a small team; spiky event-driven workloads suited to serverless; a team without capacity to operate a cluster when a PaaS or managed container service meets the need.
4. Intrinsic (domain), extraneous (tooling friction), germane (valuable learning). Reduce extraneous first.

5. A modular monolith is one deployable unit with strong internal module boundaries; microservices are separately deployed and operated services that communicate over the network, adding distributed-systems costs.

11 Further reading

- Dan McKinley, *Choose Boring Technology* (essay, boringtechnology.club).
- *Team Topologies* by Matthew Skelton and Manuel Pais — cognitive load and platform teams.
- *A Philosophy of Software Design* by John Ousterhout — managing complexity at the code level.
- Kubernetes documentation — "Production environment" considerations; and the documentation of the managed container service you would compare against.
- *Site Reliability Engineering* (Google, free online) — chapter on simplicity.

LAB NOTES What I ran, what I saw, what it means

19.7 Deployment Strategy

Topic 7 of 11 | ~4 hours | Difficulty: Advanced

1 What it is

Deployment architecture is the design of *how change reaches production*: which environments exist and what each is for, how a new version is released (all at once, rolling, blue/green, canary), how a release proceeds across regions, and how you limit the damage when a release is bad. It sits above the mechanics of individual pipelines (covered in Levels 5 and 17) and answers the architect's question: "What is the maximum harm one bad change can do, and how fast can we undo it?"

Four terms. **Environment**: a separate, isolated copy of the stack (dev, staging, production). **Release strategy**: the way traffic moves from the old version to the new one. **Progressive delivery**: exposing the new version to a growing share of traffic while automatically checking health. **Blast-radius control**: structuring releases (waves, cells, regions, feature flags) so any failure affects the smallest possible audience.

2 Why it matters in DevOps

- **Changes cause most outages.** Reliability work that ignores the release path misses the biggest cause.
- **Speed and safety are the same goal**: small, frequent, reversible changes are safer than rare big ones (DORA research reports this repeatedly).
- **Multi-region systems fail in correlated ways** if every region gets the same change at the same moment, which cancels the benefit of having multiple regions.
- **Database changes** are the hard part: you can roll back code in seconds but not data.

Related reading: Deployment Strategies §0.7, Rolling Deployments §5.10, Blue/Green Deployments §5.11, Canary Deployments §5.12, Environment Promotion §17.4.

3 Real-world system example

Amazon's automated, staged deployments. The Amazon Builders' Library article *Automating safe, hands-off deployments* describes how Amazon deploys through a pipeline that moves a change through pre-production stages, then to production in waves: a small one-box stage first, then single Availability Zones or Regions, increasing in size, with automated alarms and bake time between waves, and automatic rollback when alarms fire. Deployments are timed to avoid operating several Regions simultaneously. **Netflix** open-sourced Spinnaker and, with Google, developed Kayenta for automated canary analysis, comparing a canary against a baseline using metrics. **Google's SRE book** recommends canarying every change and describes releases that proceed in stages. **GitHub** and **Facebook** have written publicly about feature flags and gradual rollouts through ring-based exposure (employees first, then a small percentage of users). The shared pattern: *the pipeline, not a human, is the safety mechanism*.

4 Diagram

Promotion through environments and then production in waves, with a gate between each stage:

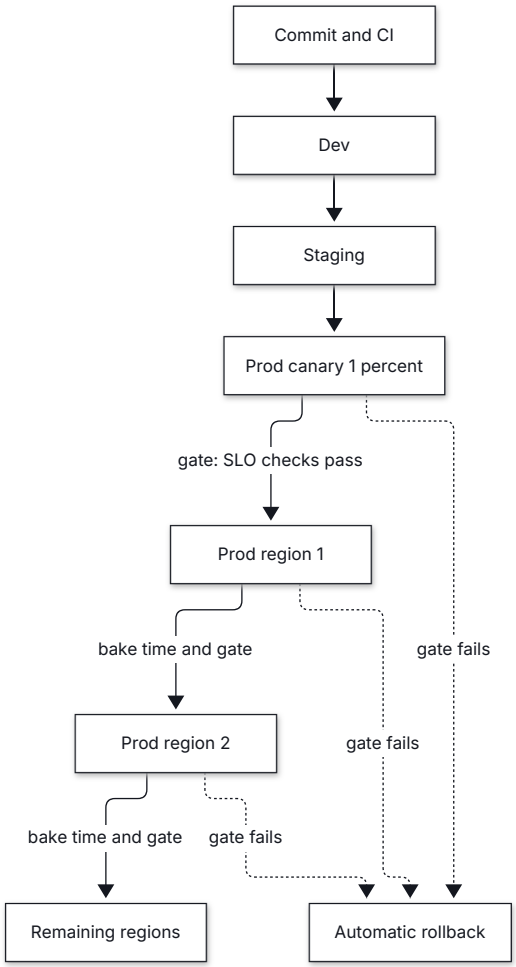


Figure 19.7-1

What a canary does at the traffic level:

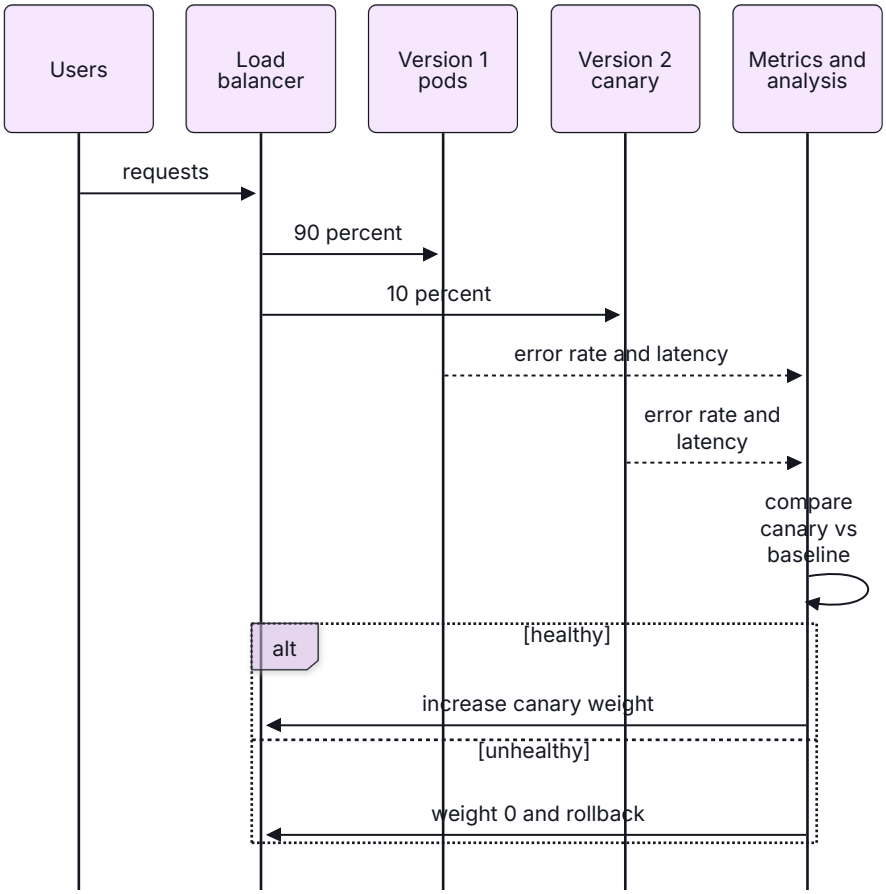


Figure 19.7-2

5 How it works

Environments

Environment	Purpose	Notes
Local / ephemeral	Developer feedback, per-PR preview environments	Cheap, auto-destroyed
Dev / integration	Merge the team's work, run integration tests	Shared, may be flaky
Staging / pre-prod	Production-like rehearsal: same config shape, same deploy path, realistic data volume	The deploy mechanism must be identical to production, otherwise you are not testing the deploy

Environment	Purpose	Notes
Production	Real users	Separate cloud account or project; stricter access

Staging can never be perfectly like production (traffic, data, scale), which is why progressive delivery in production exists. Keep environments as **code** so they do not drift (see Reusable Environments §7.6).

Release strategies compared

Strategy	How	Rollback speed	Cost	Best for	Risk
Recreate	Stop old, start new	Slow (redeploy)	Lowest	Dev, tolerable downtime	Downtime
Rolling	Replace pods gradually	Medium (roll back the roll)	Low (surge)	Default for stateless services	Mixed versions run together, slow detection
Blue/green	Two full stacks, switch traffic	Fast (switch back)	High (2x capacity during release)	Big risky changes, database-compatible changes	Costly, data sync issues
Canary	Small percentage first, grow on healthy metrics	Fast (set weight 0)	Low	High-traffic services with good metrics	Needs good SLI automation
Feature flags	Deploy dark, enable per user/group	Instant (toggle)	Low	Decoupling deploy from release	Flag debt, combinatorial states
Shadow / mirror	Copy live traffic to the new version, discard responses	N/A (no user impact)	Medium	Validating rewrites	Side effects if not isolated

Multi-region rollout

Never deploy to all regions at once. Order: internal/canary → one low-risk region → additional regions in increasing size, each with **bake time** (long enough for problems to appear, such as a memory leak or a cache expiry) and **automated gates** on SLOs. Keep each region independently deployable and independently runnable (no hard dependency on another region's control plane at runtime). Keep a global "stop" switch to halt all waves. For configuration and feature flags, treat them as deployments too: they cause outages just the same.

Blast-radius control

Layers, outermost first: separate accounts per environment → regions → **cells** (independent stacks serving subsets of customers) → availability zones → canary slice → feature flag per tenant. The smaller the unit of change, the smaller the harm. Backward-compatible data-

base migrations (**expand → migrate → contract**) mean code can be rolled back without data loss. See Database Migrations in Pipelines §5.9 and Rollback §5.7.

6 Step-by-step guide

Prerequisites: Python 3.9+. Optional: a kind cluster and `kubectl` for step 4 (Argo Rollouts is optional). No cloud cost.

1. **Simulate a staged rollout with an automatic gate.** Save as `rollout.py`:

```python
import random
random.seed(7)

# (wave name, share of global traffic). A bad release has a 5 percent error rate.
waves = [("canary (1% of traffic)", 0.01), ("us-east-1 (rest)", 0.20),
         ("eu-west-1", 0.20), ("ap-southeast-1", 0.15), ("rest of world", 0.44)]
BAD_RELEASE = True
GATE = 0.01            # halt if the error rate exceeds 1 percent
requests_per_wave = 20_000

exposed = 0.0
for name, share in waves:
    err = 0.05 if BAD_RELEASE else 0.001
    errors = sum(random.random() < err for _ in range(requests_per_wave)) /
  requests_per_wave
    exposed += share
    print(f"{name:26s} error rate {errors:6.2%}  traffic exposed so far
  {exposed:5.0%}")
    if errors > GATE:
        print(f"GATE FAILED at '{name}': rollback. Max blast radius: {exposed:.0%} of
  traffic")
        break
else:
    print("rollout complete")
```

```bash
python3 rollout.py
```

Expected with `BAD_RELEASE = True` (error rates vary slightly):

```
canary (1% of traffic)     error rate  4.98%  traffic exposed so far    1%
GATE FAILED at 'canary (1% of traffic)': rollback. Max blast radius: 1% of traffic
```

Set `BAD_RELEASE = False` and the script walks through all five waves and prints `rollout complete`. The bad release harmed 1% of traffic instead of 100%.

2. **Compare the harm of different strategies for the same bad release.** Assume 1,000 requests/s and detection plus rollback taking 5 minutes for a canary at 1% vs 2 minutes for a big-bang deploy with detection at 100% traffic:

```bash
python3 -c "
rps=1000; err=0.05
big_bang = rps*120*err
canary   = rps*0.01*300*err
print(f'big-bang failed requests: {big_bang:,.0f}')
print(f'canary failed requests:   {canary:,.0f}')"
```

   Expected: `big-bang failed requests: 6,000` and `canary failed requests: 150`. Even though the canary took longer to roll back, it harmed 40 times fewer requests.

3. **Design the environment matrix.** Fill in a table: environment, cloud account, who may deploy, data (synthetic or masked), cost-saving schedule, parity with production (what is *not* the same). Write down which differences are risks (for example staging has 1 replica, so it cannot reveal a concurrency bug).

4. **Write a canary rollout as code.** With Argo Rollouts (see its docs to install the controller), a Rollout replaces the Deployment:

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: shop-api
  namespace: shop
spec:
  replicas: 10
  selector:
    matchLabels: { app: shop-api }
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: { duration: 5m }
        - setWeight: 50
        - pause: { duration: 10m }
        - setWeight: 100
  template:
    metadata:
      labels: { app: shop-api }
    spec:
      containers:
        - name: api
          image: example.com/shop-api:1.4.0
```

`kubectl argo rollouts get rollout shop-api -n shop --watch` shows the progress. Add an `AnalysisTemplate` that queries Prometheus for the error ratio so that failing the query aborts the rollout automatically. See Argo CD and Flux §17.2.

5. **Plan database changes with expand/contract.** Write the three releases for renaming `orders.total` to `orders.amount_cents`:
   1. *Expand*: add the new column, write both, read the old. Roll back safely.
   2. *Migrate*: backfill in batches, switch reads to the new column. Roll back safely (old column is still being written).
   3. *Contract*: stop writing the old column, drop it later after a bake period. This is the only non-trivially reversible step.
6. **Define the abort and rollback rules** in writing before the release: which metrics gate each wave (error rate, p99 latency, saturation, business metric such as orders per minute), the thresholds, the bake time, who can press stop, and how to roll back (git revert and redeploy, or `kubectl argo rollouts abort`). See Rollback Through Git §17.6.

7 Real-world scenario

(Illustrative, not a documented incident.) A team ships a new connection-pool library on a Friday with a rolling update across both regions at once. Staging is fine because it has two replicas and almost no traffic. At 14:10 production p99 latency climbs and error rates rise to 12% as the new pool leaks connections under real concurrency. Both regions fail together; rollback takes 18 minutes because the pipeline needs a full rebuild and the one engineer who knows how is out.

The post-incident changes: a 1% canary with automatic analysis on error ratio and p99, regions deployed in sequence with 30 minutes of bake time, rollbacks that re-deploy the previous immutable image (not rebuild), and a "no Friday afternoon deploys without a flag" convention replaced by deploy-dark-and-flag. **Lesson:** the multi-region design was useless for reliability because the release process made both regions fail identically.

8 Common mistakes and best practices

Don't

- Deploy to every region or every cell at the same time.
- Make staging a different deployment mechanism from production.
- Treat feature flags and config changes as "not deploys"; they need the same gates.
- Ship a destructive database change in the same release as the code that depends on it.

- Rely on a human watching a dashboard as the only gate.

Do

- Deploy immutable, versioned artifacts and roll back by redeploying a previous version.
- Roll out in waves with bake time and automatic metric-based gates, smallest blast radius first.
- Separate deploy from release with feature flags, and clean flags up.
- Use expand/contract migrations so each step is reversible.
- Rehearse rollback regularly; measure time to rollback as a key metric.

9 Tasks

- [ ] **T1 — Run the staged rollout simulation** *(≈30 min, intermediate)*
  - **Goal:** see the effect of gating on blast radius.
  - **Steps:**
    1. Create `rollout.py` and run it with `BAD_RELEASE = True` and `False`.
    2. Change the gate threshold to 0.10 (10%) and run again with `BAD_RELEASE = True`.
    3. Explain what happened and why gates should be set from SLOs.
  - **Done when:** you have three outputs and a two-sentence explanation of how a loose gate lets a bad release through.

- [ ] **T2 — Compare strategies by harm and cost** *(≈45 min, intermediate)*
  - **Goal:** quantify rolling, blue/green and canary for the same bad release.
  - **Steps:**
    1. Extend the arithmetic in step 6.2 for rolling (50% of pods updated before detection), blue/green (100% switched, instant rollback after 2 minutes), canary (1%).
    2. Add the extra infrastructure cost of blue/green using the example prices from the Cost topic's model (state the numbers you use).
    3. Print failed requests and extra cost per strategy.
  - **Done when:** a table shows failed requests and extra cost for all four strategies, and you can name the best fit for `shop-api` and why.

- [ ] **T3 — Canary with automatic abort (kind)** *(≈90 min, advanced)*
  - **Goal:** build a progressive delivery pipeline locally.
  - **Steps:**

1. Create a kind cluster, install Argo Rollouts (follow its installation docs), and deploy an `nginx`-based Rollout with the steps in 6.4.
2. Update the image to a version that returns errors (for example an `http-echo` configured to return 500) and watch the rollout.
3. Add an analysis step (a Job-based or web metric check against a stub that returns failure) so the rollout aborts.

- **Done when:** `kubectl argo rollouts get rollout` shows `Degraded`/aborted with traffic back on the stable version, and you show the output.

☐ **T4 — Expand/contract migration** *(≈60 min, advanced)*

- **Goal:** make a schema change reversible at every step.
- **Steps:**
  1. Run PostgreSQL in Docker and create `orders(id, total)` with sample rows.
  2. Execute the three phases from step 6.5 as SQL scripts (add column, backfill in batches of 1,000 rows, drop old column).
  3. After each phase, write the rollback command and prove it by running it in a copy of the database.
- **Done when:** each phase has an apply script, a rollback script and a verification query, and the rollback of phases 1 and 2 loses no data.

☐ **T5 — Multi-region release plan** *(≈45 min, advanced)*

- **Goal:** write the wave plan for the reference architecture.
- **Steps:**
  1. Define waves for three regions (canary, region 1, region 2, region 3), bake time per wave, and metrics and thresholds per gate.
  2. Define the global stop switch and who can use it.
  3. Describe how config and feature-flag changes follow the same plan.
- **Done when:** the plan is a table of wave, traffic share, bake time, gates, abort action, and has no step that changes all regions at once.

☐ **T6 — Prove it: release review** *(≈45 min, advanced)*

- **Goal:** explain the release architecture and its failure modes.
- **Steps:**
  1. Without the guide, draw the pipeline from commit to the last region in a Mermaid diagram and mark each gate.
  2. List three ways a bad change could still reach all regions (for example: config bypassing the pipeline) and the control that would close each.

- **Done when:** the diagram renders, each gate names a metric, and the three bypass paths have mitigations.

10 Self-check questions

1. Why does a multi-region deployment not automatically give you multi-region reliability?
2. What are the trade-offs between rolling, blue/green and canary?
3. What does "bake time" protect against?
4. What is the expand/contract pattern and why does it help rollback?
5. Why should staging use the same deployment mechanism as production?

ANSWERS

1. If all regions receive the same change at the same time (or share a dependency), they fail together. Staggered waves keep failures independent.
2. Rolling is cheap but mixes versions and detects slowly; blue/green rolls back instantly but doubles capacity; canary limits exposure and needs reliable automated metrics.
3. Problems that take time to appear: memory leaks, cache expiry, daily traffic peaks, slow queue growth.
4. Make additive changes first, migrate data, and remove old structures last. Each step is compatible with both old and new code so code can roll back without data loss.
5. Otherwise the deploy itself is untested; many failures come from the deployment process (permissions, config, ordering), not the application code.

11 Further reading

- Amazon Builders' Library — *Automating safe, hands-off deployments*.
- *Site Reliability Engineering* (Google) — chapter on release engineering, and *The Site Reliability Workbook* — canarying releases.
- *Accelerate* by Forsgren, Humble and Kim, and the annual DORA reports.
- *Continuous Delivery* by Humble and Farley.
- Argo Rollouts and Spinnaker/Kayenta documentation.

LAB NOTES What I ran, what I saw, what it means

19.8 Monitoring

Topic 8 of 11 | ~4 hours | Difficulty: Advanced

1 What it is

Monitoring architecture is the design of how a system tells you what it is doing: what telemetry it emits (metrics, logs, traces, errors), how that data travels through a pipeline, where it is stored, how it becomes dashboards, and above all **which conditions wake a human**. Level 10 taught the tools; this topic is about the *design decisions*.

Three principles. **Design observability in**: instrumentation, correlation ids and health endpoints are part of the feature, not a follow-up ticket. **SLO-driven alerting**: page on symptoms that threaten a **Service Level Objective (SLO)** (a target such as 99.9% of requests succeed) using error-budget **burn rate**, not on every CPU spike. **Telemetry costs money**: volume, cardinality and retention are architectural parameters you must size and cap.

2 Why it matters in DevOps

- **You cannot fix what you cannot see**, and you cannot add tracing to a running incident.
- **Bad alerting destroys teams**: noisy pages cause fatigue and missed real incidents; silent systems cause long outages. See Alert Fatigue Reduction §14.8.
- **Observability is often the second or third largest line on the bill** when nobody budgets cardinality, log volume and retention.
- **Monitoring must outlive the thing it monitors**: if the monitoring stack fails with the cluster, you are blind exactly when you need it.

Related reading: Observability §0.10, The Three Pillars §10.1, SLOs §14.2, Error Budgets §14.4, What to Monitor §10.11.

3 Real-world system example

Google SRE: SLO-based alerting. *The Site Reliability Workbook* has a chapter on alerting on SLOs, which develops the multiwindow, multi-burn-rate approach used below: page when the budget is being consumed fast over both a long and a short window, open a ticket when

it is being consumed slowly. **Prometheus, Grafana, Loki and Tempo** are open source projects with public documentation that together cover metrics, logs and traces ("the Grafana stack"), and **OpenTelemetry** (a Cloud Native Computing Foundation project) standardises instrumentation and provides the Collector as a vendor-neutral pipeline component. **Uber** open-sourced the Jaeger tracing system, and **Netflix** has publicly described how enormous telemetry volume forced it to build custom metric and trace pipelines, which is a good reminder that at scale telemetry is itself a large distributed system. The practical takeaway for most teams: use the open standards (OpenTelemetry) at the edges, so storage backends can change.

4 Diagram

The telemetry pipeline of the reference architecture:

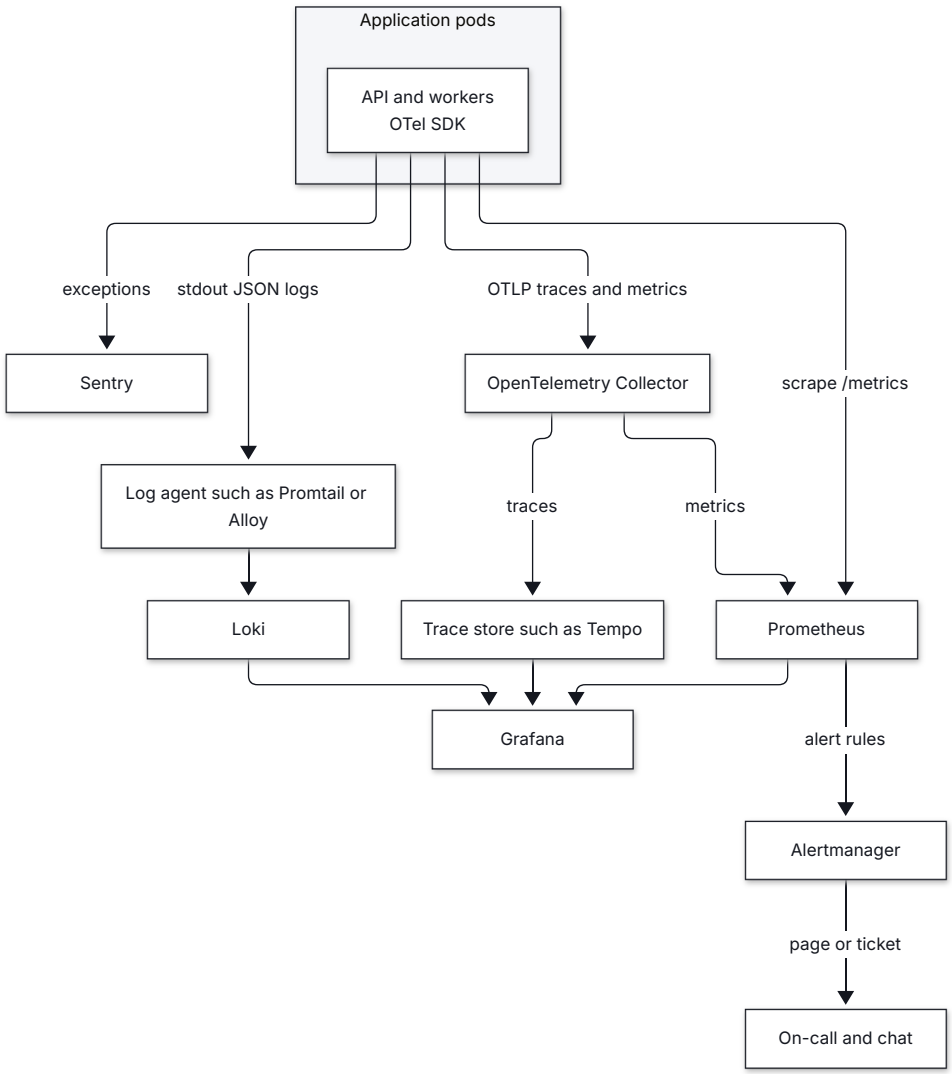


Figure 19.8-1

From SLO to alert: the decision path for every alert you write.

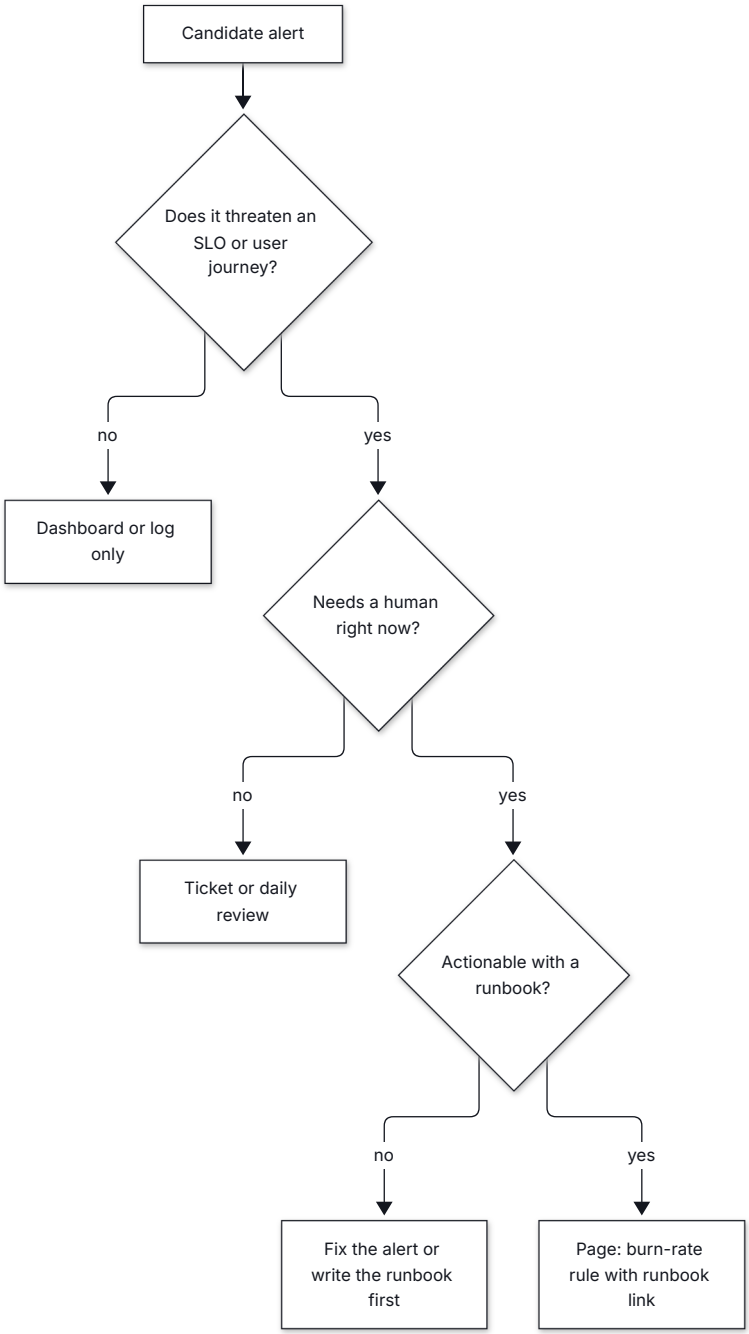


Figure 19.8-2

5 How it works

Design observability in

Layer	What to build in
Code	Structured JSON logs with a `trace_id`/request id on every line, RED metrics (rate, errors, duration) per route, spans around outbound calls
Platform	Standard sidecar-free pipeline: stdout logs, `/metrics` scrape or OTLP push, Collector as the single egress point
Service	Health and readiness endpoints that reflect real dependencies, version and build info exposed as a metric
Business	Domain metrics: orders per minute, payments succeeded; they catch failures that technical metrics miss
Runbooks	Every alert links to a runbook, and every dashboard has an owner

See Structured Logging §10.5, Correlation IDs §10.7 and OpenTelemetry §10.9.

SLO-driven alerting

Given an SLO (say 99.9% over 30 days), the **error budget** is 0.1% of requests. **Burn rate** is how many times faster than "exactly on budget" you are consuming it. A burn rate of 1 uses the whole budget in exactly 30 days; 14.4 uses 2% of the budget in one hour. The recommended pattern pairs a long window with a short one so alerts are both significant and recent:

Severity	Long window	Short window	Burn rate	Budget consumed	Action
Page	1 h	5 m	14.4	2%	Wake someone
Page	6 h	30 m	6	5%	Wake someone
Ticket	3 d	6 h	1	10%	Handle in working hours

(Numbers follow the example in the Google SRE Workbook for a 30-day window.) This replaces dozens of cause-based alerts ("CPU above 80%") with a few symptom-based alerts that map to user pain. Keep a small set of cause-based alerts only for things that lead to certain failure soon (disk filling, certificate expiring).

Telemetry pipeline design and cost

Signal	Volume driver	Cost-control levers
Metrics	**Active series**: product of label cardinalities	Avoid unbounded labels (user id, request id, full URL), recording rules, drop unused metrics, shorter retention for high-resolution data

Signal	Volume driver	Cost-control levers
Logs	GB per day: requests x lines x bytes	Log levels, no debug in production by default, sample noisy lines, short retention for verbose logs and long for audit logs, store labels sparingly in Loki
Traces	Requests x spans x sampling rate	Head sampling (for example 5%), tail sampling at the Collector (keep all errors and slow traces)
Errors	Events per month	Sentry rate limits, sampling, grouping, release tagging

Architecture choices: a **Collector tier** (agent per node and a gateway) to batch, sample, redact and route; **high availability** for Prometheus (two replicas) and long-term storage (Thanos, Mimir or a managed service); **separate failure domains** so the monitoring plane is outside the failure domain it watches (for instance an external uptime check and a dead man's switch alert).

6 Step-by-step guide

Prerequisites: Python 3.9+. Optional: Docker/Prometheus for step 5 (`promtool` is shipped with Prometheus). No cost.

1. **Compute the error budget and burn-rate thresholds.** Save as `burn.py` :

```python
slo = 0.999
budget = 1 - slo
window_days = 30
print(f"error budget: {budget:.3%} of requests, {budget*window_days*24*60:.1f} minutes
  of total outage per {window_days} days\n")
alerts = [("1h", "5m", 14.4, "page"), ("6h", "30m", 6, "page"), ("3d", "6h", 1,
  "ticket")]
for long_w, short_w, rate, action in alerts:
    hours = {"1h": 1, "6h": 6, "3d": 72}[long_w]
    consumed = rate * hours / (window_days * 24)
    print(f"burn {rate:>5}x over {long_w:>2} (and {short_w:>3}) → consumes
  {consumed:6.1%} of 30d budget → {action}; error ratio threshold {rate*budget:.3%}")
```

```bash
python3 burn.py
```

Expected output:

```
error budget: 0.100% of requests, 43.2 minutes of total outage per 30 days

burn  14.4x over 1h (and  5m) → consumes   2.0% of 30d budget → page; error ratio
  threshold 1.440%
burn     6x over 6h (and 30m) → consumes   5.0% of 30d budget → page; error ratio
  threshold 0.600%
burn     1x over 3d (and  6h) → consumes  10.0% of 30d budget → ticket; error ratio
  threshold 0.100%
```

So, for a 99.9% SLO, you page when the error ratio is above 1.44% for both the last hour and the last 5 minutes.

2. **Turn it into a Prometheus rule.** Save as `alerts.yaml`:

YAML

```yaml
groups:
  - name: shop-api-slo
    rules:
      - record: job:http_error_ratio:rate5m
        expr: |
          sum(rate(http_requests_total{job="shop-api",code=~"5.."}[5m]))
          /
          sum(rate(http_requests_total{job="shop-api"}[5m]))
      - record: job:http_error_ratio:rate1h
        expr: |
          sum(rate(http_requests_total{job="shop-api",code=~"5.."}[1h]))
          /
          sum(rate(http_requests_total{job="shop-api"}[1h]))
      - alert: ShopApiErrorBudgetFastBurn
        expr: |
          job:http_error_ratio:rate1h > (14.4 * 0.001)
          and
          job:http_error_ratio:rate5m > (14.4 * 0.001)
        for: 2m
        labels: { severity: page }
        annotations:
          summary: "shop-api is burning its 30-day error budget 14.4x too fast"
          runbook_url: "https://example.com/runbooks/shop-api-error-budget"
```

Validate it (requires `promtool` from a Prometheus release, or run it via the container image):

BASH

```
promtool check rules alerts.yaml
```

Expected: `SUCCESS: 3 rules found`.

3. **Size the telemetry pipeline.** Save as `telemetry.py` (example numbers; substitute your own):

```python
pods, series_per_pod, scrape_s, bytes_per_sample, retention_days = 60, 800, 15, 2, 15
samples_per_day = 86_400 / scrape_s
metrics_gb = pods * series_per_pod * samples_per_day * bytes_per_sample *
  retention_days / 1e9
print(f"active series: {pods*series_per_pod:,}  metrics storage ({retention_days}d):
  {metrics_gb:,.1f} GB")

rps, log_bytes_per_req = 900, 600
log_gb_day = rps * 86_400 * log_bytes_per_req / 1e9
print(f"logs: {log_gb_day:,.1f} GB/day raw; with 30d retention and 5x compression:
  {log_gb_day*30/5:,.0f} GB stored")

sampling, spans_per_trace, bytes_per_span = 0.05, 12, 500
trace_gb_day = rps * 86_400 * sampling * spans_per_trace * bytes_per_span / 1e9
print(f"traces at {sampling:.0%} sampling: {trace_gb_day:,.1f} GB/day (100% sampling
  would be {trace_gb_day/sampling:,.0f} GB/day)")

base = 5 * 40 * 8        # method x route x status
print(f"series per histogram metric (10 buckets): {base*10:,}")
print(f"adding a user_id label with 100,000 users: {base*10*100_000:,} series")
```

```bash
python3 telemetry.py
```

Expected:

```
active series: 48,000  metrics storage (15d): 8.3 GB
logs: 46.7 GB/day raw; with 30d retention and 5x compression: 280 GB stored
traces at 5% sampling: 23.3 GB/day (100% sampling would be 467 GB/day)
series per histogram metric (10 buckets): 16,000
adding a user_id label with 100,000 users: 1,600,000,000 series
```

Lessons: logs dominate volume; sampling is what makes tracing affordable; one careless label turns a small metric into a billion series and takes Prometheus down. (Bytes-per-sample, bytes-per-log and span sizes are rough assumptions; measure your own.)

4. **Write the OpenTelemetry Collector pipeline as the single egress point.** Example `otel-collector.yaml`:

```yaml
receivers:
  otlp:
    protocols:
```

```
      grpc: {}
      http: {}
processors:
  memory_limiter: { check_interval: 1s, limit_mib: 400 }
  batch: {}
  attributes/redact:
    actions:
      - { key: http.request.header.authorization, action: delete }
      - { key: user.email, action: delete }
  tail_sampling:
    decision_wait: 10s
    policies:
      - { name: errors, type: status_code, status_code: { status_codes: [ERROR] } }
      - { name: slow, type: latency, latency: { threshold_ms: 500 } }
      - { name: baseline, type: probabilistic, probabilistic: { sampling_percentage: 5
  } }
exporters:
  otlp/tempo: { endpoint: tempo:4317, tls: { insecure: true } }
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, attributes/redact, tail_sampling, batch]
      exporters: [otlp/tempo]
```

Note `tail_sampling` ships in the Collector *contrib* distribution. It keeps every error and every slow trace plus 5% of the rest, which preserves what you debug with while cutting volume by roughly an order of magnitude. See Distributed Tracing §10.8.

5. **Check the monitoring-of-monitoring.** List how you would know if (a) Prometheus is down, (b) Alertmanager cannot send, (c) the whole cluster is gone. Typical answers: a second Prometheus or managed service that scrapes the first; an always-firing "Watchdog" alert sent to an external dead man's switch; an external synthetic check (outside the cluster and region) against `https://example.com/healthz`.
6. **Produce the alert inventory.** Table: alert name, SLO or journey it protects, severity, threshold, runbook URL, owner, last time it fired and whether it was actionable. Delete or demote any alert with no runbook or no action taken in 90 days.

7 Real-world scenario

(Illustrative, not a documented incident.) An on-call engineer receives 40 pages a week, mostly "CPU above 80% on node X" and "pod restarted", and almost none lead to action. One Tuesday a payment-provider timeout change makes 3% of checkouts fail for 40 minutes; the only alert that fires is a ticket-level "5xx above 1%" that nobody reads at 2 a.m. Customers notice first.

The team replaces its cause-based pages with two SLO burn-rate pages (checkout availability and p99 latency) and two ticket alerts, links runbooks, and adds an orders-per-minute business metric. Weekly pages drop from 40 to 3, and the next payment issue pages within 6 minutes with a runbook link to the provider's status and the circuit-breaker toggle. **Lesson:** alert on user-visible symptoms measured against an SLO; keep causes on dashboards for diagnosis.

8 Common mistakes and best practices

Don't

- Page on resource thresholds that do not map to user impact.
- Put unbounded values (user id, URL with ids, request id) in metric labels.
- Log at debug level in production by default or log request bodies and secrets.
- Host your only alerting path inside the cluster it watches.
- Create dashboards no one owns; they rot and mislead.

Do

- Define SLOs per user journey first, then derive alerts from error-budget burn.
- Keep one correlation id across logs, traces and error reports so one id finds everything.
- Use the OpenTelemetry Collector for batching, redaction, sampling and routing.
- Budget telemetry (series, GB/day, sampling) per service and alert on growth.
- Give every page a runbook and review the alert inventory monthly.

9 Tasks

- [ ] **T1 — Burn-rate calculator** *(≈30 min, intermediate)*
  - **Goal:** derive alert thresholds from an SLO.
  - **Steps:**
    1. Run `burn.py` for 99.9%.
    2. Change the SLO to 99.5% and 99.99% and re-run.
    3. For 99.99%, compute the minimum error ratio at which a 14.4x burn pages, and explain whether you can reliably measure that with 5 requests per second.
  - **Done when:** three outputs are saved and you state the measurement limit for low-traffic services (hint: few requests means noisy ratios).
- [ ] **T2 — Validate an SLO alert rule** *(≈45 min, intermediate)*

- **Goal:** write and lint a burn-rate alert.
- **Steps:**
  1. Create `alerts.yaml` from step 6.2 and add the 6 h / 30 m rule at burn rate 6.
  2. Run `promtool check rules alerts.yaml` (use the `prom/prometheus` image's `promtool` entrypoint if not installed).
  3. Write a promtool unit test (`promtool test rules`) that feeds a series with 2% errors and asserts the fast-burn alert fires.
- **Done when:** `check rules` reports SUCCESS and `test rules` passes, with outputs saved.

☐ **T3 — Cardinality explosion drill** *(≈60 min, advanced)*

- **Goal:** see how a label breaks a metrics system.
- **Steps:**
  1. Run Prometheus in Docker (`docker run -p 9090:9090 prom/prometheus`) and a small script that exposes a counter with a `user_id` label created for 50,000 distinct values using `prometheus_client` for Python.
  2. Scrape it and query `count({__name__="demo_requests_total"})` and Prometheus's own `prometheus_tsdb_head_series`.
  3. Fix it by removing the label and show the series count drop.
- **Done when:** you show before and after series counts and the Prometheus memory difference (`process_resident_memory_bytes`).

☐ **T4 — Collector with tail sampling** *(≈75 min, advanced)*

- **Goal:** build and test the sampling pipeline.
- **Steps:**
  1. Run the OpenTelemetry Collector contrib image with the config from step 6.4, using the `debug` exporter instead of Tempo.
  2. Send traces from `telemetrygen` (from the OpenTelemetry project) with some marked as errors.
  3. Verify the debug output shows all error traces and a small fraction of the successful ones.
- **Done when:** counts show 100% of error traces kept and roughly 5% of normal ones, with the command and output saved.

☐ **T5 — Design the monitoring plan for the reference architecture** *(≈60 min, advanced)*

- **Goal:** produce the observability design section of a design doc.
- **Steps:**

1. List the three user journeys (browse, checkout, admin export) and one SLI and SLO for each.
2. Choose where each signal is stored and for how long, and estimate GB/day using `telemetry.py`.
3. Write the alert list with severity and runbook placeholders; include the monitoring-of-monitoring design.

- **Done when:** the plan has SLOs, a storage and retention table with volumes, an alert table with at most 6 pages, and a statement of what the monitoring plane's failure domain is.

☐ **T6 — Prove it: alert review** *(≈45 min, advanced)*

- **Goal:** justify every page.
- **Steps:**
  1. Without the guide, take a list of 10 typical alerts (CPU high, disk 90%, pod restarts, 5xx rate, p99 latency, queue depth, certificate expiry in 7 days, DB replication lag, error-budget burn, node not ready).
  2. For each decide: page, ticket, dashboard, or delete; explain with the SLO test.
- **Done when:** each of the 10 has a decision with a one-sentence reason, and at most 3 are pages.

10 Self-check questions

1. What is burn rate and why pair a long window with a short one?
2. Why is a user_id label on a Prometheus metric dangerous?
3. What is the difference between head sampling and tail sampling of traces?
4. Name two cause-based alerts that are still worth a page or ticket.
5. How do you make sure you notice when the monitoring system itself fails?

ANSWERS

1. Burn rate is the speed of error-budget consumption relative to exactly using it all over the SLO window. The long window ensures significance (real budget impact) and the short window ensures the problem is still happening, so alerts reset quickly.
2. Each distinct value creates a new time series; cardinality explodes memory, storage and query cost.
3. Head sampling decides when the trace starts (cheap, but blind to outcome); tail sampling decides after the trace completes, so it can keep all errors and slow traces.

4. Disk about to fill, certificate about to expire, replication broken: they predict certain user-facing failure.
5. An independent watcher: a dead man's switch alert that must keep arriving, external synthetic checks, and a second monitoring path outside the failure domain.

11 Further reading

- *The Site Reliability Workbook* (Google, free online) — "Alerting on SLOs".
- *Observability Engineering* by Charity Majors, Liz Fong-Jones and George Miranda.
- Prometheus, Alertmanager, Grafana, Loki documentation; OpenTelemetry documentation (opentelemetry.io), including the Collector.
- *Implementing Service Level Objectives* by Alex Hidalgo.
- Sentry documentation — sampling, rate limits, release health.

LAB NOTES What I ran, what I saw, what it means

19.9 Disaster Recovery

Topic 9 of 11 | ~4 hours | Difficulty: Advanced

1 What it is

Disaster recovery (DR) is the plan, and the architecture behind the plan, for getting a system working again after a serious failure: a lost region, a corrupted database, a deleted bucket, ransomware, a bad migration, a compromised cloud account. Two numbers drive every DR decision. **RPO (Recovery Point Objective)** is how much *data* you can afford to lose, measured as time (an RPO of 5 minutes means you may lose at most the last 5 minutes of writes). **RTO (Recovery Time Objective)** is how long you can be *down* before service must be restored.

DR is not the same as high availability. High availability (previous topics) keeps the system running through *expected, small* failures such as a lost node or zone, automatically. DR handles *large, rare, often human-caused or data-destroying* events where redundancy itself is damaged (replication faithfully copies a `DROP TABLE` to every replica). The architectural job is to choose, **per component**, the cheapest DR tier that meets that component's RPO and RTO.

2 Why it matters in DevOps

- **Replication is not backup.** Multi-AZ protects against a dead zone, not against a developer, a bug or an attacker deleting data.
- **Untested recovery does not exist.** Most failed recoveries fail on missing credentials, missing runbooks, forgotten dependencies or restore times far above estimates.
- **DR is expensive** when applied uniformly. Pricing it per component prevents both overspending and nasty surprises.
- **Regulation and customers** increasingly ask for documented RPO/RTO and evidence of restore tests.

Related reading: Disaster Recovery §0.9, RPO §15.6, RTO §15.7, Backups and Point-in-Time Recovery §13.5, Failure Scenario Drills §15.10.

Real-world system example

3

AWS disaster recovery strategies. AWS's whitepaper *Disaster Recovery of Workloads on AWS: Recovery in the Cloud* describes four strategies on a spectrum of cost and speed: **backup and restore**, **pilot light**, **warm standby** and **multi-site active/active**. It maps them to RPO/RTO ranges (from hours down to near zero) and stresses that data replication alone does not protect against corruption, so point-in-time backups are still needed. **GitLab (January 2017)** published a candid public postmortem of a production database incident in which an engineer removed the wrong data directory while repairing replication, and several of the backup mechanisms turned out not to have been working or verified; a few hours of customer data were lost. It is a classic example of why restores must be tested. Cloud providers' own post-incident summaries also repeatedly show the value of designing so that recovery does not depend on the thing that failed. (Check the primary sources for details.)

4 Diagram

The DR strategy spectrum: cost and complexity rise as RTO falls.

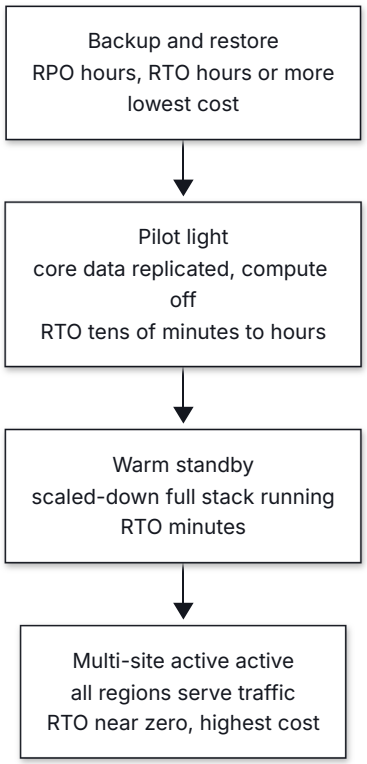


Figure 19.9-1

Per-component DR for the reference architecture, primary region and recovery region:

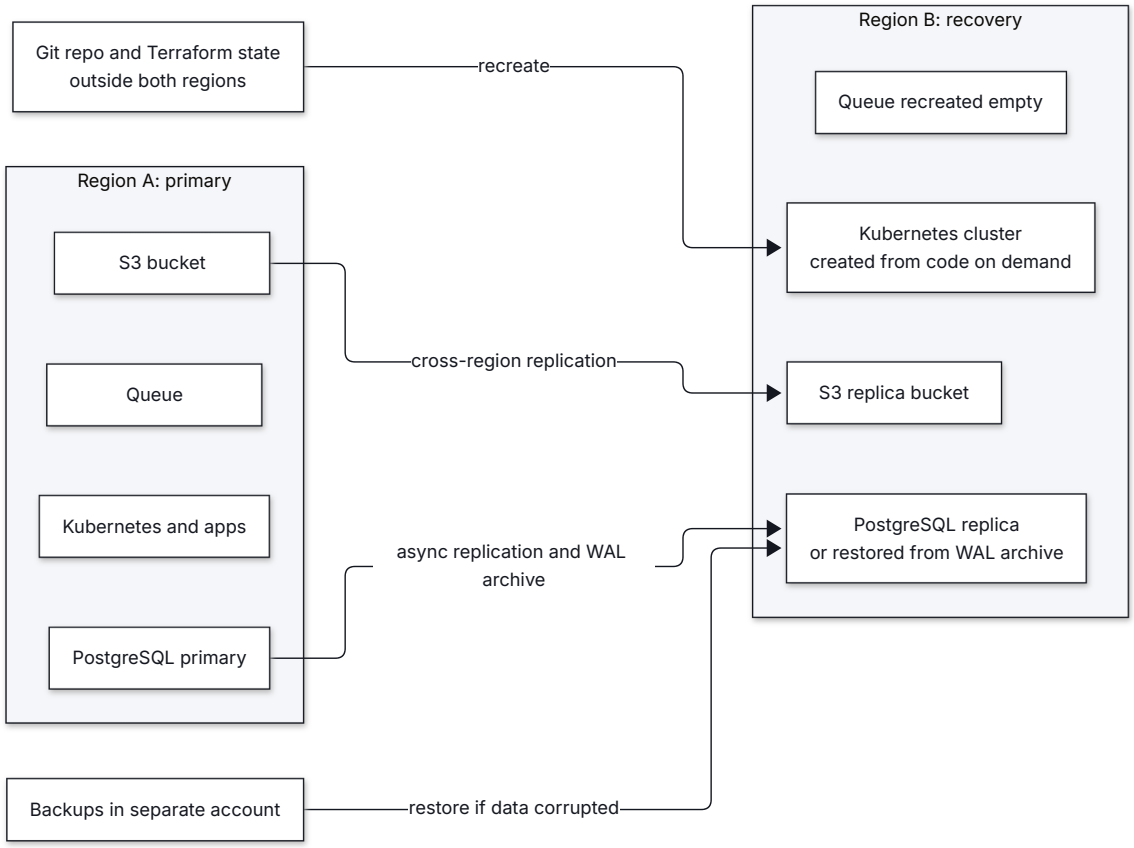


Figure 19.9-2

5 How it works

The four strategies

Strategy	What runs in the recovery region	Typical RPO	Typical RTO	Relative cost	Notes
Backup and restore	Nothing; backups stored there	Hours	Hours to a day	Lowest	Must rebuild infrastructure from code and restore data; restore time grows with data size
Pilot light	Data replicated, infrastructure defined but not running (or minimal)	Minutes	Tens of minutes to hours	Low	Bring up compute during a disaster; needs automation

Strategy	What runs in the recovery region	Typical RPO	Typical RTO	Relative cost	Notes
Warm standby	Scaled-down but complete working stack	Seconds to minutes	Minutes	Medium to high	Easy to test, always partially paid for
Multi-site active/active	Full stack serving traffic in each region	Near zero	Near zero	Highest	Data consistency and conflict resolution are hard (see CAP and consistency §13.12)

(The ranges above are typical orders of magnitude, not guarantees; your numbers come from your own measured drills.)

RPO/RTO-driven design

1. **List components and data** and classify each by business impact (not by how interesting the technology is).
2. **Set RPO and RTO per component** with the business owner; ask "what does an hour of downtime / an hour of lost orders cost?"
3. **Pick the mechanism** that meets each target (continuous replication for low RPO; backups for corruption protection; infrastructure as code for low-effort rebuilds).
4. **Compute the real restore time**: data size / restore throughput + replay time + DNS and cache warm-up + human decision time.
5. **Test**, record actual RPO/RTO, and update the design.

Some components need no DR at all beyond rebuild: caches (Redis) can be repopulated, stateless services are redeployed from images, dashboards are in Git. Concentrate budget on **stateful, irreplaceable data**: the database, object storage, secrets and keys.

Threat types and the mechanism that covers them

Threat	Does replication help?	What helps
Instance or zone failure	Yes	Multi-AZ, automatic failover
Region outage	Yes (cross-region replica)	Cross-region replication, standby or pilot light
Accidental delete or bad migration	No (replicated instantly)	Point-in-time recovery, versioning, delayed replicas
Ransomware or compromised credentials	No	Immutable/off-account backups (object lock), separate account with separate credentials
Provider or account loss	No	Backups outside the account/provider, infrastructure code in Git

See Backups §15.1, Off-site Backups §15.8 and Failover §15.5.

6 Step-by-step guide

Prerequisites: Python 3.9+; optional Docker for step 4. No cloud cost.

1. **Create a per-component RPO/RTO table and check it with code.** Save as `dr_check.py`:

```python
# minutes. Columns: name, required RPO, required RTO, mechanism RPO, mechanism RTO, mechanism
components = [
    ("PostgreSQL orders",  5,    60,  0.1, 15, "Multi-AZ sync standby + PITR (WAL) backups"),
    ("PostgreSQL region",  5,    120, 15,  57, "Cross-region WAL archive + restore (500 GB)"),
    ("Redis cache",        None, 30,  None, 10, "Rebuild from PostgreSQL, warm on start"),
    ("S3 uploads",         60,   240, 15,  60, "Versioning + cross-region replication"),
    ("Message queue",      5,    60,  0,   20, "Managed multi-AZ queue; DLQ replay"),
    ("Kubernetes cluster", None, 120, None, 90, "Recreate from Terraform and GitOps"),
    ("Grafana dashboards", 1440, 480, 60,  30, "Dashboards as code in Git"),
]
bad = 0
for name, rpo, rto, m_rpo, m_rto, mech in components:
    rpo_ok = rpo is None or (m_rpo is not None and m_rpo ⩽ rpo)
    rto_ok = m_rto ⩽ rto
    status = "OK  " if rpo_ok and rto_ok else "FAIL"
    bad += status == "FAIL"
    print(f"{status} {name:20s} RPO {str(m_rpo):>5}/{str(rpo):>5}  RTO {m_rto:>4}/{rto:>4}  {mech}")
print(f"{bad} component(s) fail their targets")
```

```bash
python3 dr_check.py
```

Expected output:

```
OK   PostgreSQL orders    RPO   0.1/    5  RTO   15/  60  Multi-AZ sync standby + PITR
  (WAL) backups
FAIL PostgreSQL region    RPO    15/    5  RTO   57/ 120  Cross-region WAL archive +
  restore (500 GB)
OK   Redis cache          RPO  None/ None  RTO   10/  30  Rebuild from PostgreSQL,
  warm on start
OK   S3 uploads           RPO    15/   60  RTO   60/ 240  Versioning + cross-region
  replication
OK   Message queue        RPO     0/    5  RTO   20/  60  Managed multi-AZ queue; DLQ
  replay
OK   Kubernetes cluster   RPO  None/ None  RTO   90/ 120  Recreate from Terraform and
  GitOps
OK   Grafana dashboards   RPO    60/ 1440  RTO   30/ 480  Dashboards as code in Git
1 component(s) fail their targets
```

The region-loss case for PostgreSQL fails its RPO: WAL archives shipped every 15 minutes cannot meet a 5-minute RPO. The fix is a continuously streaming cross-region replica (RPO seconds), which costs a running instance. The table made that trade-off visible and priceable. (The mechanism numbers are example assumptions; replace them with measured values.)

2. **Compute the real restore time.** Backups' restore time grows with data size:

```bash
python3 -c "
gb=500; mb_per_s=150; replay_min=10; decision_min=15; dns_min=5
restore=gb*1024/mb_per_s/60
print(f'data copy: {restore:.0f} min, total RTO estimate:
  {restore+replay_min+decision_min+dns_min:.0f} min')"
```

Expected: `data copy: 57 min, total RTO estimate: 87 min`. If the target RTO is 60 minutes, backup-and-restore does **not** meet it for 500 GB; you need a replica or snapshot-based approach.

3. **Choose the tier per component and price it.** For each stateful component write: the tier (backup-restore / pilot light / warm standby / active-active), the extra monthly cost (use the Cost topic's method and example prices), and the outage cost it avoids. If a component's DR costs more than the loss it prevents, lower its target and document that decision in an ADR.
4. **Rehearse a point-in-time recovery locally (PostgreSQL in Docker).** This proves that

backups and replicas are different things.

```bash
docker run -d --name pg -e POSTGRES_PASSWORD=pw postgres:16
sleep 5
docker exec pg psql -U postgres -c "CREATE TABLE orders(id int, total int); INSERT
  INTO orders SELECT g, g*10 FROM generate_series(1,1000) g;"
docker exec pg pg_dump -U postgres -Fc postgres -f /tmp/before.dump     # the "backup"
docker exec pg psql -U postgres -c "DROP TABLE orders;"                 # the
  "disaster" (a replica would copy this too)
docker exec pg pg_restore -U postgres -d postgres /tmp/before.dump
docker exec pg psql -U postgres -tc "SELECT count(*) FROM orders;"
```

Expected last line: `1000`. Clean up with `docker rm -f pg`. In production, use continuous WAL archiving for true point-in-time recovery; a dump is the simplest demonstration. See Restore Procedures §15.4.

5. **Write the DR runbook skeleton.** For the declared disaster "Region A unavailable": decision criteria and who declares it; the ordered steps (promote the database replica, apply Terraform in region B, let GitOps sync the apps, verify, switch DNS); verification checks; communication templates; and the **fail-back** procedure. Store the runbook somewhere that does not depend on Region A.
6. **Schedule tests**: quarterly restore test of backups into an empty environment, twice-yearly regional failover game day, and record actual RPO/RTO each time as the number that matters. See Testing Backup Restoration §15.9.

7 Real-world scenario

(Illustrative, not a documented incident.) A bad migration script run on a Thursday afternoon sets `orders.status = 'cancelled'` for every row in production. The Multi-AZ standby and the cross-region replica both applied the change within seconds, so "we have replicas" was irrelevant. The on-call engineer stops writes, restores the database to a new instance at a timestamp one minute before the migration using point-in-time recovery from the WAL archive, and replays the few legitimate orders from the queue's logs.

Recovery takes 70 minutes for a 400 GB database against an RTO target of 60, and loses about 2 minutes of data against an RPO of 5. The review accepts the RPO result, notes the RTO miss, and adds a delayed replica (30 minutes behind) for faster recovery, a migration review rule, and a quarterly restore drill. **Lesson:** replicas protect against hardware; point-in-time backups protect against humans, and drills turn estimates into facts.

8 Common mistakes and best practices

Don't

- Treat replication or RAID as a backup.
- Store backups in the same account, region or credentials as production.
- Assume the restore works because the backup job reports success.
- Apply one DR tier to everything; caches and dashboards do not need a warm standby.
- Keep the DR runbook, credentials or Terraform state only inside the failed region.

Do

- Set RPO and RTO per component with business owners and revisit annually.
- Keep immutable or separate-account copies of critical data (object lock, backup vault).
- Automate environment creation (Terraform, GitOps) so recovery is a pipeline run, not a heroic effort.
- Measure real recovery times in drills and record them next to the targets.
- Plan and test fail-back, not only fail-over.

9 Tasks

- [ ] **T1 — Per-component DR table** *(≈45 min, intermediate)*
  - **Goal:** assign targets and tiers to the reference architecture.
  - **Steps:**
    1. List all stateful and stateless components (CDN, LB, Kubernetes, API, worker, web, Redis, queue, PostgreSQL, S3, observability data, secrets).
    2. For each, set an RPO and RTO and choose a DR tier from the four strategies.
    3. Run the check logic of `dr_check.py` against your table.
  - **Done when:** every component has RPO, RTO, tier and mechanism, and the script reports zero failures (or you list the accepted failures with reasons).
- [ ] **T2 — Restore-time calculator** *(≈30 min, intermediate)*
  - **Goal:** find where backup-and-restore stops meeting your RTO.
  - **Steps:**
    1. Write a function `rto_minutes(gb, mb_per_s, replay, decision, dns)` and print results for 50 GB, 500 GB, 5 TB and 50 TB.
    2. Find the database size at which a 60-minute RTO is exceeded for your assumed throughput.

3. Name the alternatives for databases beyond that size (snapshot restore, replica promotion, partial restore).

- **Done when:** a table shows RTO per size and you state the crossover size.

- [ ] **T3 — Backup vs replica demonstration** *(≈45 min, intermediate)*
  - **Goal:** prove why replication is not backup.
  - **Steps:**
    1. Run two PostgreSQL containers with streaming replication (or use a docker-compose example from the PostgreSQL docs) and insert rows.
    2. `DELETE` all rows on the primary and show the replica is empty a moment later.
    3. Restore from a `pg_dump` taken before the delete and show the rows are back.
  - **Done when:** you show row counts on primary and replica after the delete (0 and 0), and after the restore (full count).

- [ ] **T4 — Point-in-time recovery drill** *(≈90 min, advanced)*
  - **Goal:** recover to a precise moment using WAL archiving.
  - **Steps:**
    1. Following the PostgreSQL documentation on continuous archiving, configure `archive_mode`, `archive_command` (to a local directory) and take a base backup with `pg_basebackup`.
    2. Insert rows, note a timestamp, then run a destructive statement.
    3. Restore the base backup, set `recovery_target_time` to just before the destructive statement and start the server.
  - **Done when:** the recovered database contains the rows from before the timestamp and not the effect of the destructive statement; save the logs and timings.

- [ ] **T5 — Write the Region-loss runbook** *(≈60 min, advanced)*
  - **Goal:** have an executable plan.
  - **Steps:**
    1. Write the runbook from step 6.5 with exact commands (Terraform workspace, `kubectl`, promotion of the replica, DNS change) as placeholders for your environment.
    2. Perform a tabletop exercise: a colleague reads the runbook aloud and challenges each step.
    3. Update the runbook with every gap found.

- **Done when:** the runbook has decision criteria, ordered steps with owners, verification checks, communications, and fail-back, and the tabletop produced at least three fixes.

☐ **T6 — Prove it: DR design defence** *(≈60 min, advanced)*

- **Goal:** justify a DR plan to a skeptical reviewer.
- **Steps:**
  1. Without the guide, present the DR plan for the reference architecture: components, targets, tiers, costs, tests.
  2. Answer: "What happens if an engineer deletes the production database at 14:00?" and "What if the whole cloud account is compromised?"
- **Done when:** both questions are answered with specific mechanisms, expected RPO/RTO, and the evidence (a drill) that supports each.

10 Self-check questions

1. What is the difference between RPO and RTO?
2. Why does a Multi-AZ replica not protect against an accidental `DROP TABLE`?
3. Name the four AWS DR strategies in order of increasing cost.
4. Which components can usually be rebuilt instead of backed up, and why?
5. Why store backups in a separate account?

ANSWERS

1. RPO is the acceptable amount of data loss (time between last recoverable write and the failure); RTO is the acceptable time to restore service.
2. Replication copies every change, including destructive ones, almost immediately. Only point-in-time backups, delayed replicas or versioning can return to a prior state.
3. Backup and restore, pilot light, warm standby, multi-site active/active.
4. Stateless services (images + code), caches, dashboards and infrastructure-as-code managed resources: their content can be regenerated from Git or from the source-of-truth database.
5. If the production account is compromised or deleted, backups in it go too; a separate account with separate credentials (and object lock) limits the attacker's reach.

11 Further reading

- AWS whitepaper *Disaster Recovery of Workloads on AWS: Recovery in the Cloud*; Well-Architected Framework — Reliability pillar.

- PostgreSQL documentation — "Backup and Restore" and "Continuous Archiving and Point-in-Time Recovery (PITR)".
- GitLab's public postmortem of the January 2017 database incident.
- *Site Reliability Engineering* (Google) — data integrity chapter.
- *Release It!* by Michael Nygard — chapters on stability and recovery.

LAB NOTES What I ran, what I saw, what it means

19.10 Reference Production Architecture Walkthrough

Topic 10 of 11 | ~5 hours | Difficulty: Advanced

1 What it is

This topic walks, hop by hop, through one complete production architecture for the `shop-api` application and explains *why* each component is there, what it costs you, how it fails, and which earlier topic of this level it exercises:

CDN → Load Balancer → Kubernetes → API / Worker / Web → Redis → Message Queue → PostgreSQL / S3, observed by **Prometheus, Grafana, Loki, OpenTelemetry and Sentry**.

It is a *reference* architecture: a sensible, widely used starting point you adapt, not a law. The aim is that after reading it you can redraw it from memory, explain every arrow, and say what you would remove for a smaller system and what you would add for a larger one. (Level 6 and Level 8 cover the AWS and Kubernetes building blocks; Level 10 covers the observability tools.)

2 Why it matters in DevOps

- **Most production systems are variations of this shape.** Recognising the pattern lets you onboard to a new company's stack in a day.
- **Each hop is a decision**: caching, queueing, state, identity, observability. Understanding the whole path prevents local optimisations that harm the system.
- **It is the shared vocabulary for design reviews and incident response**: "the worker is lagging because the queue is backed up because PostgreSQL is slow" is a sentence you can only say if the picture is in your head.

Related reading: What Happens When You Visit a URL §2.11, Cluster, Nodes and Control Plane §8.1, The Three Pillars §10.1, Integrating Argo CD, Terraform and Kubernetes §18.10.

3 Real-world system example

The same shape appears in public engineering write-ups from many companies. The Kubernetes documentation and the AWS Well-Architected reference diagrams show variations of "edge cache, load balancer, stateless compute cluster, cache, queue, relational database and object storage". Shopify's engineering blog describes a Rails monolith with MySQL, Redis and Kafka behind CDN and load balancers, deployed in "pods" (independent shards of the stack) for isolation. GitHub has described a Rails monolith with MySQL, Redis and background job workers. Stripe and many others publicly describe queue-backed asynchronous processing and idempotency keys. Grafana Labs documents the combination of Prometheus (metrics), Loki (logs) and Tempo (traces) with Grafana for visualisation, and the OpenTelemetry project documents the Collector as the central telemetry gateway. None of these is identical to the reference below, but all share the same building blocks: the differences are in scale, data tier and team structure.

4 Diagram

The request path and the asynchronous path:

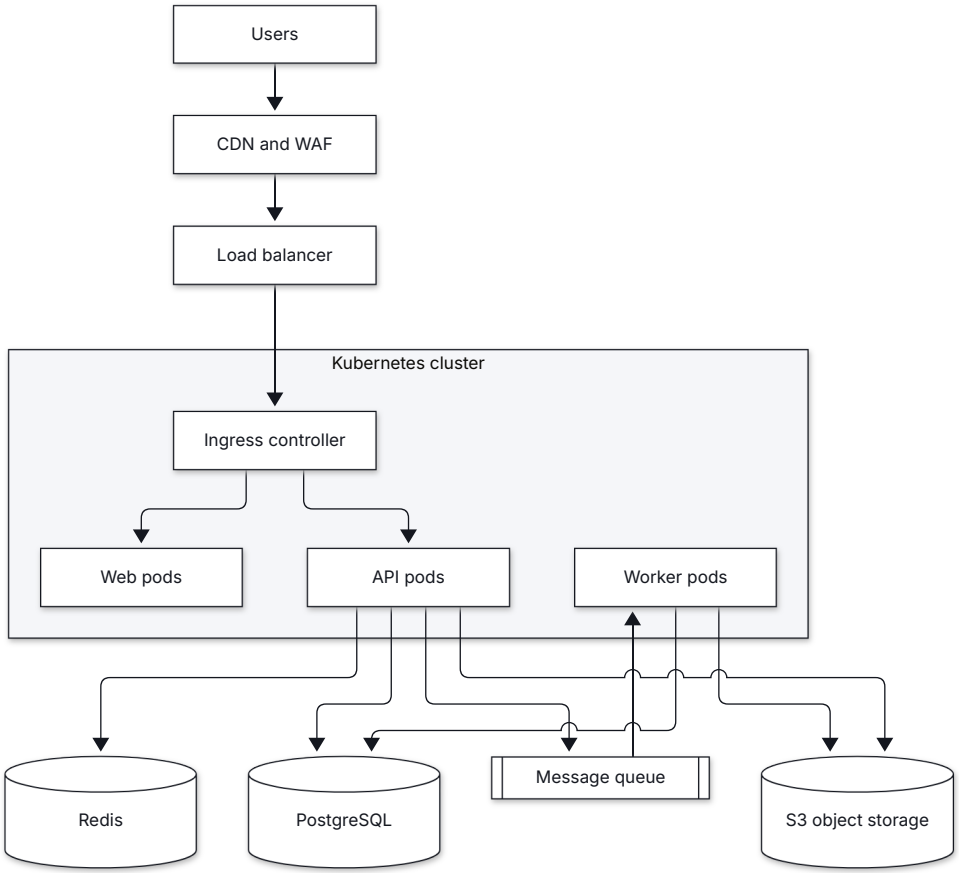


Figure 19.10-1

The observability plane (separate from the request path, but fed by every component):

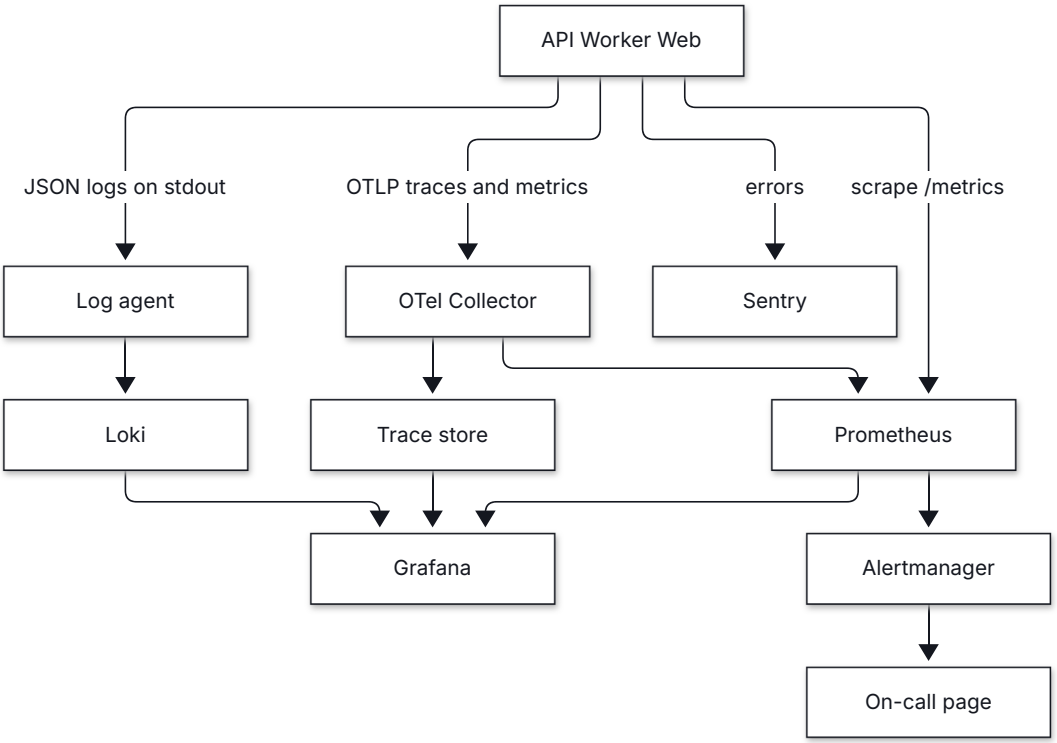


Figure 19.10-2

5 How it works

Component by component

Hop	Role	Key design decisions	Fails how	Topic exercised
CDN + WAF	Cache static assets and cacheable API responses near users, absorb DDoS, terminate TLS at the edge	Cache keys, TTLs, `stale-if-error`, origin shielding, restrict origin to CDN	Purge mistakes, caching private data, origin overload on cold cache	Performance, Security, Cost
Load balancer	Spread connections across zones and ingress nodes, health checks	L7 (ALB) vs L4 (NLB), cross-zone, idle timeouts, TLS policy	Misconfigured health check, idle-timeout mismatch causes 502/504	Availability

Hop	Role	Key design decisions	Fails how	Topic exercised
Kubernetes	Schedule and heal pods, rolling updates, autoscaling	Managed control plane, multi-AZ node groups, requests/limits, PodDisruptionBudgets, topology spread, Pod Security Standards, NetworkPolicies	Node pressure, bad requests/limits, cluster upgrade mistakes	Scalability, Ops complexity
API pods	Stateless HTTP handlers	Timeouts and retries with backoff, idempotency keys, connection pool sizing, graceful shutdown	Thread/connection exhaustion, slow dependency	Scalability, Performance
Worker pods	Consume the queue: emails, exports, webhooks, image processing	Idempotent handlers, retry with backoff, dead-letter queue, scale on queue depth	Poison messages, duplicate processing, backlog	Scalability, Availability
Web pods	Server-rendered or static frontend (often just CDN + object storage)	Do you need pods at all? Static hosting on S3 + CDN is cheaper and simpler	Rarely the problem	Cost, Ops complexity
Redis	Cache, sessions, rate-limit counters, short-lived locks	TTLs, eviction policy, HA (replica + failover), treat as cache not source of truth	Stampede on restart, eviction of hot keys	Performance, Availability
Message queue	Decouple accept from process; absorb bursts	Managed queue (SQS, RabbitMQ, Kafka), at-least-once delivery, DLQ, ordering needs	Backlog growth, poison messages, lost messages if misconfigured	Scalability, DR
PostgreSQL	System of record	Managed (RDS/Aurora/Cloud SQL), Multi-AZ, pooled connections, PITR backups, read replicas, migrations that are backward compatible	Connection exhaustion, long transactions, disk full, bad migration	Availability, DR, Performance
S3	Uploads, exports, backups, logs/trace/metric blocks	Pre-signed URLs, lifecycle rules, versioning, encryption, VPC endpoint	Public bucket by mistake, unbounded growth	Cost, Security, DR
Observability	See and alert	OTel Collector gateway, SLO burn-rate alerts, cardinality and retention budgets	Pipeline outage, runaway cost	Monitoring

Cross-cutting concerns

- **Identity:** pods use workload identity (IRSA on EKS, equivalents elsewhere) to access S3 and the queue: no static keys. Humans use SSO. See Least-privilege IAM §11.2.

- **Secrets:** External Secrets Operator or the CSI driver pulls from a secrets manager; nothing sensitive in Git. See Kubernetes Secrets §8.7.
- **Delivery:** CI builds an immutable image; GitOps (Argo CD) syncs manifests; progressive delivery with a canary gate. See The Full Flow: CI to GitOps §17.7.
- **Infrastructure:** VPC, subnets, EKS, RDS, S3, IAM defined in Terraform with remote state and locking. See Terraform CI/CD §7.7.
- **Network:** public subnets hold only the load balancer and NAT; nodes, databases and caches live in private subnets; security groups and NetworkPolicies default-deny.

Request walk-through

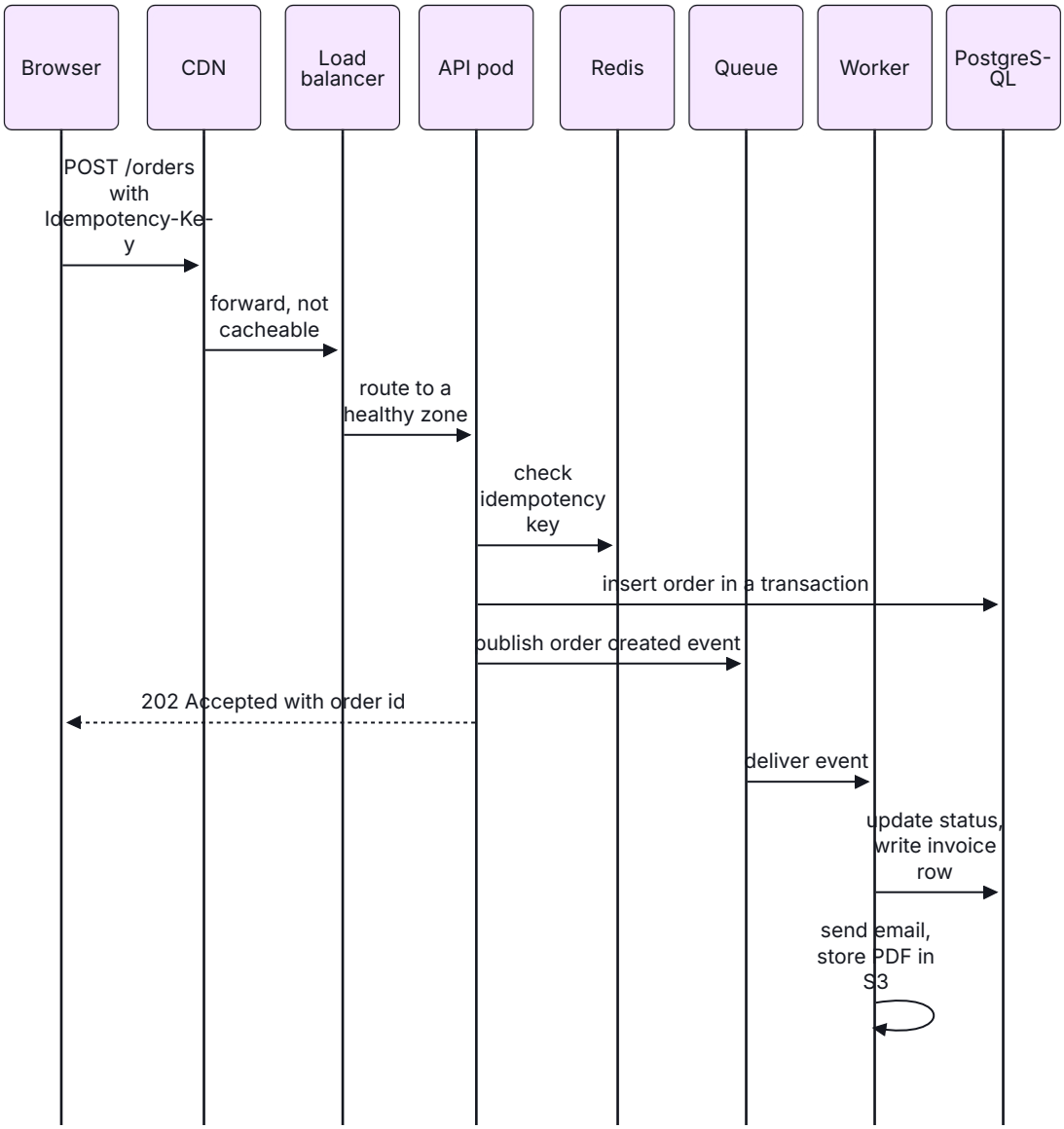


Figure 19.10-3

The user gets a fast `202`; the slow work (PDF, email) happens asynchronously with retries. The idempotency key makes client retries safe.

6 Step-by-step guide

Prerequisites: Docker with Compose v2 for steps 1 and 2; a kind cluster and `kubectl` for step 3 (optional). All local, no cloud cost. If you do not have Docker, you can still do steps 4-6 on paper.

1. **Stand up the stateful services and observability stack locally with Docker Compose.** Save as `compose.yaml`:

```yaml
services:
  postgres:
    image: postgres:16
    environment: { POSTGRES_PASSWORD: dev-only-password }
    ports: ["5432:5432"]
    healthcheck: { test: ["CMD-SHELL", "pg_isready -U postgres"], interval: 5s,
  retries: 10 }
  redis:
    image: redis:7
    ports: ["6379:6379"]
  rabbitmq:
    image: rabbitmq:3-management
    ports: ["5672:5672", "15672:15672"]
  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    environment: { MINIO_ROOT_USER: devuser, MINIO_ROOT_PASSWORD: dev-only-password }
    ports: ["9000:9000", "9001:9001"]
  prometheus:
    image: prom/prometheus
    volumes: ["./prometheus.yml:/etc/prometheus/prometheus.yml:ro"]
    ports: ["9090:9090"]
  loki:
    image: grafana/loki
    ports: ["3100:3100"]
  grafana:
    image: grafana/grafana
    ports: ["3000:3000"]
  otel-collector:
    image: otel/opentelemetry-collector-contrib
    volumes: ["./otel.yaml:/etc/otelcol-contrib/config.yaml:ro"]
    ports: ["4317:4317", "4318:4318"]
```

Create the two small config files it mounts:

```yaml
# prometheus.yml
global: { scrape_interval: 15s }
scrape_configs:
  - job_name: prometheus
    static_configs: [{ targets: ["localhost:9090"] }]
```

```yaml
# otel.yaml
receivers: { otlp: { protocols: { grpc: {}, http: {} } } }
processors: { batch: {} }
exporters: { debug: {} }
service:
  pipelines:
    traces: { receivers: [otlp], processors: [batch], exporters: [debug] }
```

```bash
docker compose up -d
docker compose ps
```

Expected: eight services in state `running` (PostgreSQL `healthy`). The queue (RabbitMQ, standing in for SQS or Kafka), the object store (MinIO, S3-compatible), and the three observability backends now exist. (Image tags are unpinned here for brevity; pin them in anything real.)

2. **Verify each component answers.**

```bash
docker compose exec postgres pg_isready -U postgres        # accepting connections
docker compose exec redis redis-cli ping                   # PONG
curl -s -o /dev/null -w "prometheus %{http_code}\n" localhost:9090/-/ready
curl -s -o /dev/null -w "grafana    %{http_code}\n" localhost:3000/api/health
curl -s -o /dev/null -w "minio      %{http_code}\n" localhost:9000/minio/health/live
curl -s -o /dev/null -w "rabbitmq   %{http_code}\n" -u guest:guest
  localhost:15672/api/overview
```

Expect `200` from each HTTP check. (Loki may take about a minute to report ready: check `curl localhost:3100/ready`.) Open Grafana at `http://localhost:3000` (default login `admin` / `admin` on first start, change when prompted) and add Prometheus (`http://prometheus:9090`) and Loki (`http://loki:3100`) as data sources.

3. **Define the compute tier as Kubernetes manifests.** In a kind or other cluster, save as `api.yaml` the Deployment, Service, HPA and Ingress that mirror the diagram:

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: shop-api, namespace: shop }
spec:
  replicas: 3
  selector: { matchLabels: { app: shop-api } }
  template:
    metadata:
      labels: { app: shop-api }
      annotations: { prometheus.io/scrape: "true", prometheus.io/port: "3000" }
    spec:
      serviceAccountName: shop-api
      securityContext: { runAsNonRoot: true, seccompProfile: { type: RuntimeDefault }
  }
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector: { matchLabels: { app: shop-api } }
      containers:
        - name: api
          image: example.com/shop-api:1.4.0
          ports: [{ containerPort: 3000 }]
          env:
            - { name: REDIS_URL, value: "redis://redis.shop.svc:6379" }
            - { name: OTEL_EXPORTER_OTLP_ENDPOINT, value: "http://otel-
  collector.observability.svc:4317" }
            - name: DATABASE_URL
              valueFrom: { secretKeyRef: { name: shop-api-db, key: url } }
            - name: SENTRY_DSN
              valueFrom: { secretKeyRef: { name: shop-api-sentry, key: dsn } }
          resources:
            requests: { cpu: 250m, memory: 256Mi }
            limits: { memory: 512Mi }
          readinessProbe: { httpGet: { path: /readyz, port: 3000 }, periodSeconds: 5 }
          livenessProbe: { httpGet: { path: /healthz, port: 3000 }, periodSeconds: 10,
  failureThreshold: 3 }
          securityContext: { allowPrivilegeEscalation: false, readOnlyRootFilesystem:
  true, capabilities: { drop: ["ALL"] } }
---
apiVersion: v1
kind: Service
metadata: { name: shop-api, namespace: shop }
spec:
  selector: { app: shop-api }
  ports: [{ port: 80, targetPort: 3000 }]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: shop-api, namespace: shop }
spec:
```

```
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: shop-api }
  minReplicas: 3
  maxReplicas: 30
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 60 } }
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  namespace: shop
  annotations: { nginx.ingress.kubernetes.io/proxy-read-timeout: "30" }
spec:
  ingressClassName: nginx
  tls: [{ hosts: [api.example.com], secretName: api-example-com-tls }]
  rules:
    - host: api.example.com
      http:
        paths:
          - { path: /, pathType: Prefix, backend: { service: { name: shop-api, port: {
  number: 80 } } } }
```

Check it parses and passes server-side validation without creating anything:

```bash
kubectl create namespace shop
kubectl apply --dry-run=server -f api.yaml
```

Expected: lines such as `deployment.apps/shop-api created (server dry run)`. (The Ingress needs an ingress controller CRD/class to exist; `ingressClassName: nginx` validates even if no controller is installed.) The worker Deployment is identical in shape except it has no Service or Ingress and scales on queue depth (KEDA, or an HPA on an external metric), and the web Deployment serves the frontend or is replaced by static hosting on S3 behind the CDN.

4. **Trace one write through the whole architecture and annotate the diagram.** Using the sequence diagram in section 5, for every arrow write the protocol, timeout, retry policy, and what the caller does if the callee is down. This table is the heart of the design doc; gaps (an arrow with no timeout) are your first findings.
5. **Run a failure-mode analysis.** For each component, answer: what happens to users if it dies, how do we detect it, what recovers it.

Component fails	User-visible effect	Detection	Recovery
One API pod	None (readiness removes it)	Pod restarts metric	Kubernetes reschedules

Component fails	User-visible effect	Detection	Recovery
One AZ	Reduced capacity, possibly slower	LB healthy-host count, burn-rate alert	Surviving AZs absorb load (headroom)
Redis	Slower responses, more DB load	Cache hit ratio, Redis up metric	Failover to replica; app bypasses cache
Queue	Orders accepted but side effects delayed, or checkout blocked	Queue depth and age	Managed service failover; outbox table replays
PostgreSQL primary	Writes fail for about a minute	DB health, error rate	Multi-AZ failover; read-only degradation
Observability stack	Blind, but users unaffected	Dead man's switch, external checks	Restore from IaC; telemetry buffered at agents
Region	Total outage	Synthetic checks, DNS health checks	DR runbook (previous topic)

6. **Tear down.**

```bash
docker compose down -v
kubectl delete namespace shop     # if you created it
```

7 Real-world scenario

(Illustrative, not a documented incident.) A Monday morning email campaign sends 80,000 customers to the site. The CDN serves most of the catalogue, but the checkout API rises from 30 to 600 requests per second. Dashboards in Grafana show API CPU modest, but the queue's oldest-message age climbs to 25 minutes and the order confirmation emails are late; support tickets begin.

Tracing the path: the API accepts orders quickly (the queue absorbed the burst, as designed), but workers are scaled to a fixed 4 replicas and each handler makes a slow synchronous call to a PDF service. The fix, within minutes, is to scale workers on queue depth (KEDA), to raise the PDF service timeout budget separately, and to move PDF generation off the critical email path. Sentry shows no errors, because nothing failed, only slowed; the **age of the oldest message** SLI is what revealed the problem. **Lesson:** each component can be healthy while the system is not; end-to-end signals (queue age, order-to-email latency) catch what per-component health checks miss.

8 Common mistakes and best practices

Don't

- Draw only the happy path: every arrow needs a timeout, a retry rule and a failure behaviour.
- Treat Redis as a durable store or make it a hard dependency of the critical path without a fallback.
- Share one PostgreSQL superuser among API, worker and migrations.
- Run workers without a dead-letter queue and idempotent handlers.
- Let the observability stack share a failure domain (and credentials) with what it monitors.

Do

- Keep the API stateless and push slow work to the queue with an outbox or idempotency keys.
- Put the database behind a connection pooler and define a read-only degraded mode.
- Scale workers on queue depth or age, not CPU alone.
- Use workload identity, default-deny network policies and non-root, read-only containers.
- Version the whole picture: Terraform for cloud, Git for manifests, dashboards and alerts as code.

9 Tasks

- [ ] **T1 — Bring up the platform locally** *(≈45 min, intermediate)*
  - **Goal:** run the stateful and observability components of the reference architecture.
  - **Steps:**
    1. Create `compose.yaml`, `prometheus.yml` and `otel.yaml` from step 6.1.
    2. Run `docker compose up -d` and the checks in step 6.2.
    3. Add Prometheus and Loki as Grafana data sources and run the query `up` in Explore.
  - **Done when:** every health check returns success and Grafana shows `up{job="prometheus"} = 1`.
- [ ] **T2 — Send telemetry through the Collector** *(≈60 min, advanced)*
  - **Goal:** prove the OTLP path works end to end.
  - **Steps:**
    1. Install the `telemetrygen` tool from the OpenTelemetry collector-contrib repository (via `go install`, or use its container image).
    2. Send 20 traces to `localhost:4317` with `--otlp-insecure`.

3. Read the Collector logs (`docker compose logs otel-collector`) and find the received spans.

- **Done when:** the Collector log shows 20 traces with their span names.

- [ ] **T3 — Validate the manifests** *(≈45 min, intermediate)*
  - **Goal:** make the compute tier machine-checked.
  - **Steps:**
    1. Create a kind cluster and the `shop` namespace.
    2. Apply `api.yaml` with `--dry-run=server`, then for real with a placeholder `nginx` image and matching port settings in a copy of the file.
    3. Break it on purpose (a nonexistent image tag, a wrong probe path) and read the resulting events with `kubectl describe pod`.
  - **Done when:** the valid version has Ready pods, the broken version shows the failure reason, and you fixed it.

- [ ] **T4 — Annotate the request path** *(≈60 min, advanced)*
  - **Goal:** reveal missing timeouts and retries.
  - **Steps:**
    1. For each arrow in the sequence diagram write: protocol, timeout, retry policy, idempotency, and behaviour on failure.
    2. Verify the budget: the timeouts on the path must nest (outer larger than inner).
    3. Mark any arrow whose failure takes down the checkout path.
  - **Done when:** the table is complete, the timeouts nest correctly, and at least two gaps are listed with fixes.

- [ ] **T5 — Failure-mode game day (paper plus local)** *(≈75 min, advanced)*
  - **Goal:** see what actually happens when a dependency disappears.
  - **Steps:**
    1. With the Compose stack running, stop Redis (`docker compose stop redis`) while a small script keeps calling a stub app that uses it with and without a fallback.
    2. Stop and restart RabbitMQ with messages in a durable queue; confirm messages survive or not depending on settings.
    3. Fill in a table: what broke, what degraded, how long recovery took.
  - **Done when:** you have measured results for both failures and you changed a setting (fallback or durability) that improved the outcome.

- [ ] **T6 — Prove it: redraw and defend** *(≈60 min, advanced)*

- **Goal:** own the architecture.
- **Steps:**
  1. Without the guide, redraw the architecture (two Mermaid diagrams: request path and observability).
  2. For three components, justify the choice, one alternative and the trade-off, and name the first thing you would drop for a team of three.
- **Done when:** both diagrams render, and the justifications cite at least one topic of this level each (for example availability, cost, operational complexity).

10 Self-check questions

1. Why does the API return `202` and publish to a queue instead of doing all the work synchronously?
2. What is the role of the OpenTelemetry Collector versus talking to each backend directly?
3. Which components must be backed up, and which can be rebuilt?
4. What end-to-end signals would you add that component health checks do not give?
5. Which parts would you remove for a three-person team, and why?

ANSWERS

1. It keeps latency low, absorbs bursts, isolates slow or failing downstream work, and allows retries; the cost is eventual consistency and the need for idempotent consumers.
2. The Collector is a single vendor-neutral egress point that batches, samples, redacts and routes telemetry, so applications do not know about backends and backends can change without redeploying apps.
3. Back up PostgreSQL, S3 data, secrets/keys and Terraform state; rebuild pods, caches, dashboards-as-code and infrastructure from Git and images.
4. Queue age, order-to-confirmation latency, synthetic checkout checks, SLO burn rate, business metrics such as orders per minute.
5. Typically Kubernetes (use a managed container platform), the separate web pods (static hosting + CDN), a self-hosted observability stack (use managed), and possibly the queue (a Postgres job table), as long as the SLOs are still met.

11 Further reading

- AWS Well-Architected Framework and the AWS Architecture Center reference diagrams.
- Kubernetes documentation — Concepts, and the production environment guidance.

- OpenTelemetry documentation — Collector architecture; Grafana documentation for Loki, Mimir/Prometheus and Tempo.
- *Designing Data-Intensive Applications* by Martin Kleppmann.
- *Building Microservices* by Sam Newman, for the trade-offs of splitting services.

19.11 Trade-offs and Architecture Decision Records

Topic 11 of 11 | ~4 hours | Difficulty: Advanced

1 What it is

Every architecture is a pile of trade-offs: consistency versus availability, speed versus cost, flexibility versus simplicity. An **Architecture Decision Record (ADR)** is a short document that captures *one* significant decision: the context that forced it, the options considered, the choice, and the consequences accepted. ADRs live in the code repository, next to the thing they describe, and are immutable once accepted (you supersede, you do not rewrite).

Around the ADR sit three supporting skills. **Trade-off matrices** compare options against weighted criteria so the debate is about the weights and facts instead of opinions. **Design reviews** are structured conversations where peers try to find the flaws in a design before production does. **Estimating** (back-of-envelope or "Fermi" calculations) gives order-of-magnitude numbers (storage, traffic, cost) quickly enough to rule options in or out.

2 Why it matters in DevOps

- **Teams forget why.** Two years later nobody remembers why you chose a queue over a database table; without a record, people either cargo-cult it or tear it out and repeat the mistake.
- **Onboarding and incident response** speed up when the reasoning is written down ("it is read-only during failover on purpose, see ADR-012").
- **Decisions get better when written**: stating alternatives and consequences exposes weak reasoning.
- **Reviews catch expensive mistakes cheaply**: an hour reviewing a design is far cheaper than a migration after launch.
- **Senior engineering is largely this skill**: choosing well under uncertainty, explaining it, and being willing to revisit it.

Related reading: Platform Engineering §0.13, Postmortems §14.7, Policy as Code §18.7.

3 Real-world system example

Michael Nygard's ADR proposal and its adoption. Michael Nygard's 2011 blog post *Documenting Architecture Decisions* proposed short, numbered, immutable records with the sections Title, Status, Context, Decision and Consequences, stored in the project repository. The idea spread widely; the ADR GitHub organisation (adr.github.io) collects templates and tooling, **MADR** (Markdown Any Decision Records) is a popular template, and Thoughtworks' Technology Radar has listed lightweight ADRs as a recommended practice. Large engineering organisations use related documents: the **Rust RFC** process, the **Python PEPs**, **Kubernetes Enhancement Proposals (KEPs)** and **Go proposals** are all public, reviewable, versioned design documents, with discussion and a recorded outcome. AWS and Google both describe design review as part of how they build services (for example Amazon's published practice of writing narrative documents, and Google's design docs, as described in public engineering-culture writing). They differ in weight, but the structure is the same: write the options down and let others attack them.

4 Diagram

The life cycle of an ADR:

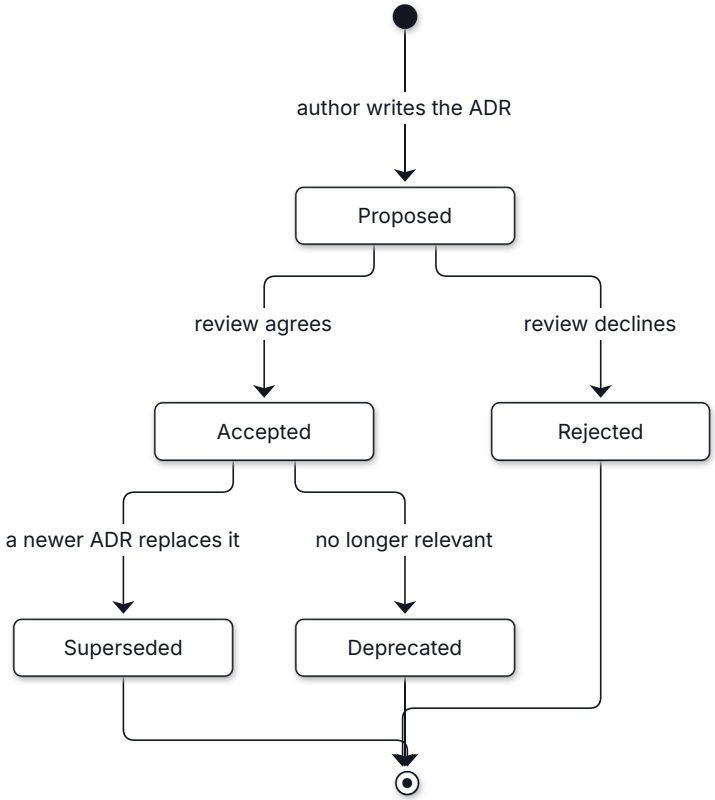


Figure 19.11-1

How a significant decision flows through the team:

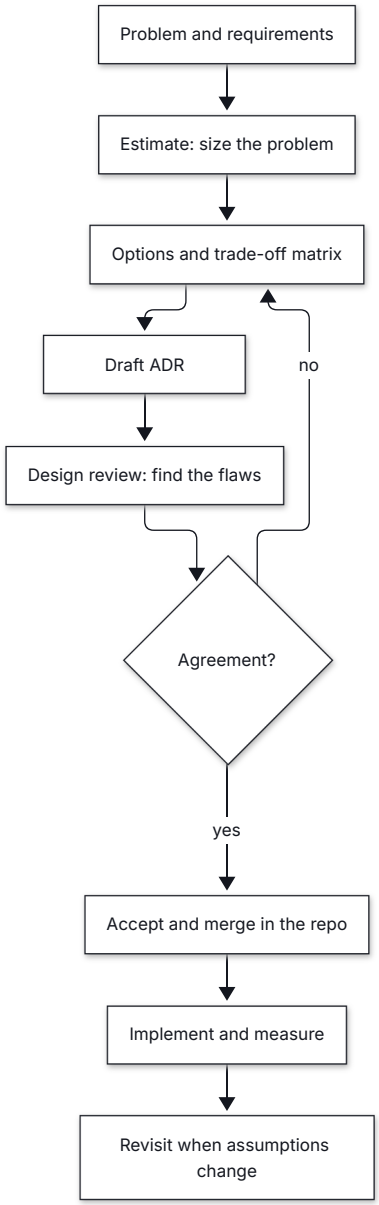


Figure 19.11-2

5 How it works

Writing ADRs

When to write one: when a decision is hard to reverse, affects several teams, has real alternatives, or you expect someone to ask "why?" later. Examples: choice of database, queue, deployment platform, multi-region strategy, authentication approach, a major library. **Do not** write one for every function name. Rule of thumb: if reversing it costs weeks, write it.

The format (Nygard / MADR style):

```markdown
# ADR-0007: Use a managed queue (SQS) for asynchronous jobs

- Status: Accepted
- Date: 2026-03-04
- Deciders: platform team, shop-api team
- Supersedes: none

## Context
What problem, what forces and constraints (requirements, SLOs, team size, budget,
  deadlines, existing systems).
State facts and numbers, with sources.

## Decision drivers
The criteria that matter, ranked: operational simplicity, durability, cost, throughput
  headroom...

## Options considered
1. Postgres job table. Pros / cons.
2. Managed queue (SQS). Pros / cons.
3. Self-hosted Kafka. Pros / cons.

## Decision
We will use ... because ... (one paragraph, in active voice: "We will ...").

## Consequences
Positive, negative and neutral. What becomes easier, what becomes harder, what we must now
  do (runbooks, monitoring, migration).
Risks and how we will notice if the decision is wrong (the trigger to revisit).

## Links
Related ADRs, issues, benchmarks, diagrams.
```

Good ADR habits: one decision per record; short (one to two pages); numbered and stored in `docs/adr/`; state the **trade-offs honestly** including what you gave up; record **why not** for the rejected options; link to evidence; and never edit an accepted ADR except to change its status (write a new one that supersedes it).

Trade-off matrices

A weighted decision matrix scores each option on each criterion, multiplies by weights and sums. It will not make the decision for you, but it forces you to (1) name the criteria, (2) say how much each matters, and (3) expose disagreement: people often agree on the scores but not the weights. Always run a **sensitivity check**: if a small change to a weight flips the winner, the options are close and the decision should be made on something else (reversibility, risk, team preference), and say so in the ADR.

Common trade-off pairs worth naming explicitly: consistency vs availability (see CAP and consistency §13.12), latency vs throughput, cost vs redundancy, build vs buy, flexibility vs simplicity, speed of delivery vs safety. Prefer **reversible** ("two-way door") decisions to be made quickly and **irreversible** ("one-way door") decisions slowly, with more review.

Design reviews

A good design review is a meeting that everyone has prepared for by reading the document. The author presents for five minutes; the rest is questions. Reviewers use a checklist (see the capstone for a full one) covering requirements, failure modes, security, operations, cost and migration. Rules: critique the design, not the person; prefer questions to assertions ("what happens when the queue is full?"); record decisions and open questions; assign owners. Reviews should end with one of: approved, approved with changes, or needs another round.

Estimating

Order-of-magnitude numbers are enough to choose between designs. Method: state assumptions, use round numbers, compute, then sanity-check against something known. Useful constants: 86,400 seconds in a day; about 2.6 million seconds in a month; 1 million requests per day is about 12 requests per second on average; typical peak is 3-10x the daily average. Keep a list of numbers you have measured in your own system; they beat any table.

6 Step-by-step guide

Prerequisites: Python 3.9+, a Git repository for your docs folder. No cost.

1. **Create the ADR folder and the template.**

```bash
mkdir -p docs/adr
cat > docs/adr/0000-template.md <<'EOF'
# ADR-0000: Short title in the form "Use X for Y"

- Status: Proposed | Accepted | Rejected | Deprecated | Superseded by ADR-NNNN
- Date: YYYY-MM-DD
- Deciders: names or teams
```

```
## Context
## Decision drivers
## Options considered
## Decision
## Consequences
## Links
EOF
ls docs/adr
```

Expected: `0000-template.md`. Commit this first so every later ADR starts from the same shape.

2. **Estimate before you decide.** Save as `fermi.py`:

```python
orders_per_day = 50_000
events_per_order = 12          # created, paid, packed, ...
bytes_per_event = 700          # JSON with metadata
retention_years = 7
replication_factor = 2         # primary + one copy
raw = orders_per_day * events_per_order * bytes_per_event * 365 * retention_years
print(f"raw: {raw/1e12:.2f} TB over {retention_years} years")
print(f"with replication: {raw*replication_factor/1e12:.2f} TB")
avg = orders_per_day * events_per_order / 86400
print(f"events per second (avg): {avg:.1f}; peak at 10x: {avg*10:.0f}")
```

```bash
python3 fermi.py
```

Expected output:

```
raw: 1.07 TB over 7 years
with replication: 2.15 TB
events per second (avg): 6.9; peak at 10x: 69
```

Seven events per second and about 2 TB over seven years mean a Postgres table (partitioned by month) is plenty for the audit log; a streaming platform would be over-engineering. The estimate just *removed an option* in two minutes.

3. **Build a weighted decision matrix with a sensitivity check.** Save as `matrix.py`:

```python
criteria = {"Operational simplicity": 30, "Durability / delivery guarantees": 25,
            "Throughput headroom": 15, "Cost": 15, "Team familiarity": 15}
options = {
```

```
    "Postgres job table":  {"Operational simplicity": 5, "Durability / delivery
  guarantees": 4, "Throughput headroom": 2, "Cost": 5, "Team familiarity": 5},
    "Managed queue (SQS)": {"Operational simplicity": 4, "Durability / delivery
  guarantees": 5, "Throughput headroom": 4, "Cost": 4, "Team familiarity": 3},
    "Self-hosted Kafka":   {"Operational simplicity": 1, "Durability / delivery
  guarantees": 5, "Throughput headroom": 5, "Cost": 2, "Team familiarity": 1},
}
def scores(weights):
    return {o: sum(weights[c] * s[c] for c in weights) / sum(weights.values()) for o,
  s in options.items()}

for o, v in sorted(scores(criteria).items(), key=lambda kv: -kv[1]):
    print(f"{o:22s} {v:4.2f}")
w2 = dict(criteria); w2["Throughput headroom"] *= 2
print("\nthroughput weight doubled:")
for o, v in sorted(scores(w2).items(), key=lambda kv: -kv[1]):
    print(f"{o:22s} {v:4.2f}")
```

BASH

```
python3 matrix.py
```

Expected:

```
Postgres job table     4.30
Managed queue (SQS)    4.10
Self-hosted Kafka      2.75

throughput weight doubled:
Managed queue (SQS)    4.09
Postgres job table     4.00
Self-hosted Kafka      3.04
```

The winner flips when throughput matters twice as much: the top two are close. The honest ADR says so, picks based on the stated drivers, and records the trigger to revisit ("if sustained load passes 500 jobs/s, move to the managed queue"). (Scores are example judgments; yours should be backed by measurements and team discussion.)

4. **Write the ADR.** Copy the template to `docs/adr/0001-use-postgres-job-table-for-async-jobs.md` and fill it in, including the estimate (step 2), the matrix result (step 3), what you gave up, and the revisit trigger. Keep it to one page.
5. **Hold a design review.** Circulate the ADR 24 hours before. In the meeting, run the review checklist (capstone, section 4, the "Design review" milestone): requirements, failure modes, security, operability, cost, migration and rollback. Record outcomes as comments in the pull request; approve by merging.

6. **Supersede, don't rewrite.** Six months later the load has grown. Write `0014-move-async-jobs-to-sqs.md` with `Supersedes: ADR-0001`, and change ADR-0001's status line to `Superseded by ADR-0014`. The history of how your thinking changed is itself valuable.

7 Real-world scenario

(Illustrative, not a documented incident.) A new engineer joins and proposes replacing the `shop-api` queue with direct database writes "to remove a moving part". The team lead points to `docs/adr/0007`, which explains that the queue was chosen to absorb flash-sale bursts that previously took the database down, with a trade-off of eventual consistency for order emails. The ADR also lists the trigger to revisit: "if flash sales are discontinued or sustained load stays below 50 jobs/s".

The engineer checks the metrics: peak is still 600 jobs/s at sale time. The proposal is dropped in ten minutes instead of causing a two-hour debate and, worse, a repeat outage. A month later the same engineer writes ADR-0015 for a genuine change (adding a dead-letter queue) that follows the same format. **Lesson:** an ADR turns institutional memory into something that survives turnover and shortens future arguments.

8 Common mistakes and best practices

Don't

- Write ADRs after the fact to justify what you already built, with straw-man alternatives.
- Hide the downsides: an ADR with no negative consequences is advertising.
- Edit accepted ADRs; the history is the point.
- Write a book. Long ADRs are not read.
- Treat the matrix score as the decision; it is an input to judgment.

Do

- Write one when a decision is costly to reverse; store ADRs in the repo and review them like code.
- Always include rejected options and why.
- State assumptions and numbers, and the trigger that would make you revisit.
- Do a sensitivity check on weights and say if the call is close.
- Separate one-way-door from two-way-door decisions and spend review effort accordingly.

9 Tasks

- [ ] **T1 — Set up ADRs in a repo** *(≈20 min, beginner)*
  - **Goal:** have the infrastructure for decision records.
  - **Steps:**
    1. Create `docs/adr/0000-template.md` as in step 6.1 and commit it in a Git repository.
    2. Add `docs/adr/README.md` explaining when to write an ADR and the status values.
  - **Done when:** `git log --oneline -- docs/adr` shows the commit and the template renders on GitHub.

- [ ] **T2 — Estimate three numbers** *(≈40 min, intermediate)*
  - **Goal:** practise Fermi estimation with stated assumptions.
  - **Steps:**
    1. For `shop-api`, estimate: peak requests per second, daily log volume in GB, and database size after 3 years.
    2. Write each assumption as a variable at the top of a script and print the results.
    3. Sanity check each against a real number you can measure (for example your own sample app's log size per request).
  - **Done when:** a script prints the three results, each assumption is labelled, and you note one sanity check per number.

- [ ] **T3 — Decision matrix with sensitivity** *(≈45 min, intermediate)*
  - **Goal:** compare real options and test the robustness of the answer.
  - **Steps:**
    1. Pick a decision (managed vs self-hosted PostgreSQL, ECS vs Kubernetes, or Redis vs in-process cache).
    2. Build the matrix as in `matrix.py` with at least 5 criteria and 3 options, with weights agreed by you and (ideally) a colleague scoring independently.
    3. Vary each weight by plus or minus 50% and record whether the winner changes.
  - **Done when:** the script prints the base ranking and a table of weight changes that flip the winner (or states that none do).

- [ ] **T4 — Write a real ADR** *(≈60 min, advanced)*
  - **Goal:** produce a reviewable ADR.
  - **Steps:**
    1. Choose a decision from the reference architecture (for example "Use a managed PostgreSQL with Multi-AZ rather than running it on Kubernetes").

2. Fill the template: context with numbers, drivers, three options, decision, consequences (positive and negative), revisit trigger.
3. Open a pull request containing only the ADR.

- **Done when:** the ADR is under two pages, has at least one honest negative consequence per chosen option, and includes a revisit trigger.

- [ ] **T5 — Run a design review** *(≈60 min, advanced)*
  - **Goal:** find flaws before production does.
  - **Steps:**
    1. Ask a colleague (or play both roles) to review the ADR from T4 using the checklist in the capstone.
    2. Record at least five questions and the answers or follow-up actions as PR comments.
    3. Update the ADR with the changes agreed.
  - **Done when:** the PR has at least five review comments, each resolved with an answer or a change, and the final ADR reflects them.

- [ ] **T6 — Prove it: supersede a decision** *(≈45 min, advanced)*
  - **Goal:** show the full life cycle.
  - **Steps:**
    1. Without the guide, write ADR-0001 for a decision with explicit revisit triggers.
    2. Invent (and label as hypothetical) a change in conditions that hits the trigger, then write ADR-0002 that supersedes it, updating the status in ADR-0001.
    3. Explain in three sentences what the history teaches a newcomer.
  - **Done when:** two linked ADRs exist with correct statuses, and the explanation mentions both the original reasoning and the changed assumption.

10 Self-check questions

1. What are the core sections of an ADR and why is it immutable once accepted?
2. When should you write an ADR and when not?
3. What does a sensitivity check on a decision matrix tell you?
4. Distinguish one-way-door and two-way-door decisions.
5. Why do estimates use round numbers and stated assumptions?

ANSWERS

1. Title, status, context, decision, consequences (often also drivers and options). It is immutable so the history of reasoning stays intact; changes are made by writing a new ADR that supersedes it.
2. Write one when the decision is costly to reverse, has real alternatives, or affects several teams. Do not for easily reversible, local choices.
3. Whether the winner depends on contested weights. If a small change flips the result, the options are close and other factors (risk, reversibility) should decide.
4. One-way-door decisions are hard or impossible to reverse and deserve deep review; two-way-door decisions are cheap to undo and should be made quickly.
5. The goal is the order of magnitude to rule options in or out; explicit assumptions make the estimate reviewable and easy to correct.

11 Further reading

- Michael Nygard, *Documenting Architecture Decisions* (2011 blog post), and adr.github.io for templates and tools.
- MADR (Markdown Any Decision Records) project documentation.
- *Fundamentals of Software Architecture* by Mark Richards and Neal Ford; *Software Architecture: The Hard Parts* by Ford et al.
- *Thinking in Systems* by Donella Meadows (trade-offs and feedback).
- Public design-proposal processes: Kubernetes Enhancement Proposals (KEPs), Rust RFCs, Python PEPs.

LEVEL 19: ARCHITECTURE

★ Design the Reference Production Architecture

Level 19 capstone ~14 hours

1 Goal and outcome

Design, on paper and in code-checked documents, a complete production architecture for `shop-api` and defend it like a senior engineer:

CDN → Load Balancer → Kubernetes → API / Worker / Web → Redis → Message Queue → PostgreSQL / S3, observed with **Prometheus, Grafana, Loki, OpenTelemetry and Sentry**.

This is a *design* capstone: you will not necessarily deploy it (Levels 6-10 and 17 do that). Instead you will produce a design package in a Git repository that a reviewer could use to build and operate the system:

Deliverable	File	Topics it exercises
Requirements, assumptions, estimates	`docs/requirements.md`, `model/model.py`	Scalability, Performance, ADRs and estimating
Architecture diagrams (Mermaid)	`docs/architecture.md`	Reference architecture walkthrough
Availability and capacity model	`model/model.py`, `docs/availability.md`	Availability, Scalability
Threat model (STRIDE) and blast-radius plan	`docs/threat-model.md`	Security architecture
Cost estimate with unit economics	`docs/cost.md`	Cost, Operational complexity
Release and environment strategy	`docs/deployment.md`	Deployment strategy
Observability design and SLOs	`docs/slo.md`	Monitoring
Disaster recovery plan	`docs/dr-plan.md`	Disaster recovery
Written ADRs (at least 6)	`docs/adr/*.md`	Trade-offs and ADRs, all topics
Design review checklist and review record	`docs/review-checklist.md`, `docs/review-record.md`	Trade-offs and ADRs

Outcome: a repository whose `python3 check_design.py .` prints `design check passed`, plus a 20-minute design-review walkthrough you can deliver without notes.

2 Architecture

The target architecture (request path):

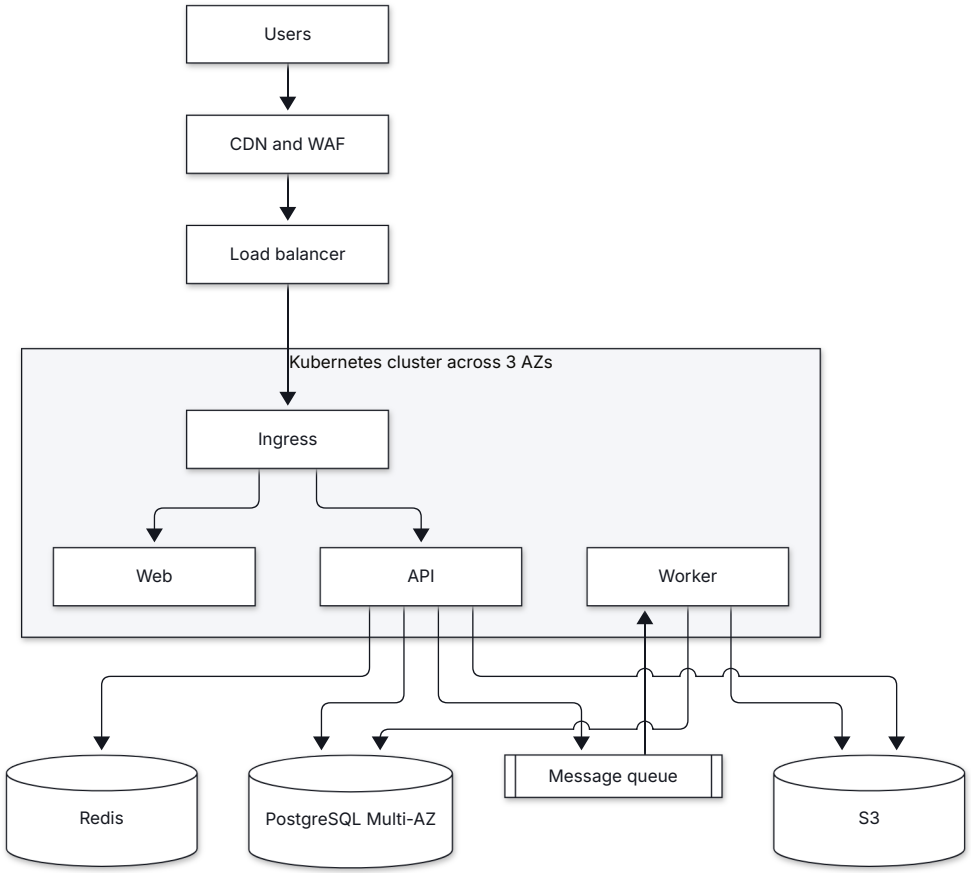


Figure 19-1

The observability plane:

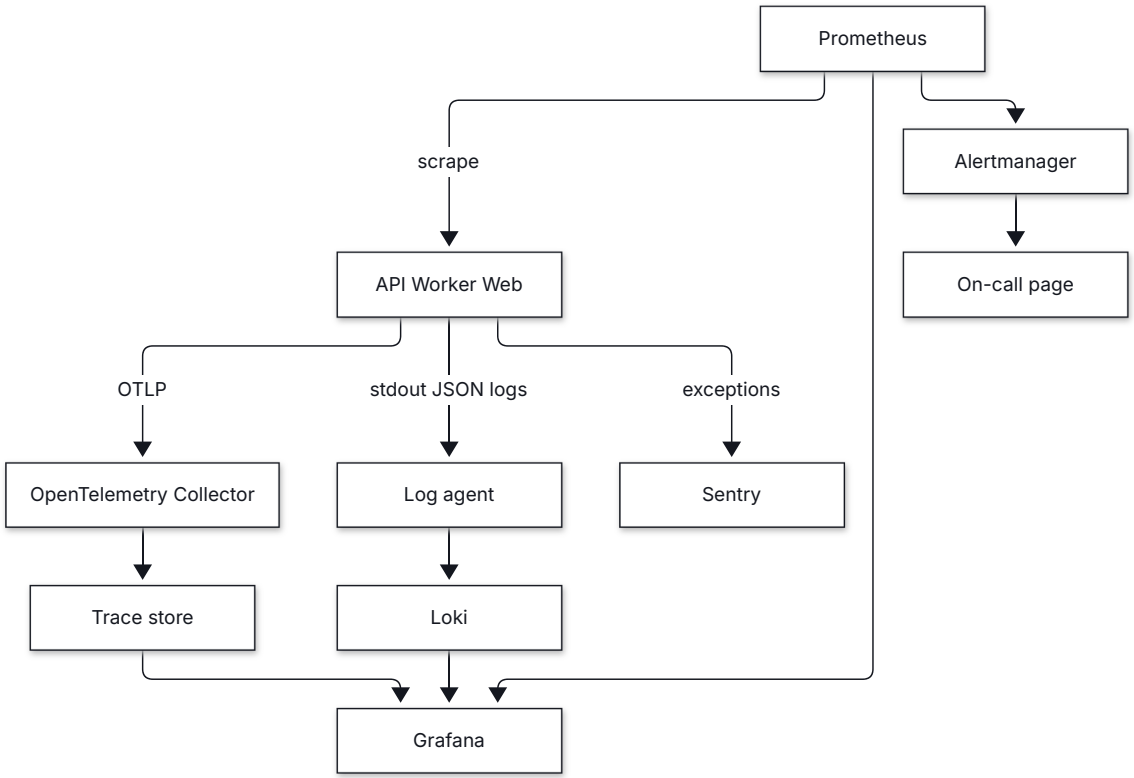


Figure 19-2

Your design package adds what these diagrams cannot show: numbers, threats, decisions, costs and recovery procedures.

3 Prerequisites and cost

- Completed topics L19.1-L19.11 (this level). Helpful: Levels 6, 8, 10, 13, 14 and 15.
- Tools: Git, Python 3.9+, a text editor with Markdown preview (GitHub renders Mermaid natively). Optional: Docker for the walkthrough Compose stack from topic 10.
- **Cost: free.** All cost figures in your design are *estimates using clearly labelled example prices*; do not create cloud resources for this capstone.
- Time budget: about 14 hours across the milestones below; the review milestone is best done with a colleague.

Build it step by step

4

Milestone 0 — Scaffold the design repository

```bash
mkdir -p reference-architecture/{docs/adr,model} && cd reference-architecture
git init
touch docs/{requirements,architecture,availability,threat-model,cost,deployment,slo,dr-
  plan,review-checklist,review-record}.md
```

Save the verifier as `check_design.py` (it checks that every deliverable exists and is well formed):

```python
import pathlib, re, sys

root = pathlib.Path(sys.argv[1] if len(sys.argv) > 1 else ".")
problems = []

def need(path):
    p = root / path
    if not p.exists():
        problems.append(f"missing {path}")
        return ""
    return p.read_text(encoding="utf-8")

arch = need("docs/architecture.md")
if len(re.findall(r"```mermaid", arch)) < 2:
    problems.append("docs/architecture.md needs at least 2 mermaid diagrams (request path
  + observability)")
for comp in ["CDN", "Load balancer", "Kubernetes", "Redis", "PostgreSQL", "S3",
  "Prometheus", "Grafana", "Loki", "OpenTelemetry", "Sentry"]:
    if comp.lower() not in arch.lower():
        problems.append(f"architecture.md never mentions {comp}")

adrs = sorted((root / "docs/adr").glob("0*.md")) if (root / "docs/adr").exists() else []
real = [a for a in adrs if not a.name.startswith("0000")]
if len(real) < 6:
    problems.append(f"need at least 6 ADRs besides the template, found {len(real)}")
for a in real:
    t = a.read_text(encoding="utf-8")
    for sec in ["Status", "Context", "Options considered", "Decision", "Consequences"]:
        if sec.lower() not in t.lower():
            problems.append(f"{a.name} lacks section {sec}")
    if "supersed" not in t.lower() and "revisit" not in t.lower():
        problems.append(f"{a.name} has no revisit trigger or supersedes note")

for f, words in {"docs/cost.md": ["per order", "assum"], "docs/dr-plan.md": ["RPO", "RTO",
  "fail-back"],
```

```
                "docs/threat-model.md": ["Spoofing", "Tampering", "Repudiation",
  "Information disclosure", "Denial of service", "Elevation of privilege"],
                "docs/slo.md": ["burn", "SLO"], "docs/review-checklist.md": ["- [
  ]"]}.items():
    t = need(f)
    for w in words:
        if t and w.lower() not in t.lower():
            problems.append(f"{f} lacks '{w}'")

print("\n".join(problems) if problems else "design check passed")
print(f"{len(problems)} problems")
sys.exit(1 if problems else 0)
```

BASH

```
python3 check_design.py .      # expect a list of problems: that is your to-do list
```

Milestone 1 — Requirements and estimates (topics 1, 4, 11)

Write `docs/requirements.md` with: functional scope (browse, cart, checkout, order history, admin export), users (2,000,000 daily active users as the working assumption), **non-functional requirements** (checkout availability SLO 99.9%, p99 checkout latency under 500 ms, RPO 5 minutes and RTO 60 minutes for orders, 7-year audit retention), constraints (team of 6, budget ceiling you choose, single cloud provider, regions `us-east-1` primary and `eu-west-1` for DR), and a list of **assumptions you will validate**.

Save the single-file model as `model/model.py` (example assumptions and example USD prices; every number is yours to change):

PYTHON

```
import math

A = dict(dau=2_000_000, req_per_user=40, peak_factor=5, rps_per_pod=150, target_util=0.6,
         pods_per_node=8, zones=3, conns_per_pod=3, hit_ratio=0.9, read_share=0.8,
         orders_per_month=400_000)

# 1. capacity (Scalability, Performance)
avg = A["dau"] * A["req_per_user"] / 86_400
peak = avg * A["peak_factor"]
pods = math.ceil(peak / (A["rps_per_pod"] * A["target_util"]))
pods_az_loss = math.ceil(pods * A["zones"] / (A["zones"] - 1))
nodes = math.ceil(pods_az_loss / A["pods_per_node"])
print(f"[capacity] peak {peak:,.0f} rps → {pods} pods ({pods_az_loss} with one-zone
  headroom) → {nodes} nodes")
print(f"[capacity] DB connections {pods_az_loss*A['conns_per_pod']} (needs pooler if >
  100); DB reads/s after cache {peak*A['read_share']*(1-A['hit_ratio']):,.0f}")

# 2. availability (Availability): critical-path components in series
chain = {"CDN": .9999, "Load balancer": .9999, "Kubernetes API pods": .9995,
```

```
          "Redis (optional, bypassed)": 1.0, "PostgreSQL Multi-AZ": .9995, "Payment
  provider (async)": 1.0}
avail = math.prod(chain.values())
TARGET = 0.999
print(f"[availability] composite {avail:.4%} (expected downtime {(1-avail)*30*24*60:.0f}
  min/30d) vs target {TARGET:.2%}: {'meets' if avail ⩾ TARGET else 'MISSES'} target")

# 3. cost (Cost): EXAMPLE USD prices, replace with current ones
H = 730
cost = {"nodes": nodes*0.17*H, "control plane": 0.10*H, "load balancer":
  (0.0225+5*0.008)*H,
        "postgres multi-az": 0.34*H + 200*0.115*2, "redis": 2*0.07*H, "s3": 2000*0.023,
        "cdn+egress": 3000*0.8*0.085 + 3000*0.2*0.09, "nat": 3*0.045*H + 1500*0.045,
        "observability storage+saas": 120}
total = sum(cost.values())
print(f"[cost] example total ${total:,.0f}/month = ${total/A['orders_per_month']:.4f} per
  order; top item: {max(cost, key=cost.get)}")

# 4. DR (Disaster Recovery): required vs designed, minutes
dr = [("PostgreSQL", 5, 60, 0.1, 15), ("S3 uploads", 60, 240, 15, 60), ("Queue", 5, 60, 0,
  20), ("Cluster", None, 120, None, 90)]
fails = [n for n, rpo, rto, mr, mt in dr if (rpo is not None and mr > rpo) or mt > rto]
print(f"[dr] {len(dr)} components checked, failing: {fails or 'none'}")
```

BASH

```
python3 model/model.py
```

Expected output with these assumptions:

```
[capacity] peak 4,630 rps → 52 pods (78 with one-zone headroom) → 10 nodes
[capacity] DB connections 234 (needs pooler if > 100); DB reads/s after cache 370
[availability] composite 99.8800% (expected downtime 52 min/30d) vs target 99.90%: MISSES
  target
[cost] example total $2,346/month = $0.0059 per order; top item: nodes
[dr] 4 components checked, failing: none
```

Read it as a reviewer: the model already tells you the database needs a pooler and that the series chain **misses** the 99.9% target, so your availability design must change something (for example improve the weakest component, add redundancy, or relax the target with a reason).

Milestone 2 — Architecture diagrams (topic 10)

In `docs/architecture.md` include the two diagrams from section 2 (extended for your decisions), a component table (role, key design decisions, how it fails), and the request walk-through as a `sequenceDiagram` for `POST /orders` with an idempotency key. Annotate every arrow with a timeout and a retry policy.

Milestone 3 — Availability and scalability design (topics 1, 2)

In `docs/availability.md`: the SPOF table (component, failure mode, redundancy, detection, recovery time), the series/parallel calculation (extend `model.py` until the composite meets the target or document why not), the degradation ladder (what users see when Redis, the queue, the payment provider or the primary database is unavailable), the scaling plan (what scales on which metric: API on CPU, workers on queue depth, database via pooler then replicas), and the first three bottlenecks you expect with the metric that will show each.

Milestone 4 — Security architecture (topic 3)

In `docs/threat-model.md`: a data flow diagram with trust boundaries (internet, edge, cluster, data zone), a STRIDE table covering every flow that crosses a boundary (at least 12 threats, each with mitigation and a decision: mitigate, accept, transfer or avoid), the identity plan (SSO for humans, workload identity for pods, no static cloud keys), the network plan (private subnets, default-deny NetworkPolicies), and a blast-radius statement (accounts, namespaces, per-service database users, optional cells). The file must contain all six STRIDE category names.

Milestone 5 — Operability, build-vs-buy and deployment (topics 6, 7)

In `docs/deployment.md`: environments (dev, staging, production in separate accounts), the pipeline from commit to production (CI builds an immutable image, GitOps syncs, canary with automatic analysis), the **multi-region wave plan** (canary 1%, primary region, DR region), the expand/contract migration policy, and rollback rules with the time-to-rollback target. Add a **complexity budget**: list the innovation tokens you spend (for example Kubernetes) and a build-vs-buy table for PostgreSQL, the queue, Redis, and the observability stack (managed vs self-hosted) with a 3-year TCO sketch. State plainly when a simpler platform would be the better choice and at what scale this design becomes justified.

Milestone 6 — Monitoring and SLOs (topic 8)

In `docs/slo.md`: three user journeys with SLIs and SLOs, the burn-rate alert table (page and ticket windows) for the checkout SLO, a Prometheus rule file (validated with `promtool check rules` if available), the telemetry pipeline (OTel Collector with tail sampling, Loki labels policy, Sentry sampling) and a telemetry volume and cost estimate. Include monitoring-of-monitoring (dead man's switch, external synthetic check).

Milestone 7 — Cost (topic 5)

In `docs/cost.md`: the monthly cost model from `model.py` extended with observability, backups and DR (standby costs), every price labelled as an example with a date, the **cost per order** and a target, the top five levers (with estimated savings) and the guardrails (tags, bud-

gets, lifecycle rules, retention caps). Include at least two what-if scenarios (for example single NAT gateway; 50% Spot workers).

Milestone 8 — Disaster recovery (topic 9)

In `docs/dr-plan.md`: per-component RPO/RTO and chosen DR tier (use the table logic of topic 9), the mechanisms (Multi-AZ, PITR, cross-region replica, S3 replication and versioning, backups in a separate account with object lock, infrastructure as code), the region-loss runbook (declare, promote, recreate, verify, switch DNS), the **fail-back** procedure, communication templates, and the test schedule with the measured results you expect to record.

Milestone 9 — Write the ADRs (topic 11)

Create `docs/adr/0000-template.md` from the template below, then write at least six ADRs, one page each. Suggested decisions: (1) Kubernetes vs a managed container service, (2) managed PostgreSQL Multi-AZ vs self-hosted, (3) queue technology, (4) Redis usage and failure behaviour, (5) multi-AZ now and warm-standby DR region vs active-active, (6) OpenTelemetry Collector plus Prometheus/Loki/Grafana vs a SaaS observability vendor, (7) canary release strategy, (8) CDN caching policy.

```markdown
# ADR-0000: Short title in the form "Use X for Y"

- Status: Proposed | Accepted | Rejected | Deprecated | Superseded by ADR-NNNN
- Date: YYYY-MM-DD
- Deciders: names or teams

## Context
The problem, constraints and numbers (requirements, SLOs, team size, budget, deadline).
  Cite your estimate.

## Decision drivers
Ranked criteria (for example: operational simplicity, durability, cost, throughput).

## Options considered
1. Option A: pros, cons.
2. Option B: pros, cons.
3. Option C: pros, cons, and why it was not chosen.

## Decision
We will ... because ...

## Consequences
Positive, negative, neutral. New work this creates (runbooks, alerts, migration).
Revisit trigger: the measurable condition under which we reconsider this decision.

## Links
Related ADRs, diagrams, benchmarks.
```

Milestone 10 — Design review (topic 11)

Save this checklist as `docs/review-checklist.md`, run it with a reviewer, and record answers, actions and the verdict (approved, approved with changes, another round) in `docs/review-record.md`.

MARKDOWN

```
# Design review checklist

## Requirements and scope
- [ ] Functional scope and non-goals are written down
- [ ] SLOs, RPO and RTO are numeric and agreed with an owner
- [ ] Assumptions and estimates are explicit and have a validation plan

## Scalability and performance
- [ ] Peak load, headroom and first three bottlenecks are identified with metrics
- [ ] Every service is stateless or its state is explained
- [ ] Latency budget exists per critical path; every call has a timeout and retry policy
- [ ] Caches have TTL, invalidation and stampede protection

## Availability and failure
- [ ] Every single point of failure is listed with a mitigation or an accepted risk
- [ ] Composite availability is calculated and meets the target
- [ ] Degraded modes are defined and tested; dependencies are classified critical or
  optional

## Security
- [ ] Trust boundaries and STRIDE threats reviewed; decisions recorded
- [ ] No static cloud keys; secrets in a manager; least-privilege roles and database users
- [ ] Network default-deny; encryption in transit and at rest; audit logging enabled

## Operations and complexity
- [ ] Owner and on-call for each component; runbooks exist for each page
- [ ] Number of new technologies is within the innovation budget; build-vs-buy is
  justified
- [ ] Upgrade and patching plan for Kubernetes, PostgreSQL and the observability stack

## Delivery
- [ ] Immutable artifacts; progressive rollout with automatic gates; rollback tested
- [ ] Migrations are backward compatible (expand and contract)
- [ ] No release step changes all regions at once

## Observability
- [ ] SLO burn-rate alerts exist; every page has a runbook link
- [ ] Telemetry volume, cardinality and retention are budgeted
- [ ] Monitoring-of-monitoring exists outside the failure domain

## Disaster recovery
- [ ] Per-component RPO/RTO met by a named mechanism; backups are in a separate account
- [ ] Restore and failover have been (or are scheduled to be) tested, with times recorded
- [ ] Fail-back is documented
```

```
## Cost
- [ ] Monthly estimate with assumptions and price dates; cost per unit defined
- [ ] Tagging, budgets, retention and lifecycle guardrails exist

## Decisions
- [ ] Each significant decision has an ADR with options, consequences and a revisit
  trigger
```

5 Tasks

- [ ] **T1 — Scaffold and baseline** *(≈45 min, intermediate; Milestones 0-1; topics 1, 4, 11)*
  - **Goal:** repository, verifier and a validated capacity model.
  - **Steps:**
    1. Create the folders and `check_design.py` (Milestone 0); run it and save the problem list.
    2. Write `docs/requirements.md` and `model/model.py`; run the model.
    3. Change two assumptions (for example `peak_factor` 5 to 10, `hit_ratio` 0.9 to 0.99) and record how the pods, DB connections and DB reads change.
  - **Done when:** `python3 model/model.py` prints four sections and your requirements file lists at least eight numeric assumptions with a note on how each will be validated.

- [ ] **T2 — Diagrams and the request walk-through** *(≈90 min, intermediate; Milestone 2; topic 10)*
  - **Goal:** an accurate, annotated architecture.
  - **Steps:**
    1. Build `docs/architecture.md` with the request-path and observability diagrams and the `POST /orders` sequence diagram.
    2. Add the component table and an annotation (timeout, retry, failure behaviour) for every arrow.
    3. Check the timeouts nest (outer larger than inner).
  - **Done when:** all diagrams render on GitHub and `check_design.py` no longer reports problems about `architecture.md`.

- [ ] **T3 — Meet the availability target** *(≈90 min, advanced; Milestone 3; topics 1, 2)*
  - **Goal:** fix the model's MISSES result with design, not wishful thinking.
  - **Steps:**
    1. Reproduce the 99.88% composite. List which components are in series on the critical path.

2. Propose two changes (for example make the payment path asynchronous, a second PostgreSQL standby, remove a hard dependency) and encode each in `model.py`.
3. Write the degradation ladder and the SPOF table.

- **Done when:** `model.py` prints `meets target` (or your documented argument for a lower target), and `docs/availability.md` contains the SPOF table and ladder.

- [ ] **T4 — Threat model and blast radius** *(≈2 hours, advanced; Milestone 4; topic 3)*
  - **Goal:** at least 12 STRIDE threats with decisions.
  - **Steps:**
    1. Draw the data flow diagram with trust boundaries.
    2. For every boundary-crossing flow work through all six STRIDE categories and record threat, mitigation and decision.
    3. Quantify blast radius for two designs (single shared stack vs cells) as in topic 3.
  - **Done when:** `docs/threat-model.md` passes the verifier, has 12 or more threats, and a blast-radius comparison table.

- [ ] **T5 — SLOs, alerts and telemetry budget** *(≈2 hours, advanced; Milestone 6; topic 8)*
  - **Goal:** an alerting design a team could run.
  - **Steps:**
    1. Define SLIs and SLOs for browse, checkout and admin export.
    2. Write and lint the fast-burn and slow-burn Prometheus rules for checkout (`promtool check rules` if you have it, otherwise YAML-parse it).
    3. Estimate the telemetry volume (metrics series, log GB per day, trace GB per day with sampling) with a script.
  - **Done when:** `docs/slo.md` has the burn-rate table, the rule file passes its check, and the volume script output is included.

- [ ] **T6 — Cost and DR plan** *(≈3 hours, advanced; Milestones 5, 7, 8; topics 5, 6, 7, 9)*
  - **Goal:** the money and recovery sections.
  - **Steps:**
    1. Extend the cost model with observability, backups and the DR region standby; label every price as an example with a date; compute cost per order and two what-if scenarios.
    2. Write the complexity budget and build-vs-buy table, and the wave plan for multi-region rollout.

3. Write the DR plan: per-component RPO/RTO check (a failing line is fine if you explain the accepted risk), region-loss runbook, fail-back and test schedule.

- **Done when:** `docs/cost.md`, `docs/deployment.md` and `docs/dr-plan.md` pass the verifier, and your DR table was produced by running code.

- [ ] **T7 — ADRs and design review** *(≈3 hours, advanced; Milestones 9-10; topic 11)*
  - **Goal:** six defensible ADRs and a recorded review.
  - **Steps:**
    1. Write six ADRs from the suggested list, each with a matrix or estimate behind it and a revisit trigger.
    2. Run the review checklist with a colleague (or in two roles with a day between).
    3. Update the ADRs and record the verdict and open actions.
  - **Done when:** `python3 check_design.py .` prints `design check passed` and `docs/review-record.md` lists at least eight review questions with outcomes.

- [ ] **T8 — Prove it: the 20-minute design defence** *(≈2 hours, advanced; all milestones; all 11 topics)*
  - **Goal:** present and defend the design without notes.
  - **Steps:**
    1. Without looking at the guide, redraw the architecture, state the three biggest risks, and the three trade-offs you made on purpose.
    2. Answer these live: "Why Kubernetes?", "What if the primary database fails at 14:00?", "What if the cloud account is compromised?", "What is the cost per order and what would you cut first?", "What is the first bottleneck at 10x?"
    3. Record yourself or write the answers; compare with your documents and fix any inconsistency you find.
  - **Done when:** each of the five questions has a specific, numeric answer that matches the documents, and a listener can name your top three risks afterwards.

6 Verification checklist

- [ ] `python3 check_design.py .` prints `design check passed`
- [ ] `python3 model/model.py` runs and its availability line says `meets target` (or the docs justify the exception)
- [ ] Every Mermaid block renders on GitHub (preview the files)
- [ ] The threat model contains all six STRIDE categories and at least 12 threats with decisions

- [ ] At least six ADRs, each with options considered, consequences and a revisit trigger
- [ ] Cost file labels all prices as examples, states assumptions and the cost per order
- [ ] DR plan has per-component RPO/RTO, region-loss runbook, fail-back and a test schedule
- [ ] SLO file has numeric SLOs and a burn-rate alert table
- [ ] No section contradicts another (for example the cost file's node count matches the model)
- [ ] The design review record exists with a verdict and actions

7 Teardown

There is nothing to tear down in the cloud. If you ran the optional Compose stack from topic 10:

```bash
docker compose down -v
```

Keep the repository: it is a portfolio piece. Remove any real credentials (there should be none) before publishing, and make sure you did not paste real account identifiers into the documents.

8 Stretch goals and what to learn next

- Convert the design into Terraform modules and a Helm chart and deploy a minimal version (Levels 6-9), then compare real costs with your estimate.
- Add a second design option (for example a PaaS-based architecture) and write the ADR comparing both with a decision matrix and sensitivity analysis.
- Add a cell-based variant and quantify blast radius and cost.
- Run a game day for one failure (drop Redis, fail the primary database in a test environment) and record measured RTO/RPO against your plan.
- Add a `CODEOWNERS` file and a pull-request check that runs `check_design.py` so the design stays valid as it evolves.
- Study cost optimisation in depth: continue to FinOps §20.1 in Level 20, then revisit your cost model with Savings Plans, Spot and rightsizing.

LAB NOTES What I ran, what I saw, what it means

LEVEL 20 · PART IX

Cost Optimization

20

Goal: Reduce cloud spend without sacrificing reliability by treating cost as an engineering metric.

RECOMMENDED TIME
2-3 weeks

TOPICS
9

PRACTICE / CAPSTONE
Audit a sample AWS/Kubernetes environment: tag costs, find waste, rightsize, add lifecycle policies and propose a savings plan with numbers.

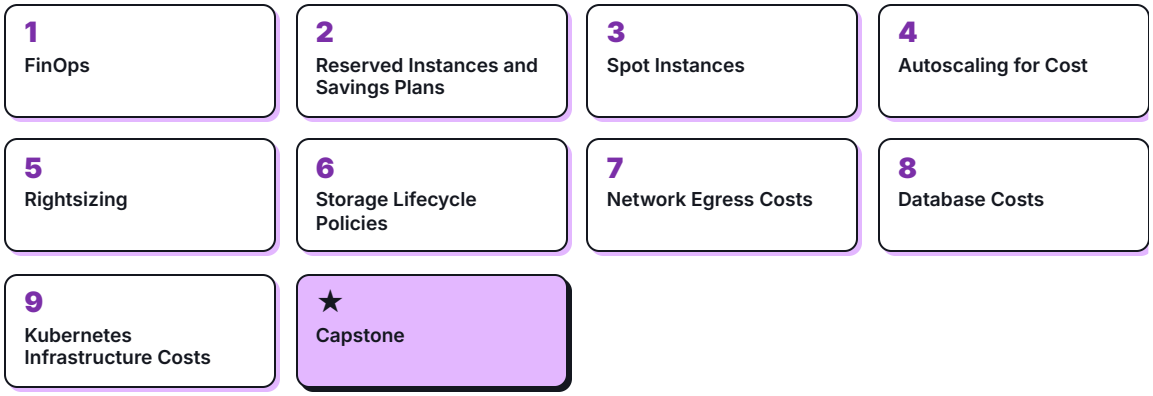
Topics

#	Topic	What you will learn
1	FinOps **p.3705**	inform/optimize/operate, tagging, showback/chargeback, unit economics, Cost Explorer
2	Reserved Instances and Savings Plans **p.3717**	commitments, coverage vs utilization, break-even
3	Spot Instances **p.3729**	interruption handling, diversification, Spot on K8s/ASG, suitable workloads
4	Autoscaling for Cost **p.3743**	scale-in, scheduled scaling, scale-to-zero, off-hours shutdown
5	Rightsizing **p.3755**	utilization analysis, Compute Optimizer, VPA recommendations, risks
6	Storage Lifecycle Policies **p.3767**	S3 tiers, Intelligent-Tiering, snapshot cleanup, log retention
7	Network Egress Costs **p.3779**	inter-AZ/region/internet transfer, NAT cost, VPC endpoints, CDN offload
8	Database Costs **p.3791**	instance sizing, storage/IOPS, replicas, Aurora Serverless, caching to reduce load
9	Kubernetes Infrastructure Costs **p.3803**	bin packing, requests inflation, Kubecost/OpenCost, node pools, Karpenter
★	Level capstone project **p.3819**	Audit a sample AWS/Kubernetes environment: tag costs, find waste, rightsize, add lifecycle policies and propose a savings plan with numbers.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



20.1 FinOps

Topic 1 of 9 | ~4 hours | Difficulty: Intermediate

1 What it is

FinOps (Cloud Financial Operations) is the practice of making cloud cost a shared engineering metric instead of a surprise on an invoice. Cloud spend is variable: every autoscaling group, every NAT gateway and every forgotten snapshot changes the bill, and the people who make those decisions (engineers) are not the people who receive the invoice (finance). FinOps closes that gap with data, shared ownership and a repeating cycle.

The FinOps Foundation (a project of the Linux Foundation) describes a lifecycle with three phases that you loop through continuously:

Phase	Question it answers	Typical activities
Inform	Who is spending what, and why?	tagging, cost allocation, dashboards, showback, anomaly alerts
Optimize	Where can we spend less for the same outcome?	rightsizing, commitments, Spot, storage lifecycle, deleting waste
Operate	How do we keep it that way?	budgets, policies as code, reviews, unit-cost targets, engineering KPIs

You cannot optimize what you cannot attribute, so this level starts with **Inform**: tag everything, allocate it to a team and a service, and express cost per unit of business value (per request, per customer, per build).

2 Why it matters in DevOps

- **Cost is an availability-grade signal.** A runaway job, a crash-looping pod that pulls images through a NAT gateway or a public bucket serving terabytes shows up on the bill before it shows up in a dashboard.
- **Untagged spend cannot be owned.** If 40 percent of the bill is "unallocated", nobody can be asked to reduce it.

- **Absolute dollars mislead.** A bill that doubled while traffic tripled is an improvement. **Unit economics** (cost per 1,000 requests, per active customer, per deployment) tells you whether the system is getting more or less efficient.
- **Engineers control the levers.** Instance size, replica count, log retention, data transfer paths and storage class are all code. FinOps works when cost feedback reaches the pull request, not a quarterly review.
- Related levels: Cost §19.5 covers cost as an architecture trade-off, and Infrastructure as Code §0.3 is where tags are enforced.

3 Real-world system example

AWS billing and cost tooling. AWS publishes the building blocks most teams use: **Cost Explorer** for interactive analysis, **AWS Budgets** for threshold alerts, **Cost Anomaly Detection** for unusual spikes, **Cost Categories** to group spend into business units, and the **Cost and Usage Report** (now offered through **Data Exports**) for line-item data you can query with Athena. Cost allocation works through **cost allocation tags**: a tag only appears in billing data after you activate it in the Billing console, and it is not retroactive. AWS Organizations adds **tag policies** to standardize tag keys and values across accounts.

The FinOps Foundation framework is the public reference for the Inform, Optimize, Operate lifecycle and for terms such as showback and chargeback. **OpenCost**, a CNCF project, applies the same allocation ideas inside Kubernetes (see Kubernetes Infrastructure Costs §20.9). Large organizations commonly run a central FinOps team that publishes dashboards and benchmarks while product teams own the actions, but the specific structure varies by company.

4 Diagram

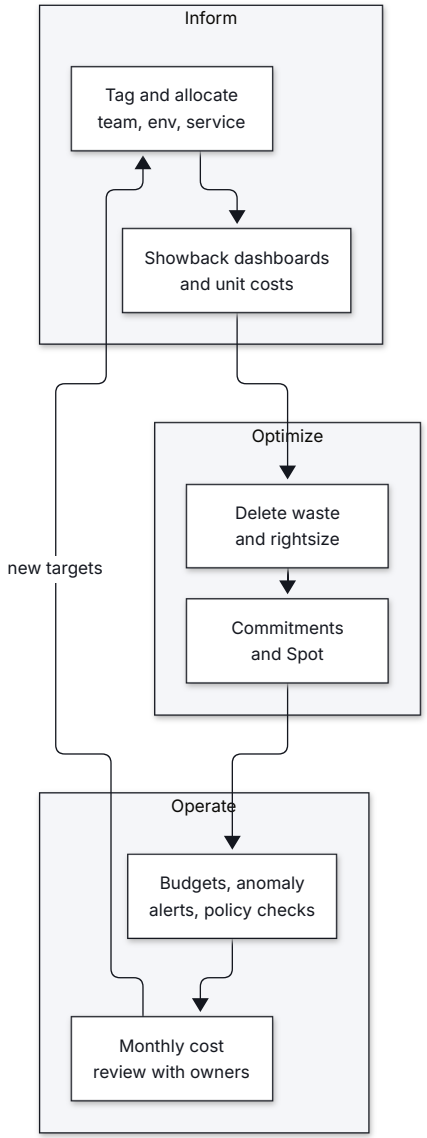


Figure 20.1-1

From a resource to a number a team can act on:

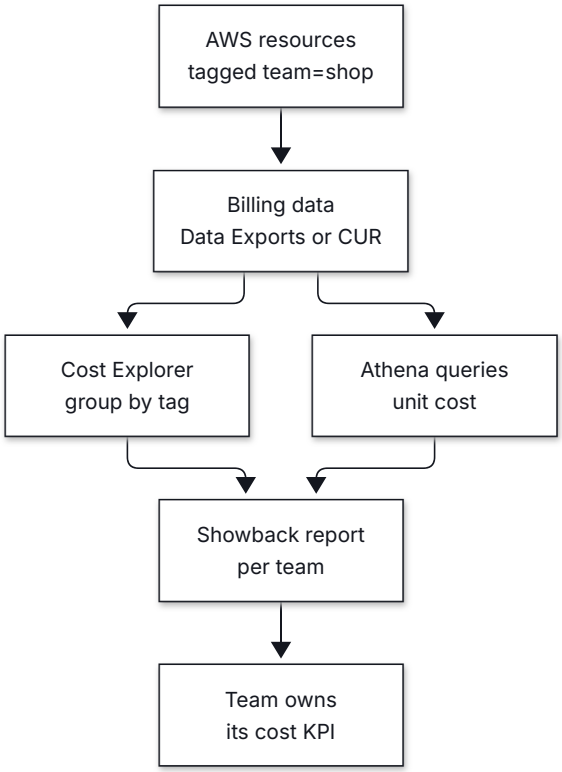


Figure 20.1-2

5 How it works

Tagging

A good tag schema is small, mandatory and enforced. A starting set for `shop-api`:

Tag key	Example value	Purpose
`team`	`shop`	who owns it and who gets the showback
`env`	`prod`, `staging`, `dev`	separate production from waste candidates
`service`	`shop-api`	roll up to a product
`cost-center`	`cc-1042`	finance mapping

Not everything can be tagged (data transfer, some support charges, shared services), so allocation also uses account structure, **Cost Categories** and, for Kubernetes, namespace or label based allocation. Aim for 80-90 percent allocated; chasing 100 percent is a poor use of time.

Showback versus chargeback

Model	What happens	When to use
Showback	Teams see their costs; no money moves	start here; builds awareness without politics
Chargeback	Costs are billed to team budgets	mature organizations with reliable allocation and a shared-cost policy

Shared costs (networking, logging, the Kubernetes control plane) need an explicit rule: split evenly, proportionally to usage, or hold centrally. Decide and document it before arguments start.

Unit economics

Pick one or two denominators that reflect value: requests served, active customers, orders, builds, GB processed. Then track `total cost / units` over time. A falling unit cost with a rising bill is healthy growth.

Useful AWS CLI calls

Command	Use
`aws ce get-cost-and-usage`	cost by service, tag or account over a period
`aws ce get-tags` / `aws ce list-cost-allocation-tags`	which tags exist and which are active
`aws budgets create-budget`	threshold alerts
`aws ce get-anomalies`	list detected anomalies

6 Step-by-step guide

Prerequisites: Python 3 for the offline steps; `opa` (Open Policy Agent) or `conftest` for the tag policy; an AWS sandbox account for the optional Cost Explorer steps. Cost Explorer is free to use in the console; each paid API request to Cost Explorer is billed at a small per-request price, so keep CLI calls few. Prices and outputs are illustrative.

1. **Compute unit economics offline.** Save as `unit_cost.py`. All numbers are **EXAMPLES**; re-

place them with your own.

```python
# unit_cost.py - unit economics from a monthly bill. All numbers are EXAMPLES.
monthly_cost = {"compute": 6200.0, "database": 2900.0, "network": 1100.0, "storage":
  450.0, "observability": 750.0}
requests_per_month = 180_000_000      # from your load balancer metrics
active_customers = 42_000             # from your product analytics

total = sum(monthly_cost.values())
print(f"total monthly cost        : ${total:,.2f}")
print(f"cost per 1M requests      : ${total / (requests_per_month / 1e6):,.2f}")
print(f"cost per active customer  : ${total / active_customers:,.4f}")
for name, cost in sorted(monthly_cost.items(), key=lambda kv: -kv[1]):
    print(f"  {name:<14} ${cost:>9,.2f}  {cost / total:>6.1%}")
```

```bash
python3 unit_cost.py
```

Expected output:

```
total monthly cost        : $11,400.00
cost per 1M requests      : $63.33
cost per active customer  : $0.2714
  compute        $ 6,200.00   54.4%
  database       $ 2,900.00   25.4%
  network        $ 1,100.00    9.6%
  observability  $   750.00    6.6%
  storage        $   450.00    3.9%
```

2. **Define the tag schema as code.** In Terraform, set tags once on the provider so every resource inherits them:

```hcl
provider "aws" {
  region = "us-east-1"
  default_tags {
    tags = {
      team        = "shop"
      env         = "prod"
      service     = "shop-api"
      "cost-center" = "cc-1042"
    }
  }
}
```

Resources created by this provider receive the tags (visible in `tags_all` in the plan).

3. **Enforce tags in CI with a policy.** This Rego policy reads `terraform show -json` output and fails when a taggable resource is created without the required keys. Save as `tags.rego`:

```rego
package main

import rego.v1

required_tags := {"team", "env", "service", "cost-center"}
taggable := {"aws_instance", "aws_s3_bucket", "aws_db_instance", "aws_ebs_volume",
  "aws_lb"}

deny contains msg if {
  some rc in input.resource_changes
  rc.type in taggable
  "create" in rc.change.actions
  tags := object.get(rc.change.after, "tags_all", object.get(rc.change.after, "tags",
  {}))
  missing := required_tags - {k | some k, _ in tags}
  count(missing) > 0
  msg := sprintf("%s is missing required cost tags: %v", [rc.address, sort(missing)])
}
```

Create a tiny fake plan to test it (`plan.json`):

```json
{
  "resource_changes": [
    {"address": "aws_instance.api", "type": "aws_instance",
     "change": {"actions": ["create"], "after": {"instance_type": "m6i.large",
       "tags": {"team": "shop", "env": "prod", "service": "shop-api", "cost-center":
  "cc-1042"}}}},
    {"address": "aws_s3_bucket.assets", "type": "aws_s3_bucket",
     "change": {"actions": ["create"], "after": {"bucket": "shop-assets-example",
       "tags": {"team": "shop"}}}}
  ]
}
```

```bash
opa eval -f pretty -d tags.rego -i plan.json 'data.main.deny'
# or, with Conftest:  conftest test --policy . plan.json
```

Expected output:

```
[
  "aws_s3_bucket.assets is missing required cost tags: [\"cost-center\", \"env\",
  \"service\"]"
]
```

In real use, run `terraform plan -out=tfplan && terraform show -json tfplan > plan.json` in the pipeline (see Policy as Code §18.7).

4. **Activate cost allocation tags (AWS sandbox).** In the Billing console open *Cost allocation tags*, select `team`, `env`, `service` and `cost-center`, and choose *Activate*. Data appears for usage after activation (allow up to about a day); earlier usage is not retroactively tagged.
5. **Query cost by tag.** Pick a past month you have data for:

```bash
aws ce get-cost-and-usage \
  --time-period Start=2026-09-01,End=2026-10-01 \
  --granularity MONTHLY \
  --metrics UnblendedCost \
  --group-by Type=TAG,Key=team
```

Output is JSON with one `Groups` entry per `team$<value>`, including an empty value (`team$`) for untagged spend. That untagged line is your allocation gap.

6. **Add a budget with an alert.**

```bash
cat > budget.json <<'EOF'
{
  "BudgetName": "shop-monthly",
  "BudgetLimit": {"Amount": "500", "Unit": "USD"},
  "TimeUnit": "MONTHLY",
  "BudgetType": "COST"
}
EOF
cat > notifications.json <<'EOF'
[{
  "Notification": {"NotificationType": "ACTUAL", "ComparisonOperator": "GREATER_THAN",
                   "Threshold": 80, "ThresholdType": "PERCENTAGE"},
  "Subscribers": [{"SubscriptionType": "EMAIL", "Address": "finops@example.com"}]
}]
EOF
aws budgets create-budget --account-id <YOUR_ACCOUNT_ID> \
  --budget file://budget.json --notifications-with-subscribers
  file://notifications.json
```

Expect no output on success; confirm in the Budgets console. Delete the budget later with `aws budgets delete-budget`.

7 Real-world scenario

(Illustrative.) The finance team asks why the AWS bill rose 38 percent in a quarter. Cost Explorer grouped by service shows EC2-Other and data transfer climbing, but grouped by `team` the largest slice is blank: 31 percent of spend is untagged. Following the untagged resources shows a data-processing job started by a contractor, running in a public subnet through a NAT gateway, copying log archives between Regions every night. The team tags it, moves the job into the right account, adds an S3 gateway endpoint (see Network Egress Costs §20.7) and sets a budget alert at 80 percent.

The lesson is not the saving, which is modest. It is that the investigation took two days because nothing was owned. After the tag policy, the next anomaly alert named the owning team in the first message.

8 Common mistakes and best practices

Don't

- Don't launch a cost-cutting program before you can attribute cost; you will optimize the loudest team, not the biggest cost.
- Don't create a dozen tag keys with free-text values; `env=Prod`, `env=production` and `env=prd` split your report.
- Don't forget to activate tags in billing; an untouched tag produces no data.
- Don't make cost a finance-only metric that engineers see once a quarter.
- Don't chase the cheapest option blindly; an outage costs more than any instance discount.

Do

- Do enforce tags in IaC and CI, and use tag policies in AWS Organizations.
- Do start with showback, then consider chargeback when allocation is trusted.
- Do track unit cost, not just totals.
- Do alert on anomalies and review spend monthly with the owners present.
- Do keep a short backlog of savings ideas with an estimated value and an owner.

9 Tasks

- [ ] **T1 — Compute unit economics** *(≈20 min, beginner)*
  - **Goal:** turn a bill into a unit cost.
  - **Steps:**
    1. Run `unit_cost.py` with the example numbers.
    2. Replace the dictionary with five categories from a bill you know (a past project is fine).
    3. Add a cost per deployment metric for CI minutes.
  - **Done when:** the script prints cost per 1M requests and a percentage breakdown, and you can explain what a good trend looks like.

- [ ] **T2 — Define a tag schema** *(≈30 min, beginner)*
  - **Goal:** a four-tag schema with allowed values.
  - **Steps:**
    1. Choose keys and allowed values for `env` (`prod`, `staging`, `dev`).
    2. Add `default_tags` to a Terraform AWS provider block.
    3. Run `terraform plan` and find the tags in `tags_all`.
  - **Done when:** the plan output shows your four tags on a new resource.

- [ ] **T3 — Enforce tags with policy** *(≈40 min, intermediate)*
  - **Goal:** a CI check that rejects untagged resources.
  - **Steps:**
    1. Save `tags.rego` and `plan.json`.
    2. Run `opa eval` (or `conftest test`) and read the failure.
    3. Fix the bucket tags in `plan.json` and re-run.
  - **Done when:** the policy returns an empty list for the fixed plan and one message for the broken plan.

- [ ] **T4 — Find the allocation gap** *(≈45 min, intermediate; AWS sandbox)*
  - **Goal:** measure untagged spend.
  - **Steps:**
    1. Activate the `team` tag.
    2. Run the `get-cost-and-usage` command grouped by `team`.
    3. Compute untagged spend as a percentage of total.

- **Done when:** you have a percentage and a list of three untagged resources to fix. Keep API calls few because Cost Explorer requests are charged.

- [ ] **T5 — Break it: remove a required tag and watch the alert** *(≈30 min, intermediate)*
  - **Goal:** see the guardrail fire in a pull request.
  - **Steps:**
    1. Add a CI step running the policy on `plan.json`.
    2. Remove `service` from one resource and push.
    3. Restore it.
  - **Done when:** a pull request check fails with the missing-tag message and passes after the fix.

- [ ] **T6 — Prove it** *(≈60 min, intermediate)*
  - **Goal:** explain the whole loop out loud.
  - **Steps:**
    1. Without the guide, define Inform, Optimize and Operate and give one concrete action for each in `shop-api`.
    2. Explain showback versus chargeback and propose how to split a shared NAT gateway bill.
    3. Write the unit metric you would put on a team dashboard and why.
  - **Done when:** you can walk a teammate through the loop in five minutes and they can name the tag keys and the unit metric.

10 Self-check questions

1. What are the three FinOps phases and which comes first?
2. Why is a tag useless until it is activated for cost allocation?
3. When would you choose showback over chargeback?
4. Why can a rising total bill still mean an improvement?
5. Name two kinds of cost that tags cannot allocate well.

ANSWERS

1. Inform, Optimize, Operate; Inform first because you need attribution before you can optimize.
2. AWS only includes a tag in billing data after you activate it as a cost allocation tag, and it is not applied retroactively.

3. Early on, or when allocation is not yet trusted; showback builds awareness without moving money.
4. If unit cost (per request or customer) is falling while volume grows faster than spend.
5. Data transfer and shared services such as networking, logging or the Kubernetes control plane; these need account structure, Cost Categories or an allocation rule.

11 Further reading

- FinOps Foundation: the FinOps Framework and the "Cloud FinOps" book (O'Reilly).
- AWS documentation: Billing and Cost Management, Cost Explorer, Budgets, Cost Anomaly Detection, Data Exports, tag policies in AWS Organizations.
- AWS Well-Architected Framework: Cost Optimization pillar.
- OpenCost project documentation (opencost.io).

20.2 Reserved Instances and Savings Plans

Topic 2 of 9 | ~4 hours | Difficulty: Intermediate

1 What it is

On-demand cloud pricing is flexible and expensive. A **commitment** trades flexibility for a lower price: you promise to pay for a certain amount of usage for one or three years, and the provider gives a discount against on-demand rates. On AWS there are two main mechanisms:

- **Reserved Instances (RIs)** commit to a specific instance configuration (for example `m6i.large`, Linux, in one Region or Availability Zone). **Standard** RIs give the deepest discount but are the least flexible; **Convertible** RIs can be exchanged for other configurations at a smaller discount. RIs also exist for RDS, ElastiCache, OpenSearch and Redshift.
- **Savings Plans (SPs)** commit to a **dollar amount per hour** (for example $12 per hour) for one or three years. **Compute Savings Plans** apply automatically across EC2 instance families, sizes, Regions, operating systems and tenancy, and also to Fargate and Lambda. **EC2 Instance Savings Plans** are tied to one instance family in one Region for a higher discount. SageMaker has its own plan type.

Each can be paid **No Upfront**, **Partial Upfront** or **All Upfront**; more upfront money means a bigger discount.

Two numbers tell you whether commitments are healthy:

Metric	Definition	Low value means
Coverage	share of eligible usage that is covered by commitments	you pay on-demand prices for steady load
Utilization	share of your commitment that was actually used	you are paying for commitment you do not use

2 Why it matters in DevOps

- Compute is usually the largest line on the bill, and its steady baseline is the easiest thing to discount.

- A commitment is a **financial lock-in**: over-commit and you pay for idle capacity for up to three years.
- Commitments interact with engineering decisions. Moving to Graviton, containers or Lambda changes what a plan covers; a Compute Savings Plan survives these changes where a Standard RI may not.
- A good commitment strategy is boring: cover the stable floor with commitments, use Spot Instances §20.3 for interruptible work, and leave the peaks on demand.
- See also Cost §19.5 for the architecture-level view, and Rightsizing §20.5: rightsize first, then commit, or you lock in waste.

3 Real-world system example

AWS Cost Explorer's Savings Plans and RI reports. AWS documents recommendations that analyze your recent usage (7, 30 or 60 days) and propose an hourly commitment, with estimated savings and a monthly commitment figure. Cost Explorer also provides **coverage** and **utilization** reports for both RIs and Savings Plans, and **AWS Budgets** can alert when utilization or coverage drops below a threshold you set. AWS also documents that commitments are applied automatically to matching usage on the bill, in an order that favors the biggest discount, so you do not tag instances to "use" a plan.

RDS and other services have their own reserved offerings, which is why a database-heavy bill is often a separate commitment exercise (see Database Costs §20.8).

Treat any specific discount percentage as an example; it varies by service, instance family, Region, term and payment option, and AWS can change it.

4 Diagram

How a Savings Plan is applied to an hour of usage:

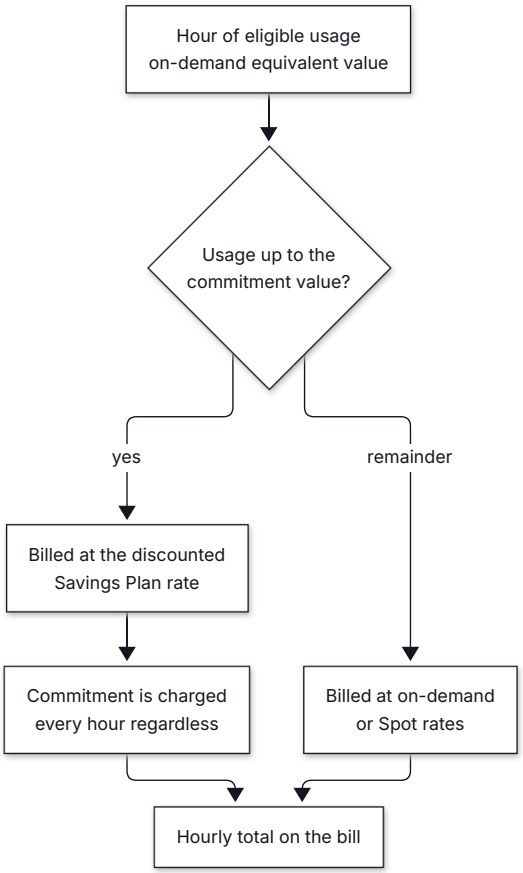


Figure 20.2-1

Choosing the commitment level against a daily usage curve:

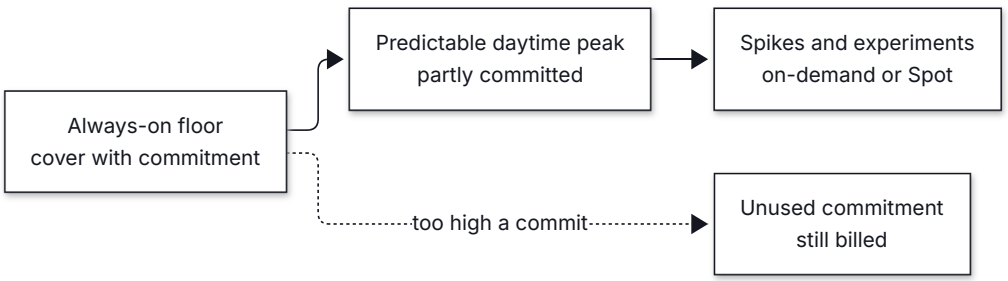


Figure 20.2-2

5 How it works

Choosing between the options

Option	Flexibility	Discount	Good for
Compute Savings Plan	any family, size, Region, OS; also Fargate and Lambda	lower than the RI types	changing fleets, containers, migration to Graviton
EC2 Instance Savings Plan	one family in one Region	higher	stable, known families
Standard RI	one configuration (and optionally one AZ for capacity reservation)	highest	very stable databases and fixed fleets
Convertible RI	exchangeable for other configurations	between	if you need RI semantics but expect change

Commitments are paid hourly even if you use nothing, so the **break-even point** is simple: if the commitment discount is `d`, the commitment is cheaper than on-demand as long as the covered resource would otherwise run more than `1 - d` of the hours. With a 34 percent discount, break-even utilization is 66 percent.

How much to commit

Cover the **steady floor**, not the average. Look at the lowest sustained hourly spend over several weeks, and commit to roughly that, then add commitments in stages (a "ladder") every quarter as the baseline grows. A coverage target of 60-80 percent of eligible compute with 95+ percent utilization is a common, conservative aim; there is no universal number.

Useful commands

Command	Use
`aws ce get-savings-plans-purchase-recommendation`	get a recommendation for a term, payment option and lookback
`aws ce get-savings-plans-coverage`	coverage over a time range
`aws ce get-savings-plans-utilization`	utilization over a time range
`aws ce get-reservation-coverage` / `get-reservation-utilization`	the same for RIs

6 Step-by-step guide

Prerequisites: Python 3. The AWS CLI steps are read-only and need Cost Explorer enabled in a sandbox account; **do not purchase a commitment while following this topic**, because commitments cannot be cancelled. All prices below are **EXAMPLES**: check current pricing on the AWS pricing pages and the Savings Plans console before deciding.

1. **Calculate the break-even.** Save as `sp_breakeven.py`:

```python
# sp_breakeven.py - Savings Plan / Reserved Instance break-even. EXAMPLE numbers only.
# Check current pricing: discounts differ by instance family, Region, term and payment option.
od_rate = 0.192          # example on-demand $/hour for one instance
discount = 0.34          # example commitment discount vs on-demand (1-year, no upfront)
hours_year = 8760

committed_rate = od_rate * (1 - discount)
print(f"committed cost per year  : ${committed_rate * hours_year:,.2f} (paid whether used or not)")
print(f"break-even utilization   : {committed_rate / od_rate:.0%} of hours must be covered")
print()
print("instance running X% of the year → on-demand vs commitment")
for util in (0.30, 0.50, 0.66, 0.80, 1.00):
    on_demand = od_rate * hours_year * util
    commit = committed_rate * hours_year
    better = "commitment" if commit < on_demand else "on-demand"
    print(f"  {util:>4.0%}  on-demand ${on_demand:>8,.2f}  commitment ${commit:>8,.2f}  → {better}")
```

```bash
python3 sp_breakeven.py
```

Expected output:

```
committed cost per year  : $1,110.07 (paid whether used or not)
break-even utilization   : 66% of hours must be covered

instance running X% of the year → on-demand vs commitment
   30%  on-demand $  504.58  commitment $1,110.07  → on-demand
   50%  on-demand $  840.96  commitment $1,110.07  → on-demand
   66%  on-demand $1,110.07  commitment $1,110.07  → commitment
   80%  on-demand $1,345.54  commitment $1,110.07  → commitment
  100%  on-demand $1,681.92  commitment $1,110.07  → commitment
```

The 66 percent row is an exact tie, shown as "commitment" only because of the comparison. The lesson: a dev instance that runs 8 hours a day (33 percent) must not be covered.

2. **Size the commitment against a usage curve.** This simulates one week of hourly usage (a 20 dollars-per-hour floor, a daytime peak, quieter weekends) and sweeps the commitment level. Save as `sp_sizing.py`:

```python
# sp_sizing.py - how big should the hourly commitment be? EXAMPLE usage and discount.
# Usage is expressed as on-demand-equivalent dollars per hour for one week (168 hours).
import math

discount = 0.30
usage = []
for hour in range(168):
    day, h = divmod(hour, 24)
    base = 20.0                                                  # always-on floor
    daytime = 12.0 * max(0.0, math.sin(math.pi * (h - 6) / 14)) if 6 ⩽ h ⩽ 20 else 0.0
    weekend = 0.6 if day ⩾ 5 else 1.0
    usage.append(base + daytime * weekend)

on_demand_total = sum(usage)
print(f"all on-demand for the week: ${on_demand_total:,.2f}  (min ${min(usage):.2f}/h, max ${max(usage):.2f}/h)")
print(f"{'commit $/h':>10} {'week cost':>11} {'saving':>8} {'coverage':>9} {'utilization':>12}")
for commit in (10, 15, 18, 20, 22, 25, 30):
    covered_od_eq = commit / (1 - discount)                      # on-demand-equivalent value the commit can cover
    cost = 0.0; used = 0.0; covered = 0.0
    for u in usage:
        c = min(u, covered_od_eq)
        covered += c
        used += c * (1 - discount)                               # commitment actually consumed
        cost += commit + (u - c)                                 # commit is always paid, remainder at on-demand
    util = used / (commit * len(usage))
    print(f"{commit:>10} {cost:>11,.2f} {1 - cost / on_demand_total:>8.1%} {covered / on_demand_total:>9.1%} {util:>12.1%}")
```

```bash
python3 sp_sizing.py
```

Expected output:

```
all on-demand for the week: $4,020.32  (min $20.00/h, max $32.00/h)
commit $/h   week cost   saving  coverage  utilization
        10    3,300.32    17.9%     59.7%       100.0%
        15    3,050.32    24.1%     86.8%        96.9%
        18    3,231.89    19.6%     94.8%        88.3%
        20    3,438.93    14.5%     98.0%        82.1%
        22    3,701.56     7.9%     99.9%        76.0%
        25    4,200.00    -4.5%    100.0%        67.0%
        30    5,040.00   -25.4%    100.0%        55.8%
```

Notice the best saving is around a 15 dollars-per-hour commitment, with high utilization, not the largest coverage. Past that point each extra commitment dollar is used less than 70 percent of the time and the saving shrinks and then turns negative. (The commit is expressed as discounted dollars, so a $15 per hour commitment covers $21.43 of on-demand-equivalent usage in this model.)

3. **Look at your real coverage and utilization (read-only).**

```bash
aws ce get-savings-plans-coverage \
  --time-period Start=2026-09-01,End=2026-10-01 --granularity MONTHLY
aws ce get-savings-plans-utilization \
  --time-period Start=2026-09-01,End=2026-10-01 --granularity MONTHLY
```

In an account with no plans, coverage is 0 percent and utilization calls report no data. That is the expected baseline.

4. **Ask AWS for a recommendation (read-only).**

```bash
aws ce get-savings-plans-purchase-recommendation \
  --savings-plans-type COMPUTE_SP \
  --term-in-years ONE_YEAR \
  --payment-option NO_UPFRONT \
  --lookback-period-in-days THIRTY_DAYS
```

Read the `HourlyCommitmentToPurchase`, `EstimatedSavingsPercentage` and `EstimatedMonthlySavingsAmount` fields in the response. An empty result means there was not enough usage to recommend anything. Compare the recommended hourly commitment with the floor you found in step 2; recommendations are based on the lookback window, so a one-off spike inflates them.

5. **Write the purchase decision down.** Record: scope (Compute SP), term (1 year), payment (No Upfront), the hourly commitment, the expected coverage and utilization, who approved it, and the review date. Commitments are financial decisions with an owner, not a console click.
6. **Set a utilization alert.** In AWS Budgets, create a *Savings Plans utilization* budget with a 95 percent threshold, so a change that strands your commitment (migrating away from EC2, say) is caught within days.

7 Real-world scenario

(Illustrative.) A team buys a 3-year Standard RI for a fleet of `m5.2xlarge` instances because the discount is the highest. Eight months later, the platform team moves to containers on Graviton-based nodes. The RIs still bill every hour, the new nodes run at on-demand prices, and utilization drops to 30 percent. The team spends weeks trying to resell and exchange what cannot be exchanged.

The fix going forward: rightsize and finish migrations first, buy a **Compute** Savings Plan for only the stable floor, ladder purchases quarterly, keep a utilization alert at 95 percent, and put the one-year term first until the roadmap is stable.

8 Common mistakes and best practices

Don't

- Don't commit before rightsizing; you lock in waste at a discount.
- Don't cover the peak. Commit to the minimum steady usage, not the average or the maximum.
- Don't buy a 3-year term for a workload with a 12-month architecture roadmap.
- Don't forget that dev and staging fleets that stop at night have low utilization and should not be covered.
- Don't buy several large commitments on one day; stagger them so you can correct course.

Do

- Do start with Compute Savings Plans for flexibility, then add Instance Savings Plans or RIs for very stable load.
- Do monitor both coverage and utilization monthly.
- Do read recommendations with the lookback window in mind.
- Do assign an owner and a review date to each commitment.
- Do treat commitments as part of the FinOps loop in FinOps §20.1.

9 Tasks

- [ ] **T1 — Compute your break-even** *(≈20 min, beginner)*
  - **Goal:** know the utilization at which a commitment pays off.
  - **Steps:**
    1. Run `sp_breakeven.py`.
    2. Change `discount` to 0.20 (a smaller discount) and 0.50.
    3. Record the break-even each time.
  - **Done when:** you can state break-even utilization for three discounts and explain the formula `1 - discount`.

- [ ] **T2 — Sweep the commitment level** *(≈30 min, intermediate)*
  - **Goal:** find the sweet spot on a usage curve.
  - **Steps:**
    1. Run `sp_sizing.py`.
    2. Change the base floor from 20 to 12 and the peak height from 12 to 25.
    3. Re-run and note how the best commitment moves.
  - **Done when:** you can explain, from the numbers, why the best commitment follows the floor and not the peak.

- [ ] **T3 — Read real coverage and utilization** *(≈30 min, intermediate; AWS sandbox)*
  - **Goal:** use the AWS CLI to read your own numbers.
  - **Steps:**
    1. Run the coverage and utilization commands for last month.
    2. Run the purchase recommendation command with a 30-day lookback.
  - **Done when:** you have the three figures (coverage, utilization, recommended hourly commitment) written down, or you can explain why they are empty.

- [ ] **T4 — Break it: strand a commitment** *(≈30 min, intermediate)*
  - **Goal:** feel the cost of over-commitment.
  - **Steps:**
    1. In `sp_sizing.py`, set the commitment to 25 and then reduce the floor to 14 (a migration removes load).
    2. Compute the weekly loss compared with all on-demand.
  - **Done when:** you have a table showing the commitment becoming a loss and a sentence on how a utilization alert would have caught it.

- [ ] **T5 — Write a commitment policy** *(≈45 min, advanced)*
  - **Goal:** a one-page policy a team could follow.
  - **Steps:**
    1. Define target coverage and utilization ranges, term preference, approval owner and review cadence.
    2. Add a rule: rightsizing review must precede any purchase.
    3. Add the budget alert from step 6.
  - **Done when:** the policy is in your repo, reviewed by a teammate, and includes numeric thresholds.
- [ ] **T6 — Prove it** *(≈45 min, advanced)*
  - **Goal:** justify a savings plan proposal.
  - **Steps:**
    1. Using your own usage data (or the simulated week), propose a commitment amount, term and payment option.
    2. Show saving, coverage, utilization and worst-case loss if load drops by 30 percent.
    3. Present it in five minutes with the break-even formula.
  - **Done when:** a listener can reproduce your numbers and name the risk. Do not purchase anything while practicing.

10 Self-check questions

1. What is the difference between coverage and utilization?
2. A commitment gives a 40 percent discount. What is the break-even utilization?
3. Why do Compute Savings Plans suit teams that are moving to containers?
4. Why should you rightsize before committing?
5. What risk comes with a 3-year Standard RI compared with a 1-year Compute Savings Plan?

ANSWERS

1. Coverage is the share of eligible usage that commitments cover; utilization is the share of the commitment you actually used.
2. 60 percent (1 minus 0.40): the resource must be used more than 60 percent of the time for the commitment to beat on-demand.
3. They apply across instance families, sizes, Regions, operating systems, and also to Fargate and Lambda, so changes in the fleet do not strand them.

4. Otherwise you commit to capacity you do not need and lock in the waste at a discount.
5. Lock-in: it ties you to a specific configuration for three years, so architecture changes can leave it unused.

11 Further reading

- AWS documentation: Savings Plans User Guide, Reserved Instances (EC2 User Guide), Cost Explorer recommendations and reports.
- AWS Well-Architected Framework: Cost Optimization pillar, "Pricing models".
- FinOps Foundation: Rate Optimization capability.

LAB NOTES What I ran, what I saw, what it means

20.3 Spot Instances

Topic 3 of 9 | ~5 hours | Difficulty: Intermediate

1 What it is

Spot Instances are spare EC2 capacity that AWS sells at a large discount compared with on-demand prices (AWS advertises discounts of up to roughly 90 percent; the real figure varies by instance type, Region and time). The trade is that AWS can **reclaim** the capacity when it needs it back. When that happens your instance receives a **two-minute interruption notice** and is then terminated or stopped.

Other clouds offer the same idea under other names: Azure Spot VMs and Google Cloud Spot VMs (formerly preemptible VMs). This topic uses the AWS terms.

The discount is not free money. It is a payment for building workloads that tolerate losing a node at any time: stateless, checkpointed, retryable or horizontally redundant.

2 Why it matters in DevOps

- Compute is typically the biggest cost line, and many workloads (CI runners, batch jobs, stateless web tiers, data processing, Kubernetes worker nodes) can run on Spot with little risk.
- Spot forces good engineering habits: graceful shutdown, idempotent jobs, health checks and diversification. Those habits improve reliability even for on-demand fleets.
- Done badly, Spot causes outages: a single-instance-type pool in one Availability Zone can disappear all at once.
- Spot stacks with other levers. Cover the stable baseline with Savings Plans §20.2, run the elastic part on Spot, and let autoscaling §20.4 match capacity to load.
- Related: Auto Scaling §6.2 and EC2 §6.1.

3 Real-world system example

AWS Auto Scaling groups with mixed instances policies. AWS documents using an Auto Scaling group with a **mixed instances policy** that combines multiple instance types and purchase options: a base of on-demand capacity plus Spot capacity above it, with the allocation strategy `price-capacity-optimized` recommended for most workloads. **Capacity Rebalancing** lets the group proactively replace a Spot instance when AWS emits a **rebalance recommendation** (an early signal that the instance is at elevated risk), before the two-minute interruption notice arrives.

Karpenter (an open-source Kubernetes node provisioner originated by AWS) chooses among many instance types and can provision Spot capacity, then consolidates nodes to reduce cost. **AWS Node Termination Handler** and the EKS managed node group Spot support watch interruption events so Kubernetes can cordon and drain nodes in time. Many CI systems (for example self-hosted runner fleets) are a classic Spot use case because jobs are short and can be retried.

4 Diagram

The life of an interruption:

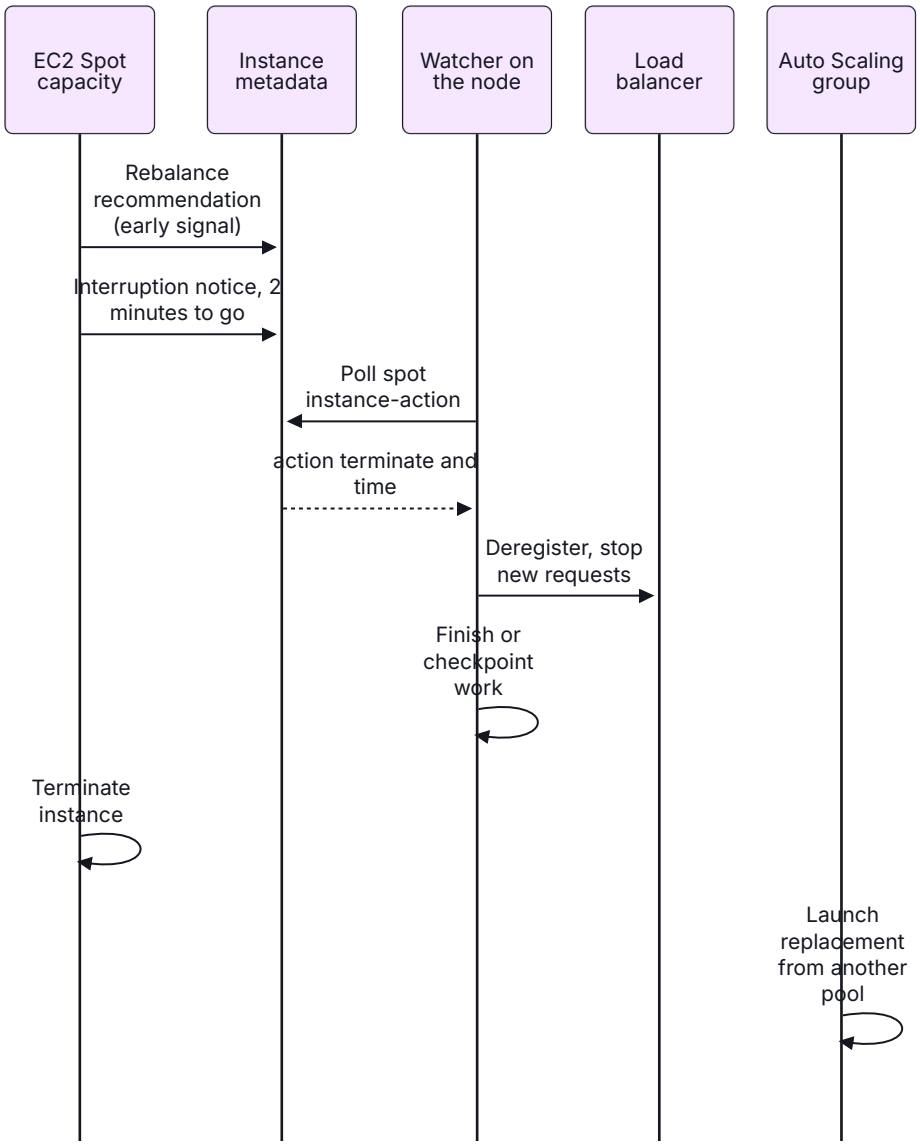


Figure 20.3-1

Diversified capacity:

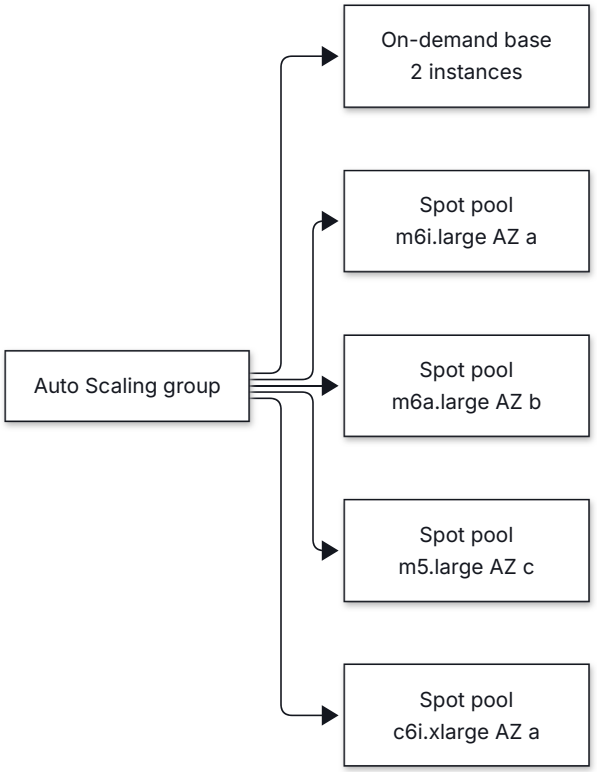


Figure 20.3-2

5 How it works

Interruption signals

Signal	When	What to do
Rebalance recommendation	may arrive well before the interruption (not guaranteed)	with Capacity Rebalancing, launch a replacement and drain the old node
Interruption notice	2 minutes before reclaim	stop taking work, checkpoint, deregister from the load balancer
Instance metadata `spot/instance-action`	HTTP 404 normally, JSON when an interruption is scheduled	polled by your agent or by the termination handler
EventBridge event `EC2 Spot Instance Interruption Warning`	same two-minute notice, as an event	trigger automation or notifications

Diversification

Spot capacity is organized in **pools**: an instance type in an Availability Zone. Pools run out independently, so the key resilience technique is **diversification**: ask for many instance types (similar vCPU and memory, across generations and families) across several AZs. Prefer `price-capacity-optimized` (chooses pools with the most spare capacity and the lowest price) over `lowest-price`, which tends to pick the pools most likely to be reclaimed.

Which workloads suit Spot

Good fit	Poor fit
CI runners, batch and ETL, rendering, big-data workers	single-instance databases and primary stateful stores
stateless web and API replicas behind a load balancer (with an on-demand base)	long jobs with no checkpoints
Kubernetes workers for stateless pods	workloads with strict single-node latency or licence binding
dev and test environments	anything where the interruption cost exceeds the saving

Break-even

If a Spot node is interrupted at a rate of `r` per hour and each interruption wastes `L` hours of work, the effective billed hours grow by `r × L`. Spot is cheaper as long as `spot_price × (1 + r·L) < on_demand_price`.

6 Step-by-step guide

Prerequisites: Python 3 for the offline steps. The Auto Scaling and Karpenter steps need an AWS sandbox account and a Kubernetes cluster respectively; Spot instances still cost money, so use small sizes, set a billing alarm and complete the cleanup. Prices are **EXAMPLES**: check current Spot pricing (`aws ec2 describe-spot-price-history`) because it moves.

1. **Estimate whether Spot is worth it.** Save as `spot_breakeven.py`:

```python
# spot_breakeven.py - is Spot still cheaper after interruptions? EXAMPLE numbers only.
od_price = 0.384            # example on-demand $/hour
spot_price = 0.115          # example Spot $/hour (varies by pool and time)
job_hours = 10.0            # useful compute the job needs
interrupt_per_hour = 0.05   # example: 5% chance per instance-hour of an interruption
lost_hours_per_interrupt = 0.5   # work redone since the last checkpoint, plus restart
  time
```

```
expected_interrupts = job_hours * interrupt_per_hour
effective_hours = job_hours + expected_interrupts * lost_hours_per_interrupt
print(f"expected interruptions     : {expected_interrupts:.2f}")
print(f"effective hours billed     : {effective_hours:.2f} (vs {job_hours:.2f}
  useful)")
print(f"on-demand cost             : ${od_price * job_hours:,.2f}")
print(f"spot cost                  : ${spot_price * effective_hours:,.2f}")
print(f"saving                     : {1 - spot_price * effective_hours / (od_price *
  job_hours):.1%}")
# the Spot price at which you only break even
be = od_price * job_hours / effective_hours
print(f"break-even Spot price      : ${be:,.4f}/h ({be / od_price:.0%} of on-demand)")
print()
print("how much does lost work per interruption matter? (no checkpoints = more lost
  work)")
for lost in (0.5, 2.0, 4.0):
    eff = job_hours + expected_interrupts * lost
    print(f"  lost {lost:>3}h per interrupt → spot ${spot_price * eff:>6.2f} vs on-
  demand ${od_price * job_hours:.2f}")
```

BASH

```
python3 spot_breakeven.py
```

Expected output:

```
expected interruptions     : 0.50
effective hours billed     : 10.25 (vs 10.00 useful)
on-demand cost             : $3.84
spot cost                  : $1.18
saving                     : 69.3%
break-even Spot price      : $0.3746/h (98% of on-demand)

how much does lost work per interruption matter? (no checkpoints = more lost work)
  lost 0.5h per interrupt → spot $  1.18 vs on-demand $3.84
  lost 2.0h per interrupt → spot $  1.27 vs on-demand $3.84
  lost 4.0h per interrupt → spot $  1.38 vs on-demand $3.84
```

Even with a lot of lost work, a deep discount wins. The risk is not the arithmetic but **whole-fleet** interruption and work that cannot be redone, which the arithmetic does not show.

2. **Write an interruption watcher.** On EC2 the metadata service is at `169.254.169.254` and IMDSv2 needs a session token. Save as `spot_watcher.py`:

PYTHON

```
# spot_watcher.py - poll the instance metadata service for a Spot interruption notice.
# On EC2 the base URL is http://169.254.169.254 ; pass another URL to test locally.
import json, sys, time, urllib.request, urllib.error
```

```
BASE = sys.argv[1] if len(sys.argv) > 1 else "http://169.254.169.254"

def get(path, token=None, method="GET", headers=None):
    req = urllib.request.Request(BASE + path, method=method, headers=headers or {})
    if token:
        req.add_header("X-aws-ec2-metadata-token", token)
    with urllib.request.urlopen(req, timeout=2) as r:
        return r.read().decode()

def token():
    try:   # IMDSv2 session token
        return get("/latest/api/token", method="PUT",
                   headers={"X-aws-ec2-metadata-token-ttl-seconds": "60"})
    except urllib.error.URLError:
        return None

def drain():
    print("interruption notice received: stop taking new work, checkpoint, deregister
  from the load balancer")

for _ in range(10):                       # a real watcher loops forever, every 5
  seconds
    tok = token()
    try:
        body = get("/latest/meta-data/spot/instance-action", tok)
        action = json.loads(body)
        print(f"action={action['action']} time={action['time']}")
        drain()
        break
    except urllib.error.HTTPError as e:
        if e.code != 404:                 # 404 means no interruption is scheduled
            raise
        print("no interruption scheduled")
    time.sleep(0.2)
```

3. **Test it locally with a fake metadata server.** Save as `fake_imds.py`, which returns 404 twice and then an interruption:

```
PYTHON
# fake_imds.py - tiny local stand-in for the metadata service. Interruption appears
  after 3 requests.
import http.server, json
count = 0
class H(http.server.BaseHTTPRequestHandler):
    def do_PUT(self):
        self.send_response(200); self.end_headers(); self.wfile.write(b"FAKE-TOKEN")
    def do_GET(self):
        global count
        if self.path.endswith("/spot/instance-action"):
            count += 1
```

```
            if count < 3:
                self.send_response(404); self.end_headers(); return
            self.send_response(200); self.end_headers()
            self.wfile.write(json.dumps({"action": "terminate", "time": "2026-10-
  09T12:00:00Z"}).encode()); return
        self.send_response(404); self.end_headers()
    def log_message(self, *a): pass
http.server.HTTPServer(("127.0.0.1", 8169), H).serve_forever()
```

BASH

```
python3 fake_imds.py &
python3 spot_watcher.py http://127.0.0.1:8169
kill %1
```

Expected output:

```
no interruption scheduled
no interruption scheduled
action=terminate time=2026-10-09T12:00:00Z
interruption notice received: stop taking new work, checkpoint, deregister from the
  load balancer
```

4. **Create a diversified Auto Scaling group (AWS sandbox).** Assuming a launch template `shop-api-worker` and three private subnets already exist, save as `asg.json` (replace the subnet IDs):

JSON

```
{
  "AutoScalingGroupName": "shop-api-workers",
  "MinSize": 2,
  "MaxSize": 6,
  "DesiredCapacity": 3,
  "VPCZoneIdentifier": "subnet-aaaa1111,subnet-bbbb2222,subnet-cccc3333",
  "CapacityRebalance": true,
  "MixedInstancesPolicy": {
    "LaunchTemplate": {
      "LaunchTemplateSpecification": {"LaunchTemplateName": "shop-api-worker",
  "Version": "$Latest"},
      "Overrides": [
        {"InstanceType": "m6i.large"},
        {"InstanceType": "m6a.large"},
        {"InstanceType": "m5.large"},
        {"InstanceType": "m5a.large"},
        {"InstanceType": "m7i.large"}
      ]
    },
    "InstancesDistribution": {
      "OnDemandBaseCapacity": 1,
```

```
      "OnDemandPercentageAboveBaseCapacity": 0,
      "SpotAllocationStrategy": "price-capacity-optimized"
    }
  }
}
```

```bash
aws autoscaling create-auto-scaling-group --cli-input-json file://asg.json
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names shop-api-
  workers \
  --query 'AutoScalingGroups[0].Instances[*].[InstanceId,InstanceType,LifecycleState]'
  --output table
```

You should see a table of three instances, one on-demand and two Spot, across several instance types (output varies). Check which are Spot with `aws ec2 describe-instances --instance-ids <ID> --query 'Reservations[].Instances[].InstanceLifecycle'` (returns `spot` for Spot instances).

5. **Run Spot on Kubernetes with Karpenter.** A `NodePool` that allows both capacity types and many instance types, with consolidation (assumes Karpenter and an `EC2NodeClass` named `default` are installed, for example on EKS):

```yaml
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: spot-first
spec:
  template:
    spec:
      nodeClassRef:
        group: karpenter.k8s.aws
        kind: EC2NodeClass
        name: default
      requirements:
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["spot", "on-demand"]
        - key: karpenter.k8s.aws/instance-category
          operator: In
          values: ["c", "m", "r"]
        - key: karpenter.k8s.aws/instance-generation
          operator: Gt
          values: ["4"]
  limits:
    cpu: "100"
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
```

```
    consolidateAfter: 1m
```

When both capacity types are allowed, Karpenter prefers Spot. Pin critical pods to on-demand with `nodeSelector: {karpenter.sh/capacity-type: on-demand}`.

6. **Make a workload interruption-friendly.** A Deployment that spreads across zones and a PodDisruptionBudget so a drain never takes down too many replicas at once:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
spec:
  replicas: 4
  selector:
    matchLabels: {app: shop-api}
  template:
    metadata:
      labels: {app: shop-api}
    spec:
      terminationGracePeriodSeconds: 60
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels: {app: shop-api}
      containers:
        - name: shop-api
          image: nginx:1.27
          resources:
            requests: {cpu: 250m, memory: 256Mi}
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: shop-api
spec:
  minAvailable: 3
  selector:
    matchLabels: {app: shop-api}
```

```bash
kubectl apply -f shop-api.yaml
kubectl get pdb shop-api
```

Expect `ALLOWED DISRUPTIONS` of `1` once all four pods are ready.

7. **Clean up.**

```bash
aws autoscaling delete-auto-scaling-group --auto-scaling-group-name shop-api-workers --force-delete
kubectl delete -f shop-api.yaml
```

Confirm no instances remain with `aws ec2 describe-instances --filters Name=tag:aws:autoscaling:groupName,Values=shop-api-workers`.

7 Real-world scenario

(Illustrative.) A team moves its CI runner fleet to Spot and saves most of its runner bill. Then a Friday-afternoon capacity crunch in the one instance type they requested (`c5.2xlarge`, one zone) interrupts the entire fleet. Builds fail, the queue grows, and deploys stall for an hour.

Investigation shows the Auto Scaling group had a single override and one subnet. The fix: eight instance types across three zones, `price-capacity-optimized`, Capacity Rebalancing, a small on-demand base, and a job retry policy in CI so an interrupted build restarts automatically. The next capacity event cost two retried jobs and no outage.

8 Common mistakes and best practices

Don't

- Don't run a Spot fleet with one instance type or one zone.
- Don't put databases, single-replica stateful services or the Kubernetes control plane on Spot.
- Don't ignore the two-minute notice; an unhandled interruption drops in-flight requests.
- Don't use `lowest-price` as the allocation strategy for important workloads.
- Don't forget that a hard Spot price limit can stop you from getting capacity at all; the default maximum is the on-demand price.

Do

- Do diversify across many instance types and zones.
- Do keep a small on-demand base for the critical floor.
- Do checkpoint long jobs and make them idempotent.
- Do use PodDisruptionBudgets and topology spread on Kubernetes.
- Do monitor the interruption rate and the time to replace capacity.

9 Tasks

- [ ] **T1 — Run the break-even model** *(≈20 min, beginner)*
  - **Goal:** quantify when Spot pays.
  - **Steps:**
    1. Run `spot_breakeven.py`.
    2. Set `interrupt_per_hour` to 0.3 and `lost_hours_per_interrupt` to 4.
    3. Note whether Spot still wins.
  - **Done when:** you can state the condition `spot × (1 + r·L) < on-demand` and show a case where it fails.
- [ ] **T2 — Handle a simulated interruption** *(≈30 min, intermediate)*
  - **Goal:** a watcher that reacts to the notice.
  - **Steps:**
    1. Run `fake_imds.py` and `spot_watcher.py`.
    2. Change `drain()` to write a file `/tmp/drained` and exit with code 0.
  - **Done when:** the file exists after the fake interruption and the watcher printed the action.
- [ ] **T3 — Build a diversified fleet** *(≈45 min, intermediate; AWS sandbox, small cost)*
  - **Goal:** a mixed on-demand and Spot Auto Scaling group.
  - **Steps:**
    1. Create the group from `asg.json`.
    2. List the instance types and lifecycle (`spot` or on-demand).
    3. Delete the group.
  - **Done when:** the table shows at least two instance types and `InstanceLifecycle` is `spot` for non-base instances, and the group is deleted.
- [ ] **T4 — Break it: a one-pool fleet** *(≈30 min, intermediate)*
  - **Goal:** understand concentrated risk.
  - **Steps:**
    1. Draw (or script) a fleet of 10 nodes in 1 pool and one in 8 pools.
    2. Simulate a pool reclaim removing 100 percent of one pool.
  - **Done when:** you can show the one-pool fleet loses 100 percent and the eight-pool fleet loses about 12.5 percent.
- [ ] **T5 — Spot-friendly Kubernetes workload** *(≈45 min, advanced; any cluster)*

- **Goal:** survive a node drain.
- **Steps:**
  1. Apply the Deployment and PDB.
  2. Run `kubectl drain <node> --ignore-daemonsets --delete-emptydir-data` on a worker.
  3. Watch the pods reschedule while at least three stay ready.
- **Done when:** `kubectl get pods -w` shows the service never drops below `minAvailable`.

☐ **T6 — Prove it** *(≈45 min, advanced)*

- **Goal:** choose and defend what runs on Spot.
- **Steps:**
  1. List five workloads from `shop-api` and decide Spot, on-demand or mixed for each.
  2. For each Spot workload, write the interruption behaviour (retry, drain, checkpoint).
  3. Explain diversification and the two-minute notice.
- **Done when:** a reviewer agrees with the classification and you can recite the signals without notes.

10 Self-check questions

1. How much warning does a Spot interruption give, and where do you read it?
2. Why is `price-capacity-optimized` preferred over `lowest-price`?
3. Name two workloads that suit Spot and two that do not.
4. What is a Spot pool, and why does diversification matter?
5. What does Capacity Rebalancing do?

ANSWERS

1. Two minutes, from the instance metadata `spot/instance-action` path or the EventBridge interruption warning event.
2. It picks pools with deeper spare capacity as well as low price, so interruptions are less likely; `lowest-price` tends to pick the most reclaimed pools.
3. Suit: CI runners, stateless web replicas, batch jobs. Do not suit: primary databases, long jobs without checkpoints.
4. A pool is an instance type in an Availability Zone. Pools fail independently, so spreading across many reduces the share lost at once.

5. It launches a replacement when a rebalance recommendation signals elevated risk, so capacity is replaced before the interruption notice.

11 Further reading

- AWS documentation: EC2 Spot Instances user guide, Auto Scaling groups with multiple instance types and purchase options, Capacity Rebalancing.
- Karpenter documentation (karpenter.sh): NodePools, disruption and Spot.
- AWS Node Termination Handler documentation (GitHub: aws/aws-node-termination-handler).
- AWS Well-Architected Framework: Cost Optimization pillar.

20.4 Autoscaling for Cost

Topic 4 of 9 | ~4 hours | Difficulty: Intermediate

1 What it is

Autoscaling is usually taught as a reliability tool: add capacity when load rises. Used for cost, the more important half is the opposite: **remove capacity when load falls** and, for workloads that are idle for hours, remove it entirely. Four techniques cover most of the savings:

Technique	Idea	Example
Scale-in	shrink quickly but safely after a peak	HPA scale-down policy, ASG target tracking scale-in
Scheduled scaling	change min and max capacity at known times	more replicas at 08:00, fewer at 20:00
Scale-to-zero	run no replicas when there is no work	Lambda, KEDA with `minReplicaCount: 0`, Knative
Off-hours shutdown	stop non-production environments overnight and at weekends	EventBridge Scheduler, Instance Scheduler, a CronJob

You pay for capacity by the second or hour whether it serves traffic or not. A fleet sized for the daily peak and left running 24 hours a day wastes the gap between the peak and the average.

2 Why it matters in DevOps

- Most services have a daily and weekly rhythm. A production fleet that is flat all day is paying for the 3 a.m. peak at 3 p.m.
- Non-production environments are the cheapest saving: dev and staging used 12 hours a day, five days a week are idle about 64 percent of the time.
- Aggressive scale-in is also a reliability risk. Scale in too fast and the next spike hits a cold, small fleet, so the settings matter as much as the idea.
- Autoscaling interacts with commitments: see Reserved Instances and Savings Plans §20.2 (cover only the floor) and Spot Instances §20.3 (the elastic part).

- Related: Auto Scaling §6.2, Horizontal Pod Autoscaling §8.16 and HPA, VPA and Cluster Autoscaler §16.7.

3 Real-world system example

AWS Auto Scaling and Application Auto Scaling document three scaling modes: dynamic (target tracking and step scaling), **scheduled** actions with a recurrence, and predictive scaling, which forecasts load from history for EC2 Auto Scaling groups. AWS also publishes the **Instance Scheduler on AWS** solution, which starts and stops EC2 and RDS instances on a schedule, and Amazon EventBridge Scheduler can call EC2, ECS and Lambda APIs on cron expressions.

Kubernetes documents the HorizontalPodAutoscaler with configurable `behavior` for scale-up and scale-down, including a stabilization window that prevents flapping. **KEDA** (Kubernetes Event-driven Autoscaling, a CNCF project) extends it with event sources such as queues and cron schedules and can scale a Deployment from zero to N and back. **AWS Lambda** and **Knative** are the standard examples of scale-to-zero. The **Cluster Autoscaler** and **Karpenter** remove empty or under-used nodes so that pod scale-in becomes a lower node bill (see Kubernetes Infrastructure Costs §20.9).

4 Diagram

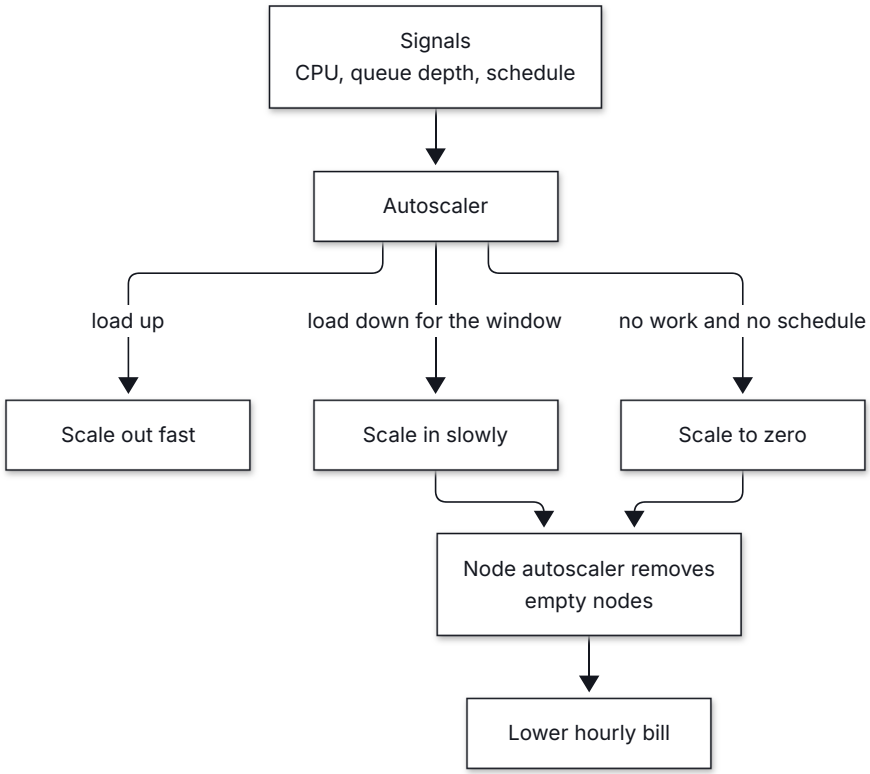


Figure 20.4-1

A weekday for a development environment:

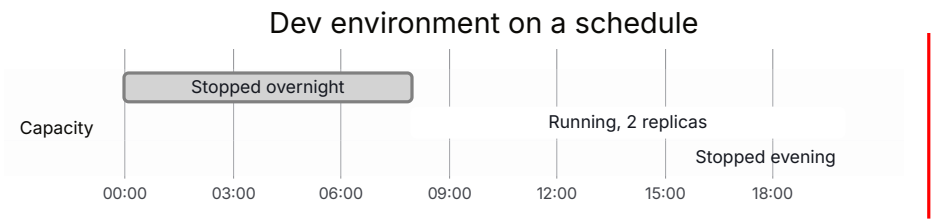


Figure 20.4-2

5 How it works

Scale-in without hurting users

Mechanism	Setting	Effect
HPA `behavior.scaleDown.stabilizationWindowSeconds`	for example 300	uses the highest recommendation in the window, so a brief dip does not shrink the fleet
HPA `behavior.scaleDown.policies`	for example 20 percent per minute	limits the rate of removal
ASG target tracking	target 55-65 percent CPU	scales in when load stays well below target
Connection draining / `terminationGracePeriodSeconds`	30-120 seconds	in-flight requests finish before a node goes away
PodDisruptionBudget	`minAvailable`	blocks too many pods leaving at once

Scale out fast and scale in slowly: the asymmetry is the whole design.

Scheduled scaling

When you know the pattern (traffic doubles at 08:00, a nightly batch), set **minimum** capacity ahead of time rather than waiting for metrics to catch up. In an Auto Scaling group, a scheduled action changes `MinSize`, `MaxSize` or `DesiredCapacity` at a cron time. Combine it with dynamic scaling so spikes still scale out.

Scale-to-zero

A workload can reach zero only if something wakes it when work arrives, and the first request tolerates a cold start. Fine for queue workers, cron-like processors, preview environments and internal tools; wrong for latency-sensitive user paths. The standard Kubernetes HPA does not scale to zero by default (the feature is gated and not generally available), so use KEDA or Knative.

Off-hours shutdown

Stopping an EC2 instance stops compute charges but **not** attached EBS volumes, Elastic IPs or NAT gateways. Bigger savings come from tearing down the whole environment (infrastructure as code `destroy` or scaling node groups to zero) rather than only stopping VMs. Make the environment reproducible so shutdown is safe.

6 Step-by-step guide

Prerequisites: Python 3 for the first step; a Kubernetes cluster (kind is fine) with `kubectl` for the Kubernetes steps; an AWS sandbox only for the optional ASG steps. Prices are **EXAMPLES**: check current pricing.

1. **Calculate the off-hours saving.** Save as `offhours.py`:

```python
# offhours.py - savings from shutting non-production down outside working hours.
  EXAMPLE price.
hourly_cost_all_envs = 4.80      # example total $/hour for dev+staging compute when
  running
hours_per_week_on = 12 * 5       # 08:00-20:00 Monday-Friday
hours_per_week = 168
always_on = hourly_cost_all_envs * hours_per_week * 52 / 12
scheduled = hourly_cost_all_envs * hours_per_week_on * 52 / 12
print(f"always on  : ${always_on:,.2f}/month")
print(f"scheduled  : ${scheduled:,.2f}/month ({hours_per_week_on}/{hours_per_week}
  hours)")
print(f"saving     : ${always_on - scheduled:,.2f}/month = {1 - scheduled /
  always_on:.0%}")
print("note: EBS volumes, Elastic IPs and NAT gateways keep billing while instances
  are stopped")
```

```bash
python3 offhours.py
```

Expected output:

```
always on  : $3,494.40/month
scheduled  : $1,248.00/month (60/168 hours)
saving     : $2,246.40/month = 64%
note: EBS volumes, Elastic IPs and NAT gateways keep billing while instances are
  stopped
```

2. **Tune HPA scale-in.** A Deployment and an HPA that scales out fast and in slowly:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
spec:
  replicas: 2
  selector:
    matchLabels: {app: shop-api}
```

```
  template:
    metadata:
      labels: {app: shop-api}
    spec:
      containers:
        - name: shop-api
          image: nginx:1.27
          resources:
            requests: {cpu: 100m, memory: 128Mi}
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: shop-api
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: shop-api
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
        - type: Percent
          value: 100
          periodSeconds: 30
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 20
          periodSeconds: 60
```

BASH

```
kubectl apply -f shop-api-hpa.yaml
kubectl get hpa shop-api
```

Expect `TARGETS` such as `cpu: 1%/60%` (a metrics server must be installed, as in kind with `metrics-server`) and `REPLICAS 2`. Without a metrics server, `TARGETS` shows `<unknown>`.

3. **Scale a worker to zero with KEDA.** Install KEDA (`helm repo add kedacore https://kedacore.github.io/charts`, `helm install keda kedacore/keda -n keda --create-namespace`), then:

```yaml
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: shop-worker
spec:
  scaleTargetRef:
    name: shop-worker
  minReplicaCount: 0
  maxReplicaCount: 10
  cooldownPeriod: 300
  triggers:
    - type: cron
      metadata:
        timezone: America/New_York
        start: "0 8 * * 1-5"
        end: "0 20 * * 1-5"
        desiredReplicas: "2"
```

```bash
kubectl apply -f shop-worker-scaledobject.yaml
kubectl get scaledobject shop-worker
```

Outside the 08:00-20:00 weekday window the Deployment `shop-worker` is scaled to 0 replicas; inside it holds 2. For queue-driven workers, replace the trigger with a queue scaler such as `aws-sqs-queue` so replicas follow backlog and drop to zero when the queue is empty (check the KEDA scaler documentation for the exact metadata and authentication fields).

4. **Shut dev down at night with a CronJob (no extra tooling).** A pair of CronJobs scale a dev namespace down at 20:00 and up at 08:00 (UTC here; adjust). It needs a ServiceAccount allowed to scale deployments:

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: scaler
  namespace: dev
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
```

```
  name: scaler
  namespace: dev
rules:
  - apiGroups: ["apps"]
    resources: ["deployments", "deployments/scale"]
    verbs: ["get", "list", "patch", "update"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: scaler
  namespace: dev
subjects:
  - kind: ServiceAccount
    name: scaler
    namespace: dev
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: scaler
---
apiVersion: batch/v1
kind: CronJob
metadata:
  name: dev-scale-down
  namespace: dev
spec:
  schedule: "0 20 * * 1-5"
  jobTemplate:
    spec:
      template:
        spec:
          serviceAccountName: scaler
          restartPolicy: OnFailure
          containers:
            - name: kubectl
              image: registry.k8s.io/kubectl:v1.31.0   # any image that contains
  kubectl works
              command: ["kubectl", "scale", "deployment", "--all", "--replicas=0", "-
  n", "dev"]
```

Create the matching `dev-scale-up` CronJob with `schedule: "0 8 * * 1-5"` and `--replicas=2`. Test immediately without waiting:

```bash
kubectl -n dev create job --from=cronjob/dev-scale-down test-down
kubectl -n dev get deploy
```

Expect every Deployment to show `0/0`. Remember that a pure scale-to-zero leaves the nodes running unless a node autoscaler removes them.

5. **Scale an EC2 Auto Scaling group on a schedule (AWS sandbox).**

```bash
aws autoscaling put-scheduled-update-group-action \
  --auto-scaling-group-name shop-api-workers \
  --scheduled-action-name weekday-morning \
  --recurrence "0 8 * * 1-5" --time-zone "America/New_York" \
  --min-size 4 --desired-capacity 4
aws autoscaling put-scheduled-update-group-action \
  --auto-scaling-group-name shop-api-workers \
  --scheduled-action-name weeknight \
  --recurrence "0 20 * * 1-5" --time-zone "America/New_York" \
  --min-size 1 --desired-capacity 1
aws autoscaling describe-scheduled-actions --auto-scaling-group-name shop-api-workers
```

   The output lists both actions with their recurrence. Use scheduled actions to raise or lower the **floor**; target tracking handles the rest between min and max.

6. **Clean up.** Delete the test objects and any sandbox resources:

```bash
kubectl delete -f shop-api-hpa.yaml --ignore-not-found
kubectl -n dev delete job test-down --ignore-not-found
aws autoscaling delete-scheduled-action --auto-scaling-group-name shop-api-workers --
  scheduled-action-name weekday-morning
aws autoscaling delete-scheduled-action --auto-scaling-group-name shop-api-workers --
  scheduled-action-name weeknight
```

7 Real-world scenario

(Illustrative.) A startup runs identical dev, staging and demo environments, 24 hours a day, and the three together cost about as much as production. The team adds an off-hours CronJob for the namespaces and a node group minimum of zero, then discovers the Monday 08:00 start takes 12 minutes because images are pulled to fresh nodes and a database restore job runs before the app becomes ready. Engineers arriving early are blocked.

They move the start to 07:30, keep a pre-pulled image cache in a small always-on node group, and add a `/start-env` chat command for weekend work. The monthly saving is over half of the non-production bill, and a documented "wake-up time" becomes part of the environment contract.

8 Common mistakes and best practices

Don't

- Don't scale in on a single low CPU sample; use a stabilization window.
- Don't scale to zero on a user-facing path with a strict latency target.
- Don't assume stopped instances are free; volumes, IPs and NAT gateways still bill.
- Don't schedule shutdowns for resources with in-memory state you cannot rebuild.
- Don't forget the node layer: pods at zero with nodes still running saves nothing.

Do

- Do scale out fast and in slowly, and use PodDisruptionBudgets for safe drains.
- Do use scheduled scaling for known daily and weekly patterns.
- Do tag environments with their schedule so an off-hours policy can find them.
- Do measure savings from the bill, not from the schedule.
- Do keep a documented way to wake an environment on demand.

9 Tasks

- [ ] **T1 — Price the off-hours schedule** *(≈15 min, beginner)*
  - **Goal:** a number for the idea.
  - **Steps:**
    1. Run `offhours.py`.
    2. Change hours on to 10 hours (09:00-19:00) and weekends on for one environment.
  - **Done when:** you can state the monthly saving for both schedules.
- [ ] **T2 — Tune scale-in** *(≈40 min, intermediate)*
  - **Goal:** observe the scale-down window.
  - **Steps:**
    1. Apply the HPA and generate load with `kubectl run load --image=busybox:1.36 --restart=Never -- /bin/sh -c "while true; do wget -q -O- http://shop-api; done"` (create a Service named `shop-api` first).
    2. Delete the load pod and watch `kubectl get hpa shop-api -w`.
  - **Done when:** you see replicas rise quickly, then fall back no sooner than about five minutes after the load stops.
- [ ] **T3 — Scale a worker to zero** *(≈45 min, intermediate)*
  - **Goal:** a Deployment that returns to zero.
  - **Steps:**
    1. Install KEDA and a `shop-worker` Deployment.

2. Apply the `ScaledObject` with a cron window that covers the next 10 minutes, then wait for it to end.

- **Done when:** `kubectl get deploy shop-worker` shows 2 replicas inside the window and 0 after it (allowing for `cooldownPeriod`).

- [ ] **T4 — Break it: scale-in too fast** *(≈30 min, intermediate)*
  - **Goal:** feel the flapping risk.
  - **Steps:**
    1. Set `scaleDown.stabilizationWindowSeconds: 0` with a 100 percent policy.
    2. Create bursty load (30 seconds on, 30 seconds off).
  - **Done when:** `kubectl describe hpa shop-api` shows repeated scale events, and restoring the 300-second window stops the flapping.
- [ ] **T5 — Automate environment shutdown** *(≈60 min, advanced)*
  - **Goal:** the CronJob pair works.
  - **Steps:**
    1. Create the ServiceAccount, Role, binding and both CronJobs in a `dev` namespace.
    2. Run `test-down`, then run a matching `test-up`.
  - **Done when:** all Deployments in `dev` go to 0 and return to 2 on demand.
- [ ] **T6 — Prove it** *(≈45 min, advanced)*
  - **Goal:** propose an autoscaling cost plan.
  - **Steps:**
    1. Without the guide, list for `shop-api` which components get HPA scale-in, schedules, scale-to-zero and off-hours shutdown.
    2. State the node-layer change that makes each saving real.
    3. Estimate the saving with `offhours.py` numbers.
  - **Done when:** you can defend each choice and name one risk per technique.

10 Self-check questions

1. Why scale out fast and in slowly?
2. What does an HPA stabilization window do?
3. Why can stopping instances still leave a bill?
4. What must exist for scale-to-zero to work?
5. Why does a pod-level saving need a node autoscaler?

ANSWERS

1. Late scale-out hurts users while late scale-in only costs a little; the asymmetry protects reliability.
2. It looks back over a time window and uses the highest recommended replica count, preventing flapping after brief dips.
3. EBS volumes, Elastic IPs, NAT gateways and snapshots keep charging.
4. A trigger that wakes the workload (queue, schedule, request) and a workload that tolerates a cold start.
5. Pods at zero still occupy nodes; only when nodes are removed does the hourly compute bill drop.

11 Further reading

- Kubernetes documentation: Horizontal Pod Autoscaling and its `behavior` section.
- KEDA documentation (keda.sh): scalers and `ScaledObject`.
- AWS documentation: Amazon EC2 Auto Scaling scheduled scaling and predictive scaling, Instance Scheduler on AWS, EventBridge Scheduler.
- Knative Serving documentation: scale to zero.

20.5 Rightsizing

Topic 5 of 9 | ~4 hours | Difficulty: Intermediate

1 What it is

Rightsizing is matching the size of a resource (an EC2 instance, an RDS database, a Kubernetes container request) to what the workload actually needs. Engineers size for fear: a new service gets `m6i.2xlarge` "to be safe", a container gets `cpu: 2`, and nothing is ever revisited. The result is capacity that is paid for every hour and used a small fraction of the time.

The method is the same everywhere:

1. **Measure** real utilization (CPU, memory, network, disk I/O) over a window that includes your peaks.
2. **Choose** a smaller size, or a different family, that keeps a safe margin at the high percentile.
3. **Change** in a controlled way (canary, one node first, off-peak).
4. **Verify** performance and SLOs, then repeat on a schedule.

Rightsizing is the cheapest optimization to reason about and the easiest to get wrong, because shrinking a resource removes the safety margin that was hiding other problems.

2 Why it matters in DevOps

- It is often the largest single saving that needs no architectural change: many fleets run at low average CPU.
- It must precede commitments. Buying a Savings Plan §20.2 for an oversized fleet locks in the waste.
- In Kubernetes it affects the whole cluster: inflated requests reserve capacity that nothing uses, which makes the cluster autoscaler add nodes (see Kubernetes Infrastructure Costs §20.9).
- Wrong rightsizing causes incidents: out-of-memory kills, CPU throttling and latency spikes. The risk is why this is a practice with measurements, not a one-off cleanup.

- Related: EC2 §6.1, Resource Requests and Limits §8.14 and Capacity Planning §14.9.

3 Real-world system example

AWS Compute Optimizer is the managed rightsizing service. After you opt in, it analyses CloudWatch metrics (by default the previous 14 days, with an option for longer lookback) and classifies EC2 instances, Auto Scaling groups, EBS volumes, Lambda functions and ECS services on Fargate as **over-provisioned**, **under-provisioned**, **optimized** or similar, with alternative instance types and a projected performance risk. A key limitation AWS documents: memory metrics are only available if you install the CloudWatch agent (or have the data from another supported source), so without it recommendations are CPU- and network-based.

Kubernetes Vertical Pod Autoscaler (VPA) watches actual container usage and publishes recommended requests. In `updateMode: "Off"` it only recommends and does not evict pods, which is the safe way to use it for analysis. **Goldilocks**, an open-source project from Fairwinds, creates a VPA in recommendation mode for each Deployment in a labelled namespace and shows the results in a dashboard.

4 Diagram

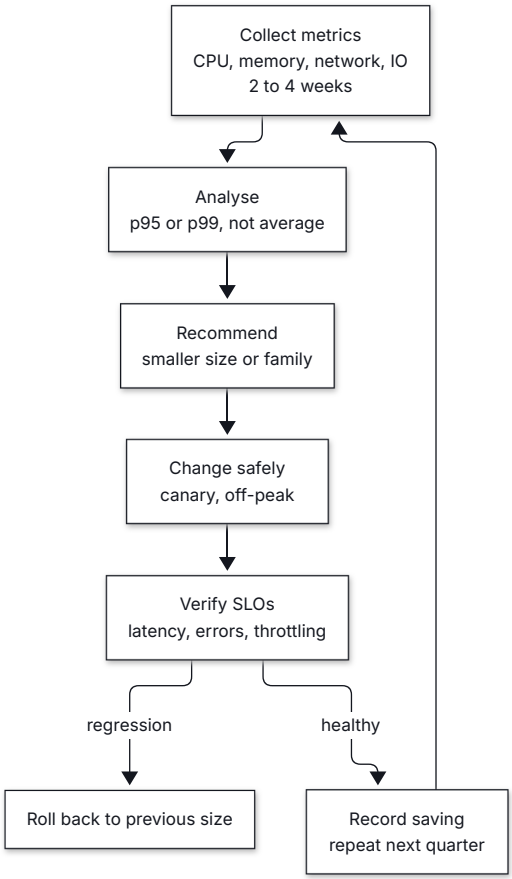


Figure 20.5-1

Why averages mislead:

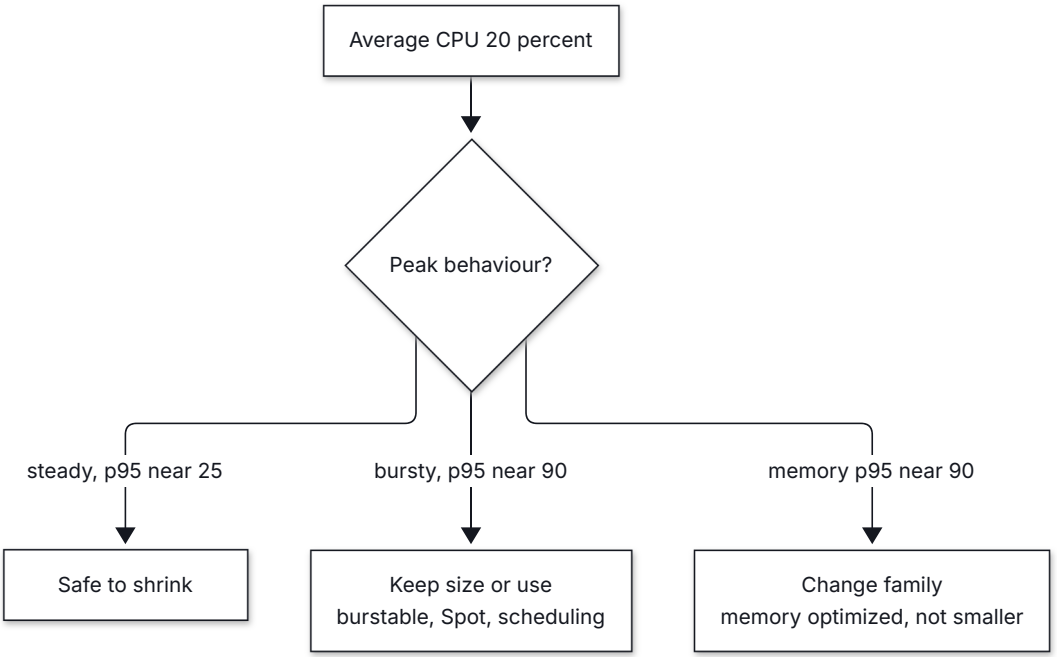


Figure 20.5-2

5 How it works

What to look at

Resource	Metrics to use	Percentile and margin
EC2 compute	CPU, memory (needs agent), network in/out	size for p95 CPU around 60 percent, memory p95 around 80 percent
EBS volumes	IOPS, throughput, queue length	match gp3 IOPS and throughput to need, not to size
RDS / Aurora	CPU, freeable memory, connections, IOPS	see Database Costs §20.8
Kubernetes containers	usage versus requests (CPU, memory)	request near p90-p95 usage plus headroom; memory limits stay safe
Lambda	memory used versus configured, duration	a bigger memory setting can be cheaper if duration drops

Choose a window that contains your real peaks: month-end jobs, weekly batch, a campaign. Two weeks is the minimum; four to six is better. Use the high percentile because a service that averages 12 percent CPU and hits 90 percent each day at 09:00 is not oversized.

Risks and how to control them

Risk	Control
Memory pressure (OOM kills) after shrinking	install memory metrics first; shrink memory conservatively; watch `OOMKilled`
CPU throttling on Kubernetes	keep limits generous or remove CPU limits; watch `container_cpu_cfs_throttled_periods_total`
Network bandwidth is tied to size	check the family's baseline and burst network limits
Burstable instances (T family) running out of credits	monitor CPU credit balance; prefer unlimited mode or a fixed-performance family if sustained
Licences or software that scale by core	involve the owner before changing
Change needs a restart	roll one instance at a time behind the load balancer
Seasonality hidden by the window	extend the window, compare year-on-year, keep an event headroom plan

Moving down, sideways and across

Rightsizing is not only "smaller". Moving from an older generation to a newer one, to ARM-based Graviton, or from a general-purpose to a memory-optimized family can lower the cost for the same work. Test on a canary because architecture changes need a compatible image (multi-arch container images, for example).

6 Step-by-step guide

Prerequisites: Python 3 for the offline analysis; an AWS sandbox with some running instances for Compute Optimizer (it is free for its basic recommendations; enhanced infrastructure metrics are paid, so avoid enabling them for the exercise); a Kubernetes cluster with the Vertical Pod Autoscaler CRDs for the Kubernetes steps. Prices are **EXAMPLES**: check current pricing.

1. **Analyse utilization samples offline.** This script reads per-instance CPU and memory samples, takes the 95th percentile, and chooses the smallest size on an example ladder that keeps 60 percent CPU and 80 percent memory at that percentile. Save as `rightsize.py`:

   ```
   PYTHON
   ```

```
# rightsize.py - suggest a smaller instance from utilization samples. EXAMPLE data and
  prices.
import statistics

# 20 hourly samples per instance; real data comes from CloudWatch (CPU) and the
  CloudWatch agent (memory)
DATA = {
    "i-api-1":   {"cpu": [8, 10, 12, 9, 11, 14, 17, 13, 10, 8, 9, 12, 16, 15, 11, 9,
  8, 10, 13, 19],
                  "mem": [22, 23, 23, 24, 25, 26, 26, 25, 24, 23, 23, 24, 26, 27, 25,
  24, 23, 24, 25, 28]},
    "i-batch-1": {"cpu": [3, 4, 3, 5, 88, 91, 4, 3, 5, 4, 3, 85, 90, 4, 3, 4, 5, 3, 4,
  3],
                  "mem": [10, 11, 10, 12, 55, 58, 11, 10, 12, 11, 10, 54, 57, 11, 10,
  11, 12, 10, 11, 10]},
    "i-db-1":    {"cpu": [40, 45, 48, 52, 50, 47, 44, 46, 49, 51, 53, 48, 45, 44, 47,
  50, 52, 49, 46, 45],
                  "mem": [86, 88, 89, 90, 90, 89, 88, 88, 89, 90, 91, 90, 89, 88, 89,
  90, 90, 89, 88, 88]},
}
# example size ladder: name -> (vCPU, GiB, $/hour)
LADDER = [("m6i.large", 2, 8, 0.096), ("m6i.xlarge", 4, 16, 0.192), ("m6i.2xlarge", 8,
  32, 0.384)]
CURRENT = "m6i.2xlarge"

def p95(values):
    values = sorted(values)
    return values[min(len(values) - 1, int(round(0.95 * (len(values) - 1))))]

cur = next(x for x in LADDER if x[0] == CURRENT)
for inst, d in DATA.items():
    need_vcpu = cur[1] * p95(d["cpu"]) / 100 / 0.6      # aim for 60% CPU at the p95
    need_gib = cur[2] * p95(d["mem"]) / 100 / 0.8       # aim for 80% memory at the
  p95
    pick = next((x for x in LADDER if x[1] >= need_vcpu and x[2] >= need_gib), cur)
  # keep if nothing fits
    saving = (cur[3] - pick[3]) * 730
    print(f"{inst:<10} cpu p95={p95(d['cpu']):>3}% avg=
  {statistics.mean(d['cpu']):>3.0f}%  "
          f"mem p95={p95(d['mem']):>3}%  {cur[0]} -> {pick[0]:<12} saves
  ${saving:>4,.0f}/month")
```

BASH

```
python3 rightsize.py
```

Expected output:

```
i-api-1    cpu p95= 17% avg= 12%  mem p95= 27%  m6i.2xlarge → m6i.xlarge   saves $
  140/month
i-batch-1  cpu p95= 90% avg= 21%  mem p95= 57%  m6i.2xlarge → m6i.2xlarge  saves $
  0/month
i-db-1     cpu p95= 52% avg= 48%  mem p95= 90%  m6i.2xlarge → m6i.2xlarge  saves $
  0/month
```

Read the three results as three different stories. The API is a clear shrink. The batch host looks idle on average (21 percent) but spikes to 90 percent, so keep its size and consider scheduling or Spot instead. The database is **memory-bound** at 90 percent; the fix, if any, is a memory-optimized family, not a smaller general-purpose one.

2. **Install the CloudWatch agent for memory (EC2).** Without it, memory is invisible. A minimal configuration file `amazon-cloudwatch-agent.json`:

```json
{
  "metrics": {
    "append_dimensions": {"InstanceId": "${aws:InstanceId}"},
    "metrics_collected": {
      "mem": {"measurement": ["mem_used_percent"], "metrics_collection_interval": 60},
      "disk": {"measurement": ["used_percent"], "resources": ["/"],
  "metrics_collection_interval": 60}
    }
  }
}
```

Install and start the agent following the AWS guide for your OS (Amazon Linux example: `sudo dnf install amazon-cloudwatch-agent`, then `sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config -m ec2 -c file:/path/to/amazon-cloudwatch-agent.json -s`). After a few minutes the `CWAgent` namespace shows `mem_used_percent`. The instance role needs the `CloudWatchAgentServerPolicy` managed policy.

3. **Opt in to Compute Optimizer and read recommendations.**

```bash
aws compute-optimizer update-enrollment-status --status Active
aws compute-optimizer get-ec2-instance-recommendations \
  --query 'instanceRecommendations[*].
  [instanceArn,finding,currentInstanceType,recommendationOptions[0].instanceType]' \
  --output table
```

The first call returns the status immediately; recommendations need about 12 hours of metrics to appear and the table is empty until then. Findings look like `OVER_PROVISIONED`, `OPTIMIZED` or `UNDER_PROVISIONED`. Treat the options as hypotheses: check the `performanceRisk` score and test before changing.

4. **Get Kubernetes recommendations with a VPA in recommendation-only mode.** Assumes the VPA components are installed (see the Kubernetes autoscaler project). This VPA never evicts pods:

```yaml
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: shop-api
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: shop-api
  updatePolicy:
    updateMode: "Off"
  resourcePolicy:
    containerPolicies:
      - containerName: "*"
        minAllowed: {cpu: 50m, memory: 64Mi}
        maxAllowed: {cpu: "2", memory: 2Gi}
```

```bash
kubectl apply -f shop-api-vpa.yaml
kubectl describe vpa shop-api | sed -n '/Recommendation/,$p'
```

After some minutes of traffic you see `Lower Bound`, `Target`, `Uncapped Target` and `Upper Bound` per container. Compare **Target** with the current requests. (Output is similar to `Target: Cpu: 25m Memory: 262144k` for a lightly loaded pod.)

5. **See requests versus usage with Prometheus.** If you run Prometheus with kube-state-metrics and cAdvisor, these queries compare what pods reserve with what they use:

```promql
# CPU requested versus used, by namespace
sum by (namespace) (kube_pod_container_resource_requests{resource="cpu"})
sum by (namespace) (rate(container_cpu_usage_seconds_total{container≠""}[5m]))

# memory requested versus working set
sum by (namespace) (kube_pod_container_resource_requests{resource="memory"})
sum by (namespace) (container_memory_working_set_bytes{container≠""})
```

A ratio of used to requested below about 0.3 over weeks is a rightsizing candidate (see Prometheus and PromQL §10.2).

6. **Apply a change safely and verify.** Change one replica or one instance first, then watch the SLO signals:

```bash
kubectl set resources deployment shop-api --requests=cpu=250m,memory=256Mi
kubectl rollout status deployment/shop-api
kubectl top pods -l app=shop-api
kubectl get pods -l app=shop-api -o jsonpath='{range .items[*]}{.metadata.name}{" "}
  {.status.containerStatuses[0].lastState.terminated.reason}{"\n"}{end}'
```

   The last command prints `OOMKilled` if memory was set too low. To roll back: `kubectl rollout undo deployment/shop-api`. For EC2, stop the instance, change the type, start it, and keep the old type in your runbook.

7 Real-world scenario

(Illustrative.) The platform team finds that every `shop-api` pod requests 2 CPUs and 4 GiB, a value copied from a template. Usage is around 150m CPU and 600 MiB. They halve requests across the fleet in one change. The cluster autoscaler consolidates nodes and the monthly bill drops, until a Monday morning when memory-heavy report generation pods are `OOMKilled` and customers see errors.

The investigation shows the VPA window was a quiet fortnight that excluded the month-end report job. The fix: restore the memory request for that workload, extend the analysis window to cover a month-end, add an `OOMKilled` alert, and roll out requests changes by workload class rather than fleet-wide.

8 Common mistakes and best practices

Don't

- Don't rightsize from averages or from a window that excludes your busiest period.
- Don't shrink without memory data; CPU-only recommendations can cause OOM kills.
- Don't change every service in one release.
- Don't treat tool recommendations as instructions; verify and test.
- Don't buy commitments before rightsizing.

Do

- Do collect memory metrics (the CloudWatch agent, cAdvisor).

- Do size at p95 or p99 with explicit headroom.
- Do change gradually and verify SLOs.
- Do automate a quarterly review and record savings.
- Do consider changing family or architecture, not only size.

9 Tasks

- [ ] **T1 — Run the rightsizing analysis** *(≈20 min, beginner)*
  - **Goal:** classify instances from numbers.
  - **Steps:**
    1. Run `rightsize.py`.
    2. Change the `i-batch-1` samples to steady 30 percent CPU and re-run.
  - **Done when:** the batch host moves from "keep" to a smaller size and you can explain why p95 drove the first result.
- [ ] **T2 — Add a memory-bound case** *(≈30 min, intermediate)*
  - **Goal:** see why smaller is not always right.
  - **Steps:**
    1. Add a memory-optimized entry to `LADDER` (for example 4 vCPU, 32 GiB at a price you choose).
    2. Re-run the database case and decide if the cheaper option exists.
  - **Done when:** the script suggests a memory-optimized size for the database or you can explain why it does not.
- [ ] **T3 — Use Compute Optimizer** *(≈40 min, intermediate; AWS sandbox)*
  - **Goal:** read a real recommendation.
  - **Steps:**
    1. Opt in and leave a test instance running with the CloudWatch agent.
    2. Query recommendations after a day.
    3. Record the finding, the proposed type and the performance risk.
  - **Done when:** you have one recommendation and a note on whether you would act, and why. Terminate the test instance afterwards.
- [ ] **T4 — Run a VPA in recommendation mode** *(≈45 min, intermediate)*
  - **Goal:** compare requests with VPA targets.
  - **Steps:**

1. Apply the VPA with `updateMode: "Off"`.
2. Generate light load and read the recommendation.

- **Done when:** you can compare the current request with `Target` and compute the ratio.

☐ **T5 — Break it: shrink memory too far** *(≈30 min, intermediate)*

- **Goal:** observe an OOM kill and recover.
- **Steps:**
  1. Set a pod's memory limit to 20Mi with a memory-hungry command (for example `stress`-like allocation).
  2. Observe `OOMKilled` in `kubectl describe pod`.
  3. Fix the limit and roll back with `kubectl rollout undo`.
- **Done when:** you show the `OOMKilled` reason and the recovered healthy pod.

☐ **T6 — Prove it** *(≈45 min, advanced)*

- **Goal:** a rightsizing proposal with risk controls.
- **Steps:**
  1. Without the guide, choose three workloads, list the metrics, window and percentile you would use.
  2. Write the rollout and rollback plan for one of them.
  3. Estimate the monthly saving using example prices.
- **Done when:** a reviewer can follow your plan and the saving is traceable to numbers.

10 Self-check questions

1. Why is the average a poor basis for rightsizing?
2. What does Compute Optimizer need to include memory in recommendations?
3. What does the VPA `updateMode: "Off"` do?
4. Give two ways a rightsizing change can cause an incident.
5. Why rightsize before buying a Savings Plan?

ANSWERS

1. It hides peaks; a service averaging 15 percent CPU may hit 90 percent at known times. Use p95 or p99 over a window that includes peaks.
2. Memory metrics, usually from the CloudWatch agent; without them recommendations rely on CPU, network and disk metrics.
3. It computes and shows recommendations but does not evict pods or apply changes.

4. OOM kills from too little memory, CPU throttling or credit exhaustion, or a missed seasonal peak.
5. Otherwise you commit to capacity you do not need and lock in the waste for one to three years.

11 Further reading

- AWS documentation: AWS Compute Optimizer User Guide, CloudWatch agent, EC2 instance types.
- Kubernetes autoscaler project documentation: Vertical Pod Autoscaler.
- Fairwinds Goldilocks documentation.
- *Site Reliability Engineering* (Google) chapters on capacity planning and load.

20.6 Storage Lifecycle Policies

Topic 6 of 9 | ~4 hours | Difficulty: Intermediate

1 What it is

Storage is cheap per gigabyte and expensive in aggregate, because nothing deletes itself. Logs, build artifacts, database snapshots, container images, old object versions and half-finished uploads accumulate forever unless a rule removes them. A **lifecycle policy** is that rule: it moves data to cheaper storage classes as it ages and deletes it when it is no longer needed.

Three families of lifecycle controls cover most of a bill:

Area	Mechanism	What it does
Object storage	S3 lifecycle rules, S3 Intelligent-Tiering	transition objects between storage classes, expire old objects and versions
Block storage	EBS snapshot lifecycle (Amazon Data Lifecycle Manager), AWS Backup	create and retain a fixed number of snapshots, delete the rest
Logs and registries	CloudWatch Logs retention, Loki or Elasticsearch retention, ECR lifecycle policies	stop keeping data past its useful life

2 Why it matters in DevOps

- Unbounded growth is silent. A bucket that gains a few hundred gigabytes a month never triggers an alert, but after two years it is a large line item.
- CloudWatch Logs groups are created with **no expiration** by default, so log volume you stopped reading years ago is still billed every month.
- Compliance sets a floor, not a ceiling: you may *have* to keep audit logs for a year, but not for ten.
- Wrong lifecycle rules lose data or add cost: moving tiny objects to an infrequent-access class can cost more than leaving them, and deleting a snapshot you needed is an outage.
- Related: S3 §6.8, EBS and EFS §6.9, CloudWatch §6.18, Backups §15.1 and Log Aggregation with Loki §10.6.

3 Real-world system example

Amazon S3 storage classes are a published ladder of price versus retrieval: S3 Standard for frequent access, Standard-IA and One Zone-IA for infrequent access, Glacier Instant Retrieval, Glacier Flexible Retrieval and Glacier Deep Archive for archives. AWS documents limits that matter when you design a rule: infrequent-access and archive classes have **minimum storage durations** (30 days for the IA classes, 90 for Glacier Instant and Flexible Retrieval, 180 for Deep Archive), the IA classes have a **128 KB minimum billable object size**, and retrieval from colder classes costs per GB. **S3 Intelligent-Tiering** moves objects between access tiers automatically based on observed access, charges a small monitoring and automation fee per object, and has no retrieval charge for its automatic tiers.

Amazon Data Lifecycle Manager automates EBS snapshot creation and retention by tag. **Grafana Loki** and other log systems expose a retention setting so old chunks are deleted, and **Amazon ECR** supports lifecycle policies that expire images by count or age. The principle is the same everywhere: define the useful life of the data, then automate the transition and the deletion.

Treat every price in this topic as an example and check current pricing.

4 Diagram

An object's life under a lifecycle policy:

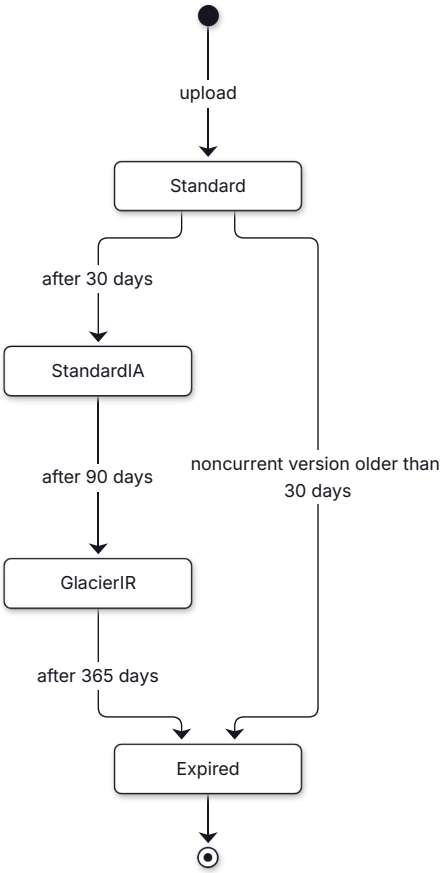


Figure 20.6-1

Where to apply which control:

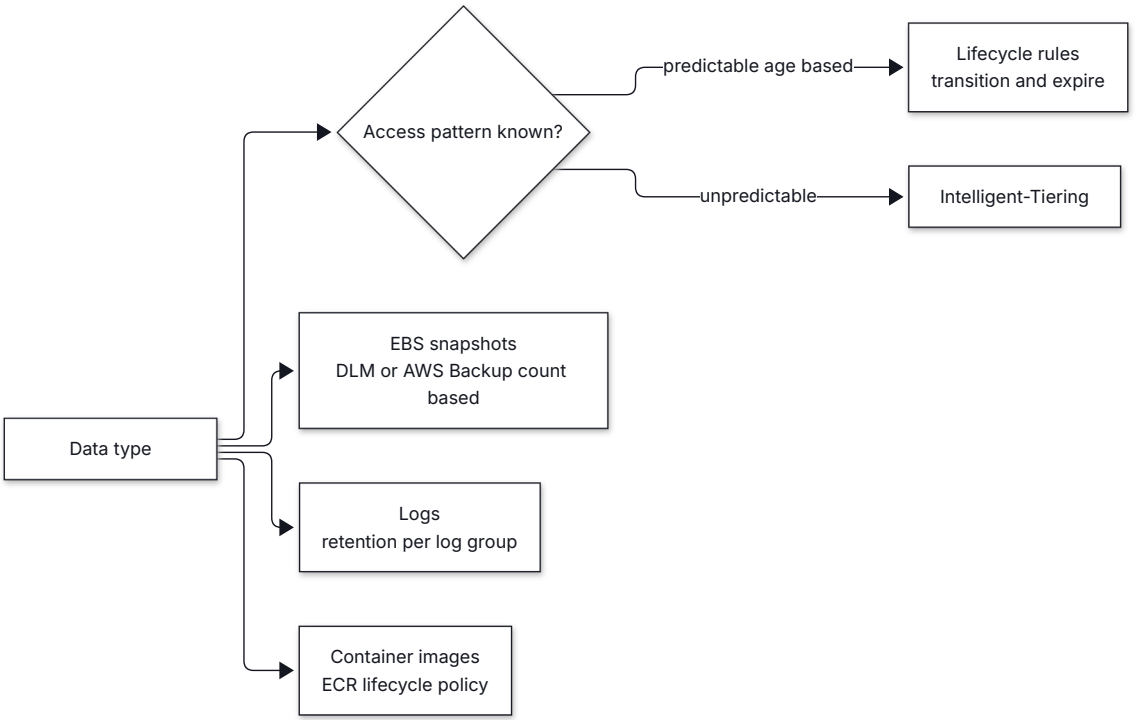


Figure 20.6-2

5 How it works

S3 lifecycle rules

A rule has a **filter** (prefix, tags, object size), **transitions** (move after N days to a class) and **expiration** (delete after N days). Separate settings control **noncurrent versions** in versioned buckets and **incomplete multipart uploads** (invisible partial uploads that keep costing money). Transitions run asynchronously, roughly daily.

Rule element	Purpose
`Transitions`	move current objects to a cheaper class after N days
`Expiration`	delete current objects (or add a delete marker in a versioned bucket)
`NoncurrentVersionExpiration`	delete old versions after N days
`AbortIncompleteMultipartUpload`	remove abandoned multipart upload parts
`Filter`	scope by `Prefix`, `Tag`, `ObjectSizeGreaterThan`, `ObjectSizeLessThan`

Tier or Intelligent-Tiering?

Choice	Use when
Explicit lifecycle transitions	access is predictable by age (logs are rarely read after a week)
Intelligent-Tiering	access is unknown or changes (user uploads), objects are larger than 128 KB and live for more than a month
Leave it in Standard	objects are tiny, short-lived or read constantly

Retrieval costs matter: if you read the dataset frequently, the colder class can cost more than Standard.

Snapshots, logs and images

- **EBS snapshots** are incremental, but deleting the middle of a chain is safe: AWS keeps the data other snapshots need. Cost still accumulates from forgotten snapshots of deleted volumes and old AMIs.
- **Log retention** should be set per log group: days for debug logs, months for application logs, a year or more for audit logs (follow your compliance obligation).
- **ECR lifecycle policies** keep the last N images and expire untagged ones.

6 Step-by-step guide

Prerequisites: Python 3 for the first step; the AWS CLI and a sandbox account for the rest. Storage in the exercise is a few small test objects, so the cost is negligible; still clean up at the end. Prices are **EXAMPLES**: check current pricing.

1. **Compare storage classes for 10 TB.** Save as `tiers.py`:

```python
# tiers.py - S3 storage-class comparison for 10 TB. EXAMPLE prices only (us-east-1
  style), check current pricing.
tb = 10
gb = tb * 1024
prices = {   # $/GB-month, retrieval $/GB, min storage days
    "STANDARD":       (0.023, 0.00, 0),
    "STANDARD_IA":    (0.0125, 0.01, 30),
    "GLACIER_IR":     (0.004, 0.03, 90),
    "DEEP_ARCHIVE":   (0.00099, 0.02, 180),
}
print(f"{'class':<14}{'storage/mo':>12}{'retrieve 1x':>13}{'min days':>10}")
for name, (store, retr, mind) in prices.items():
    print(f"{name:<14}${store * gb:>10,.2f} ${retr * gb:>11,.2f}{mind:>10}")
# how often can you read the full dataset per month before IA costs more than
  STANDARD?
```

```
std, ia = prices["STANDARD"], prices["STANDARD_IA"]
saved = (std[0] - ia[0]) * gb
per_read = ia[1] * gb
print(f"\nSTANDARD_IA saves ${saved:,.2f}/month; each full read costs
  ${per_read:,.2f}")
print(f"break-even: reading the whole dataset {saved / per_read:.2f}x per month")
```

BASH

```
python3 tiers.py
```

Expected output:

```
class           storage/mo  retrieve 1x  min days
STANDARD      $    235.52 $       0.00         0
STANDARD_IA   $    128.00 $     102.40        30
GLACIER_IR    $     40.96 $     307.20        90
DEEP_ARCHIVE  $     10.14 $     204.80       180

STANDARD_IA saves $107.52/month; each full read costs $102.40
break-even: reading the whole dataset 1.05x per month
```

The point: with these example prices Standard-IA only wins if you read the entire dataset about once a month or less. Archives win only for data you almost never touch.

2. **Write an S3 lifecycle configuration.** Save as `lifecycle.json`:

JSON

```
{
  "Rules": [
    {
      "ID": "logs-tiering-and-expiry",
      "Status": "Enabled",
      "Filter": {"Prefix": "logs/"},
      "Transitions": [
        {"Days": 30, "StorageClass": "STANDARD_IA"},
        {"Days": 90, "StorageClass": "GLACIER_IR"}
      ],
      "Expiration": {"Days": 365}
    },
    {
      "ID": "cleanup-noncurrent-and-multipart",
      "Status": "Enabled",
      "Filter": {},
      "NoncurrentVersionExpiration": {"NoncurrentDays": 30},
      "AbortIncompleteMultipartUpload": {"DaysAfterInitiation": 7}
    },
    {
      "ID": "uploads-intelligent-tiering",
```

```
      "Status": "Enabled",
      "Filter": {"Prefix": "uploads/"},
      "Transitions": [{"Days": 0, "StorageClass": "INTELLIGENT_TIERING"}]
    }
  ]
}
```

Check the JSON parses before applying it:

```bash
python3 -c "import json; print(len(json.load(open('lifecycle.json'))['Rules']),
  'rules')"
```

Expected: `3 rules`.

3. **Apply it to a test bucket.** (Bucket names are global; use your own.)

```bash
BUCKET=shop-lifecycle-lab-<YOUR_SUFFIX>
aws s3api create-bucket --bucket "$BUCKET" --region us-east-1
aws s3api put-bucket-versioning --bucket "$BUCKET" --versioning-configuration
  Status=Enabled
aws s3api put-bucket-lifecycle-configuration --bucket "$BUCKET" \
  --lifecycle-configuration file://lifecycle.json
aws s3api get-bucket-lifecycle-configuration --bucket "$BUCKET" \
  --query 'Rules[*].[ID,Status]' --output table
```

The table lists the three rule IDs with `Enabled`. Transitions do not happen immediately; they run based on object age.

4. **Find the log groups that never expire.**

```bash
aws logs describe-log-groups \
  --query 'logGroups[?retentionInDays==`null`].[logGroupName,storedBytes]' --output
  table
```

Every row is a log group billed forever. Set a retention period for one:

```bash
aws logs put-retention-policy --log-group-name /aws/lambda/shop-api --retention-in-
  days 30
```

Valid values are a fixed list (1, 3, 5, 7, 14, 30, 60, 90, 120, 150, 180, 365 and more); an invalid number returns an error.

5. **Automate EBS snapshots with Data Lifecycle Manager.** Save the policy details as `dlm.json`:

```json
{
  "ResourceTypes": ["VOLUME"],
  "TargetTags": [{"Key": "backup", "Value": "daily"}],
  "Schedules": [{
    "Name": "daily-keep-14",
    "CreateRule": {"Interval": 24, "IntervalUnit": "HOURS", "Times": ["03:00"]},
    "RetainRule": {"Count": 14},
    "CopyTags": true
  }]
}
```

```bash
aws dlm create-lifecycle-policy \
  --description "daily snapshots keep 14" \
  --state ENABLED \
  --execution-role-arn arn:aws:iam::
  <YOUR_ACCOUNT_ID>:role/AWSDataLifecycleManagerDefaultRole \
  --policy-details file://dlm.json
```

   The command returns a `PolicyId`. Volumes tagged `backup=daily` now get a snapshot at 03:00 UTC daily and only the latest 14 are kept. The role must exist (create it with `aws dlm create-default-role` if needed).

6. **Find orphaned snapshots.**

```bash
aws ec2 describe-snapshots --owner-ids self \
  --query "Snapshots[?StartTime<='2026-07-01'].
  [SnapshotId,StartTime,VolumeSize,Description]" --output table
```

   Check each against live volumes and AMIs before deleting (`aws ec2 describe-images --owners self`); a snapshot used by an AMI cannot be deleted until the AMI is deregistered. Delete with `aws ec2 delete-snapshot --snapshot-id <ID>`.

7. **Expire old container images with an ECR lifecycle policy.** Save as `ecr-policy.json`:

```json
{
  "rules": [
    {
      "rulePriority": 1,
      "description": "Keep the last 20 images",
      "selection": {"tagStatus": "any", "countType": "imageCountMoreThan",
  "countNumber": 20},
      "action": {"type": "expire"}
    }
  ]
}
```

```bash
aws ecr put-lifecycle-policy --repository-name shop-api --lifecycle-policy-text
  file://ecr-policy.json
```

   The response echoes the policy. ECR evaluates policies about once a day and the preview command `aws ecr start-lifecycle-policy-preview` shows what would be expired without deleting.

8. **Clean up the lab bucket.**

```bash
aws s3 rb "s3://$BUCKET" --force
```

   If you uploaded test objects to the versioned bucket, `--force` removes only the current objects, so `rb` fails while old versions or delete markers remain. Use the console "Empty bucket" action, which removes all versions in one step, then delete the bucket.

7 Real-world scenario

(Illustrative.) A team notices S3 is 22 percent of the bill. One bucket holds 90 TB of CloudFront and ALB access logs, kept forever because "we might need them". An engineer proposes moving everything to Glacier Deep Archive immediately. The numbers look good until they read the minimum duration (180 days) and the retrieval rules: any security investigation would need a slow restore and a fee.

The team settles on a tiered rule: 30 days Standard, then Glacier Instant Retrieval until 180 days, then expire at one year (the audit requirement), plus a lifecycle rule for incomplete multipart uploads that cleans 4 TB of orphan parts nobody knew existed. They test the poli-

cy on one prefix first. Monthly storage cost falls by roughly half, and the audit team confirms a one-year window meets the policy.

8 Common mistakes and best practices

Don't

- Don't move small objects (under 128 KB) to infrequent-access classes; the minimum billable size makes them cost more.
- Don't archive data you query often; retrieval fees and delays erase the saving.
- Don't delete without checking compliance, legal hold and restore needs.
- Don't leave CloudWatch log groups on "never expire".
- Don't forget versioned buckets: delete markers and noncurrent versions keep costing money.

Do

- Do start with a filtered prefix, observe, then widen the rule.
- Do add `AbortIncompleteMultipartUpload` to every bucket.
- Do use Intelligent-Tiering for unpredictable access and large objects.
- Do set retention in infrastructure as code, with a default for every log group.
- Do review storage with S3 Storage Lens or Storage Class Analysis before choosing thresholds.

9 Tasks

- [ ] **T1 — Compare classes with numbers** *(≈20 min, beginner)*
  - **Goal:** know the break-even read frequency.
  - **Steps:**
    1. Run `tiers.py`.
    2. Change the dataset to 1 TB and Standard-IA retrieval to 0.02 and compare.
  - **Done when:** you can state the break-even reads per month for both inputs.
- [ ] **T2 — Build and apply a lifecycle policy** *(≈40 min, intermediate; AWS sandbox)*
  - **Goal:** a policy live on a bucket.
  - **Steps:**
    1. Create the lab bucket with versioning.
    2. Apply `lifecycle.json` and read it back.

- **Done when:** `get-bucket-lifecycle-configuration` lists all three rules.

- [ ] **T3 — Set log retention across an account** *(≈40 min, intermediate)*
  - **Goal:** no log group without retention.
  - **Steps:**
    1. List groups with no retention.
    2. Write a short loop that sets 30 days on non-audit groups (excluding names that contain `audit`).
  - **Done when:** the listing query returns only the groups you deliberately excluded.

- [ ] **T4 — Break it: a rule that costs more** *(≈30 min, intermediate)*
  - **Goal:** show a bad transition.
  - **Steps:**
    1. Model 10 million 20 KB objects moved to Standard-IA using the 128 KB minimum billable size.
    2. Compare the billed size to Standard.
  - **Done when:** you show the IA bill is higher than Standard for that dataset and write the `ObjectSizeGreaterThan` filter that prevents it.

- [ ] **T5 — Automate snapshot retention** *(≈45 min, advanced)*
  - **Goal:** hands-off snapshot cleanup.
  - **Steps:**
    1. Tag a test volume `backup=daily` and apply the DLM policy.
    2. Run `aws ec2 describe-snapshots --owner-ids self --filters Name=tag:backup,Values=daily` after the first scheduled run.
  - **Done when:** snapshots appear and the policy shows `RetainRule` of 14; delete the policy and test volume afterwards.

- [ ] **T6 — Prove it** *(≈45 min, advanced)*
  - **Goal:** write a storage retention plan.
  - **Steps:**
    1. For `shop-api`, list logs, uploads, backups, images and artifacts, and set an age-based plan for each.
    2. Justify each threshold with access pattern and compliance.
    3. Show the rules as code.
  - **Done when:** a reviewer can apply your policies and the plan names one risk and one rollback per data type.

10 Self-check questions

1. Why can moving tiny objects to Standard-IA increase the bill?
2. What does `AbortIncompleteMultipartUpload` clean up?
3. When is Intelligent-Tiering better than explicit rules?
4. What is the default retention of a CloudWatch Logs group?
5. Why test a lifecycle rule on one prefix first?

ANSWERS

1. The IA classes bill a 128 KB minimum per object, so objects smaller than that are charged as if they were 128 KB.
2. Abandoned upload parts that are not visible as objects but still take storage space.
3. When access patterns are unknown or change, and objects are large and long-lived enough to justify the per-object monitoring fee.
4. Never expire.
5. Lifecycle actions, especially expiration, are not easily reversible; a small scope limits the impact of a mistake.

11 Further reading

- AWS documentation: Amazon S3 User Guide (Managing storage lifecycle, Storage classes, Intelligent-Tiering), Amazon Data Lifecycle Manager, CloudWatch Logs retention, Amazon ECR lifecycle policies.
- S3 Storage Lens and Storage Class Analysis documentation.
- Grafana Loki documentation: retention.

20.7 Network Egress Costs

Topic 7 of 9 | ~4 hours | Difficulty: Intermediate

1 What it is

Cloud providers charge for moving data, and the charge depends on **where the bytes go**. Traffic *into* AWS from the internet is generally free. Traffic *out* to the internet, between Regions and even between **Availability Zones (AZs)** in one Region is billed per gigabyte. Some services add their own processing fee on top: a **NAT gateway** charges both an hourly price and a per-GB data-processing price for everything that passes through it.

Because it is metered in the background, network cost is the line item most often discovered after the fact. Nobody chooses to spend money on a NAT gateway; it happens because a container image pull, an S3 download or a chatty service call routes through a paid hop.

The main cost paths to know:

Path	Billed how (pattern, check current pricing)
Internet to AWS	usually free
AWS to internet	per GB, tiered, with a small monthly free allowance
Region to Region	per GB, cheaper than internet
AZ to AZ in one Region	per GB, charged in each direction
Through a NAT gateway	hourly price plus per-GB processing, on top of transfer charges
VPC gateway endpoint (S3, DynamoDB)	no hourly or data processing charge
VPC interface endpoint (PrivateLink)	hourly price per AZ plus per-GB processing
Origin to CloudFront	free from AWS origins; CloudFront to viewers is billed per GB

2 Why it matters in DevOps

- A multi-AZ design (the recommended availability pattern) turns every cross-zone call into a charge. Chatty microservices and replicated datastores can quietly add four-figure monthly bills.

- NAT gateways are a classic surprise: private subnets pulling images, packages and S3 objects all pay per GB.
- Architecture decisions made for resilience (multi-AZ, multi-Region replication) have a data-transfer price. Your design needs to include it.
- Egress also shapes vendor choices: moving large datasets out of a cloud, or serving video straight from a compute fleet, is expensive.
- Related: Route Tables, Internet Gateway and NAT Gateway §6.5, VPC and Subnets §6.4, CloudFront §6.13, CDNs §12.7 and Cost §19.5.

3 Real-world system example

AWS data transfer pricing and VPC endpoints. AWS publishes the data transfer price structure for EC2: free inbound, tiered outbound to the internet, a per-GB rate between Regions and a per-GB rate in each direction between AZs. AWS also documents **gateway endpoints** for S3 and DynamoDB, which are free and route traffic over the AWS network instead of through a NAT gateway, and **interface endpoints** (AWS PrivateLink) for other services, which cost an hourly fee per AZ plus a data processing charge. The AWS Well-Architected cost pillar specifically calls out data transfer costs and the use of CDNs and endpoints to reduce them.

CloudFront is documented as free for data transfer from AWS origins (S3, ALB, EC2) to CloudFront, with CloudFront-to-viewer prices that are tiered and include a monthly free allowance, so a high cache hit ratio reduces both origin load and the volume billed at the more expensive origin egress rates. Many public engineering write-ups describe the same lesson: caching and keeping traffic inside a zone is cheaper than moving bytes.

4 Diagram

Where egress charges appear for a `shop-api` stack:

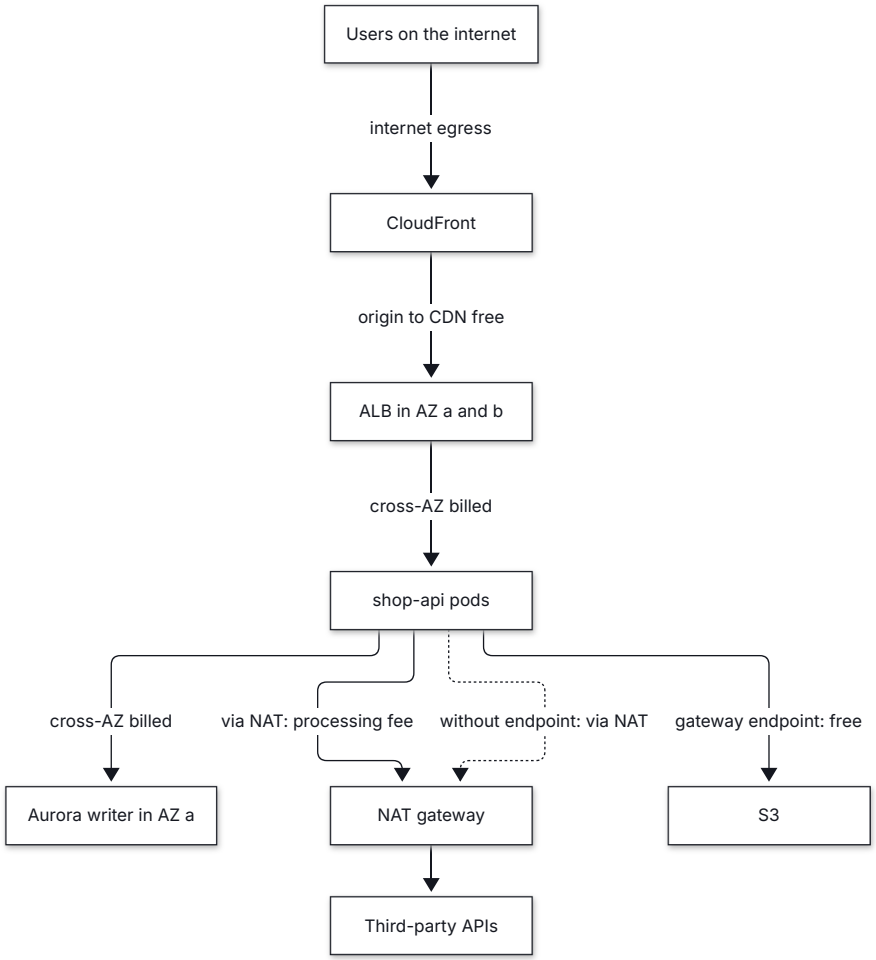


Figure 20.7-1

Gateway endpoint versus NAT for S3 traffic:

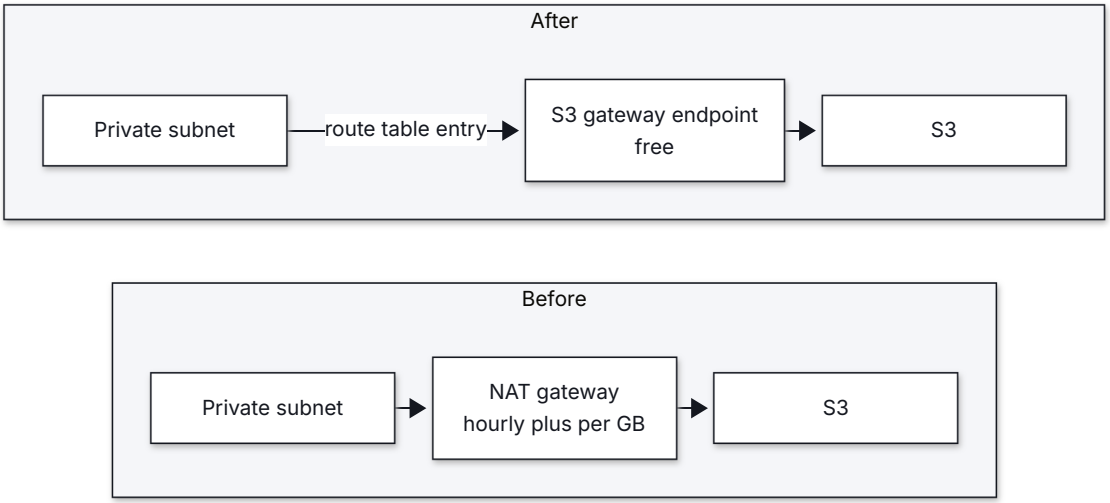


Figure 20.7-2

5 How it works

NAT gateway

A NAT gateway lets instances in private subnets reach the internet. Costs: an hourly charge per gateway (you usually run one per AZ for resilience) plus a per-GB charge for **all** data it processes, in either direction. Common hidden consumers:

- Image pulls from public registries and ECR.
- Package manager updates and CI downloads.
- Access to S3, DynamoDB and other AWS APIs (avoidable with endpoints).
- Calls to SaaS APIs and telemetry vendors.

Endpoints

Endpoint type	Services	Cost model	Use for
Gateway	S3 and DynamoDB	free	any private subnet that talks to S3 or DynamoDB
Interface (PrivateLink)	ECR, CloudWatch, STS, Secrets Manager, SQS, and many others	hourly per AZ plus per-GB	high-volume AWS API traffic; also a security benefit

An interface endpoint pays off only above a volume threshold, because of its fixed hourly cost per AZ. Compute the break-even before adding a dozen of them.

Inter-AZ traffic

Common sources: a load balancer sending requests to targets in other zones (cross-zone load balancing), a service calling another service in a different AZ, database replication, and reads hitting a replica in another zone. Mitigations: topology-aware routing in Kubernetes (zone-aware service routing), keeping chatty pairs co-located, reading from a same-zone replica, and checking whether cross-zone load balancing is needed for each load balancer. Do not give up multi-AZ resilience to save these costs; reduce the unnecessary part.

Internet and CDN offload

Serving large static content (images, video, downloads) from a CDN lowers egress cost per GB, adds a free allowance, reduces origin requests and improves latency. A cache hit ratio of 85-95 percent is a typical target for static assets (the achievable ratio depends on your content). Compression (gzip, Brotli) and image optimization reduce bytes at the source.

6 Step-by-step guide

Prerequisites: Python 3 and `opa` (or `conftest`) for the offline steps; an AWS sandbox for the rest. Endpoints and NAT gateways cost money while they exist, so create only what you need and run the cleanup. All prices are **EXAMPLES**; check current pricing.

1. **Model a month of network cost.** Save as `egress.py`:

```python
# egress.py - monthly network cost model for shop-api. All prices are EXAMPLES; check current pricing.
price = {
    "nat_hour": 0.045,        # $/hour per NAT gateway
    "nat_gb": 0.045,          # $/GB processed by NAT gateway
    "inter_az_gb": 0.01,      # $/GB each direction between AZs in a Region
    "internet_gb": 0.09,      # $/GB to the internet (first tier, ignoring free allowance)
    "cdn_gb": 0.085,          # $/GB from the CDN to viewers (example, varies by geography)
}
hours = 730

# 1. NAT: 3 AZs, each with a NAT gateway; 8 TB/month of S3 and ECR pulls leave private subnets
nat_gb_month = 8 * 1024
nat_fixed = 3 * price["nat_hour"] * hours
nat_var = nat_gb_month * price["nat_gb"]
print(f"NAT gateways fixed        : ${nat_fixed:>9,.2f}")
print(f"NAT data processing       : ${nat_var:>9,.2f}  ({nat_gb_month} GB)")
print(f"with S3 gateway endpoint  : ${nat_fixed + nat_var * 0.25:>9,.2f}  (if 75% of that traffic is S3, endpoint is free)")
```

```
# 2. Chatty cross-AZ service calls: 5 TB/month crossing AZs, charged in both
  directions
x_gb = 5 * 1024
print(f"\ncross-AZ traffic          : ${x_gb * price['inter_az_gb'] * 2:>9,.2f}
  ({x_gb} GB, both directions)")

# 3. Serving 20 TB/month of images from the origin vs through a CDN
out_gb = 20 * 1024
direct = out_gb * price["internet_gb"]
cdn_free_gb = 1024            # example: first 1 TB/month free on the CDN
cdn = (out_gb - cdn_free_gb) * price["cdn_gb"]
hit = 0.85                    # cache hit ratio; origin only serves the misses
print(f"\n20 TB direct to internet  : ${direct:>9,.2f}")
print(f"20 TB via CDN             : ${cdn:>9,.2f}  (origin→CDN transfer is free when
  the origin is in the same cloud)")
print(f"origin traffic and compute drop to {1 - hit:.0%} of the previous load; that
  saving is not priced here")
```

BASH

```
python3 egress.py
```

Expected output:

```
NAT gateways fixed        : $    98.55
NAT data processing       : $   368.64  (8192 GB)
with S3 gateway endpoint  : $   190.71  (if 75% of that traffic is S3, endpoint is
  free)

cross-AZ traffic          : $   102.40  (5120 GB, both directions)

20 TB direct to internet  : $ 1,843.20
20 TB via CDN             : $ 1,653.76  (origin→CDN transfer is free when the origin
  is in the same cloud)
origin traffic and compute drop to 15% of the previous load; that saving is not priced
  here
```

The NAT line shows the pattern: the per-GB processing charge dwarfs the hourly one. For the CDN line, the visible price difference is small; most of the benefit is the work your origin no longer does.

2. **Enforce a gateway endpoint with policy.** A Conftest/OPA rule that fails a Terraform plan which creates a VPC and NAT gateway without an S3 gateway endpoint. Save as `endpoints.rego`:

REGO

```
package main
```

```
import rego.v1

created(type) := {rc.address |
  some rc in input.resource_changes
  rc.type == type
  "create" in rc.change.actions
}

s3_endpoints := {rc.address |
  some rc in input.resource_changes
  rc.type == "aws_vpc_endpoint"
  endswith(rc.change.after.service_name, ".s3")
}

deny contains msg if {
  some vpc in created("aws_vpc")
  some nat in created("aws_nat_gateway")
  count(s3_endpoints) == 0
  msg := sprintf("%s has NAT gateway %s but no S3 gateway endpoint: S3 traffic will be
  billed per GB by NAT", [vpc, nat])
}
```

A minimal fake plan `plan_bad.json`:

JSON

```
{"resource_changes": [
  {"address": "aws_vpc.main", "type": "aws_vpc", "change": {"actions": ["create"],
  "after": {"cidr_block": "10.0.0.0/16"}}},
  {"address": "aws_nat_gateway.a", "type": "aws_nat_gateway", "change": {"actions":
  ["create"], "after": {}}}
]}
```

BASH

```
opa eval -f pretty -d endpoints.rego -i plan_bad.json 'data.main.deny'
```

Expected output:

```
[
  "aws_vpc.main has NAT gateway aws_nat_gateway.a but no S3 gateway endpoint: S3
  traffic will be billed per GB by NAT"
]
```

Add an `aws_vpc_endpoint` resource with `service_name` ending in `.s3` to the plan and the result becomes `[]`. With Conftest the same file works as `conftest test --policy . plan_bad.json`.

3. **Find out how much your NAT gateways carry.** In CloudWatch, the NAT gateway metric `BytesOutToDestination` (traffic to the internet) and `BytesInFromDestination` (responses) show volume:

```bash
aws cloudwatch get-metric-statistics --namespace AWS/NATGateway \
  --metric-name BytesOutToDestination \
  --dimensions Name=NatGatewayId,Value=<NAT_GATEWAY_ID> \
  --start-time 2026-09-01T00:00:00Z --end-time 2026-10-01T00:00:00Z \
  --period 2592000 --statistics Sum
```

   The `Sum` value is bytes for the month; divide by 1,073,741,824 for GB and multiply by the NAT processing price. To see *who* is talking, enable VPC Flow Logs on the NAT's network interface and query by source address and destination (Athena or CloudWatch Logs Insights).

4. **Add a free S3 gateway endpoint.** In Terraform:

```hcl
resource "aws_vpc_endpoint" "s3" {
  vpc_id            = aws_vpc.main.id
  service_name      = "com.amazonaws.us-east-1.s3"
  vpc_endpoint_type = "Gateway"
  route_table_ids   = [aws_route_table.private.id]
}
```

   Or with the CLI:

```bash
aws ec2 create-vpc-endpoint --vpc-id <VPC_ID> --service-name com.amazonaws.us-east-
  1.s3 \
  --vpc-endpoint-type Gateway --route-table-ids <PRIVATE_ROUTE_TABLE_ID>
```

   Afterwards the private route table has a route to a prefix list (`pl-...`) targeting the endpoint. Verify from a private instance: `aws s3 ls` still works, and NAT `BytesOutToDestination` for S3 traffic falls.

5. **Decide on an interface endpoint with arithmetic.** An interface endpoint has an hourly charge per AZ. With an example $0.01 per hour per AZ and $0.01 per GB processed, three AZs cost about 3 × 0.01 × 730 = $21.90 per month in fixed fees. The NAT route costs $0.045 per GB; the endpoint route costs $0.01 per GB. The saving per GB is $0.035, so the break-even is 21.90 / 0.035, about 626 GB per month through the endpoint. Only add it above that volume (or for security reasons).

6. **Reduce cross-AZ traffic in Kubernetes.** Ask for zone-aware routing on a Service. Kubernetes supports the annotation `service.kubernetes.io/topology-mode: Auto` on a Service, which tries to keep traffic within the caller's zone when there are enough endpoints:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: shop-api
  annotations:
    service.kubernetes.io/topology-mode: Auto
spec:
  selector: {app: shop-api}
  ports:
    - port: 80
      targetPort: 8080
```

```bash
kubectl apply -f shop-api-svc.yaml
kubectl get endpointslice -l kubernetes.io/service-name=shop-api -o yaml | grep -A2
  hints
```

When hints are enabled you see `forZones` entries in the EndpointSlice (this needs endpoints spread across zones; with too few endpoints the hints are not applied). Check that load stays balanced, since zone routing can overload a smaller zone.

7. **Offload to a CDN and compress.** Put CloudFront in front of the static origin, and enable compression:

```bash
curl -sI -H 'Accept-Encoding: br, gzip' https://example.com/static/app.js | grep -i -E
  'content-encoding|x-cache|age'
```

You want `content-encoding: br` or `gzip`, and on repeat requests `x-cache: Hit from cloudfront` with a growing `age`. In CloudFront reports, watch the *cache hit rate*; below roughly 80 percent for static assets, look for query strings, cookies or `Vary` headers that fragment the cache. (Replace `example.com` with your own distribution domain.)

8. **Clean up.** Delete anything you created: the endpoint (`aws ec2 delete-vpc-endpoints --vpc-endpoint-ids <ID>`), NAT gateways (`aws ec2 delete-nat-gateway --nat-gateway-id <ID>` and release the Elastic IP), the CloudFront distribution (disable, then delete) and test buckets.

7 Real-world scenario

(Illustrative.) Finance flags a 60 percent month-on-month rise in "EC2-Other". Cost Explorer shows NAT gateway data processing as the driver. Flow logs show that most NAT traffic is a nightly job pulling a few terabytes from S3 and a Kubernetes cluster pulling container images from a public registry on every new node.

The team adds an S3 gateway endpoint (an hour of work, zero ongoing cost), mirrors images to ECR with an interface endpoint for the highest-volume cluster, and caches images on nodes. NAT processing falls by about two thirds, and they add the endpoint policy check from step 2 so new VPCs are created with the endpoint by default. Cross-AZ costs remain, so they schedule a separate review of zone-aware routing.

8 Common mistakes and best practices

Don't

- Don't send S3 and DynamoDB traffic through a NAT gateway when a free gateway endpoint exists.
- Don't create interface endpoints for every service by default; each has fixed hourly fees per AZ.
- Don't disable multi-AZ resilience to save transfer fees; reduce unnecessary cross-zone calls instead.
- Don't serve large static files straight from application servers.
- Don't forget the other side: cross-Region replication and backups cost transfer on every byte.

Do

- Do check NAT metrics and flow logs when "EC2-Other" or data transfer grows.
- Do add gateway endpoints to every VPC template (see Infrastructure Templates §18.6).
- Do use a CDN with a healthy cache hit ratio and compress responses.
- Do co-locate chatty services and use zone-aware routing where it fits.
- Do put data-transfer estimates in design reviews.

9 Tasks

- [ ] **T1 — Model your network bill** *(≈25 min, beginner)*
  - **Goal:** price each path.
  - **Steps:**

1. Run `egress.py`.
2. Change the NAT volume to 30 TB and the cross-AZ volume to 50 TB.

- **Done when:** you can name the largest cost and explain which of the three levers (endpoint, topology, CDN) addresses it.

- [ ] **T2 — Enforce an endpoint with policy** *(≈30 min, intermediate)*
  - **Goal:** the policy fails and then passes.
  - **Steps:**
    1. Run `opa eval` on `plan_bad.json`.
    2. Add a gateway endpoint resource to the plan and re-run.
  - **Done when:** the first run prints one message and the second prints `[]`.
- [ ] **T3 — Find NAT volume** *(≈40 min, intermediate; AWS sandbox)*
  - **Goal:** measure real NAT traffic.
  - **Steps:**
    1. Query `BytesOutToDestination` for a NAT gateway.
    2. Convert to GB and cost at your NAT processing price.
  - **Done when:** you have a monthly GB figure and a dollar estimate (or you explain that the sandbox has no NAT).
- [ ] **T4 — Break it: remove the endpoint** *(≈30 min, intermediate)*
  - **Goal:** watch traffic fall back onto NAT.
  - **Steps:**
    1. From a private instance, download a large S3 object with and without the endpoint route.
    2. Compare NAT metrics for the two runs.
  - **Done when:** NAT bytes rise only when the endpoint is removed. Remove the endpoint and the NAT afterwards to avoid ongoing cost.
- [ ] **T5 — Compute an interface endpoint break-even** *(≈30 min, advanced)*
  - **Goal:** decide with numbers.
  - **Steps:**
    1. Use your own prices for hourly, per-GB and NAT rates.
    2. Compute break-even GB per month for 2 and for 3 AZs.
  - **Done when:** you have a table and a recommendation for a service at 100 GB per month and at 2 TB per month.

- [ ] **T6 — Prove it** *(≈45 min, advanced)*
  - **Goal:** audit a design for egress.
  - **Steps:**
    1. Draw `shop-api` with its traffic paths and mark which hops are billed.
    2. Propose three changes with estimated savings.
    3. Explain why multi-AZ stays.
  - **Done when:** the audit lists billed hops and a reviewer agrees with the three changes.

10 Self-check questions

1. Which direction of internet traffic is usually free, and which is billed?
2. What two charges does a NAT gateway add?
3. Why is an S3 gateway endpoint a no-brainer when a NAT gateway exists?
4. What makes an interface endpoint worth it?
5. How can Kubernetes reduce cross-AZ traffic?

ANSWERS

1. Inbound is generally free; outbound to the internet is billed per GB.
2. An hourly charge and a per-GB data processing charge.
3. It is free and removes S3 traffic from the NAT per-GB charge.
4. Enough monthly volume that the per-GB saving exceeds the fixed hourly cost, or a security reason such as keeping traffic off the internet path.
5. Zone-aware routing (topology-aware routing hints), co-locating chatty services, and reading from same-zone replicas.

11 Further reading

- AWS documentation: EC2 On-Demand data transfer pricing, NAT gateway pricing, VPC endpoints and AWS PrivateLink, CloudFront pricing.
- AWS Well-Architected Framework: Cost Optimization pillar.
- Kubernetes documentation: Topology Aware Routing.

20.8 Database Costs

Topic 8 of 9 | ~5 hours | Difficulty: Advanced

1 What it is

Managed databases (Amazon RDS, Aurora, ElastiCache, DynamoDB) are usually the second largest compute-like line on a cloud bill, and they are the hardest to cut because they hold state and sit on the critical path. Their cost has five parts:

Part	What drives it
Instance	class and size, count of instances, Multi-AZ standby, read replicas
Storage	provisioned GB, storage type (gp3, io1/io2), snapshots and backups
I/O and throughput	provisioned IOPS, Aurora I/O requests (or the I/O-Optimized option)
Data transfer	cross-AZ replication and reads, cross-Region replicas, egress
Extras	Performance Insights retention, RDS Proxy, enhanced monitoring, extended support for old engine versions

The levers are: size the instance to real load, choose the right storage type, count replicas deliberately, let capacity follow load where that is safe (Aurora Serverless v2), and **reduce the load itself** with caching and better queries so you can run smaller instances.

2 Why it matters in DevOps

- A database sized for peak that never comes is the classic idle resource, and the bill repeats every hour.
- The cheapest database query is the one you never run. A cache or an index can remove load that would otherwise need a larger instance (see PostgreSQL Indexes §13.1 and Redis §13.7).
- Cutting the wrong thing is dangerous. Removing a Multi-AZ standby or a replica saves money and takes away your availability, so the trade-off must be explicit (see Replication §13.4 and Disaster Recovery §0.9).

- Reserved pricing for databases is a separate commitment decision from compute; see Reserved Instances and Savings Plans §20.2.
- Related: RDS and Aurora §6.10, ElastiCache §6.11 and Rightsizing §20.5.

3 Real-world system example

Amazon Aurora is documented with two cluster storage configurations: **Aurora Standard** (you pay for storage plus per-request I/O) and **Aurora I/O-Optimized** (no per-request I/O charge, with higher instance and storage prices). AWS guidance says I/O-Optimized tends to be the better choice when I/O charges make up a large share of the Aurora bill (AWS cites roughly 25 percent as the threshold), and you can switch between them with limits on how often. **Aurora Serverless v2** scales capacity in fine-grained **Aurora Capacity Units (ACUs)** between a configured minimum and maximum, and AWS documents scaling to zero with automatic pause on supported engine versions. Older "provisioned" Aurora instances, by contrast, are fixed size.

Amazon ElastiCache (Redis or Memcached engines, and Valkey) is documented as a cache-aside layer in front of RDS to reduce read load. **RDS Proxy** pools connections so that many small application instances (Lambda, for example) do not force a bigger database just to handle connection counts. Both patterns are described in AWS database blogs and the Well-Architected guidance. Costs and thresholds change; check current pricing.

4 Diagram

Read path with a cache-aside layer:

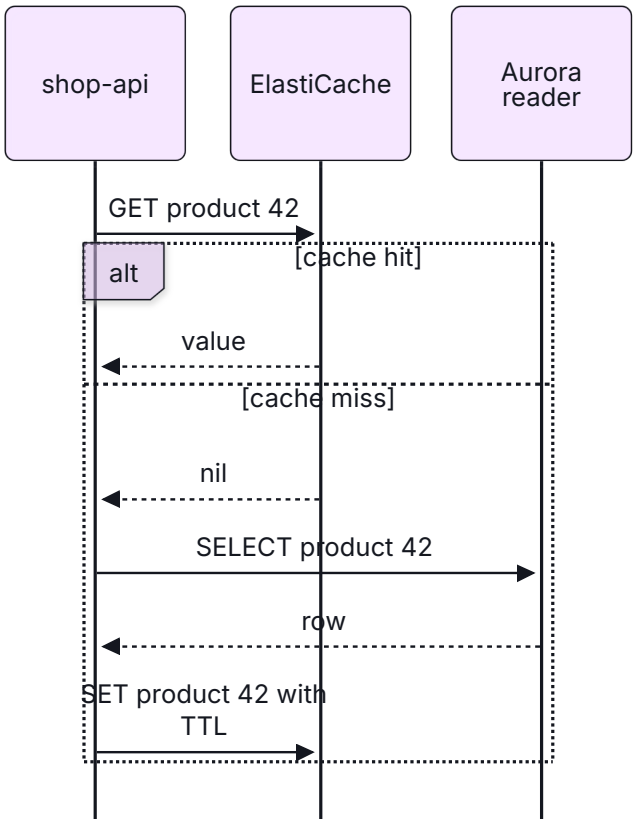


Figure 20.8-1

Deciding where database money goes:

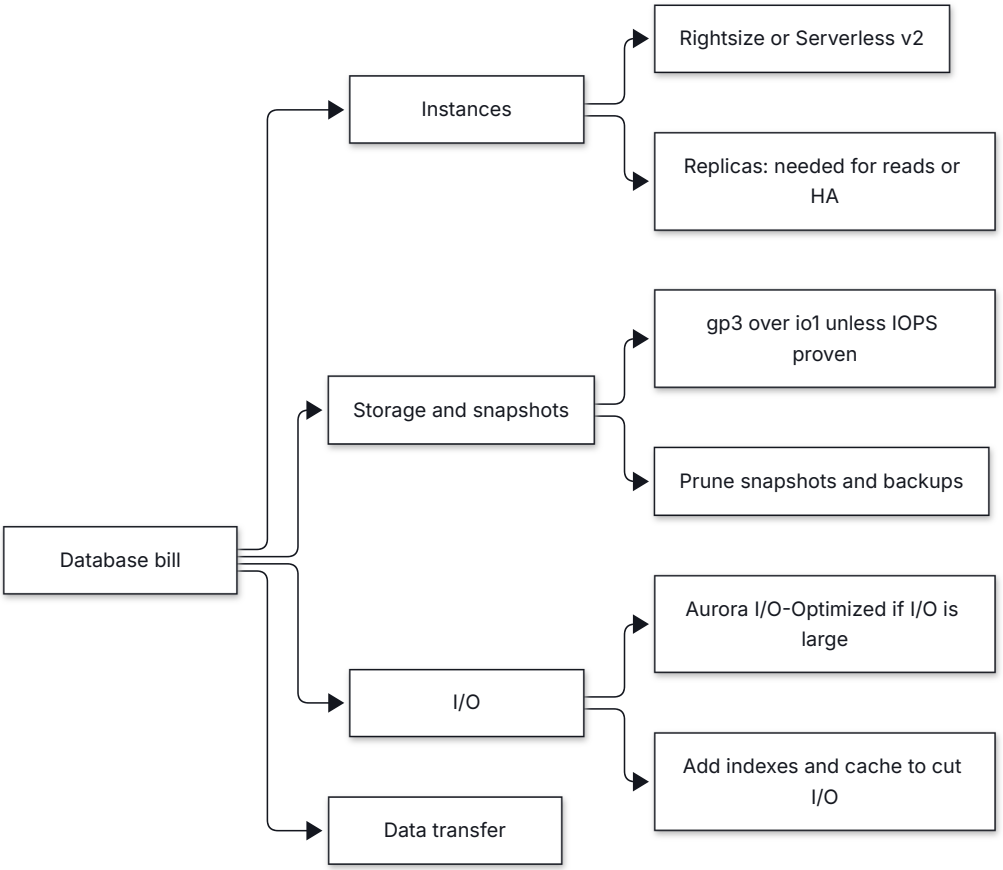


Figure 20.8-2

5 How it works

Instance sizing and replicas

Use the metrics from Rightsizing §20.5: `CPUUtilization`, `FreeableMemory`, `DatabaseConnections`, read and write IOPS and latency. Databases are often **memory-bound** (the working set should fit in the buffer cache), so a lower CPU does not mean you can take a smaller class. Each **read replica** is a full instance with its own cost. Keep replicas that serve real read load or provide failover, and remove idle ones. Multi-AZ doubles the instance cost for a standby; for production data that is usually correct, for dev and staging it usually is not.

Storage and IOPS

Option	Notes
gp3	baseline IOPS and throughput are independent of size; usually cheaper than gp2 for the same performance; the default choice
io1/io2 (provisioned IOPS)	pay for IOPS you provision; use only when gp3 limits are proven insufficient
Storage autoscaling	grows storage on demand; it never shrinks, so set a sensible maximum
Snapshots/backups	free up to the size of your provisioned storage within the retention period; manual snapshots and extra retention are billed

Aurora Serverless v2

Good fit: spiky, unpredictable or intermittent load (dev environments, internal tools, multi-tenant bursts). Poor fit: a steady high load where provisioned instances with a commitment are cheaper per unit of capacity. Set `MinCapacity` high enough that buffer cache and connection limits are not starved, and a `MaxCapacity` that bounds your worst-case bill.

Reduce the load

Order of attack, cheapest first: fix slow queries and missing indexes; add connection pooling; cache hot reads (cache-aside with a TTL); move analytics to a replica or warehouse; archive old rows; only then scale the instance down.

6 Step-by-step guide

Prerequisites: Python 3 for the offline steps; the AWS CLI and a sandbox for the rest. Databases are billed per hour while they exist, so keep the lab short and **delete everything** in the cleanup step. Set a billing alarm first. Prices are **EXAMPLES**: check current pricing.

1. **Estimate whether a cache pays for itself.** Save as `db_cost.py`:

```python
# db_cost.py - will a cache pay for itself? EXAMPLE prices only.
db_node_hour = 0.50           # example price of one database replica/instance size
cache_node_hour = 0.17        # example price of one cache node
hours = 730
reads_per_sec = 4000
one_db_node_can_serve = 1500  # reads/sec per node from load test
hit_ratio = 0.9

import math
nodes_no_cache = math.ceil(reads_per_sec / one_db_node_can_serve)
db_reads_with_cache = reads_per_sec * (1 - hit_ratio)
nodes_cache = max(1, math.ceil(db_reads_with_cache / one_db_node_can_serve))
```

```
cache_nodes = 2   # primary + replica
before = nodes_no_cache * db_node_hour * hours
after = nodes_cache * db_node_hour * hours + cache_nodes * cache_node_hour * hours
print(f"no cache : {nodes_no_cache} DB nodes → ${before:,.2f}/month")
print(f"cache    : {nodes_cache} DB node(s) + {cache_nodes} cache nodes →
  ${after:,.2f}/month at {hit_ratio:.0%} hit ratio")
print(f"saving   : ${before - after:,.2f}/month")
print("note: keep at least one DB replica for availability; the saving is smaller if
  Multi-AZ already needs 2 nodes")
```

BASH

```
python3 db_cost.py
```

Expected output:

```
no cache : 3 DB nodes → $1,095.00/month
cache    : 1 DB node(s) + 2 cache nodes → $613.20/month at 90% hit ratio
saving   : $481.80/month
note: keep at least one DB replica for availability; the saving is smaller if Multi-AZ
  already needs 2 nodes
```

Change `hit_ratio` to 0.5 and the saving changes sign or disappears. The cache only pays when the hit ratio is high enough to remove whole instances.

2. **See how cache size drives hit ratio.** This deterministic simulation sends 200,000 requests, with popularity skewed so a few keys are hot, through a FIFO cache. Save as `cache_sim.py`:

PYTHON

```
# cache_sim.py - how much database load does a cache-aside cache remove? Pure Python,
  deterministic.
import random
random.seed(42)

KEYS = 10_000
REQUESTS = 200_000
# popularity: a few hot keys get most traffic (Zipf-like weights)
weights = [1 / (i + 1) ** 1.1 for i in range(KEYS)]

def run(cache_size):
    cache = {}                        # key → None; evict the oldest key when full
  (simple FIFO)
    order = []
    hits = db_reads = 0
    for key in random.choices(range(KEYS), weights=weights, k=REQUESTS):
        if key in cache:
            hits += 1
        else:
```

```
            db_reads += 1
            cache[key] = None
            order.append(key)
            if len(order) > cache_size:
                cache.pop(order.pop(0))
    return hits / REQUESTS, db_reads

print(f"{'cache entries':>14} {'hit ratio':>10} {'db reads':>10} {'db load vs no
  cache':>20}")
for size in (0, 100, 500, 1000, 3000):
    ratio, db = run(size)
    print(f"{size:>14} {ratio:>10.1%} {db:>10} {db / REQUESTS:>20.1%}")
```

BASH

```
python3 cache_sim.py
```

Expected output (identical on any machine with the same Python random implementation, otherwise very similar):

```
 cache entries  hit ratio   db reads  db load vs no cache
             0       0.0%     200000               100.0%
           100      47.5%     105049                52.5%
           500      66.5%      67082                33.5%
          1000      74.3%      51472                25.7%
          3000      85.9%      28170                14.1%
```

Caching just 1 percent of the keys (100 entries) halves the database reads; diminishing returns follow. Your real workload decides the curve, so measure the hit ratio (`keyspace_hits` / `keyspace_misses` in Redis `INFO stats`).

3. **List your databases and their size, class and Multi-AZ setting.**

BASH

```
aws rds describe-db-instances \
  --query 'DBInstances[*].
  [DBInstanceIdentifier,DBInstanceClass,Engine,MultiAZ,StorageType,AllocatedStorage]'
  \
  --output table
```

Look for: large classes in non-production, `MultiAZ=True` outside production, `io1` storage without a measured need, and read replicas nobody queries.

4. **Check whether a database is actually busy.**

```bash
aws cloudwatch get-metric-statistics --namespace AWS/RDS --metric-name CPUUtilization
  \
  --dimensions Name=DBInstanceIdentifier,Value=<DB_INSTANCE_ID> \
  --start-time 2026-09-25T00:00:00Z --end-time 2026-10-09T00:00:00Z \
  --period 3600 --statistics Maximum Average --query 'Datapoints | sort_by(@,
  &Timestamp)[-5:]'
```

   Repeat for `FreeableMemory` and `DatabaseConnections`. Low CPU with low freeable memory means memory-bound: do not shrink without checking the cache hit rate (`BufferCacheHitRatio` on Aurora, `pg_stat_database` blks_hit/blks_read on PostgreSQL).

5. **Create an Aurora Serverless v2 cluster for a dev database (billed while running).**

```bash
aws rds create-db-cluster \
  --db-cluster-identifier shop-dev \
  --engine aurora-postgresql \
  --master-username shopadmin --manage-master-user-password \
  --serverless-v2-scaling-configuration MinCapacity=0.5,MaxCapacity=2
aws rds create-db-instance \
  --db-instance-identifier shop-dev-1 \
  --db-cluster-identifier shop-dev \
  --engine aurora-postgresql \
  --db-instance-class db.serverless
aws rds wait db-instance-available --db-instance-identifier shop-dev-1
```

   The `wait` returns once the instance is available (several minutes). `--manage-master-user-password` stores the password in Secrets Manager (see Secrets Manager §6.16). Check the capacity with the `ServerlessDatabaseCapacity` metric. Zero-ACU auto-pause is configured with a minimum capacity of 0 and a pause delay on supported engine versions; confirm the current option names in the AWS documentation before using it.

6. **Cap your risk and prune snapshots.**

```bash
aws rds describe-db-snapshots --snapshot-type manual \
  --query 'DBSnapshots[*].[DBSnapshotIdentifier,SnapshotCreateTime,AllocatedStorage]'
  --output table
aws rds describe-db-instances --query 'DBInstances[*].
  [DBInstanceIdentifier,BackupRetentionPeriod]' --output table
```

Manual snapshots never expire; delete the ones past your retention policy with `aws rds delete-db-snapshot`. For automated backups, set `BackupRetentionPeriod` to the shortest value your recovery objectives allow (see RPO §15.6).

7. **Run a local cache-aside exercise (optional, needs Docker).**

```bash
docker run -d --name cache -p 6379:6379 redis:7
docker exec cache redis-cli SET product:42 '{"name":"Widget"}' EX 60
docker exec cache redis-cli GET product:42
docker exec cache redis-cli INFO stats | grep -E 'keyspace_(hits|misses)'
```

`GET` returns the JSON string; the second call to `INFO stats` shows hits and misses growing. Remove it with `docker rm -f cache`.

8. **Clean up.**

```bash
aws rds delete-db-instance --db-instance-identifier shop-dev-1 --skip-final-snapshot
aws rds delete-db-cluster --db-cluster-identifier shop-dev --skip-final-snapshot
```

Confirm with `aws rds describe-db-clusters --query 'DBClusters[*].DBClusterIdentifier'` that the list is empty or no longer contains the lab cluster.

7 Real-world scenario

(Illustrative.) The `shop-api` database is a Multi-AZ Aurora cluster with a writer and two readers, sized for a promotional event that happens twice a year. CPU averages 18 percent. A tempting proposal is to drop to one reader and a smaller class.

Investigation of slow-query logs shows that 70 percent of read traffic is the same product-detail query. The team adds a cache-aside layer with a 60-second TTL, a 91 percent hit rate, and lets Performance Insights confirm the load drop. They then remove one reader, keep Multi-AZ for production, move staging to Aurora Serverless v2 with a low maximum, and schedule a temporary capacity increase before the next promotion. The saving funds the cache with money to spare, and availability is unchanged.

8 Common mistakes and best practices

Don't

- Don't downsize a database because CPU is low without checking memory and cache hit ratio.
- Don't run Multi-AZ and large instances in dev and staging by default.
- Don't pay for `io1` / `io2` IOPS you cannot show you use; start with gp3.
- Don't leave manual snapshots, old replicas and unused clusters running.
- Don't add a cache without a plan for invalidation, TTLs and cache-down behaviour.

Do

- Do cut load first (indexes, query fixes, caching), then resize.
- Do tag databases with team and environment so idle ones are found.
- Do evaluate Aurora I/O-Optimized when I/O is a large share of the bill.
- Do use Serverless v2 for spiky or non-production workloads and cap `MaxCapacity`.
- Do review reservations for databases separately from compute.

9 Tasks

- [ ] **T1 — Price a cache** *(≈20 min, beginner)*
  - **Goal:** know when a cache saves money.
  - **Steps:**
    1. Run `db_cost.py`.
    2. Try hit ratios of 0.5, 0.7 and 0.9.
  - **Done when:** you can state the hit ratio below which the cache does not pay for itself in this model.
- [ ] **T2 — Measure a hit-ratio curve** *(≈30 min, intermediate)*
  - **Goal:** see diminishing returns.
  - **Steps:**
    1. Run `cache_sim.py`.
    2. Change the Zipf exponent from 1.1 to 0.6 (flatter popularity) and re-run.
  - **Done when:** you can explain why a flatter distribution needs a much larger cache for the same hit ratio.
- [ ] **T3 — Audit your databases** *(≈40 min, intermediate; AWS sandbox)*
  - **Goal:** produce a finding list.
  - **Steps:**
    1. Run the `describe-db-instances` query.

2. Record any Multi-AZ non-production databases, `io1` storage and idle replicas.

- **Done when:** you have a table of findings with a proposed action and an owner for each.

☐ **T4 — Break it: shrink a memory-bound database** *(≈45 min, advanced)*

- **Goal:** feel the risk safely.
- **Steps:**
  1. On a local PostgreSQL in Docker, load a dataset larger than `shared_buffers` (for example with `pgbench -i -s 100`).
  2. Run `pgbench` with a small and then a large `shared_buffers` and compare latency.
- **Done when:** you show that latency worsens with less memory even though CPU stays low, and you explain what that means for downsizing.

☐ **T5 — Serverless v2 for a dev database** *(≈60 min, advanced; billed while running)*

- **Goal:** a dev database that scales with load.
- **Steps:**
  1. Create the cluster with a low maximum.
  2. Run a load generator briefly and watch `ServerlessDatabaseCapacity` rise.
  3. Delete the cluster.
- **Done when:** you have a metric graph that shows capacity changing, and the cluster no longer exists.

☐ **T6 — Prove it** *(≈45 min, advanced)*

- **Goal:** a defensible database savings plan.
- **Steps:**
  1. Without the guide, list five database levers and their risks.
  2. Pick three for `shop-api` and estimate the saving with example numbers.
  3. State what you would not change and why.
- **Done when:** a reviewer sees an explicit availability trade-off for each change.

10 Self-check questions

1. Why does a low CPU not prove a database is oversized?
2. When does Aurora I/O-Optimized make sense?
3. What does a cache-aside cache cost and save?
4. What is the downside of removing a read replica or Multi-AZ standby?

5. Why is gp3 usually the starting point for storage?

ANSWERS

1. Databases are often memory- or I/O-bound; the working set must fit the buffer cache, so memory and cache hit ratio matter as much as CPU.
2. When per-request I/O charges are a large share of the Aurora bill (AWS suggests around a quarter) so removing them outweighs the higher instance and storage price.
3. It adds cache nodes and code complexity, and saves database instance capacity when the hit ratio is high enough to remove whole instances.
4. Less read capacity and lower availability or slower failover; it must be a deliberate trade-off.
5. Its performance baseline is independent of size and it is typically cheaper than provisioned IOPS types unless you need more than its limits.

11 Further reading

- AWS documentation: Amazon RDS User Guide, Amazon Aurora User Guide (Aurora Serverless v2, I/O-Optimized), Amazon ElastiCache User Guide, RDS Proxy.
- PostgreSQL documentation: monitoring statistics (`pg_stat_database`) and `shared_buffers`.
- *Designing Data-Intensive Applications* by Martin Kleppmann.

20.9 Kubernetes Infrastructure Costs

Topic 9 of 9 | ~5 hours | Difficulty: Advanced

1 What it is

Kubernetes does not have a bill; your cloud account does. The bill is for the **nodes** (virtual machines) the cluster runs on, plus the load balancers, volumes, NAT traffic and control plane fees around them. Kubernetes cost optimization is the work of making the cluster **use fewer or cheaper nodes for the same workloads**, and of attributing node cost back to the teams whose pods occupy them.

Three ideas explain most waste:

Idea	Meaning
Requests, not usage, reserve capacity	The scheduler places pods by their resource *requests*. A pod that requests 2 CPUs and uses 100m still blocks 2 CPUs on a node.
Bin packing	Packing pods tightly onto the fewest nodes. Poor packing leaves fragmented, half-empty nodes you still pay for.
Allocation	Splitting node cost across namespaces, labels and teams (showback), including idle capacity that nobody uses.

Requests inflation (the gap between what was requested and what is used) is the single biggest source of waste in most clusters. The fixes combine Rightsizing §20.5, the autoscaling in Autoscaling for Cost §20.4 and the purchasing options in Spot Instances §20.3 and Savings Plans §20.2.

2 Why it matters in DevOps

- Clusters hide cost behind shared nodes. Without allocation, "the cluster costs $40k" is a number nobody owns.
- Inflated requests make the cluster autoscaler add nodes for capacity that is not used, so waste scales with the number of teams.
- Fragmentation, oversized nodes for small pods, and idle node pools quietly accumulate. A policy that bounds requests keeps the problem from returning.

- Resource and cost decisions live in YAML (requests, limits, node selectors), so they can be reviewed in pull requests and enforced as code (see Policy as Code §18.7).
- Related: Resource Requests and Limits §8.14, Scheduling §8.13, HPA, VPA and Cluster Autoscaler §16.7 and FinOps §20.1.

3 Real-world system example

OpenCost is a CNCF project that implements a vendor-neutral specification for Kubernetes cost allocation. It reads cluster metrics (typically from Prometheus) and cloud pricing data and exposes cost by namespace, controller, label, pod and other dimensions, including idle cost. **Kubecost** is a commercial product that is built on OpenCost and adds features such as multi-cluster views, savings recommendations and alerts. Both are documented publicly and install with Helm.

Karpenter (open source, a CNCF project) is documented as a node provisioner that looks at pending pods, selects suitable instance types from a wide set, launches them quickly, and **consolidates** nodes by deleting empty or under-used nodes and replacing them with cheaper ones where possible. The older **Cluster Autoscaler** scales predefined node groups up and down. **Goldilocks** (Fairwinds) and the Vertical Pod Autoscaler produce request recommendations. Many public engineering write-ups describe the same pattern: reduce requests inflation first, then let node autoscaling realise the saving.

4 Diagram

How requests become node cost:

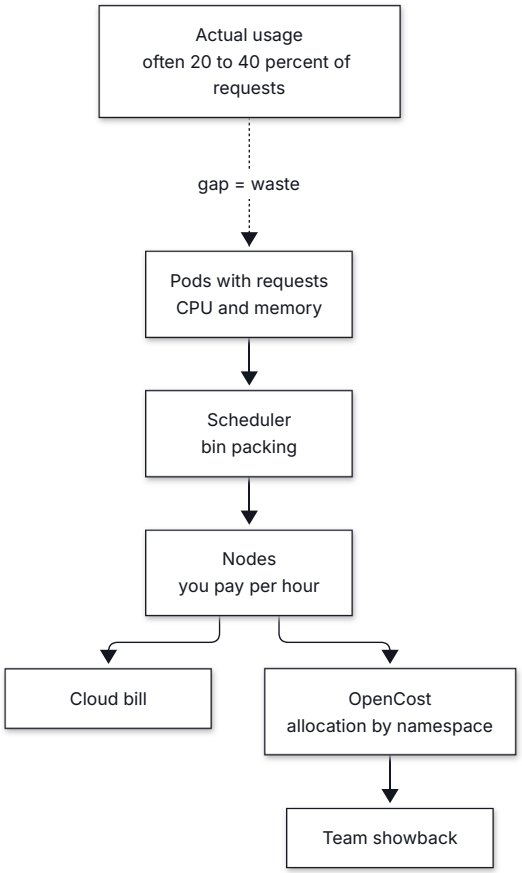


Figure 20.9-1

Karpenter consolidation:

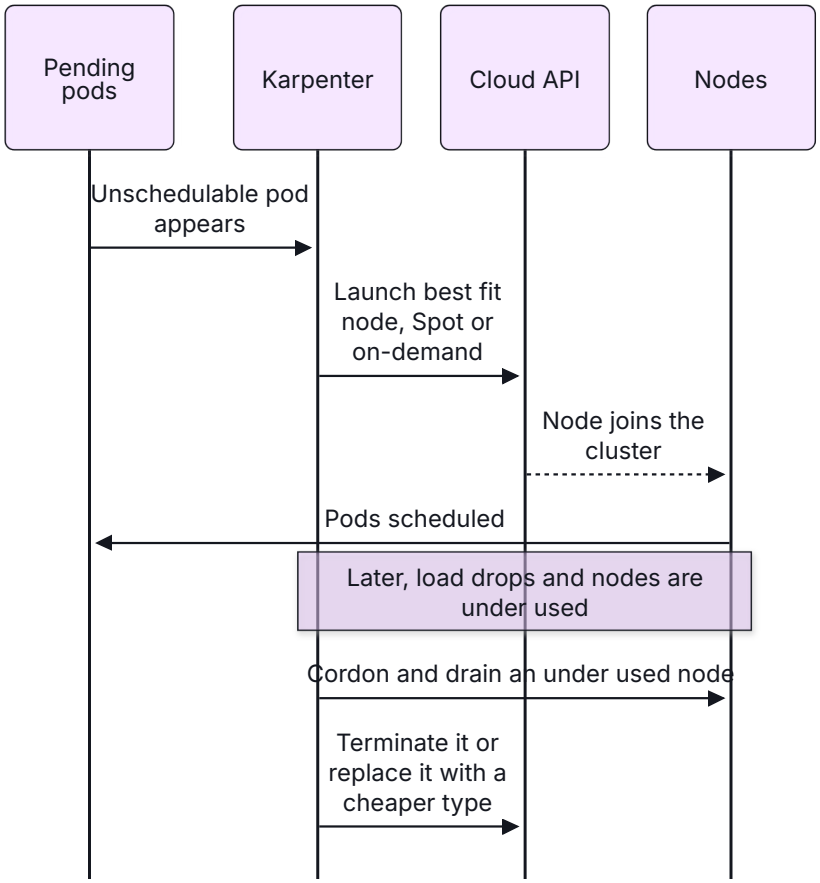


Figure 20.9-2

5 How it works

Where the waste comes from

Source	Symptom	Fix
Requests inflation	usage is 25-40 percent of requests	rightsize with VPA recommendations, set a policy cap
No requests at all	pods are "free" to the scheduler and overcommit nodes, causing noisy neighbours	`LimitRange` defaults, policy requiring requests
Oversized nodes for small pods	unallocatable leftover on every node	right node size, more pools, Karpenter picks per workload

Source	Symptom	Fix
Fragmentation	free CPU on one node, free memory on another	consolidation, requests that match the instance shape
Idle capacity	node pools or environments nobody uses	scale to zero, off-hours schedules, delete
Always on-demand	stateless pods on the most expensive purchase option	Spot node pools with tolerations, see Spot Instances §20.3
Hidden costs	NAT, cross-AZ, load balancers per Service, unattached volumes	see Network Egress Costs §20.7, one ingress controller

Node pools and purchase options

Separate pools by purpose: a small **on-demand pool** for system components and stateful or critical pods, a **Spot pool** for stateless and batch pods (tainted, so only tolerating pods land there), and special pools for GPU or memory-heavy workloads. Karpenter replaces fixed pools with `NodePool` requirements, but the principle (match workload to cheapest suitable capacity) is the same.

Allocation

OpenCost allocates a node's cost to the pods running on it in proportion to the greater of their requests and usage, then adds an **idle** share. Labels such as `team` and `app` let you aggregate cost by team. Shared costs (control plane fee, ingress, monitoring stack) need an explicit rule; see FinOps §20.1.

Guardrails

A `LimitRange` provides default requests, a `ResourceQuota` caps a namespace, and an admission policy can reject outsized requests. The Kubernetes cost lever you control in code review is the `resources:` block.

6 Step-by-step guide

Prerequisites: Python 3 and `opa` (or `conftest`) for the offline steps; a Kubernetes cluster (kind or minikube is enough) with `kubectl` and `helm` for the cluster steps. OpenCost needs Prometheus for full data and shows approximate costs on non-cloud clusters. Karpenter requires AWS (EKS) and incurs real cost, so steps are shown as manifests only. Prices are **EXAMPLES**: check current pricing. Outputs are similar to what is shown.

1. **Quantify requests inflation.** Save as `k8s_waste.py`:

```python
# k8s_waste.py - requests inflation and node cost. EXAMPLE numbers.
```

```
# Each row: (workload, replicas, cpu_request_m, cpu_p95_used_m, mem_request_mi,
  mem_p95_used_mi)
rows = [
    ("shop-api", 6, 1000, 310, 1024, 520),
    ("worker",   4, 2000, 450, 4096, 1500),
    ("nginx",    3,  500,  80,  256,  90),
]
node_cpu_m, node_mem_mi, node_hour = 4000, 16384, 0.17   # example 4 vCPU / 16 GiB
  node
hours = 730
req_cpu = sum(r[1] * r[2] for r in rows)
use_cpu = sum(r[1] * r[3] for r in rows)
req_mem = sum(r[1] * r[4] for r in rows)
use_mem = sum(r[1] * r[5] for r in rows)
import math
nodes_now = max(math.ceil(req_cpu / (node_cpu_m * 0.85)), math.ceil(req_mem /
  (node_mem_mi * 0.85)))
want_cpu = use_cpu * 1.3      # request = p95 plus 30% headroom
want_mem = use_mem * 1.3
nodes_right = max(math.ceil(want_cpu / (node_cpu_m * 0.85)), math.ceil(want_mem /
  (node_mem_mi * 0.85)))
print(f"CPU requested {req_cpu}m, used {use_cpu}m → {use_cpu / req_cpu:.0%} of
  requests used")
print(f"MEM requested {req_mem}Mi, used {use_mem}Mi → {use_mem / req_mem:.0%} of
  requests used")
print(f"nodes needed now        : {nodes_now}  (${nodes_now * node_hour *
  hours:,.2f}/month)")
print(f"nodes after rightsizing : {nodes_right}  (${nodes_right * node_hour *
  hours:,.2f}/month)")
```

BASH

```
python3 k8s_waste.py
```

Expected output:

```
CPU requested 15500m, used 3900m → 25% of requests used
MEM requested 23296Mi, used 9390Mi → 40% of requests used
nodes needed now        : 5  ($620.50/month)
nodes after rightsizing : 2  ($248.20/month)
```

(The model uses 85 percent of each node as allocatable after system overhead, and ignores spreading and availability needs, which will keep more nodes in practice.)

2. **Guard requests with policy.** A Rego policy that requires requests and caps them unless an approval annotation exists. Save as `requests.rego`:

REGO

```
package main
```

```
import rego.v1

max_cpu_millicores := 1000
max_memory_mi := 2048

millicores(q) := n if {
  endswith(q, "m")
  n := to_number(trim_suffix(q, "m"))
}

millicores(q) := n if {
  not endswith(q, "m")
  n := to_number(q) * 1000
}

mebibytes(q) := n if {
  endswith(q, "Gi")
  n := to_number(trim_suffix(q, "Gi")) * 1024
}

mebibytes(q) := n if {
  endswith(q, "Mi")
  n := to_number(trim_suffix(q, "Mi"))
}

deny contains msg if {
  input.kind == "Deployment"
  some c in input.spec.template.spec.containers
  not c.resources.requests
  msg := sprintf("%s/%s: container %q has no resource requests", [input.kind,
  input.metadata.name, c.name])
}

deny contains msg if {
  input.kind == "Deployment"
  some c in input.spec.template.spec.containers
  millicores(c.resources.requests.cpu) > max_cpu_millicores
  not input.metadata.annotations["platform.example.com/large-request-approved"]
  msg := sprintf("%s/%s: container %q requests %s CPU (max %dm without approval
  annotation)", [input.kind, input.metadata.name, c.name, c.resources.requests.cpu,
  max_cpu_millicores])
}

deny contains msg if {
  input.kind == "Deployment"
  some c in input.spec.template.spec.containers
  mebibytes(c.resources.requests.memory) > max_memory_mi
  not input.metadata.annotations["platform.example.com/large-request-approved"]
  msg := sprintf("%s/%s: container %q requests %s memory (max %dMi without approval
  annotation)", [input.kind, input.metadata.name, c.name, c.resources.requests.memory,
  max_memory_mi])
}
```

And a deliberately oversized `deploy.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shop-api
spec:
  replicas: 3
  selector:
    matchLabels: {app: shop-api}
  template:
    metadata:
      labels: {app: shop-api}
    spec:
      containers:
        - name: shop-api
          image: nginx:1.27
          resources:
            requests: {cpu: "2", memory: 4Gi}
```

```bash
opa eval -f pretty -d requests.rego -i deploy.yaml 'data.main.deny'
# or:  conftest test --policy . deploy.yaml
```

Expected output:

```
[
  "Deployment/shop-api: container \"shop-api\" requests 2 CPU (max 1000m without
  approval annotation)",
  "Deployment/shop-api: container \"shop-api\" requests 4Gi memory (max 2048Mi without
  approval annotation)"
]
```

Change the requests to `cpu: 250m, memory: 256Mi` and the result is `[]`. Run the same policy in CI (and as a Kyverno or Gatekeeper rule at admission) so inflation cannot sneak back in.

3. **Set namespace defaults and a quota.** Defaults prevent "no request" pods; a quota caps what a team can reserve:

```yaml
apiVersion: v1
kind: LimitRange
metadata:
```

```
  name: defaults
  namespace: team-shop
spec:
  limits:
    - type: Container
      defaultRequest: {cpu: 100m, memory: 128Mi}
      default: {cpu: 500m, memory: 512Mi}
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-shop-quota
  namespace: team-shop
spec:
  hard:
    requests.cpu: "8"
    requests.memory: 16Gi
    limits.memory: 32Gi
```

BASH

```
kubectl create namespace team-shop
kubectl apply -f quota.yaml
kubectl -n team-shop describe quota team-shop-quota
```

The `Used` column starts at 0 and grows as pods are scheduled; a pod that would exceed `requests.cpu` is rejected at admission.

4. **Install OpenCost and read allocation.** OpenCost reads Prometheus; install a Prometheus first (for example the kube-prometheus-stack chart; see Prometheus and PromQL §10.2), then:

BASH

```
helm repo add opencost https://opencost.github.io/opencost-helm-chart
helm repo update
helm install opencost opencost/opencost --namespace opencost --create-namespace \
  --set opencost.prometheus.internal.serviceName=<PROMETHEUS_SERVICE> \
  --set opencost.prometheus.internal.namespaceName=<PROMETHEUS_NAMESPACE>
kubectl -n opencost rollout status deploy/opencost
kubectl -n opencost port-forward service/opencost 9003 9090
```

Chart values change between releases, so confirm the Prometheus settings against the chart's `values.yaml`. In another terminal, ask the API for cost by namespace over the last 7 days:

```bash
curl -s "http://localhost:9003/allocation/compute?window=7d&aggregate=namespace" |
  python3 -m json.tool | head -40
```

The response lists namespaces with fields such as `cpuCost`, `ramCost`, `totalCost` and an `__idle__` entry for unallocated capacity. On a local cluster the prices are defaults, so use the relative proportions, not the dollar amounts. The UI is on port 9090.

5. **Compare requests with usage in PromQL.** These give the ratio per namespace, the number to bring to a rightsizing meeting:

```promql
# fraction of requested CPU that is actually used, per namespace
sum by (namespace) (rate(container_cpu_usage_seconds_total{container≠""}[1h]))
/
sum by (namespace) (kube_pod_container_resource_requests{resource="cpu"})
```

A value consistently below 0.3 is a rightsizing target.

6. **Define a Karpenter NodePool with consolidation and a Spot-only batch pool (EKS, illustrative manifests).** Karpenter must already be installed with an `EC2NodeClass` named `default`:

```yaml
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: general
spec:
  template:
    spec:
      nodeClassRef:
        group: karpenter.k8s.aws
        kind: EC2NodeClass
        name: default
      requirements:
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["on-demand"]
        - key: karpenter.k8s.aws/instance-category
          operator: In
          values: ["m", "c"]
  limits:
    cpu: "200"
```

```
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
    consolidateAfter: 2m
---
apiVersion: karpenter.sh/v1
kind: NodePool
metadata:
  name: batch-spot
spec:
  template:
    spec:
      nodeClassRef:
        group: karpenter.k8s.aws
        kind: EC2NodeClass
        name: default
      taints:
        - key: workload
          value: batch
          effect: NoSchedule
      requirements:
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["spot"]
  limits:
    cpu: "100"
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
    consolidateAfter: 1m
```

Pods tolerate the batch taint to use the Spot pool:

YAML

```
tolerations:
  - key: workload
    operator: Equal
    value: batch
    effect: NoSchedule
```

Apply with `kubectl apply -f nodepools.yaml` and watch `kubectl get nodeclaims` and `kubectl get nodes -L karpenter.sh/capacity-type` as workloads change. Karpenter launches real instances, so scale workloads back to zero and delete the NodePools afterwards.

7. **Find other hidden Kubernetes costs.**

```bash
kubectl get svc -A --field-selector spec.type=LoadBalancer
kubectl get pvc -A
kubectl get pv --no-headers | awk '$5=="Released" || $5=="Available"'
```

   Each `LoadBalancer` Service is a billed cloud load balancer (use one ingress controller where you can); released or available PersistentVolumes may still be billed volumes (see Storage Lifecycle Policies §20.6).

7 Real-world scenario

(Illustrative.) A 40-node cluster is fully packed according to the scheduler, but average node CPU use is 22 percent. The platform team ranks namespaces by the OpenCost efficiency numbers (usage divided by requests) and meets the three worst teams. The `worker` team copied `cpu: 2` from a load test; their p95 is 300m. The team halves requests in staged rollouts, the cluster autoscaler empties nodes, and Karpenter consolidation replaces large half-empty nodes with smaller ones.

Two things go wrong: a team's memory request is cut too far and pods are `OOMKilled` (they roll back, see Rightsizing §20.5), and a stateful pod blocks a consolidation until a PodDisruptionBudget is relaxed. The platform team adds the request cap policy and a monthly efficiency report per namespace. Node count falls by about a third.

8 Common mistakes and best practices

Don't

- Don't copy requests from examples or load tests without measuring real usage.
- Don't run pods without requests; they overcommit nodes and break scheduling assumptions.
- Don't give every team its own load balancer and node pool by default.
- Don't put critical or stateful pods on Spot pools.
- Don't trust cost dashboards on a local cluster for absolute dollars.

Do

- Do measure usage versus requests per namespace and publish the ratio.
- Do enforce defaults and caps with `LimitRange`, `ResourceQuota` and policy.
- Do separate on-demand, Spot and special-purpose node pools, with taints.

- Do let a node autoscaler (Cluster Autoscaler or Karpenter) remove the capacity you free up.
- Do allocate costs by label and report idle cost.

9 Tasks

- [ ] **T1 — Measure inflation** *(≈20 min, beginner)*
  - **Goal:** a requests-to-usage ratio.
  - **Steps:**
    1. Run `k8s_waste.py`.
    2. Replace the rows with three of your own workloads (or invented ones).
  - **Done when:** you can state the used-to-requested percentage and the nodes saved.
- [ ] **T2 — Enforce a request cap** *(≈30 min, intermediate)*
  - **Goal:** a failing and then passing policy check.
  - **Steps:**
    1. Run the Rego policy against `deploy.yaml`.
    2. Fix the requests and add the approval annotation to a second file to see the exception.
  - **Done when:** one run prints two messages, one prints `[]`, and the annotated deployment passes.
- [ ] **T3 — Set defaults and quotas** *(≈30 min, intermediate)*
  - **Goal:** pods without requests get defaults; over-quota pods are rejected.
  - **Steps:**
    1. Apply the `LimitRange` and `ResourceQuota`.
    2. Run a pod with no resources and inspect `kubectl get pod -o yaml`.
    3. Scale a Deployment past the quota.
  - **Done when:** the pod shows default requests, and the scale-up reports a `forbidden: exceeded quota` event.
- [ ] **T4 — Break it: starve a node** *(≈40 min, intermediate)*
  - **Goal:** feel fragmentation.
  - **Steps:**
    1. On kind, create pods that each request 60 percent of a node's CPU.
    2. Observe `Pending` pods while nodes show low usage.

3. Reduce the requests and watch the pods schedule.

- **Done when:** `kubectl describe pod` shows `Insufficient cpu` before the change and the pods run after it.

☐ **T5 — Allocate cost with OpenCost** *(≈60 min, advanced)*

- **Goal:** cost by namespace.
- **Steps:**
  1. Install Prometheus and OpenCost.
  2. Create two namespaces with different workloads.
  3. Query the allocation API aggregated by namespace.
- **Done when:** you have a table of namespaces with relative cost and the idle share, with a note that local prices are approximate.

☐ **T6 — Prove it** *(≈60 min, advanced)*

- **Goal:** a Kubernetes cost plan.
- **Steps:**
  1. Without the guide, list the causes of waste in a cluster, and the metric that exposes each.
  2. Propose guardrails (policy, quota, node pools) for `shop-api`.
  3. Explain why lowering requests alone does not reduce the bill.
- **Done when:** you can explain the node autoscaler's role and name one risk per change.

10 Self-check questions

1. Why do requests, not usage, determine node count?
2. What is requests inflation and how do you detect it?
3. What does Karpenter consolidation do?
4. Why taint a Spot node pool?
5. What does OpenCost add that the cloud bill does not show?

ANSWERS

1. The scheduler places pods by requested resources, so requested capacity is reserved on nodes whether or not it is used.
2. Requests far above real usage; detect with used-to-requested ratios from Prometheus, VPA recommendations or OpenCost efficiency.

3. It removes empty or under-used nodes and can replace them with cheaper or smaller ones after draining pods safely.
4. So only workloads that tolerate interruption (via tolerations) land there, and critical pods stay on stable nodes.
5. Cost allocated by namespace, workload and label, including the idle share, instead of only per-instance charges.

11 Further reading

- OpenCost documentation (opencost.io) and the CNCF project page.
- Karpenter documentation (karpenter.sh): NodePools and disruption.
- Kubernetes documentation: Resource Management for Pods and Containers, Limit Ranges, Resource Quotas.
- FinOps Foundation: material on container and Kubernetes cost allocation.

LAB NOTES What I ran, what I saw, what it means

★ Audit an AWS and Kubernetes Environment

Level 20 capstone ~10 hours

1 Goal and outcome

Run a complete cost audit of a sample environment and produce a savings proposal with numbers. You will not need to spend money to do it: the audit runs on a synthetic inventory of an AWS and Kubernetes environment for `shop-api` (EC2, EBS, RDS, S3, CloudWatch Logs, NAT, Kubernetes namespaces), and every step also tells you how to repeat it against a real sandbox account. By the end you will have:

- a **tag compliance** report and an enforced tag policy;
- a **waste report** that finds idle instances, unattached volumes, orphaned snapshots, immortal log groups and idle replicas;
- **rightsizing** decisions for EC2 and Kubernetes requests, with the risks written down;
- **lifecycle policies** (S3, logs, snapshots) and a **network** fix (S3 gateway endpoint);
- a **Savings Plan** proposal with coverage, utilization and break-even;
- a short **savings proposal** that totals monthly and yearly savings, ranks them by effort, and states the risk of each.

This exercises every topic: FinOps §20.1, Reserved Instances and Savings Plans §20.2, Spot Instances §20.3, Autoscaling for Cost §20.4, Rightsizing §20.5, Storage Lifecycle Policies §20.6, Network Egress Costs §20.7, Database Costs §20.8 and Kubernetes Infrastructure Costs §20.9.

2 Architecture

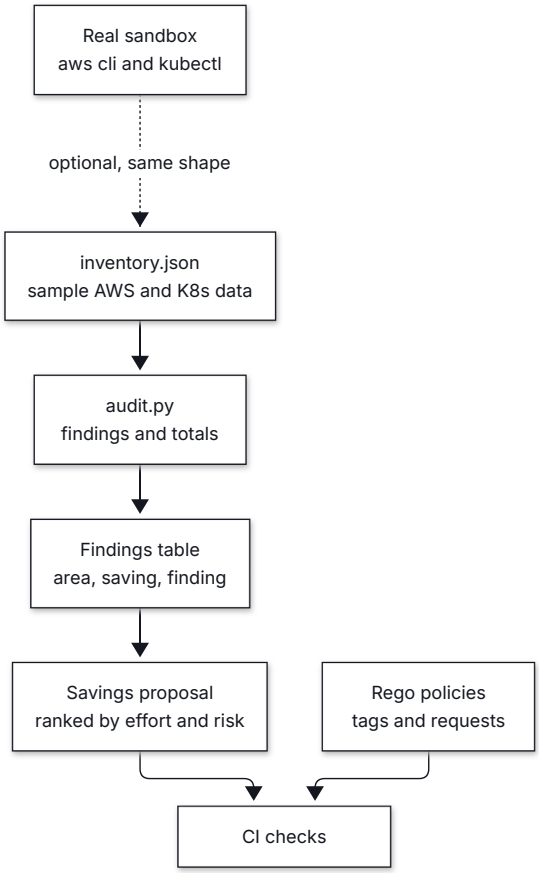


Figure 20-1

The audit loop (Inform, Optimize, Operate from FinOps §20.1):

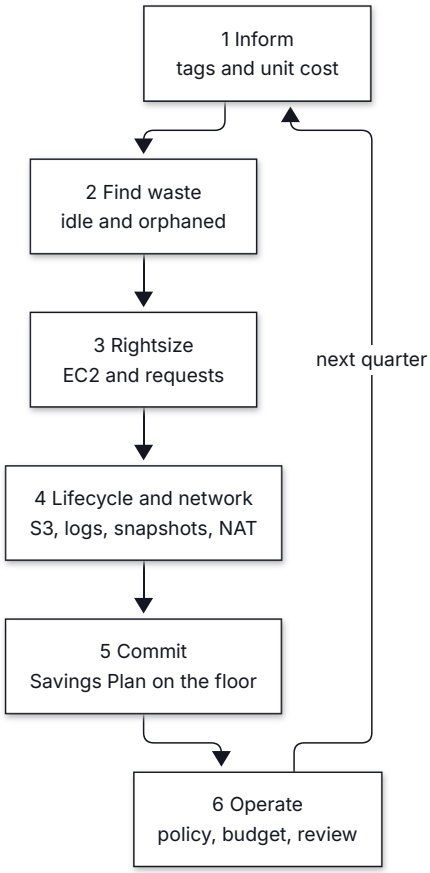


Figure 20-2

3 Prerequisites and cost

- Python 3 (standard library only). Optional: `opa` or `conftest` for the policy tasks, `kubectl` and a kind cluster for the Kubernetes task, the AWS CLI and a sandbox account for the "repeat on real data" tasks.
- **Cost:** the synthetic audit is free. If you repeat steps against a real AWS sandbox, the commands used here are read-only, except where a task says it creates something (a lifecycle policy on a test bucket, a gateway endpoint). Set a billing alarm first and use the teardown section.
- **Do not purchase a Savings Plan or Reserved Instance** while doing this capstone: commitments cannot be cancelled. The commitment step is a proposal only.

- All prices in the scripts are **EXAMPLES** (check current pricing). Output below is from running the scripts on the sample data; with real data your numbers will differ.

4 Build it step by step

Milestone 1 — Create the sample environment (topics 1, 5)

Save as `make_sample_data.py`. It writes `inventory.json`, a fake environment with deliberate problems hidden in it:

```python
# make_sample_data.py - writes inventory.json, a fake AWS/Kubernetes environment for the audit. EXAMPLE data and prices.
import json

inventory = {
    "ec2": [
        {"id": "i-web-1",    "type": "m6i.xlarge",  "price_hr": 0.192, "env": "prod", "cpu_p95": 38, "mem_p95": 52, "tags": {"team": "shop", "env": "prod", "service": "shop-api", "cost-center": "cc-1042"}},
        {"id": "i-web-2",    "type": "m6i.xlarge",  "price_hr": 0.192, "env": "prod", "cpu_p95": 41, "mem_p95": 55, "tags": {"team": "shop", "env": "prod", "service": "shop-api", "cost-center": "cc-1042"}},
        {"id": "i-batch-1",  "type": "m6i.2xlarge", "price_hr": 0.384, "env": "prod", "cpu_p95": 12, "mem_p95": 18, "tags": {"team": "shop", "env": "prod", "service": "reports", "cost-center": "cc-1042"}},
        {"id": "i-dev-1",    "type": "m6i.2xlarge", "price_hr": 0.384, "env": "dev",  "cpu_p95": 9,  "mem_p95": 20, "tags": {"team": "shop", "env": "dev"}},
        {"id": "i-stage-1",  "type": "m6i.xlarge",  "price_hr": 0.192, "env": "staging", "cpu_p95": 15, "mem_p95": 30, "tags": {}},
        {"id": "i-mystery",  "type": "c6i.4xlarge", "price_hr": 0.68,  "env": "unknown", "cpu_p95": 2,  "mem_p95": 8,  "tags": {}},
    ],
    "ebs": [
        {"id": "vol-a", "size_gb": 200, "attached": True,  "type": "gp3"},
        {"id": "vol-b", "size_gb": 500, "attached": False, "type": "gp2"},
        {"id": "vol-c", "size_gb": 1000, "attached": False, "type": "io1"},
        {"id": "vol-d", "size_gb": 100, "attached": True,  "type": "gp2"},
    ],
    "snapshots": [
        {"id": "snap-1", "age_days": 12,  "size_gb": 200, "volume_exists": True},
        {"id": "snap-2", "age_days": 400, "size_gb": 500, "volume_exists": False},
        {"id": "snap-3", "age_days": 700, "size_gb": 1000, "volume_exists": False},
        {"id": "snap-4", "age_days": 365, "size_gb": 100, "volume_exists": True},
    ],
    "log_groups": [
        {"name": "/aws/lambda/shop-api", "stored_gb": 340, "retention_days": None},
        {"name": "/ecs/reports",         "stored_gb": 120, "retention_days": None},
        {"name": "/audit/cloudtrail",    "stored_gb": 900, "retention_days": 365},
    ],
```

```
    "s3": [
        {"bucket": "shop-assets",   "size_gb": 4000,  "has_lifecycle": False, "pattern":
  "unpredictable"},
        {"bucket": "shop-logs",     "size_gb": 12000, "has_lifecycle": False, "pattern":
  "write-once-read-rarely"},
        {"bucket": "shop-backups",  "size_gb": 3000,  "has_lifecycle": True,  "pattern":
  "write-once-read-rarely"},
    ],
    "rds": [
        {"id": "shop-prod", "class": "db.r6g.xlarge", "node_price_hr": 0.52, "multi_az":
  True, "env": "prod", "cpu_p95": 35,
         "read_replicas": [{"id": "shop-prod-r1", "cpu_p95": 3}]},
        {"id": "shop-dev", "class": "db.r6g.xlarge", "node_price_hr": 0.52, "multi_az":
  True, "env": "dev", "cpu_p95": 6, "read_replicas": []},
    ],
    "network": {"nat_gateways": 3, "nat_gb_month": 9000, "s3_share_of_nat": 0.6,
  "has_s3_gateway_endpoint": False},
    "k8s_namespaces": [
        {"ns": "team-shop", "cpu_requested_m": 12000, "cpu_used_p95_m": 3000,
  "mem_requested_mi": 24576, "mem_used_p95_mi": 9000},
        {"ns": "team-pay",  "cpu_requested_m": 8000,  "cpu_used_p95_m": 5200,
  "mem_requested_mi": 16384, "mem_used_p95_mi": 11000},
        {"ns": "dev",       "cpu_requested_m": 6000,  "cpu_used_p95_m": 400,
  "mem_requested_mi": 12288, "mem_used_p95_mi": 1500},
    ],
    "prices": {"ebs_gb": {"gp3": 0.08, "gp2": 0.10, "io1": 0.125}, "snapshot_gb": 0.05,
  "logs_gb": 0.03,
               "s3_standard_gb": 0.023, "s3_ia_gb": 0.0125, "nat_hour": 0.045, "nat_gb":
  0.045, "node_hour_per_vcpu": 0.048},
    "hourly_usage_od_equiv": [round(18 + 10 * max(0, (h % 24 - 7) / 11 if 7 ⩽ h % 24 ⩽
  18 else 0), 2) for h in range(168)],
}
with open("inventory.json", "w") as f:
    json.dump(inventory, f, indent=2)
print("wrote inventory.json with", {k: len(v) for k, v in inventory.items() if
  isinstance(v, list)})
```

BASH

```
mkdir cost-audit && cd cost-audit
python3 make_sample_data.py
```

Expected output:

```
wrote inventory.json with {'ec2': 6, 'ebs': 4, 'snapshots': 4, 'log_groups': 3, 's3': 3,
  'rds': 2, 'k8s_namespaces': 3, 'hourly_usage_od_equiv': 168}
```

Milestone 2 — Run the audit (topics 1, 4, 5, 6, 7, 8, 9)

Save as `audit.py`. Each numbered block is one lever from the level:

PYTHON

```python
# audit.py - find waste in inventory.json and total the monthly saving. EXAMPLE prices;
  check current pricing.
import json, math

HOURS = 730
inv = json.load(open("inventory.json"))
P = inv["prices"]
REQUIRED = {"team", "env", "service", "cost-center"}
findings = []          # (area, finding, monthly_saving)

# 1. Tag compliance (Inform)
untagged = [i for i in inv["ec2"] if REQUIRED - set(i["tags"])]
cost_untagged = sum(i["price_hr"] * HOURS for i in untagged)
total_ec2 = sum(i["price_hr"] * HOURS for i in inv["ec2"])
print(f"tag compliance: {len(inv['ec2']) - len(untagged)}/{len(inv['ec2'])} EC2 instances
  fully tagged; "
      f"untagged spend ${cost_untagged:,.0f} of ${total_ec2:,.0f} ({cost_untagged /
  total_ec2:.0%})")

# 2. Rightsizing (halve the size when p95 CPU and memory both low; terminate near-idle
  unknowns)
rightsized = set()
for i in inv["ec2"]:
    if i["cpu_p95"] < 5 and i["env"] == "unknown":
        findings.append(("idle", f"{i['id']} {i['type']} CPU p95 {i['cpu_p95']}%, no
  owner: stop after confirming", i["price_hr"] * HOURS))
    elif i["cpu_p95"] < 20 and i["mem_p95"] < 40:
        rightsized.add(i["id"])
        findings.append(("rightsize", f"{i['id']} {i['type']} CPU p95 {i['cpu_p95']}% mem
  {i['mem_p95']}%: one size down", i["price_hr"] * HOURS * 0.5))

# 3. Off-hours for non-prod (running 60 of 168 hours); computed on the already-rightsized
  price to avoid double counting
for i in inv["ec2"]:
    if i["env"] in ("dev", "staging"):
        base = i["price_hr"] * (0.5 if i["id"] in rightsized else 1.0)
        findings.append(("schedule", f"{i['id']} ({i['env']}) off outside 08:00-20:00
  weekdays", base * HOURS * (1 - 60 / 168)))

# 4. Storage
for v in inv["ebs"]:
    if not v["attached"]:
        findings.append(("storage", f"{v['id']} unattached {v['type']} {v['size_gb']} GB:
  snapshot then delete", v["size_gb"] * P["ebs_gb"][v["type"]]))
    elif v["type"] == "gp2":
        findings.append(("storage", f"{v['id']} gp2 {v['size_gb']} GB: migrate to gp3",
  v["size_gb"] * (P["ebs_gb"]["gp2"] - P["ebs_gb"]["gp3"])))
for s in inv["snapshots"]:
    if s["age_days"] > 180 and not s["volume_exists"]:
        findings.append(("storage", f"{s['id']} {s['age_days']} days old, volume gone:
  delete after review", s["size_gb"] * P["snapshot_gb"]))
```

```
for g in inv["log_groups"]:
    if g["retention_days"] is None:
        findings.append(("logs", f"{g['name']} {g['stored_gb']} GB never expires: keep 30
  days (assume 8% retained)", g["stored_gb"] * P["logs_gb"] * 0.92))
for b in inv["s3"]:
    if not b["has_lifecycle"] and b["pattern"] == "write-once-read-rarely":
        findings.append(("s3", f"{b['bucket']} {b['size_gb']} GB: lifecycle to Standard-IA
  after 30 days", b["size_gb"] * (P["s3_standard_gb"] - P["s3_ia_gb"])))

# 5. Databases: Multi-AZ outside production, idle read replicas
for d in inv["rds"]:
    if d["multi_az"] and d["env"] != "prod":
        findings.append(("database", f"{d['id']} ({d['env']}) Multi-AZ standby not needed:
  single AZ", d["node_price_hr"] * HOURS))
    for r in d["read_replicas"]:
        if r["cpu_p95"] < 10:
            findings.append(("database", f"{r['id']} read replica CPU p95 {r['cpu_p95']}%:
  confirm no readers, then remove", d["node_price_hr"] * HOURS))

# 6. Network
n = inv["network"]
if not n["has_s3_gateway_endpoint"]:
    findings.append(("network", f"S3 gateway endpoint: {n['s3_share_of_nat']:.0%} of
  {n['nat_gb_month']} GB leaves NAT", n["nat_gb_month"] * n["s3_share_of_nat"] *
  P["nat_gb"]))

# 7. Kubernetes requests (target request = p95 * 1.3)
for ns in inv["k8s_namespaces"]:
    new_cpu = ns["cpu_used_p95_m"] * 1.3
    saved_vcpu = max(0, ns["cpu_requested_m"] - new_cpu) / 1000
    findings.append(("k8s", f"{ns['ns']}: cpu requests {ns['cpu_requested_m']}m vs p95
  {ns['cpu_used_p95_m']}m → {new_cpu:.0f}m", saved_vcpu * P["node_hour_per_vcpu"] *
  HOURS))

print(f"\n{'area':<10} {'monthly saving':>14}  finding")
for area, text, saving in sorted(findings, key=lambda f: -f[2]):
    print(f"{area:<10} ${saving:>12,.0f}  {text}")
total = sum(f[2] for f in findings)
print(f"\ntotal identified saving: ${total:,.0f}/month (${total * 12:,.0f}/year) before
  commitments")

# 8. Savings plan on the remaining steady floor (30% example discount)
usage = inv["hourly_usage_od_equiv"]
discount, best = 0.30, None
for commit in range(5, 30):
    cap = commit / (1 - discount)
    cost = sum(commit + max(0, u - cap) for u in usage)
    best = min(best, (cost, commit)) if best else (cost, commit)
od = sum(usage)
print(f"savings plan: commit ${best[1]}/h gives ${od - best[0]:,.0f} saving per week on
  ${od:,.0f} of on-demand usage ({1 - best[0] / od:.1%})")
```

```bash
python3 audit.py
```

Expected output:

```
tag compliance: 3/6 EC2 instances fully tagged; untagged spend $917 of $1,478 (62%)

area       monthly saving  finding
idle       $         496  i-mystery c6i.4xlarge CPU p95 2%, no owner: stop after
  confirming
database   $         380  shop-prod-r1 read replica CPU p95 3%: confirm no readers, then
  remove
database   $         380  shop-dev (dev) Multi-AZ standby not needed: single AZ
k8s        $         284  team-shop: cpu requests 12000m vs p95 3000m → 3900m
network    $         243  S3 gateway endpoint: 60% of 9000 GB leaves NAT
k8s        $         192  dev: cpu requests 6000m vs p95 400m → 520m
rightsize  $         140  i-batch-1 m6i.2xlarge CPU p95 12% mem 18%: one size down
rightsize  $         140  i-dev-1 m6i.2xlarge CPU p95 9% mem 20%: one size down
s3         $         126  shop-logs 12000 GB: lifecycle to Standard-IA after 30 days
storage    $         125  vol-c unattached io1 1000 GB: snapshot then delete
schedule   $          90  i-dev-1 (dev) off outside 08:00-20:00 weekdays
rightsize  $          70  i-stage-1 m6i.xlarge CPU p95 15% mem 30%: one size down
storage    $          50  vol-b unattached gp2 500 GB: snapshot then delete
storage    $          50  snap-3 700 days old, volume gone: delete after review
schedule   $          45  i-stage-1 (staging) off outside 08:00-20:00 weekdays
k8s        $          43  team-pay: cpu requests 8000m vs p95 5200m → 6760m
storage    $          25  snap-2 400 days old, volume gone: delete after review
logs       $           9  /aws/lambda/shop-api 340 GB never expires: keep 30 days (assume
  8% retained)
logs       $           3  /ecs/reports 120 GB never expires: keep 30 days (assume 8%
  retained)
storage    $           2  vol-d gp2 100 GB: migrate to gp3

total identified saving: $2,894/month ($34,730/year) before commitments
savings plan: commit $13/h gives $884 saving per week on $3,444 of on-demand usage (25.7%)
```

Read it before changing anything. The biggest items are rarely the ones people guess: an unowned instance, an idle replica, a non-production Multi-AZ database and inflated Kubernetes requests. The savings are **estimates**; several overlap (rightsizing and scheduling the same instance, for example) and the script nets the obvious one. A real audit confirms each line with its owner before acting.

Milestone 3 — Enforce the tag policy (topic 1)

The audit reports 3 of 6 instances fully tagged. Fix the cause by checking Terraform plans in CI. Use the `tags.rego` policy from FinOps §20.1 step 3 and a fake plan:

```bash
opa eval -f pretty -d tags.rego -i plan.json 'data.main.deny'
```

You should see one message naming each resource missing `cost-center`, `env` or `service`. Add `default_tags` to your Terraform provider block so every resource inherits the schema.

Milestone 4 — Rightsize EC2 and Kubernetes (topics 4, 5, 9)

For each `rightsize` line, decide with the p95 rule from Rightsizing §20.5: one size down only if CPU and memory at p95 both fit with headroom. For the Kubernetes lines, the target request is the p95 plus 30 percent headroom. Add the request cap policy from Kubernetes Infrastructure Costs §20.9 step 2 so inflation cannot return:

```bash
opa eval -f pretty -d requests.rego -i deploy.yaml 'data.main.deny'
```

For the non-production instances, add the off-hours schedule from Autoscaling for Cost §20.4 (a CronJob pair for Kubernetes or scheduled actions for an Auto Scaling group).

Milestone 5 — Lifecycle policies and network fix (topics 6, 7)

Write an S3 lifecycle configuration for `shop-logs` (Standard-IA after 30 days, expiry at a retention you justify), the multipart-upload cleanup rule, a retention period for the two immortal log groups, and an S3 gateway endpoint for the VPC. Use the JSON and commands from Storage Lifecycle Policies §20.6 and Network Egress Costs §20.7. Validate the JSON before applying anywhere:

```bash
python3 -c "import json; print(len(json.load(open('lifecycle.json'))['Rules']), 'rules')"
```

Milestone 6 — Databases and Spot (topics 3, 8)

For `shop-prod-r1` and `shop-dev`, apply the questions from Database Costs §20.8: is the replica serving reads, is Multi-AZ needed here, and could a cache remove the load? Run `db_cost.py` with your own numbers. Then list the workloads in the inventory that could run on Spot (batch, CI, stateless web with an on-demand base) using the criteria from Spot Instances §20.3, and run `spot_breakeven.py`.

Milestone 7 — Commitment proposal (topic 2)

The last line of the audit sizes a Savings Plan against the sample week of hourly usage: the best commitment is near the always-on floor, not the peak. After the other savings are applied, usage is lower, so **recompute the commitment on the reduced usage**: committing before you remove waste locks the waste in. Write the proposal: scope (Compute), term, payment option, hourly commitment, expected coverage and utilization, worst case if usage falls 30 percent, owner and review date.

Milestone 8 — The savings proposal (all topics)

Write a one-page document `proposal.md` using this outline:

```markdown
# Cost savings proposal: shop-api AWS and Kubernetes
## Summary
Identified monthly saving: $X (Y percent of the example bill). First wave (low risk): $A.
## Findings ranked by value
| # | Area | Finding | Saving/mo | Effort | Risk | Owner |
## Plan
Wave 1 (this week): delete confirmed waste, set log retention, gateway endpoint.
Wave 2 (this month): rightsize, schedules, lifecycle policies.
Wave 3 (next quarter): Savings Plan on the recomputed floor, Spot for batch, database
  changes.
## Risks and rollback
## Operating model
Tag policy in CI, budget alerts, monthly review, unit cost metric (cost per 1M requests).
```

5 Tasks

- [ ] **T1 — Run the audit** *(≈30 min, beginner; Milestones 1-2; topics 1, 5)*
  - **Goal:** produce the findings table.
  - **Steps:**
    1. Create `make_sample_data.py` and `audit.py` and run both.
    2. Copy the findings into a spreadsheet or Markdown table.
  - **Done when:** you have a ranked table and the total monthly saving figure.
- [ ] **T2 — Make tags enforceable** *(≈45 min, intermediate; Milestone 3; topic 1)*
  - **Goal:** untagged resources fail CI.
  - **Steps:**
    1. Write `tags.rego` and a fake plan with one untagged bucket.
    2. Run the policy, then fix the plan.
    3. Add `default_tags` to a Terraform provider block.

- **Done when:** the policy output goes from one message to `[]` and the tag schema is documented.

- [ ] **T3 — Rightsize with evidence** *(≈60 min, intermediate; Milestone 4; topics 4, 5, 9)*
  - **Goal:** defend each size change.
  - **Steps:**
    1. For each rightsize line, state the p95 CPU and memory and the new size.
    2. Add one instance to `inventory.json` with `cpu_p95` 10 and `mem_p95` 85 (memory-bound) and confirm the audit does not suggest shrinking it.
    3. Write the rollback step for each change.
  - **Done when:** each change has numbers, a risk and a rollback, and the memory-bound instance is kept with a note explaining why.

- [ ] **T4 — Lifecycle and endpoint** *(≈60 min, intermediate; Milestone 5; topics 6, 7)*
  - **Goal:** storage and network savings as code.
  - **Steps:**
    1. Write `lifecycle.json` for `shop-logs` and validate it parses.
    2. Write the Terraform for the S3 gateway endpoint.
    3. Add the `endpoints.rego` check from Network Egress Costs §20.7 and test it on a plan without the endpoint.
  - **Done when:** the JSON validates and the policy flags a VPC with a NAT gateway and no endpoint.

- [ ] **T5 — Database and Spot review** *(≈60 min, advanced; Milestone 6; topics 3, 8)*
  - **Goal:** availability-aware recommendations.
  - **Steps:**
    1. For each database finding, write the availability impact and what evidence you need before acting.
    2. Run `spot_breakeven.py` for the batch workload with your own interruption assumptions.
    3. Decide which instances stay on-demand and why.
  - **Done when:** each change lists impact, evidence and a decision, and at least one tempting saving is rejected with a reason.

- [ ] **T6 — Size a commitment on reduced usage** *(≈45 min, advanced; Milestone 7; topic 2)*
  - **Goal:** a Savings Plan that does not lock in waste.
  - **Steps:**

1. Take `hourly_usage_od_equiv` from `inventory.json` and subtract the compute savings you will apply (for example 15 percent).
2. Re-run the sweep from Reserved Instances and Savings Plans §20.2 and pick a commitment.
3. Compute the loss if usage falls a further 30 percent.

- **Done when:** you have a recommendation with coverage, utilization, saving and the downside figure. Do not buy anything.

- [ ] **T7 — Repeat one lever on real data** *(≈60 min, intermediate; Milestones 2-5; topics 1, 6)*
  - **Goal:** connect the sample to reality.
  - **Steps:**
    1. In a sandbox account, list log groups without retention and unattached volumes using the commands from the topics.
    2. Compare against the sample audit's findings.
  - **Done when:** you have a list of real findings (or a statement that the account is clean) and have not deleted anything without review.

- [ ] **T8 — Prove it** *(≈90 min, advanced; all milestones; all topics)*
  - **Goal:** present the proposal.
  - **Steps:**
    1. Without the guide, write `proposal.md` for a different environment (change the inventory numbers) and justify the ranking.
    2. Present it in ten minutes to a peer, covering the unit cost, the three waves, the risks, and the operating model.
    3. Answer: what would you not cut, and why?
  - **Done when:** the peer can restate the three biggest savings, the biggest risk and what keeps the saving from decaying.

6 Verification checklist

- [ ] `python3 audit.py` runs and prints a total saving and a Savings Plan line.
- [ ] The findings table is ranked, and each finding has an owner, a risk and an action.
- [ ] The tag policy returns messages on a bad plan and `[]` on a good one.
- [ ] `lifecycle.json` parses and contains the multipart cleanup rule.
- [ ] The request-cap policy rejects an oversized Deployment.

- [ ] The Savings Plan proposal was sized on **reduced** usage and states the downside.
- [ ] `proposal.md` states monthly and yearly savings, three waves and a rollback for risky changes.
- [ ] Nothing was purchased and nothing in a real account was deleted without review.

7 Teardown

The synthetic audit creates only local files:

```bash
cd .. && rm -rf cost-audit
```

If you repeated steps against a real sandbox account, remove what you created:

```bash
aws s3 rb "s3://<YOUR_TEST_BUCKET>" --force            # test bucket from the lifecycle task (empty versions first in the console)
aws ec2 delete-vpc-endpoints --vpc-endpoint-ids <ID>    # only if you created a lab endpoint
aws budgets delete-budget --account-id <YOUR_ACCOUNT_ID> --budget-name shop-monthly
kubectl delete namespace team-shop --ignore-not-found
```

Check the Billing console the next day to confirm nothing is still accruing.

8 Stretch goals and what to learn next

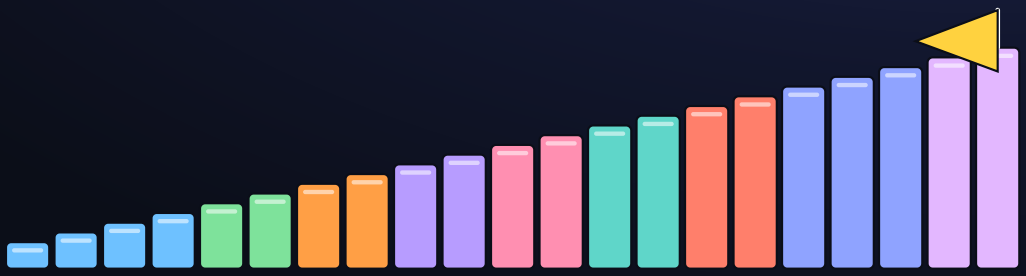
- Replace the sample inventory with output from `aws ec2 describe-instances`, `aws ec2 describe-volumes`, `aws logs describe-log-groups` and a Prometheus query, and keep the audit as a scheduled job that posts a weekly report.
- Add a cost-per-1M-requests dashboard (see FinOps §20.1) and an anomaly alert.
- Add a cost estimate to pull requests with a policy gate on large increases (see Policy as Code §18.7).
- Turn the findings into guardrails in the platform templates (Infrastructure Templates §18.6).
- Next: Level 19: Architecture §19.5 covers cost as a first-class architecture trade-off, and Level 14: SRE §14.9 covers capacity planning that keeps savings from becoming outages.

LAB NOTES What I ran, what I saw, what it means

PART X

The Summit Projects

Nine projects that chain every level into one evolving production system, from a hand-built server to a complete platform.



IN THIS PART

·	Practical Project Progression	3835
P1	Basic Server	3837
P2	Docker	3849
P3	CI/CD	3863
P4	Terraform	3875
P5	Kubernetes	3889
P6	Production Kubernetes	3903
P7	Observability	3917
P8	Disaster Recovery	3929
P9	Senior Architecture	3943

ONE SYSTEM, NINE STAGES OF GROWTH

P1	Ubuntu + Nginx + Node.js + PostgreSQL deployed manually.
P2	Node.js + PostgreSQL + Redis + Nginx containerized with Docker Compose.
P3	GitHub → GitHub Actions → tests → Docker → registry → deployment.
P4	Provision AWS VPC, subnets, EC2, ALB and RDS using Terraform.
P5	Move the application to Kubernetes.
P6	Add Ingress, TLS, HPA, RBAC, Secrets, NetworkPolicies, Helm and Argo CD.
P7	Add Prometheus, Grafana, Loki, OpenTelemetry and Sentry.
P8	Simulate infrastructure and database failures and recover from them.
P9	Build a complete production-style platform using AWS, Terraform, Docker, Kubernetes, Helm, GitOps and observability.

PART X

Practical Project Progression

Keep evolving **one realistic production project** as you learn each technology. This turns individual tools into a coherent DevOps skill set.

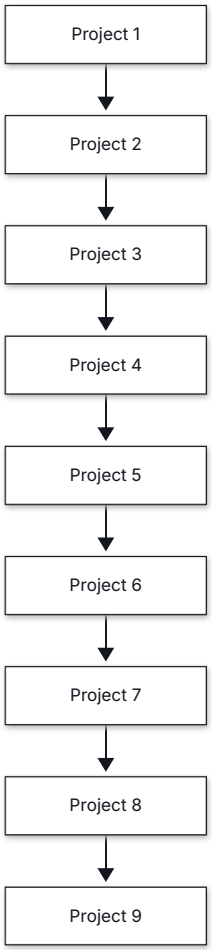


Figure 1

#	Project	What you build	Levels used
1	Project 1 - Basic Server	Ubuntu + Nginx + Node.js + PostgreSQL deployed manually.	Levels 0-2

#	Project	What you build	Levels used
2	Project 2 - Docker	Node.js + PostgreSQL + Redis + Nginx containerized with Docker Compose.	Level 4
3	Project 3 - CI/CD	GitHub → GitHub Actions → tests → Docker → registry → deployment.	Levels 3, 5
4	Project 4 - Terraform	Provision AWS VPC, subnets, EC2, ALB and RDS using Terraform.	Levels 6-7
5	Project 5 - Kubernetes	Move the application to Kubernetes.	Level 8
6	Project 6 - Production Kubernetes	Add Ingress, TLS, HPA, RBAC, Secrets, NetworkPolicies, Helm and Argo CD.	Levels 8, 9, 16, 17
7	Project 7 - Observability	Add Prometheus, Grafana, Loki, OpenTelemetry and Sentry.	Level 10
8	Project 8 - Disaster Recovery	Simulate infrastructure and database failures and recover from them.	Levels 13-15
9	Project 9 - Senior Architecture	Build a complete production-style platform using AWS, Terraform, Docker, Kubernetes, Helm, GitOps and observability.	Levels 6-20

THE SUMMIT PROJECTS

P1 Basic Server

Project 1 of 9 | Uses levels: Levels 0-2 | ~10 hours

1 Goal and outcome

Deploy `shop-api` **manually** on one Ubuntu server: Nginx in front, a Node.js API supervised by systemd, PostgreSQL behind it, SSH locked down, firewall on, a nightly database backup, and a script that proves it all works. No Docker, no pipeline, no cloud tooling. This is the baseline that **every later project automates, containerizes, codes or orchestrates**. If you have done it once by hand, you know what each of those tools is doing for you, and you can debug it when it does it wrong.

This project is the integration of three levels. The Level 1 capstone Capstone 1 builds the server step by step; this page tells you what "done" means for the **whole project series**, adds the pieces later projects depend on (a stable app contract, health endpoints, a backup you can restore, a verification script), and connects the server to what you learned about networking in Level 2 Capstone 2. Use the capstone for the long command listings and this page as the project brief.

You finish with:

- `curl http://<server-ip>/products` returning JSON from PostgreSQL through Nginx;
- SSH by key only, a deny-by-default firewall, and the app and database not reachable from the internet except through Nginx;
- the API as a systemd service under its own user, restarting on failure, logging to the journal;
- a nightly `pg_dump` backup and a proven restore into a scratch database;
- a `verify.sh` that prints `ALL CHECKS PASSED`, and a one-page "what I deployed" note you could hand to a colleague.

2 Architecture

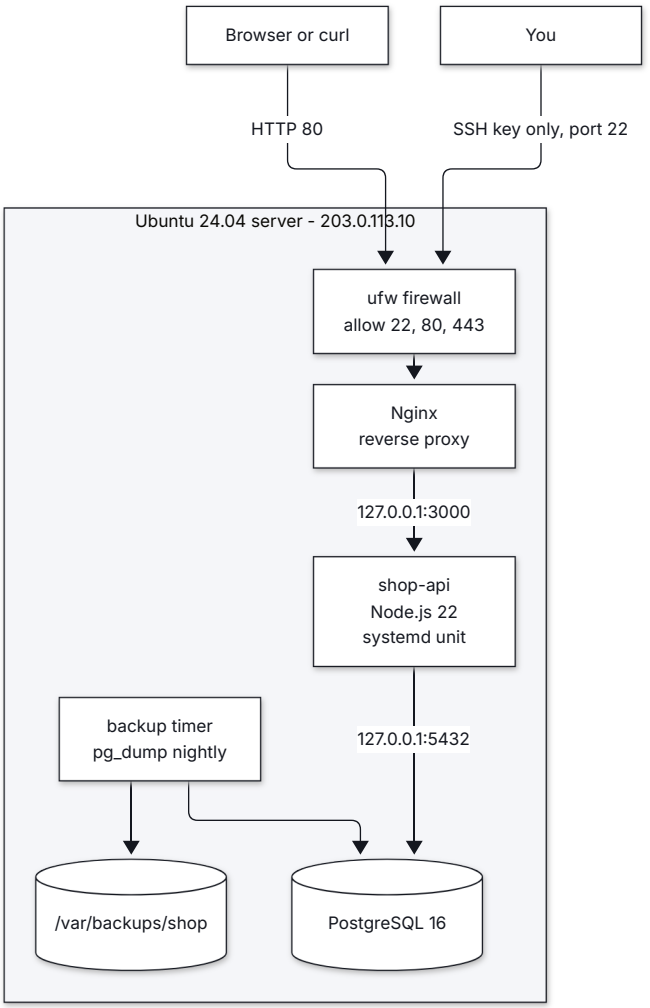


Figure 1-1

This is the starting point, so everything is new. In the next projects the boxes change but the arrows stay:

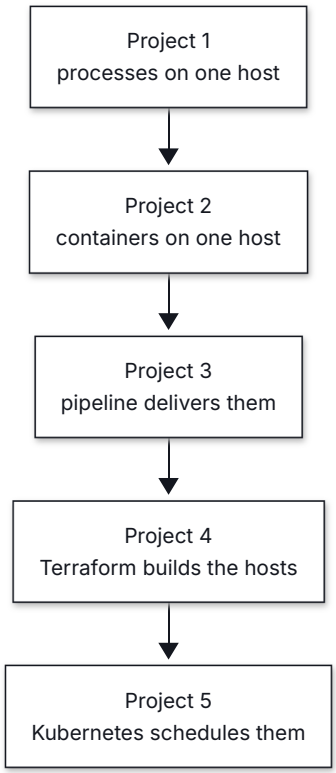


Figure 1-2

The application contract below is shared by **all nine projects**. Keep it stable; the infrastructure changes around it.

Item	Value
Listens on	`PORT` (default `3000`), bound to `127.0.0.1` here, `0.0.0.0` in containers
Configuration	environment variables only: `DATABASE_URL`, `PORT`, later `REDIS_URL`
`GET /healthz`	`200 {"status":"ok"}` as long as the process runs (liveness)
`GET /readyz`	`200` when PostgreSQL answers `SELECT 1`, otherwise `503` (readiness)
`GET /products`	JSON list from table `products`
Logs	one JSON object per line on stdout
Shutdown	on `SIGTERM`: stop accepting, drain, close the pool, exit 0

3 Prerequisites

- A blank **Ubuntu 24.04 LTS** VPS (1 vCPU / 1 GB RAM is enough; any provider; about 5 to 10 USD per month, or a local VM at no cost) and an SSH key pair. Destroy it when finished.
- Linux: filesystem hierarchy §1.1, permissions §1.6, users and sudo §1.7, systemd §1.9, journalctl §1.11, SSH keys §1.12.
- Networking: ports and firewalls §2.4, reverse proxies §2.8, network debugging tools §2.10.
- Concepts: what DevOps is Level 0, especially the SDLC §0.1 and infrastructure as code §0.3, which is what you will be doing instead of this in Project 4.
- On your laptop: `ssh`, `curl`, `git`. Replace `203.0.113.10` with your server's IP everywhere.

4 Build it step by step

Milestone 1 — Server, user and SSH hardening (≈1.5 h)

Follow Milestones M0 and M1 of the Level 1 capstone Capstone 1. The end state to reach:

```bash
# on the server, as the admin user "deploy"
sudo sshd -T | grep -Ei '^(permitrootlogin|passwordauthentication|pubkeyauthentication)'
# permitrootlogin no
# passwordauthentication no
# pubkeyauthentication yes
```

Keep a second SSH session open while you change `sshd_config`, and run `sudo sshd -t` before reloading. A typo can lock you out.

Milestone 2 — Packages: Nginx, Node.js, PostgreSQL (≈45 min)

```bash
sudo apt-get update
sudo apt-get install -y nginx postgresql ufw curl ca-certificates gnupg
# Node.js 22 LTS from NodeSource (Ubuntu's own nodejs package is older); see the L1
  capstone for the key/repo commands
node --version          # v22.x
psql --version          # psql (PostgreSQL) 16.x
nginx -v                # nginx version: nginx/1.24.x
```

Milestone 3 — Database, schema and a least-privilege role (≈45 min)

```bash
DB_PASS="$(openssl rand -base64 24 | tr -d '/+=')"
```

```
sudo -u postgres psql <<SQL
CREATE ROLE shop LOGIN PASSWORD '${DB_PASS}';
CREATE DATABASE shop OWNER shop;
SQL
sudo -u postgres psql -d shop <<'SQL'
CREATE TABLE products (
  id          serial PRIMARY KEY,
  name        text    NOT NULL,
  price_cents integer NOT NULL CHECK (price_cents ⩾ 0)
);
INSERT INTO products (name, price_cents) VALUES ('Keyboard', 4999), ('Mouse', 1999),
  ('Monitor', 17999);
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO shop;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO shop;
SQL
echo "DATABASE_URL=postgres://shop:${DB_PASS}@127.0.0.1:5432/shop" | sudo tee /etc/shop-
  api.env >/dev/null
# the env file gets its owner and mode in milestone 4, once the service account exists
ss -ltnp | grep 5432      # must show 127.0.0.1:5432 only, never 0.0.0.0
```

PostgreSQL on Ubuntu listens on loopback by default. Confirm that; do not "fix" it by opening `listen_addresses`.

Milestone 4 — The application and its systemd unit (≈1.5 h)

Create the service account, deploy directory and application (the full `server.js` is in the Level 1 capstone; add `/healthz` and `/readyz` so the contract matches):

BASH

```
sudo useradd --system --home /opt/shop-api --shell /usr/sbin/nologin shop-api
sudo install -d -o shop-api -g shop-api /opt/shop-api
sudo chown root:shop-api /etc/shop-api.env && sudo chmod 640 /etc/shop-api.env
# copy package.json, package-lock.json and server.js to /opt/shop-api, then:
sudo -u shop-api bash -c 'cd /opt/shop-api && npm ci --omit=dev --no-audit --no-fund'
```

The two health routes to add to your request handler:

JS

```
if (req.method === 'GET' && url.pathname === '/healthz') return send(res, 200, { status:
  'ok' });
if (req.method === 'GET' && url.pathname === '/readyz') {
  await pool.query('SELECT 1');                          // throws → caught below → 503
  return send(res, 200, { status: 'ready' });
}
```

`/etc/systemd/system/shop-api.service`:

INI

```
[Unit]
Description=shop-api (Node.js)
After=network-online.target postgresql.service
Wants=network-online.target

[Service]
User=shop-api
Group=shop-api
WorkingDirectory=/opt/shop-api
EnvironmentFile=/etc/shop-api.env
Environment=PORT=3000 HOST=127.0.0.1 NODE_ENV=production
ExecStart=/usr/bin/node /opt/shop-api/server.js
Restart=on-failure
RestartSec=2
# hardening: cheap, and the habit matters
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

BASH

```
sudo systemctl daemon-reload
sudo systemctl enable --now shop-api
systemctl is-active shop-api                    # active
curl -s http://127.0.0.1:3000/readyz            # {"status":"ready"}
journalctl -u shop-api -n 5 --no-pager          # JSON log lines
```

Milestone 5 — Nginx reverse proxy (≈45 min)

`/etc/nginx/sites-available/shop-api`:

NGINX

```
upstream shop_api { server 127.0.0.1:3000; keepalive 16; }

server {
    listen 80 default_server;
    server_name _;

    location / {
        proxy_pass http://shop_api;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 2s;
```

```
        proxy_read_timeout 15s;
    }
}
```

```bash
sudo ln -sf /etc/nginx/sites-available/shop-api /etc/nginx/sites-enabled/shop-api
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
curl -s http://127.0.0.1/products                  #
  [{"id":1,"name":"Keyboard","price_cents":4999}, ...]
```

Milestone 6 — Firewall (≈20 min)

```bash
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp          # for later; nothing listens yet
sudo ufw --force enable
sudo ufw status verbose
```

From your **laptop**, prove the exposure is what you intend: `curl -m 3 http://203.0.113.10:3000/` must time out or be refused, and `nc -vz -w 3 203.0.113.10 5432` must fail. (A closed port gives "refused", a filtered one gives a timeout. Level 2 explains why.)

Milestone 7 — Backup and verification (≈1 h)

`/usr/local/sbin/shop-backup.sh`:

```bash
#!/usr/bin/env bash
set -euo pipefail
DEST=/var/backups/shop
STAMP="$(date -u +%Y%m%dT%H%M%SZ)"
install -d -m 700 -o postgres -g postgres "$DEST"
sudo -u postgres pg_dump --format=custom --file="$DEST/shop-$STAMP.dump" shop
find "$DEST" -name 'shop-*.dump' -mtime +7 -delete       # keep a week
echo "backup ok: $DEST/shop-$STAMP.dump"
```

Run it from a systemd timer (`shop-backup.service` with `Type=oneshot` , `shop-backup.timer` with `OnCalendar=*-*-* 02:30:00` and `Persistent=true`), then restore it into a scratch database to prove it works:

```bash
sudo chmod 750 /usr/local/sbin/shop-backup.sh && sudo /usr/local/sbin/shop-backup.sh
sudo -u postgres createdb shop_restore_test
sudo -u postgres pg_restore --dbname=shop_restore_test /var/backups/shop/shop-*.dump
sudo -u postgres psql -d shop_restore_test -tAc 'SELECT count(*) FROM products;'     # 3
sudo -u postgres dropdb shop_restore_test
```

A minimal `verify.sh` (run it on the server; extend it as you go):

```bash
#!/usr/bin/env bash
fail=0
check() { if eval "$2" >/dev/null 2>&1; then echo "PASS  $1"; else echo "FAIL  $1";
  fail=1; fi; }
check "shop-api active"            "systemctl is-active --quiet shop-api"
check "nginx active"               "systemctl is-active --quiet nginx"
check "readyz through nginx"       "curl -fsS http://127.0.0.1/readyz"
check "products returns JSON"      "curl -fsS http://127.0.0.1/products | python3 -c
  'import json,sys; assert len(json.load(sys.stdin))>0'"
check "app only on loopback"       "! ss -ltn | grep -E '0\.0\.0\.0:3000|\*:3000'"
check "postgres only on loopback"  "! ss -ltn | grep -E '0\.0\.0\.0:5432|\*:5432'"
check "ssh passwords disabled"     "sudo sshd -T | grep -qi '^passwordauthentication no'"
check "ufw active"                 "sudo ufw status | grep -q 'Status: active'"
check "recent backup exists"       "sudo find /var/backups/shop -name '*.dump' -mmin -1500
  | grep -q ."
[ $fail -eq 0 ] && echo "ALL CHECKS PASSED" || { echo "SOME CHECKS FAILED"; exit 1; }
```

5 Tasks

- [ ] **T1 — Build the server from a blank image** *(≈3 h, beginner)*
  - **Goal:** a hardened host with an admin user, key-only SSH and a firewall.
  - **Steps:**
    1. Create the VPS and complete Milestone 1 and Milestone 6.
    2. From a second terminal, confirm `ssh -o PreferredAuthentications=password deploy@203.0.113.10` is rejected.
    3. Record `sudo ufw status verbose` in your notes.
  - **Done when:** key login works, password login fails with `Permission denied (publickey)` , and `ufw` shows only 22, 80, 443 allowed.

- [ ] **T2 — Get the API serving data through Nginx** *(≈2 h, beginner)*
  - **Goal:** the request path browser, Nginx, Node.js, PostgreSQL works end to end.
  - **Steps:**
    1. Complete Milestones 2 to 5.
    2. From your laptop run `curl -i http://203.0.113.10/products` and note the `Server: nginx` header.
    3. Run `sudo journalctl -u shop-api -f` while you curl, and find your request in the JSON log.
  - **Done when:** `/products` returns the three products from your laptop and the matching log line appears in the journal.
- [ ] **T3 — Prove the trust boundaries** *(≈30 min, intermediate)*
  - **Goal:** show that only Nginx is reachable.
  - **Steps:**
    1. From the laptop, probe ports 22, 80, 3000, 5432: `nc -vz -w 3 203.0.113.10 <port>`.
    2. On the server run `sudo ss -ltnp` and map each listener to a process and an address.
    3. Draw the trust boundaries on paper: which addresses can each listener be reached from?
  - **Done when:** 3000 and 5432 fail from outside, and `ss` shows them bound to `127.0.0.1`, and your drawing matches.
- [ ] **T4 — Change something safely** *(≈45 min, intermediate)*
  - **Goal:** make a code change and roll it out by hand, the way you will automate it in Project 3.
  - **Steps:**
    1. Add `GET /products/:id` (or change a price in the table).
    2. Copy the new `server.js` with `scp`, `sudo systemctl restart shop-api`, and re-test.
    3. Write the exact commands you ran into `deploy.sh` so it is repeatable (copy, restart, `curl /readyz` loop, print a failure and exit non-zero if it never becomes ready).
  - **Done when:** `./deploy.sh` redeploys and exits 0 only if `/readyz` returns 200 within 30 seconds.
- [ ] **T5 — Back up and restore** *(≈45 min, intermediate)*
  - **Goal:** a restore you have actually performed.
  - **Steps:**

1. Install the backup script and timer (Milestone 7); run `systemctl start shop-backup.service`.
2. `sudo -u postgres psql -d shop -c "DELETE FROM products WHERE id = 3;"` to simulate a mistake.
3. Restore the dump into `shop_restore_test`, and copy the missing row back into `shop`.

- **Done when:** `SELECT count(*) FROM products` in `shop` returns 3 again, and `systemctl list-timers` shows `shop-backup.timer` scheduled.

☐ **T6 — Kill it and watch it recover** *(≈30 min, intermediate)*

- **Goal:** understand supervision.
- **Steps:**
  1. `sudo systemctl kill -s SIGKILL shop-api` and watch `systemctl status shop-api` and the journal.
  2. Run `sudo systemctl stop postgresql`, then `curl -i http://127.0.0.1/readyz`, then start it again.
  3. Note which of `Restart=on-failure`, `/readyz` and Nginx's behaviour each part of this relied on.
- **Done when:** the unit restarts within about 2 seconds after SIGKILL, and `/readyz` returned 503 (not a hang) while PostgreSQL was down.

☐ **T7 — Automate your own verification** *(≈45 min, advanced)*

- **Goal:** turn "it works" into a repeatable test.
- **Steps:**
  1. Create `verify.sh` from Milestone 7 and run it with `sudo ./verify.sh`.
  2. Add two checks of your own (for example TLS expiry later, disk usage under 80 percent, `nginx -t` passing).
  3. Break one thing on purpose and confirm the script fails with a useful message.
- **Done when:** `sudo ./verify.sh` prints `ALL CHECKS PASSED` on a healthy host and a `FAIL` line naming the broken item otherwise.

☐ **T8 — Prove it: rebuild from your notes in one sitting** *(≈2 h, advanced)*

- **Goal:** show you can reproduce the host, not just that it once worked.
- **Steps:**
  1. Destroy the VPS. Create a new one. Using only your notes and `deploy.sh`, rebuild everything without reading this page.
  2. Time the rebuild and list every step you had to look up.

3. Explain aloud (or in writing) what happens from `curl` to the database row, naming each process, port and file involved.

- **Done when:** `sudo ./verify.sh` passes on the rebuilt host and you can name at least five manual steps that Projects 2 to 4 will remove.

6 Definition of done

- [ ] `curl http://<server-ip>/products` returns JSON through Nginx; `/readyz` returns 200.
- [ ] SSH password authentication and root login are disabled; the firewall allows only 22, 80 and 443.
- [ ] `shop-api` runs as the unprivileged `shop-api` user under systemd with `Restart=on-failure`; secrets are in `/etc/shop-api.env` with mode `640`.
- [ ] Ports 3000 and 5432 listen on `127.0.0.1` only and are unreachable from the internet.
- [ ] A backup timer exists, and you restored a backup into a scratch database and counted the rows.
- [ ] `verify.sh` prints `ALL CHECKS PASSED`.
- [ ] A one-page note lists the packages, files, users, ports and commands of the deployment (you are writing the first runbook).

7 Break it on purpose

Do each drill, write down the symptom you saw, the command that found the cause, and the fix.

1. **Wrong ownership on the env file.** `sudo chown root:root /etc/shop-api.env && sudo chmod 600 /etc/shop-api.env`, then restart. The service fails; find the permission error with `journalctl -u shop-api`. Fix with the correct group. *(Lesson: permissions and service accounts.)*
2. **Nginx points at the wrong port.** Change `server 127.0.0.1:3000` to `3001`, `nginx -t && systemctl reload nginx`. You get `502 Bad Gateway`; confirm in `/var/log/nginx/error.log` ("connect() failed … Connection refused"). *(Lesson: a 502 means the proxy is fine and the upstream is not.)*
3. **Break the SSH config, safely.** Keep a second SSH session open. Add the typo `PermitRootLogin noo` to a file in `/etc/ssh/sshd_config.d/`, then run `sudo sshd -t`. The validator reports the bad value before anything reloads. Remove the file. *(Lesson: validate before reload; keep a second session.)*

4. **Fill the disk.** Create a large file with `sudo fallocate -l 2G /var/tmp/filler` (size it to leave your small VPS only a little free space), watch `df -h`, and check what fails first (journal writes, PostgreSQL writes, backups). Delete the file. *(Lesson: disks are shared fate; monitoring arrives in Project 7.)*

8 How this evolves into the next project

Everything you did by hand has a cost: the server is a **pet**. Its state lives in package versions, files in `/etc`, and your memory. If it dies you rebuild it from notes, and two engineers will rebuild it differently.

Project 2 — Docker keeps the same application contract and the same request path, but puts Nginx, the API and PostgreSQL in **containers** described by a Compose file, and adds Redis as a cache. The `systemd` unit becomes a `restart:` policy and a health check, `/etc/shop-api.env` becomes environment variables, `apt install` becomes `image:`, and `verify.sh` becomes a smoke test you can run on any laptop. Keep your `deploy.sh` and `verify.sh`: you will reuse the checks and later run them from a pipeline in Project 3.

THE SUMMIT PROJECTS

P2 Docker

Project 2 of 9 | Uses levels: Level 4 | ~8 hours

1 Goal and outcome

Take the `shop-api` you deployed by hand in Project 1 and run the **whole stack in containers** with Docker Compose: Nginx, the Node.js API, PostgreSQL and a new Redis cache. One command (`docker compose up -d --wait`) replaces an afternoon of `apt install`, systemd units and file edits, and works the same on your laptop, a colleague's laptop and a CI runner.

The full containerization story, including the monitoring stack, lives in the Level 4 capstone Capstone 4. This project is the **integration step**: it keeps the exact application contract from Project 1, changes only how it is packaged and wired, and defines what you must be able to prove before moving on to a pipeline.

You finish with:

- a multi-stage, non-root, health-checked image `shop-api:dev`;
- `compose.yaml` bringing up four services in dependency order, gated by health checks;
- PostgreSQL data in a named volume that survives `docker compose down` and image upgrades;
- a private `data` network that has no route to the internet or the host, with only Nginx published on `127.0.0.1:8080`;
- Redis used as a cache-aside layer in front of `/products`, visible in an `X-Cache` response header;
- a `smoke.sh` test that replaces Project 1's `verify.sh` and runs anywhere Docker runs.

2 Architecture

What changed from Project 1: the server is gone as the unit of deployment. Processes became containers, `localhost` became service names on private networks, systemd became Compose, and Redis is new (marked NEW).

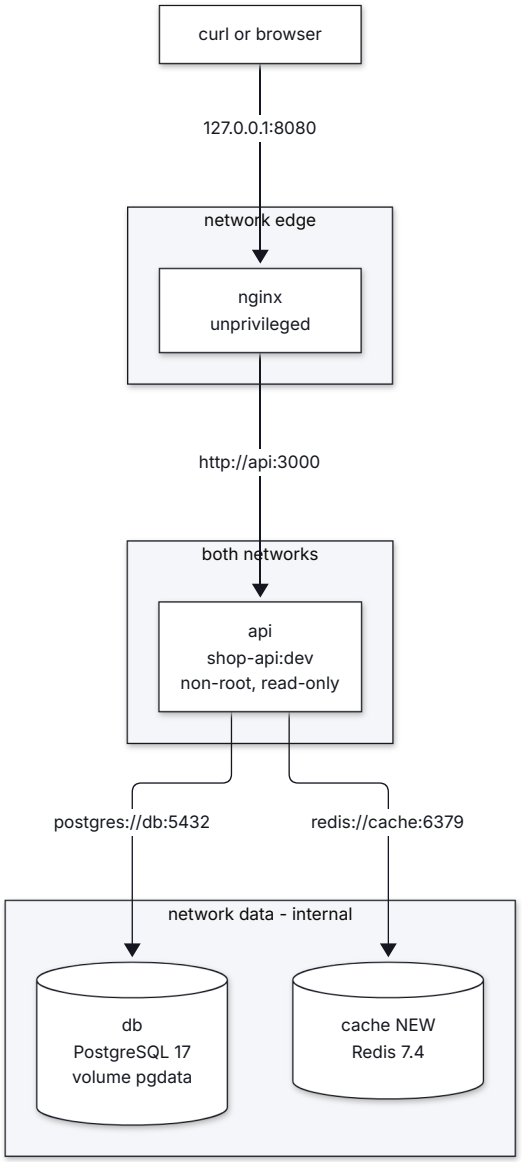


Figure 2-1

Request path for `GET /products` after this project (cache-aside):

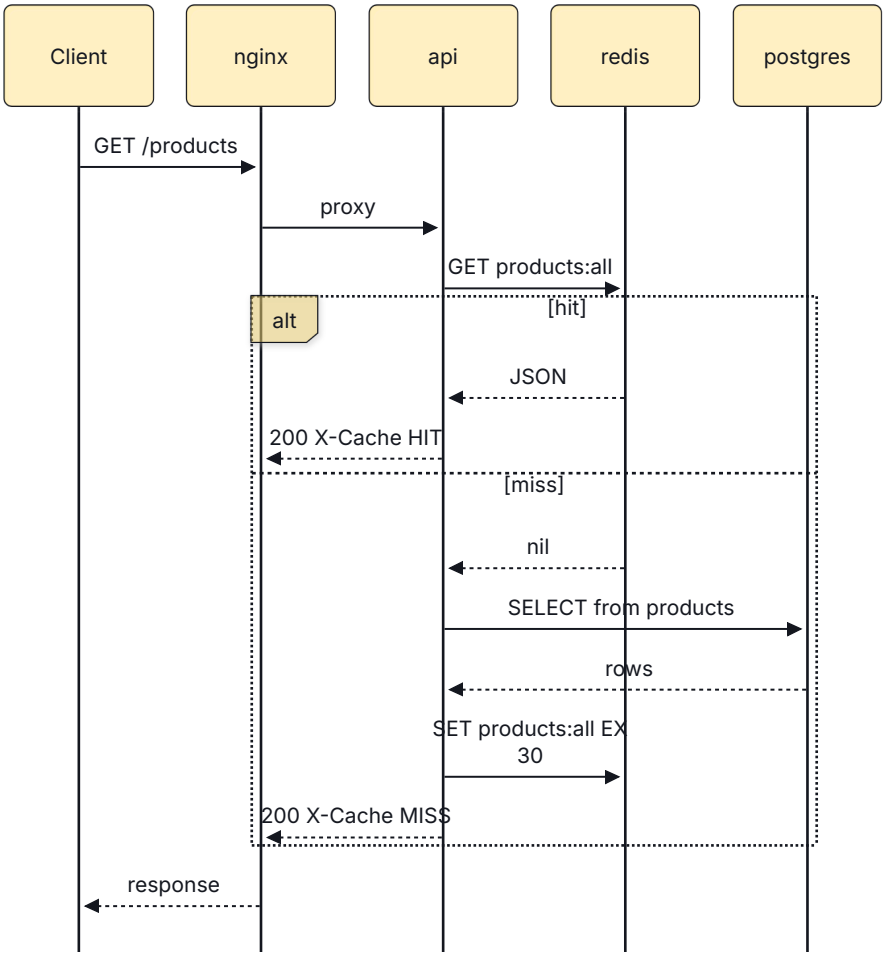


Figure 2-2

Project 1	Project 2
`apt install nginx`	`image: nginxinc/nginx-unprivileged:1.28-alpine`
systemd unit with `Restart=on-failure`	`restart: unless-stopped` plus a `healthcheck`
`/etc/shop-api.env`, mode 640	`.env` file (git-ignored) interpolated by Compose
PostgreSQL on loopback	PostgreSQL on an `internal` network, named volume `pgdata`
`ufw` allows 22, 80, 443	only `127.0.0.1:8080` is published; nothing else is reachable
`verify.sh` on the server	`smoke.sh` from your laptop

3 Prerequisites

- Docker Engine 25 or newer (or Docker Desktop) with Compose v2: `docker compose version`. About 2 GB of RAM and 2 GB of disk. Cost: nothing.
- Containers: Docker internals §4.1, images and layers §4.2, containers §4.3, volumes §4.5, networks §4.6, Dockerfile §4.7, Compose §4.8.
- Hardening topics used here: multi-stage builds §4.10, non-root containers §4.11, health checks §4.12.
- The Project 1 application source (`server.js`, `package.json`) and the SQL you used to create the `products` table.
- Redis background: Redis §13.7 (optional now, essential in Project 8).

4 Build it step by step

Create a working directory `shop/` containing your app. Files you will add are shown with their path in the first line.

Milestone 1 — Make the app container-friendly (≈45 min)

Containers need configuration from the environment, to bind to all interfaces, and to log to stdout. Check the contract:

- `HOST` defaults to `127.0.0.1` on a server but must be `0.0.0.0` in a container (Compose sets it).
- `DATABASE_URL` and `REDIS_URL` come from the environment; the app exits with a clear message if `DATABASE_URL` is missing.
- Add Redis caching to `/products` (cache-aside, 30 s TTL) and the `X-Cache` header:

```bash
npm install redis@^4 --save --no-audit --no-fund
```

```js
const { createClient } = require('redis');
const cache = createClient({ url: process.env.REDIS_URL });
cache.on('error', (e) => log('warn', 'redis error', { error: e.message }));   // never
  crash on cache trouble
cache.connect().catch(() => {});

async function cachedProducts() {
  try {
    const hit = cache.isReady ? await cache.get('products:all') : null;
    if (hit) return { rows: JSON.parse(hit), cache: 'HIT' };
```

```
  } catch (_) { /* fall through to the database */ }
  const { rows } = await pool.query('SELECT id, name, price_cents FROM products ORDER BY
  id');
  try { if (cache.isReady) await cache.set('products:all', JSON.stringify(rows), { EX: 30
  }); } catch (_) {}
  return { rows, cache: 'MISS' };
}
// in the /products handler:
//   const { rows, cache: c } = await cachedProducts();
//   res.writeHead(200, { 'Content-Type': 'application/json', 'X-Cache': c });
```

The cache is an optimisation, not a dependency: if Redis is down, `/products` must still answer from PostgreSQL. `/readyz` checks only PostgreSQL.

Milestone 2 — The image (≈1 h)

`Dockerfile`:

DOCKERFILE

```
# syntax=docker/dockerfile:1
FROM node:22-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev --no-audit --no-fund

FROM node:22-alpine AS runtime
ENV NODE_ENV=production
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY package.json server.js ./
USER node
EXPOSE 3000
CMD ["node", "server.js"]
```

`.dockerignore`:

```
node_modules
.git
.env
*.log
```

BASH

```
docker build -t shop-api:dev .
docker image ls shop-api:dev              # roughly 150-200 MB with Node 22 on Alpine
docker run --rm shop-api:dev id           # uid=1000(node) gid=1000(node)
```

Why this shape: dependencies are installed in their own layer, so editing `server.js` does not reinstall packages (layer caching §4.9); the final image has no build tools and runs as a non-root user.

Milestone 3 — Database initialisation and secrets (≈30 min)

`db/init.sql` (runs once, only when the data volume is empty):

```sql
CREATE TABLE IF NOT EXISTS products (
  id          serial PRIMARY KEY,
  name        text    NOT NULL,
  price_cents integer NOT NULL CHECK (price_cents ⩾ 0)
);
INSERT INTO products (name, price_cents) VALUES ('Keyboard', 4999), ('Mouse', 1999),
  ('Monitor', 17999);
```

`.env` (add to `.gitignore`; Compose reads it automatically):

```bash
echo "DB_PASS=$(openssl rand -hex 16)" > .env
echo ".env" >> .gitignore
```

Milestone 4 — Compose (≈1 h)

`compose.yaml` (validated with `docker compose config`):

```yaml
name: shop

services:
  nginx:
    image: nginxinc/nginx-unprivileged:1.28-alpine
    ports:
      - "127.0.0.1:8080:8080"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      api:
        condition: service_healthy
    networks: [edge]
    restart: unless-stopped

  api:
    build: .
    image: shop-api:dev
    environment:
      PORT: "3000"
      HOST: "0.0.0.0"
```

```
      DATABASE_URL: postgres://shop:${DB_PASS:?set DB_PASS in .env}@db:5432/shop
      REDIS_URL: redis://cache:6379
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "node", "-e",
  "fetch('http://127.0.0.1:3000/healthz').then(r⇒process.exit(r.ok?
  0:1)).catch(()⇒process.exit(1))"]
      interval: 10s
      timeout: 3s
      retries: 3
      start_period: 10s
    read_only: true
    tmpfs: [/tmp]
    cap_drop: [ALL]
    security_opt: ["no-new-privileges:true"]
    networks: [edge, data]
    restart: unless-stopped

  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_USER: shop
      POSTGRES_PASSWORD: ${DB_PASS:?set DB_PASS in .env}
      POSTGRES_DB: shop
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/01-init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U shop -d shop"]
      interval: 5s
      timeout: 3s
      retries: 10
    networks: [data]
    restart: unless-stopped

  cache:
    image: redis:7.4-alpine
    command: ["redis-server", "--save", "", "--appendonly", "no", "--maxmemory", "64mb",
  "--maxmemory-policy", "allkeys-lru"]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10
    networks: [data]
    restart: unless-stopped

networks:
  edge: {}
```

```
  data:
    internal: true          # no route out: db and cache cannot reach the internet

volumes:
  pgdata: {}
```

`nginx/default.conf`:

```nginx
upstream shop_api { server api:3000; keepalive 16; }
server {
    listen 8080;
    server_name _;
    location = /nginx-health { access_log off; return 200 "ok\n"; }
    location = /metrics { return 404; }          # never expose metrics publicly (used in Project 7)
    location / {
        proxy_pass http://shop_api;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 2s;
        proxy_read_timeout 15s;
    }
}
```

```bash
docker compose config --quiet && echo "compose file valid"
docker compose up -d --build --wait
docker compose ps                          # all four services "healthy" / "running"
curl -si http://127.0.0.1:8080/products | grep -iE 'HTTP/|x-cache'    # X-Cache: MISS
curl -si http://127.0.0.1:8080/products | grep -i x-cache             # X-Cache: HIT
```

Milestone 5 — Smoke test (≈30 min)

`smoke.sh`:

```bash
#!/usr/bin/env bash
set -euo pipefail
BASE="${BASE:-http://127.0.0.1:8080}"
fail() { echo "FAIL: $*" >&2; exit 1; }
curl -fsS "$BASE/readyz" >/dev/null                              || fail "readyz"
n=$(curl -fsS "$BASE/products" | python3 -c 'import json,sys;
  print(len(json.load(sys.stdin)))')
[ "$n" -ge 1 ]                                                   || fail "no products"
curl -fsS "$BASE/products" -D - -o /dev/null | grep -qi '^x-cache: HIT' || fail "cache not
  hit on second call"
[ "$(curl -s -o /dev/null -w '%{http_code}' "$BASE/metrics")" = 404 ]   || fail "/metrics
  must not be public"
echo "SMOKE OK"
```

Run `chmod +x smoke.sh && ./smoke.sh`. Add `docker compose down` followed by `up` and prove the data persisted. `docker compose down -v` is what deletes the volume; know the difference.

5 Tasks

- [ ] **T1 — Build and inspect the image** *(≈45 min, beginner)*
  - **Goal:** a small, non-root image whose layers you can explain.
  - **Steps:**
    1. Build `shop-api:dev`, then run `docker history shop-api:dev` and `docker image ls`.
    2. Run `docker run --rm shop-api:dev id` and `docker run --rm shop-api:dev ls -la /app`.
    3. Change one line of `server.js` and rebuild; observe which layers are cached.
  - **Done when:** the container reports a non-root uid, and the rebuild shows `CACHED` for the dependency layer.

- [ ] **T2 — Bring the stack up in one command** *(≈45 min, beginner)*
  - **Goal:** four healthy services and a working request path.
  - **Steps:**
    1. Create `.env`, `db/init.sql`, `nginx/default.conf` and `compose.yaml` as above.
    2. `docker compose up -d --build --wait`, then `docker compose ps`.
    3. `curl -i http://127.0.0.1:8080/products` twice.

- **Done when:** all services are healthy, and the first response has `X-Cache: MISS` and the second `X-Cache: HIT`.

- [ ] **T3 — Prove data persistence** *(≈20 min, beginner)*
  - **Goal:** distinguish container lifecycle from data lifecycle.
  - **Steps:**
    1. Insert a product with `docker compose exec db psql -U shop -d shop -c "INSERT INTO products (name, price_cents) VALUES ('Webcam', 5999);"`.
    2. `docker compose down` then `docker compose up -d --wait`; query again.
    3. Then run `docker compose down -v`, bring it up again, and query.
  - **Done when:** the Webcam row survives step 2 and is gone after step 3, and you can explain why.

- [ ] **T4 — Prove the network boundaries** *(≈30 min, intermediate)*
  - **Goal:** show that `db` and `cache` are unreachable from the host and the internet.
  - **Steps:**
    1. `nc -vz -w 2 127.0.0.1 5432` and `6379` from your host must fail (nothing is published).
    2. `docker compose exec db sh -c 'wget -T 3 -qO- http://example.com || echo no-route'`.
    3. `docker compose exec nginx sh -c 'getent hosts db || echo nginx-cannot-resolve-db'`.
  - **Done when:** the host cannot connect, `db` has no outbound route, and `nginx` cannot resolve `db` (it is not on the `data` network).

- [ ] **T5 — Cache behavior and failure** *(≈30 min, intermediate)*
  - **Goal:** confirm the cache is optional.
  - **Steps:**
    1. Watch hits and misses with `docker compose exec cache redis-cli monitor` in another terminal.
    2. `docker compose stop cache`, then `curl -i http://127.0.0.1:8080/products`.
    3. Start it again and wait for the 30 s TTL to expire to see a MISS followed by a HIT.
  - **Done when:** `/products` returns 200 with `X-Cache: MISS` while Redis is stopped, and the app log shows a warning but no crash.

- [ ] **T6 — Change the version without losing data** *(≈30 min, intermediate)*
  - **Goal:** upgrade the API image while the database stays up.

- **Steps:**
  1. Add an `APP_VERSION` build argument or env var returned by `/readyz`.
  2. `docker compose up -d --build api` and watch `docker compose ps`.
  3. Run `docker compose logs --tail 20 api` and confirm graceful stop of the old container (exit code 0).
- **Done when:** `/readyz` shows the new version, the database container was not recreated (`docker compose ps` uptime), and no request returned an error during the roll in a `while true; do curl -s -o /dev/null -w '%{http_code}\n' ...; done` loop (a brief failure is acceptable, and you can explain it).

☐ **T7 — Scan and slim the image** *(≈45 min, advanced)*

- **Goal:** find and reduce known vulnerabilities.
- **Steps:**
  1. Scan with Trivy or Docker Scout (`docker scout quickview shop-api:dev` or `trivy image shop-api:dev`).
  2. Compare against `node:22-slim` and a distroless runtime stage; read minimal base images §4.13 and image scanning §4.14.
  3. Record image sizes and critical or high findings for each variant in a table.
- **Done when:** you have a table of size and findings for at least two variants and can justify which one you would ship.

☐ **T8 — Prove it: from clean clone to green smoke test** *(≈1 h, advanced)*

- **Goal:** a new teammate can run the stack with two commands.
- **Steps:**
  1. Commit everything except `.env`; write a 10-line `README` (prerequisites, `cp .env.example .env`, `docker compose up -d --build --wait`, `./smoke.sh`).
  2. Delete all local images and volumes (`docker compose down -v --rmi local`) and follow only your README on a clean directory.
  3. Explain aloud how a request flows through the networks and which container could be compromised without exposing the database to the internet.
- **Done when:** `./smoke.sh` prints `SMOKE OK` from a clean clone, and your README needed no extra steps.

6 Definition of done

☐ `docker compose up -d --build --wait` finishes with four healthy services.

- [ ] `./smoke.sh` prints `SMOKE OK`, including a cache HIT and `/metrics` returning 404 through Nginx.
- [ ] The API image runs as a non-root user, has a read-only root file system and drops all capabilities.
- [ ] Only `127.0.0.1:8080` is published; `db` and `cache` live on an `internal` network.
- [ ] Data survives `docker compose down`; you can explain exactly what `-v` removes.
- [ ] No secret is in the image, in `compose.yaml` or in Git; `.env` is ignored and `.env.example` documents the keys.
- [ ] Stopping Redis does not break `/products`; stopping PostgreSQL turns `/readyz` into 503 within a few seconds.

7 Break it on purpose

1. **Remove the health check ordering.** Delete the `condition: service_healthy` lines and run `docker compose down -v && docker compose up -d`. The API may start before PostgreSQL accepts connections. Watch `docker compose logs api` and see why health-gated `depends_on` matters. Restore it.
2. **Change the password after first start.** Edit `DB_PASS` in `.env` and `docker compose up -d`. The API fails to authenticate because PostgreSQL only reads `POSTGRES_PASSWORD` when it initialises an empty volume. Fix it with `ALTER ROLE shop PASSWORD '...'` (or by recreating the volume in a throwaway lab).
3. **Run as root with a writable file system.** Remove `USER node`, `read_only` and `cap_drop`, then `docker compose exec api sh -c 'touch /app/x && id'`. Compare with the hardened version and write down what an attacker with code execution could do in each.
4. **Exhaust the cache memory.** Lower the limit at runtime with `docker compose exec cache redis-cli config set maxmemory 1mb`, load many keys with `docker compose exec cache sh -c 'for i in $(seq 1 20000); do redis-cli set k$i xxxxxxxxxxxxxxxxxxxxxxxx >/dev/null; done'`, and read `redis-cli info stats | grep evicted_keys`. Explain why `allkeys-lru` is right for a cache and wrong for a primary store.

8 How this evolves into the next project

You now have a reproducible artifact (the image) and a reproducible environment (Compose). But a human still builds and starts it, and nothing checks a change before it reaches the stack. Project 3 — CI/CD puts the repository on GitHub and lets GitHub Actions

lint, test, build, scan and push this exact image to a registry on every merge, then deploy it to a server using the same `compose.yaml`. Keep `smoke.sh`: it becomes the post-deploy gate in the pipeline.

LAB NOTES What I ran, what I saw, what it means

THE SUMMIT PROJECTS

P3 CI/CD

Project 3 of 9 | Uses levels: Levels 3, 5 | ~10 hours

1 Goal and outcome

Make a `git push` the only way `shop-api` reaches production. A pull request runs lint and tests against real PostgreSQL and Redis; a merge to `main` builds the Project 2 image, scans it, pushes it to the GitHub Container Registry (GHCR) and deploys it to a server, with a smoke test after the deploy and an automatic rollback if the smoke test fails.

The pipeline concepts are covered in depth in Level 5 Capstone 5 (which builds a blue/green pipeline) and the repository mechanics in the Level 3 capstone Capstone 3. This project is the **simplest pipeline that is still honest**: one repository, one workflow, one environment, and a deploy you can roll back by re-running a tag.

You finish with:

- a GitHub repository `shop-api` with `main` protected: changes arrive only through pull requests with a green `ci` check;
- a workflow with three jobs: `test` (services: PostgreSQL, Redis), `image` (build, scan, push) and `deploy` (SSH to the server, gated by the `production` environment);
- images in GHCR tagged by commit (`sha-<full sha>`), never deployed by mutable tag;
- a `deploy.sh` on the server that pulls a specific tag, waits for health, runs `smoke.sh` and rolls back on failure;
- a **one-command rollback**: re-run the deploy with the previous tag.

2 Architecture

What changed from Project 2: the same Compose stack now runs on a **server** (re-use the Project 1 host with Docker installed, or any Docker-capable VM), the image is built by

GitHub's runners rather than your laptop, and a registry sits between build and run (NEW).

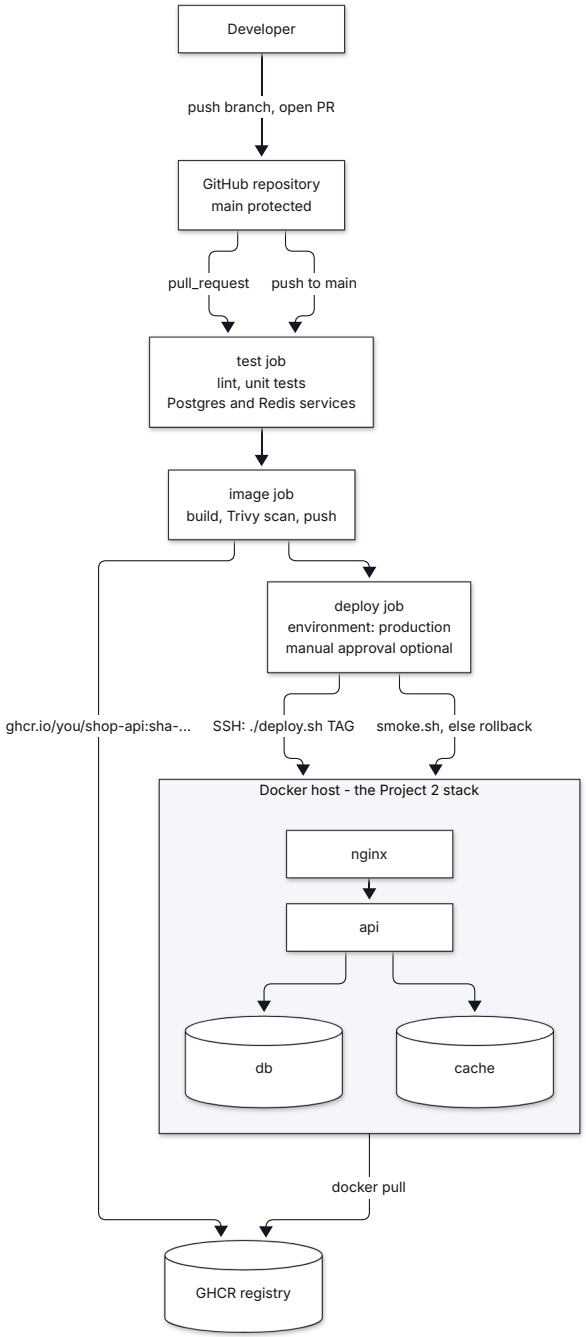


Figure 3-1

Deploy sequence on the server:

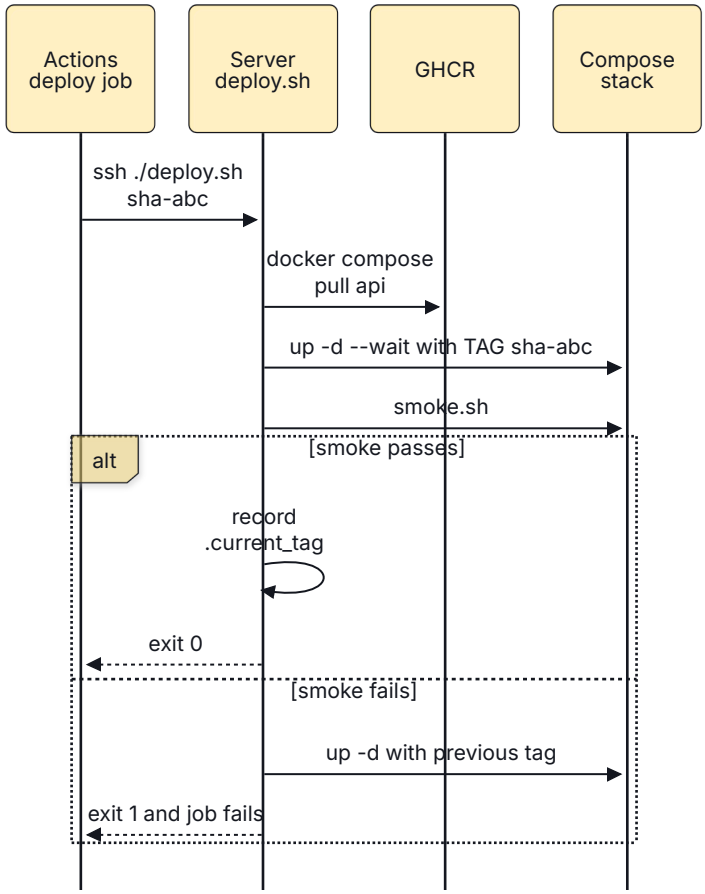


Figure 3-2

3 Prerequisites

- Everything from Project 2 working locally, and a GitHub account (free tier is enough for a public or private repository).
- A server that runs Docker: your Project 1 VPS (stop Nginx and the systemd service first so port 8080 and your data do not collide, or use a fresh VM). Cost: the VPS you already pay for.
- Git and GitHub: branching §3.1, tags §3.6, protected branches and pull requests §3.8, Actions workflows §3.11, environments and secrets §3.12.
- CI/CD: pipeline design §5.1, GitHub Actions pipelines §5.2, automated testing §5.3, Docker builds and registries §5.4, secrets §5.5, rollback §5.7.

- Tools: `git`, `gh` (GitHub CLI, optional), `ssh`.

4 Build it step by step

Milestone 1 — Repository and protected main (≈45 min)

```bash
cd shop && git init -b main && git add . && git commit -m "shop-api: containerized stack"
gh repo create shop-api --private --source=. --remote=origin --push      # or create it in the web UI
```

Under Settings, Branches (or Rulesets), protect `main`: require a pull request, require the status check `ci / test` and `ci / image`, block force pushes. Check that `.env` and `node_modules` are in `.gitignore` before the first push.

Tests need a script: add `"lint": "node --check server.js"` and `"test"` (for example `node --test`) to `package.json`, with at least one test that calls `/readyz` and `/products` against the CI database.

Milestone 2 — The workflow (≈2 h)

`.github/workflows/ci.yml` (syntax-checked as YAML; action versions are major tags, so in a real repository also pin them to commit SHAs and let Dependabot update them):

```yaml
name: ci

on:
  pull_request:
  push:
    branches: [main]

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-24.04
    services:
      postgres:
        image: postgres:17-alpine
        env:
          POSTGRES_USER: shop
```

```
          POSTGRES_PASSWORD: ci-only-not-a-secret
          POSTGRES_DB: shop_test
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U shop -d shop_test"
          --health-interval 5s --health-timeout 3s --health-retries 10
      redis:
        image: redis:7.4-alpine
        ports: ["6379:6379"]
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 5s --health-timeout 3s --health-retries 10
    env:
      DATABASE_URL: postgres://shop:ci-only-not-a-secret@localhost:5432/shop_test
      REDIS_URL: redis://localhost:6379
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: psql "$DATABASE_URL" -f db/init.sql
      - run: npm test

  image:
    needs: test
    runs-on: ubuntu-24.04
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - id: meta
        uses: docker/metadata-action@v5
        with:
          images: ghcr.io/${{ github.repository }}
          tags: |
            type=sha,format=long
      - name: Build (not pushed yet)
        uses: docker/build-push-action@v6
        with:
          context: .
          load: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
      - name: Scan before publishing
        uses: aquasecurity/trivy-action@0.28.0
        with:
```

```
          image-ref: ${{ fromJSON(steps.meta.outputs.json).tags[0] }}
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"
      - name: Log in to GHCR
        if: github.event_name == 'push'
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - name: Push
        if: github.event_name == 'push'
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha

  deploy:
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    needs: image
    runs-on: ubuntu-24.04
    environment: production
    concurrency: deploy-production
    steps:
      - name: Configure SSH
        env:
          KEY: ${{ secrets.DEPLOY_SSH_KEY }}
          KNOWN_HOSTS: ${{ vars.DEPLOY_KNOWN_HOSTS }}
        run: |
          install -m 700 -d ~/.ssh
          printf '%s\n' "$KEY" > ~/.ssh/id_ed25519 && chmod 600 ~/.ssh/id_ed25519
          printf '%s\n' "$KNOWN_HOSTS" > ~/.ssh/known_hosts
      - name: Deploy
        env:
          HOST: ${{ vars.DEPLOY_HOST }}
          TAG: sha-${{ github.sha }}
        run: ssh deploy@"$HOST" "/opt/shop/deploy.sh $TAG"
```

Two design points. First, the image is **scanned before it is pushed**, so a vulnerable image never reaches the registry from `main`. Second, `permissions:` is least privilege at the top and widened only where the job needs it (secrets and permissions §5.5).

Milestone 3 — Prepare the deploy target (≈1 h)

On the server: install Docker Engine, create `/opt/shop` containing `compose.yaml`, `nginx/`, `db/`, `smoke.sh`, a `.env` with `DB_PASS`, and this override `compose.deploy.yaml` (replace `you`):

```yaml
services:
  api:
    image: ghcr.io/you/shop-api:${TAG:?TAG is required}
    pull_policy: always
```

Create a deploy user with a dedicated key, and allow it to run Docker (membership in the `docker` group is root-equivalent, so restrict the key with `command=` in `authorized_keys` once you are comfortable). If the package is private, run `docker login ghcr.io` once with a fine-grained token that has only `read:packages`. In GitHub, create the `production` environment (add yourself as a required reviewer if you want a manual gate), then store the private key as the secret `DEPLOY_SSH_KEY` and the variables `DEPLOY_HOST` and `DEPLOY_KNOWN_HOSTS` (from `ssh-keyscan -t ed25519 <host>` checked against the host's real fingerprint).

`/opt/shop/deploy.sh`:

```bash
#!/usr/bin/env bash
set -euo pipefail
cd /opt/shop
NEW="${1:?usage: deploy.sh <tag>}"
PREV="$(cat .current_tag 2>/dev/null || echo none)"
dc() { TAG="$1" docker compose -f compose.yaml -f compose.deploy.yaml "${@:2}"; }

echo "deploying $NEW (previous: $PREV)"
dc "$NEW" pull api
dc "$NEW" up -d --no-build --wait --wait-timeout 90 || ok=no
if [ "${ok:-yes}" = yes ] && BASE=http://127.0.0.1:8080 ./smoke.sh; then
  echo "$NEW" > .current_tag
  [ "$PREV" ≠ none ] && echo "$PREV" > .previous_tag
  echo "deploy ok"
else
  echo "deploy FAILED" >&2
  if [ "$PREV" ≠ none ]; then
    echo "rolling back to $PREV" >&2
    dc "$PREV" up -d --no-build --wait --wait-timeout 90
  fi
  exit 1
fi
```

Milestone 4 — First pipeline run, then a change through a PR (≈1 h)

```bash
git switch -c feature/product-by-id
# edit server.js: add GET /products/:id, add a test
git commit -am "Add product by id" && git push -u origin HEAD
gh pr create --fill
gh pr checks --watch            # test and image pass; deploy does not run on PRs
gh pr merge --squash --delete-branch
gh run watch                    # test, image (push), deploy
curl -s http://<server>:8080/products/1
```

On the server `cat /opt/shop/.current_tag` should show the new `sha-...`; `docker compose ps` shows the new image digest.

Milestone 5 — Rollback (≈30 min)

Because every image is addressable by commit tag, rollback is a deploy of an older tag. Two ways:

```bash
ssh deploy@<server> '/opt/shop/deploy.sh "$(cat /opt/shop/.previous_tag)"'      # fastest, by hand
git revert -m 1 <merge-sha> && git push                                         # via Git, goes through CI again
```

Add a `workflow_dispatch` input `tag` and a manual `rollback` workflow (see the Level 5 roll-back topic §5.7) so anyone with access can do the first one from the Actions tab.

5 Tasks

- [ ] **T1 — Create the repository and protect `main`** *(≈30 min, beginner)*
  - **Goal:** nothing reaches `main` without review and a green check.
  - **Steps:**
    1. Push the project and configure the branch rule (PR required, status checks, no force-push).
    2. Try `git push origin main` directly from a clone.
  - **Done when:** the direct push is rejected with a protected-branch message.
- [ ] **T2 — Get `test` green on a PR** *(≈1 h, beginner)*
  - **Goal:** lint and tests run against real PostgreSQL and Redis service containers.
  - **Steps:**

1. Add `npm run lint`, `npm test` and a test hitting `/readyz` and `/products`.
2. Open a PR and watch `gh pr checks --watch`.
3. Introduce a failing test and confirm the PR cannot merge.

- **Done when:** a green PR can be merged, and a red PR is blocked by the required check.

☐ **T3 — Build, scan and publish an image** *(≈1 h, intermediate)*

- **Goal:** an immutable, traceable image in GHCR.
- **Steps:**
  1. Merge to `main` and open the package page for `shop-api`.
  2. Find the tag `sha-<full commit sha>` and the OCI labels linking back to the repository.
  3. Pull it locally: `docker pull ghcr.io/<you>/shop-api:sha-<sha>` and run `docker image inspect` to read its labels.
- **Done when:** the image exists for the merge commit, and you can trace it from the tag to the commit and the workflow run.

☐ **T4 — Deploy automatically with a gate** *(≈1.5 h, intermediate)*

- **Goal:** merge to `main` updates the running service.
- **Steps:**
  1. Configure the `production` environment, secrets and variables; add yourself as a required reviewer.
  2. Merge a visible change (add a field to `/readyz`), approve the deployment in the Actions UI.
  3. Check `curl -s http://<server>:8080/readyz` and `cat /opt/shop/.current_tag`.
- **Done when:** the new field is visible on the server within minutes of approving, and the deploy job log shows `deploy ok`.

☐ **T5 — Make the pipeline fail safely** *(≈1 h, intermediate)*

- **Goal:** a broken release does not stay deployed.
- **Steps:**
  1. Commit a change that makes `/readyz` return 500 only in production (for example a missing env var read at runtime), and merge it.
  2. Watch `deploy.sh` fail its smoke test and roll back.
  3. Read `docker compose ps` and the job log to explain which step failed.
- **Done when:** the deploy job is red, the previous tag is still serving traffic, and `.current_tag` was not updated.

- [ ] **T6 — Roll back by tag** *(≈30 min, intermediate)*
  - **Goal:** recover in under two minutes without touching Git.
  - **Steps:**
    1. Deploy a known-bad but passing change (a wrong price in the init data is fine).
    2. Run `deploy.sh` with the previous tag from `.previous_tag`.
    3. Time it with `time`.
  - **Done when:** the old behavior is back, and you have a measured rollback time written down.
- [ ] **T7 — Speed it up and harden it** *(≈1.5 h, advanced)*
  - **Goal:** a faster, less trusting pipeline.
  - **Steps:**
    1. Compare run times before and after enabling `cache: npm` and `type=gha` Docker cache.
    2. Pin every action to a commit SHA and add `dependabot.yml` for `github-actions` and `npm`.
    3. Add a Gitleaks or secret-scanning check and confirm a fake token in a PR is caught (secrets scanning §11.11).
  - **Done when:** warm-cache runs are measurably faster, all actions use SHAs, and a planted fake secret fails the build.
- [ ] **T8 — Prove it: ship a change end to end without the guide** *(≈1.5 h, advanced)*
  - **Goal:** demonstrate the whole flow and explain every trust relationship.
  - **Steps:**
    1. From a fresh clone, branch, change, PR, review (ask a friend or use a second account), merge, approve and verify on the server.
    2. Draw which credentials each job uses (`GITHUB_TOKEN`, SSH key, registry token) and what each can do if stolen.
    3. Explain why deploys use `sha-...` tags and never `latest`.
  - **Done when:** the change is live, you have the diagram, and your explanation names at least three ways the pipeline limits the damage of a compromised token.

6 Definition of done

- [ ] `main` is protected; the required checks are `ci / test` and `ci / image`.

- [ ] A PR shows lint, tests and a scan result; a merge produces a `sha—<commit>` image in GHCR.
- [ ] The deploy job runs only from `main`, uses the `production` environment, and authenticates with a dedicated key.
- [ ] No secret appears in the repository, workflow logs or image; CI-only credentials are obviously fake.
- [ ] A failing smoke test rolls back automatically; a manual rollback to a previous tag takes under two minutes.
- [ ] A new change takes a few minutes from merge to production without anyone logging in to the server.
- [ ] Workflow permissions are least privilege (`contents: read` by default).

7 Break it on purpose

1. **Poison the lockfile.** Change a dependency version in `package.json` without updating `package-lock.json`. `npm ci` fails in CI. Explain why `npm ci` is stricter than `npm install` and why that is good.
2. **Expire the deploy key.** Remove the public key from `authorized_keys` and merge a change. Read the failure in the deploy job. Then restore it. Note that the `image` job already published the image, so retry only the deploy.
3. **Race two deploys.** Merge two PRs quickly. The `concurrency` group serialises the deploys; remove it temporarily in a branch and observe how two `docker compose up` calls can collide. Restore it.
4. **Publish a vulnerable base.** Change the Dockerfile base to an old tag (for example `node:18-alpine`) in a PR. Trivy should block the merge. Decide, with the report open, when you would accept a finding and how to record that decision.

8 How this evolves into the next project

The pipeline still deploys onto **one hand-made server** whose IP, firewall rules and Docker install exist only in your memory and your notes. Project 4 — Terraform replaces that server with infrastructure defined as code on AWS: a VPC with public and private subnets, EC2 instances behind an Application Load Balancer, and a managed RDS PostgreSQL. The workflow you built here stays; only the deploy target changes, and the Terraform change itself will flow through its own pull-request pipeline.

THE SUMMIT PROJECTS

P4 Terraform

Project 4 of 9 | Uses levels: Levels 6-7 | ~14 hours

1 Goal and outcome

Replace the hand-made server with **infrastructure defined as code**: a VPC with public, private and data subnets, two EC2 instances behind an Application Load Balancer (ALB), PostgreSQL on RDS and Redis on ElastiCache, all created by `terraform apply` and removed by `terraform destroy`. The instances run the **same container image** your Project 3 pipeline publishes, so the application, its health endpoints and its smoke test do not change at all.

The AWS building blocks are taught in the Level 6 capstone Capstone 6 (built by hand in the console) and the Terraform module structure, remote state and pipeline in the Level 7 capstone Capstone 7. This project is the integration: **take the Level 7 modules, run the real `shop-api` image on them, and let a pipeline plan and apply every change**.

You finish with:

- a repository `shop-api-infra` with `modules/` (network, data, app) and thin `envs/dev` and `envs/prod` roots, using remote state with native S3 locking;
- `curl http://<alb-dns-name>/products` returning rows from RDS through the ALB, with `X-Cache` flipping from MISS to HIT via ElastiCache;
- instances that fetch the database password from Secrets Manager at boot (no password in code, state outputs, user data or the image);
- a pull-request workflow that runs `fmt`, `validate`, a security scan and `plan`, and applies on merge using GitHub OIDC (no stored AWS keys);
- a cost estimate you wrote down **before** applying, a billing alarm, and a verified `destroy`.

2 Architecture

What changed from Project 3: the Docker host is no longer a hand-built VM. Networking, firewalling, compute, load balancing and the database are resources in code (NEW boxes), the database and cache are managed services instead of containers, and the ALB replaces

Nginx as the public front door.

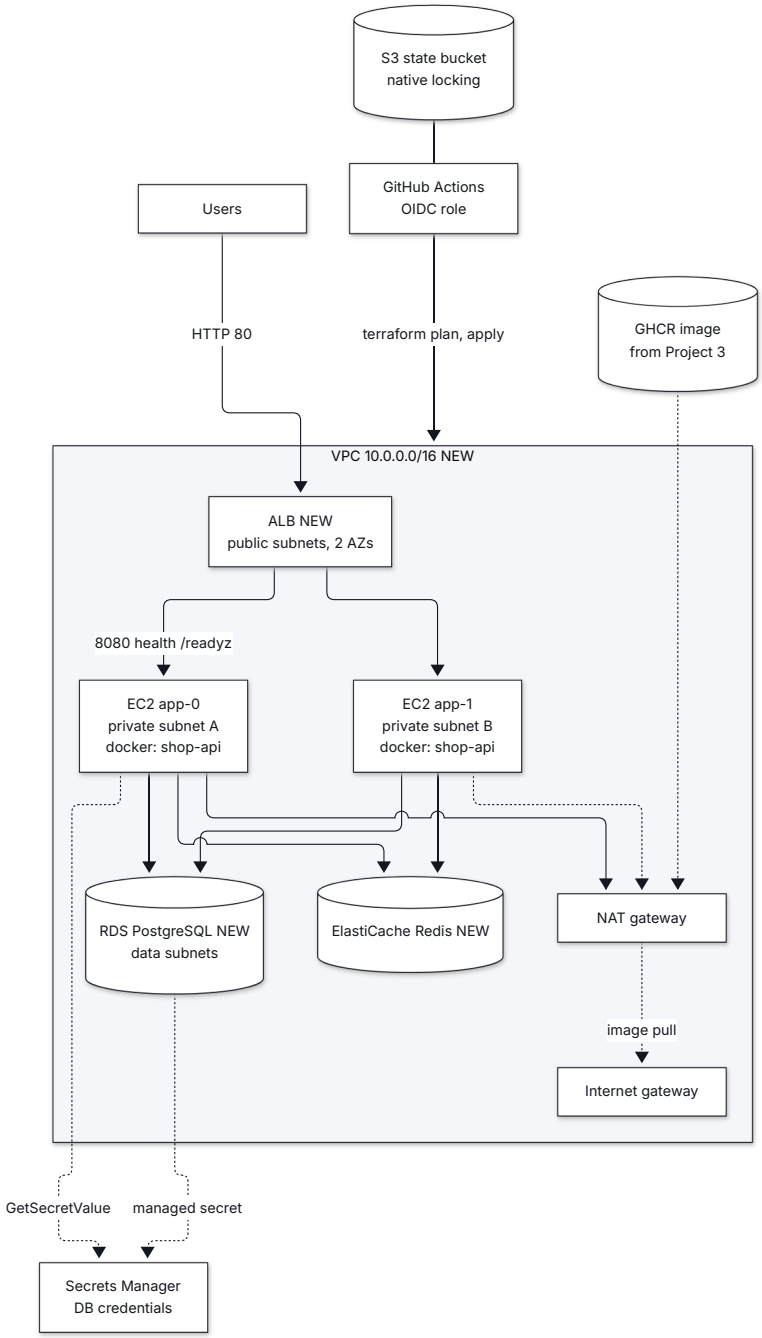


Figure 4-1

How a deployment of a new image tag works in this project (and why it is not good enough yet):

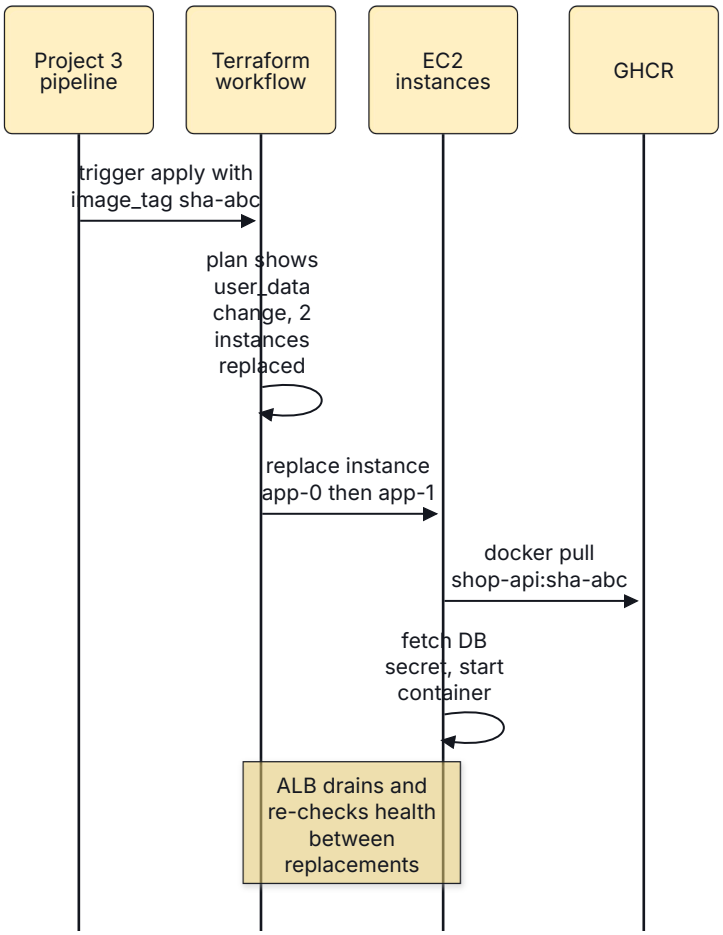


Figure 4-2

Project 3	Project 4
One VM you built by hand	Two EC2 instances from a launch definition in code
Nginx container in front	ALB with a target group and `/readyz` health check
PostgreSQL and Redis containers	RDS PostgreSQL (Secrets Manager password) and ElastiCache Redis
`.env` file on disk	IAM instance profile plus Secrets Manager
`deploy.sh` over SSH	change `image_tag`, plan in a PR, apply on merge
Firewall: `ufw`	Security groups that reference each other

3 Prerequisites

- An **AWS sandbox account** (never production), MFA on the root user, a **billing alarm** and a budget before you create anything (CloudWatch §6.18). Terraform 1.11+ (OpenTofu works), AWS CLI v2, a GitHub repository.
- **Cost warning:** the NAT gateway, ALB, RDS, ElastiCache and two instances bill per hour, typically a few US dollars per day for a small dev stack (check current regional pricing; free-tier eligibility varies). Apply, test, and destroy the same day. Region: `us-east-1`.
- AWS: EC2 §6.1, VPC and subnets §6.4, route tables, IGW and NAT §6.5, security groups §6.6, IAM §6.7, RDS §6.10, ElastiCache §6.11, ALB §6.14, Secrets Manager §6.16.
- Terraform: providers and resources §7.1, variables and outputs §7.2, modules §7.3, state and backends §7.4, reusable environments §7.6, Terraform CI/CD §7.7, state locking §7.8.
- A **public** GHCR package for `shop-api` (set the package visibility in GitHub) so instances can pull it anonymously in this lab. In real use you would push to ECR and grant the instance role pull access.

4 Build it step by step

Milestone 1 — Stand up the Level 7 stack as the baseline (≈4 h)

Follow Milestones 0 to 4 of the Level 7 capstone Capstone 7: bootstrap state (remote, locked, versioned), then the `network`, `data` and `app` modules and `envs/dev`. That version runs a tiny Python stand-in on port 8080. Confirm the baseline:

```bash
cd envs/dev && terraform init && terraform apply
curl -s "http://$(terraform output -raw alb_dns_name)/"      # JSON from the stand-in
  server
```

Write the cost estimate now (ALB hours, NAT gateway hours plus data, RDS, ElastiCache, two instances, EBS) in `COST.md` and compare it with the AWS Billing console tomorrow.

Milestone 2 — Teach the application about TLS to RDS (≈30 min)

RDS for PostgreSQL 15 and later requires TLS by default (`rds.force_ssl` defaults to on), so a plain connection that worked against the Compose database is rejected. Add a switch to the pool in `server.js`:

```js
const ssl = process.env.PGSSL === '1' ? { rejectUnauthorized: false } : false;   // lab shortcut
const pool = new Pool({ connectionString: process.env.DATABASE_URL, max: 10, ssl });
```

`rejectUnauthorized: false` encrypts the connection but does not verify the server. Treat it as a lab shortcut and replace it with the RDS CA bundle (`ssl: { ca: fs.readFileSync(...) }`) once the stack works (TLS §12.4). Commit, let Project 3's pipeline publish the new image, and note its `sha-...` tag.

Milestone 3 — Run the real image on the instances (≈2 h)

In `modules/app`, add variables `image` (full reference including tag), `db_endpoint`, `db_secret_arn`, `redis_endpoint`, and grant the instance role access to exactly one secret:

```hcl
data "aws_iam_policy_document" "read_db_secret" {
  statement {
    actions   = ["secretsmanager:GetSecretValue"]
    resources = [var.db_secret_arn]
  }
}

resource "aws_iam_role_policy" "read_db_secret" {
  name   = "read-db-secret"
  role   = aws_iam_role.app.id
  policy = data.aws_iam_policy_document.read_db_secret.json
}
```

Replace the inline Python server with a template file `modules/app/userdata.sh.tftpl`:

```bash
#!/bin/bash
set -euo pipefail
dnf install -y docker jq
systemctl enable --now docker

SECRET="$(aws secretsmanager get-secret-value --region ${region} --secret-id
  ${db_secret_arn} \
  --query SecretString --output text)"
DB_USER="$(echo "$SECRET" | jq -r .username)"
```

```
DB_PASS="$(echo "$SECRET" | jq -rj .password | jq -sRr @uri)"   # URL-encode special
  characters

docker network create shop || true
docker run -d --name api --restart unless-stopped --network shop -p 8080:3000 \
  -e PORT=3000 -e HOST=0.0.0.0 -e PGSSL=1 \
  -e DATABASE_URL="postgres://$DB_USER:$DB_PASS@${db_endpoint}:5432/shop" \
  -e REDIS_URL="redis://${redis_endpoint}:6379" \
  ${image}
```

and render it in the instance resource:

```hcl
resource "aws_instance" "app" {
  # ... as in the Level 7 capstone ...
  user_data_replace_on_change = true
  user_data = templatefile("${path.module}/userdata.sh.tftpl", {
    region         = data.aws_region.current.name
    db_secret_arn  = var.db_secret_arn
    db_endpoint    = var.db_endpoint
    redis_endpoint = var.redis_endpoint
    image          = var.image
  })
}
```

Wire the data module outputs into the app module in `envs/dev/main.tf`:

```hcl
module "app" {
  source         = "../../modules/app"
  # ... network and security group inputs ...
  db_endpoint    = module.data.db_address          # aws_db_instance.this.address
  db_secret_arn  = module.data.db_secret_arn       #
  aws_db_instance.this.master_user_secret[0].secret_arn
  redis_endpoint = module.data.redis_endpoint
  image          = "ghcr.io/you/shop-api:${var.image_tag}"
}

variable "image_tag" { type = string }
```

Point the ALB target group health check at `/readyz` (matcher `200`, interval 15 s, healthy threshold 2), keep the target port at 8080 as in Level 7, and mark the database secret output `sensitive` or output only its ARN (an ARN is not a secret).

```bash
terraform plan  -var image_tag=sha-<full sha>
terraform apply -var image_tag=sha-<full sha>
ALB="$(terraform output -raw alb_dns_name)"
until curl -fs "http://$ALB/readyz"; do sleep 10; done          # instances take a few minutes to bootstrap
curl -si "http://$ALB/products" | grep -iE 'HTTP/|x-cache'
```

If the target group shows `unhealthy`, SSH is not needed: use Session Manager (`aws ssm start-session --target <instance-id>`) and `sudo cat /var/log/cloud-init-output.log`, `docker logs api`.

Milestone 4 — Seed the schema (≈30 min)

The managed database starts empty. Do it as a one-off from an instance (the only place that can reach the private database), not by opening the database to the internet:

```bash
aws ssm start-session --target <instance-id>
# inside the session: read the secret as the user data does, then run psql from a throwaway container
SECRET="$(aws secretsmanager get-secret-value --region us-east-1 --secret-id <db-secret-arn> --query SecretString --output text)"
sudo docker run --rm -i -e PGSSLMODE=require \
  -e PGPASSWORD="$(echo "$SECRET" | jq -rj .password)" postgres:17-alpine \
  psql -h <rds-endpoint> -U "$(echo "$SECRET" | jq -r .username)" -d shop <<'SQL'
CREATE TABLE IF NOT EXISTS products (id serial PRIMARY KEY, name text NOT NULL,
  price_cents integer NOT NULL CHECK (price_cents ≥ 0));
INSERT INTO products (name, price_cents) VALUES ('Keyboard', 4999), ('Mouse', 1999),
  ('Monitor', 17999);
SQL
```

Better: run it as a one-shot container with the SQL baked into a migration image or a CI-controlled job (database migrations §5.9). You will formalise this in Project 5.

Milestone 5 — The pipeline (≈2 h)

Create the GitHub OIDC provider, a read-only `plan` role and an `apply` role limited to the `main` branch, as in Terraform CI/CD §7.7. The workflow (abbreviated):

```yaml
name: terraform
on:
```

```
  pull_request: { paths: ["envs/**", "modules/**"] }
  push: { branches: [main], paths: ["envs/**", "modules/**"] }
  workflow_dispatch:
    inputs:
      image_tag: { description: "Image tag to deploy", required: true }
permissions: { id-token: write, contents: read, pull-requests: write }
jobs:
  plan:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - uses: aws-actions/configure-aws-credentials@v4
        with: { role-to-assume: "${{ vars.PLAN_ROLE_ARN }}", aws-region: us-east-1 }
      - run: terraform -chdir=envs/dev fmt -check -recursive
      - run: terraform -chdir=envs/dev init -input=false
      - run: terraform -chdir=envs/dev validate
      - run: terraform -chdir=envs/dev plan -input=false -var image_tag=${{
  inputs.image_tag || vars.IMAGE_TAG }} -out=tfplan
```

An `apply` job (with `environment: dev` and the apply role) runs only on `main`. Keep the image tag in a repository variable (or a tracked `dev.auto.tfvars`) so that a deploy is a reviewed change.

Milestone 6 — Operate and destroy (≈1 h)

```bash
terraform plan -detailed-exitcode        # 0 = clean, 2 = drift, after you change a tag by
  hand in the console
terraform destroy -var image_tag=sha-<sha>
aws elbv2 describe-load-balancers --query 'LoadBalancers[].LoadBalancerArn'   # empty
aws ec2 describe-nat-gateways --filter Name=state,Values=available --query
  'NatGateways[].NatGatewayId'   # empty
```

Then check the Billing console and Cost Explorer for the next day, not just the Terraform output.

5 Tasks

- [ ] **T1 — Estimate before you apply** *(≈45 min, beginner)*
  - **Goal:** know the bill before creating anything.
  - **Steps:**
    1. List each billable resource in `COST.md` with its hourly or monthly price from the AWS pricing pages.
    2. Create a budget and a billing alarm at an amount you can afford.

3. After the lab, compare your estimate with the actual cost in Cost Explorer.

- **Done when:** `COST.md` exists with a total, an alarm exists, and you recorded the estimate-versus-actual difference.

- [ ] **T2 — Bootstrap remote, locked state** *(≈1 h, intermediate)*
  - **Goal:** state in S3 with native locking, never on your laptop.
  - **Steps:**
    1. Create the state bucket (versioned, encrypted, public access blocked) as in the Level 7 capstone.
    2. Configure `backend "s3"` with `use_lockfile = true` and run `terraform init`.
    3. Start a long `plan` in one terminal and a second `plan` in another.
  - **Done when:** the second run fails with a lock error and `terraform state list` works from a fresh clone.

- [ ] **T3 — Build the network and prove the boundaries** *(≈2 h, intermediate)*
  - **Goal:** only the ALB is public.
  - **Steps:**
    1. Apply the `network` module and security groups; read `terraform plan` before applying.
    2. From your laptop try to reach an instance's private IP and the RDS endpoint (both must fail).
    3. Draw the security group references (`alb`, `app`, `db`, `redis`).
  - **Done when:** nothing but the ALB answers from the internet, and your drawing matches the rules `aws ec2 describe-security-groups` shows.

- [ ] **T4 — Run the real image behind the ALB** *(≈2 h, intermediate)*
  - **Goal:** `/products` served from RDS through the ALB.
  - **Steps:**
    1. Implement Milestones 2 to 4.
    2. Wait for both targets to become healthy in the target group.
    3. Curl `/products` twice and confirm `X-Cache: MISS` then `HIT`.
  - **Done when:** `aws elbv2 describe-target-health` lists two healthy targets and the cache header changes as expected.

- [ ] **T5 — Prove no secret is exposed** *(≈45 min, intermediate)*
  - **Goal:** the database password lives only in Secrets Manager and in process memory.
  - **Steps:**

1. `grep -ri password . --include='*.tf' --include='*.tftpl' --include='*.tfvars'` and read `terraform output`.
2. Inspect the state: `terraform state pull | grep -i '"password"'` (note what *is* in state and why state needs protection).
3. Check `aws ec2 describe-instance-attribute --attribute userData` does not contain the password.

- **Done when:** no password appears in code, outputs or user data, and you wrote down what the state file contains and who can read it.

- [ ] **T6 — Ship a new image by changing one variable** *(≈1 h, intermediate)*
  - **Goal:** a deployment is a reviewed Terraform change.
  - **Steps:**
    1. Merge an application change so Project 3 publishes a new tag.
    2. Open a PR changing `image_tag`; read the plan (instances replaced).
    3. Merge to apply, while running a `curl` loop against the ALB to record any failed requests.
  - **Done when:** the new version answers on the ALB, and you counted the errors during the replacement and can explain them.

- [ ] **T7 — Detect and repair drift** *(≈45 min, advanced)*
  - **Goal:** find out-of-band changes.
  - **Steps:**
    1. In the console, change a tag and add a security group ingress rule on the app group.
    2. Run `terraform plan -detailed-exitcode` and read the diff.
    3. Decide for each change: revert with apply or adopt into code, and do so. Add the scheduled `drift.yml` from drift detection §7.9.
  - **Done when:** `plan` exits 0 again, and a scheduled workflow would have reported the drift.

- [ ] **T8 — Prove it: create, verify and destroy from scratch** *(≈2 h, advanced)*
  - **Goal:** the whole environment is reproducible and removable.
  - **Steps:**
    1. Destroy everything, then create it again with only `terraform apply -var image_tag=...` and one documented manual step (the schema seed).
    2. Run Project 2's `smoke.sh` with `BASE=http://<alb-dns-name>`.

3. Destroy again and verify with the CLI that no ALB, NAT gateway, RDS instance or ElastiCache cluster remains.

- **Done when:** `smoke.sh` prints `SMOKE OK` against the ALB, and the post-destroy CLI checks return empty lists.

6 Definition of done

- [ ] `terraform apply` builds the whole environment from an empty account (apart from the bootstrap state bucket and the OIDC role).
- [ ] `smoke.sh` passes against `http://<alb-dns-name>`; both targets are healthy; `/readyz` is the health check.
- [ ] State is remote, encrypted, versioned and locked; no state file or `.terraform/` directory is committed.
- [ ] No secret in code, outputs, user data, logs or CI variables; the pipeline uses OIDC and a plan-only role for pull requests.
- [ ] The database and cache are reachable only from the app security group.
- [ ] `fmt`, `validate` and a security scanner (for example Checkov or Trivy config) run in CI, and a PR shows the plan before merge.
- [ ] A cost estimate exists, and `terraform destroy` plus the CLI checks prove nothing is left billing.

7 Break it on purpose

1. **Manual console change.** Delete a route in a route table by hand. Watch the ALB lose its targets or the instances lose outbound access (image pulls fail), then repair it with `terraform apply`. *(Lesson: drift is real, and plans are the repair tool.)*
2. **Wrong secret ARN.** Point `db_secret_arn` at a non-existent secret, replace the instances, and read `/var/log/cloud-init-output.log` through Session Manager. Targets stay unhealthy and the ALB serves 503. *(Lesson: health checks turn a bad deploy into a visible failure instead of a bad response.)*
3. **Crash during apply.** Interrupt `terraform apply` with Ctrl+C partway through, then run `terraform plan`. See what the state believes, and recover (import or re-apply). Release a stale lock with `terraform force-unlock` only after proving no run is active. *(Lesson: state is the source of truth for what Terraform owns.)*

4. **Delete the NAT gateway.** Instances can no longer pull `ghcr.io` images or reach AWS APIs through the internet. Observe a new instance failing to bootstrap. Compare the cost and failure trade-off of a NAT gateway with VPC endpoints for Secrets Manager. *(Lesson: private subnets need an egress plan.)*

8 How this evolves into the next project

Replacing instances to ship a version is slow and blunt: two VMs, a cloud-init script, and a replace-both change that can drop requests. The cluster of EC2 instances is also sized for the peak rather than the load. Project 5 — Kubernetes moves the same image onto a cluster that does rolling updates, self-healing and autoscaling natively. You keep the VPC, the managed data tier and the pipeline discipline; the application tier becomes Kubernetes objects (Deployment, Service, Ingress, ConfigMap, Secret) and later (Project 6) is managed through Helm and Argo CD.

LAB NOTES What I ran, what I saw, what it means

THE SUMMIT PROJECTS

P5 Kubernetes

Project 5 of 9 | Uses levels: Level 8 | ~12 hours

1 Goal and outcome

Run `shop-api` on **Kubernetes**: the same container image your Project 3 pipeline publishes, the same `/healthz` and `/readyz` contract, the same Nginx-style request path, but scheduled, healed, scaled and rolled out by a cluster instead of by `docker compose` or a Terraform instance replacement.

Level 8's capstone Capstone 8 is the complete, topic-by-topic build with every manifest. This project is the integration step: it starts from that foundation, plugs in **your** image, ConfigMaps and Secrets from the earlier projects, and changes the deployment step in your pipeline from SSH to `kubectl`. Everything runs on a free local cluster (kind), so there is no cloud cost.

You finish with:

- a kind cluster with an Ingress controller and a `shop` namespace;
- PostgreSQL (StatefulSet with a PersistentVolumeClaim) and Redis (Deployment) in the cluster for the lab, and a clear note on where RDS from Project 4 would replace them;
- the API as a Deployment with two replicas, probes, resource requests and limits, a hardened security context, and a Service;
- `curl -H 'Host: shop.example.com' http://localhost:8080/products` working through the Ingress, with the cache header flipping from MISS to HIT;
- a rolling update that causes no failed requests, a rollback with one command, and a pipeline step `kubectl apply -k` that replaces `deploy.sh`.

2 Architecture

What changed from Project 4: the EC2 instances, user data and ALB target group are replaced by Kubernetes objects (marked NEW). Self-healing, rolling updates and service discovery move from scripts and cloud services into the cluster.

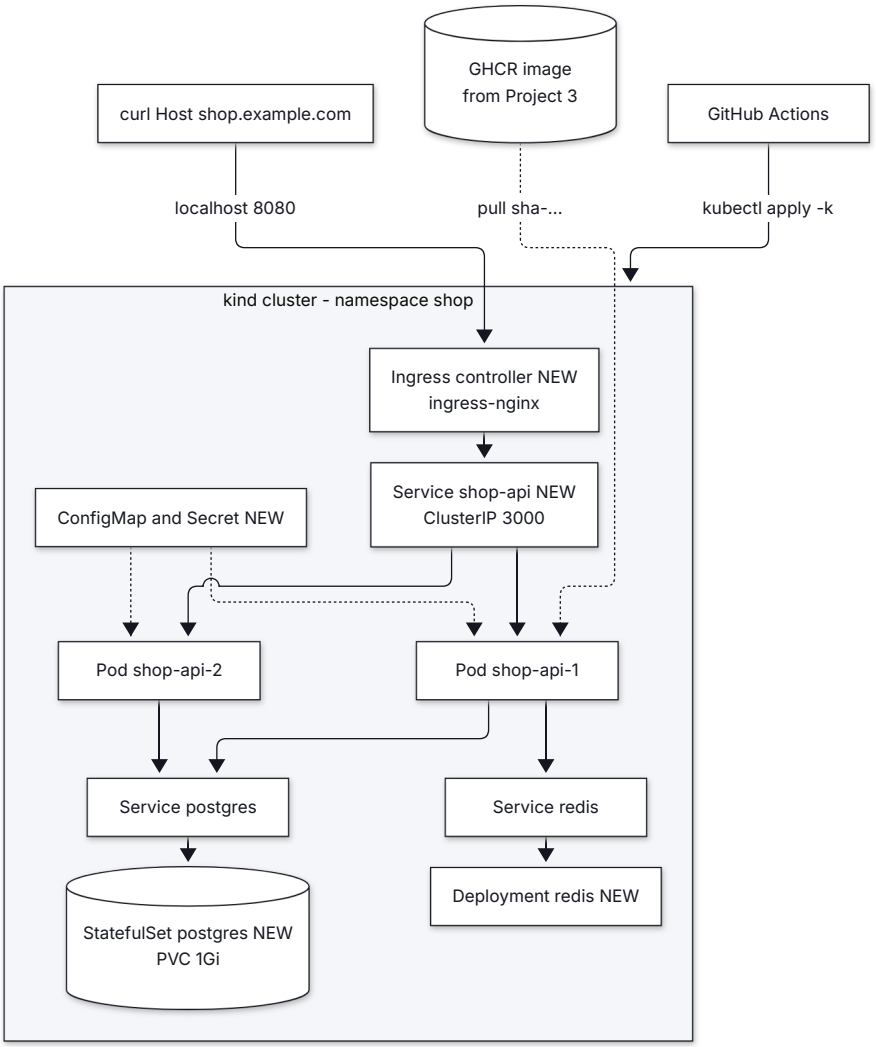


Figure 5-1

What happens when you roll out a new version:

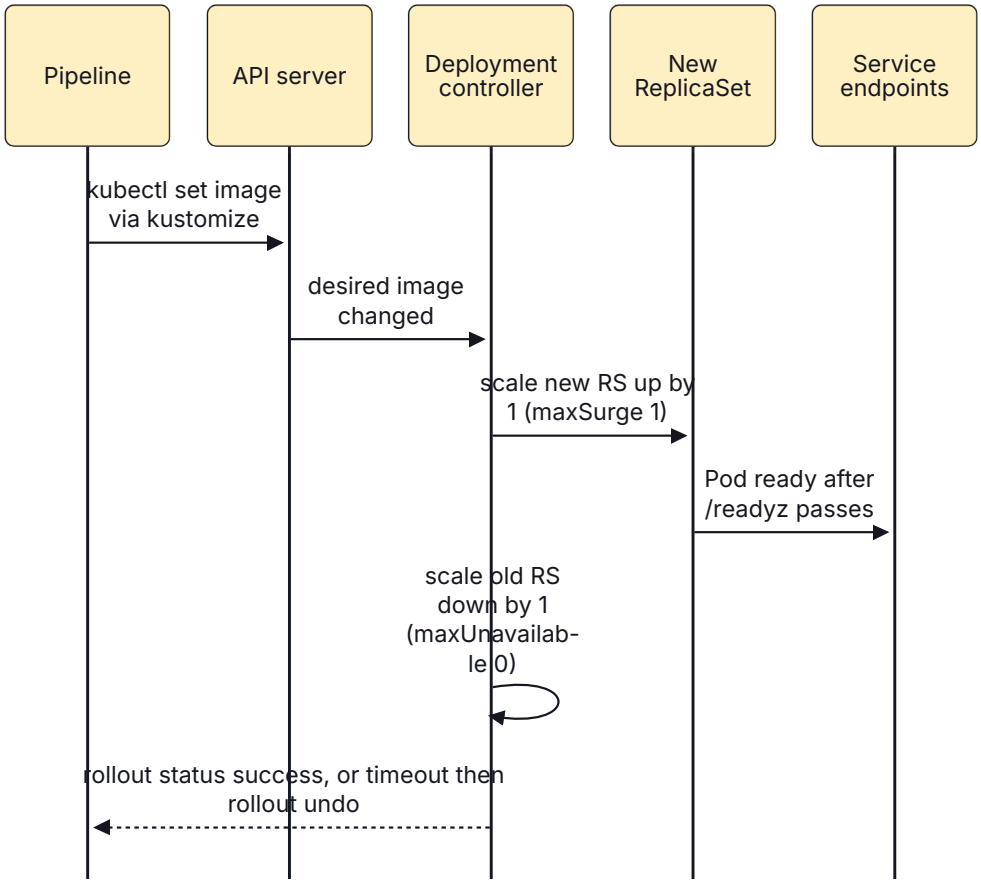


Figure 5-2

Project 4 (AWS)	Project 5 (Kubernetes)
2 EC2 instances, replaced on change	Deployment with rolling updates, `maxUnavailable: 0`
ALB health check `/readyz`	readiness probe `/readyz`, liveness probe `/healthz`
Security groups	NetworkPolicies (Project 6) and Pod security context
Secrets Manager plus instance role	Kubernetes Secret (hardened in Project 6)
`user_data` and env vars	ConfigMap and Secret
ASG or manual scaling	`replicas` now, HPA in Project 6
Terraform deploys the tag	`kustomize edit set image` plus `kubectl apply -k`

3 Prerequisites

- A laptop with Docker, kind (`kind.sigs.k8s.io`), `kubectl`, `curl`; about 8 GB RAM free. Cost: nothing. You may later repeat this on EKS using Project 4's VPC (cost: the EKS control plane and nodes bill hourly).
- The image `ghcr.io/<you>/shop-api:sha-<sha>` from Project 3 (public package, or `kind load docker-image` it).
- Kubernetes: cluster architecture §8.1, Pods §8.2, Deployments §8.3, Services §8.4, Ingress §8.5, ConfigMaps §8.6, Secrets §8.7, namespaces §8.8, volumes, PV and PVC §8.9, StatefulSets §8.10, requests and limits §8.14, probes §8.15, service discovery §8.17.
- Do not start here without comfortable Linux, networking and Docker skills. If `kubectl describe` output is hard to read, revisit Level 2 and Level 4 first.

4 Build it step by step

Work in a repository folder `k8s/` (in the `shop-api` repo) so that the manifests are versioned with the code.

Milestone 1 — Cluster and ingress controller (≈45 min)

Use the kind configuration and controller install from Milestone 1 of the Level 8 capstone Capstone 8 (a kind cluster with ports 80/443 mapped to `localhost:8080/8443`, a CNI that enforces NetworkPolicies such as Calico, and ingress-nginx). Note the capstone's caveat: ingress-nginx is retired upstream, so treat it as a learning tool and keep `ingressClassName` configurable. Verify:

```bash
kubectl get nodes                                  # all Ready
kubectl -n ingress-nginx get pods                  # controller Running
```

Milestone 2 — Namespace, configuration and secrets (≈45 min)

`k8s/namespace.yaml`:

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: shop
  labels:
    pod-security.kubernetes.io/enforce: baseline
    pod-security.kubernetes.io/warn: restricted
```

`k8s/configmap.yaml`:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: shop-api-config
  namespace: shop
data:
  PORT: "3000"
  HOST: "0.0.0.0"
  REDIS_URL: "redis://redis:6379"
```

Secrets must not be committed. Create the database credentials imperatively (Project 6 replaces this with a proper secret workflow):

```bash
kubectl apply -f k8s/namespace.yaml -f k8s/configmap.yaml
DB_PASS="$(openssl rand -hex 16)"
kubectl -n shop create secret generic shop-db \
  --from-literal=POSTGRES_USER=shop --from-literal=POSTGRES_PASSWORD="$DB_PASS" \
  --from-literal=DATABASE_URL="postgres://shop:${DB_PASS}@postgres:5432/shop"
kubectl -n shop get secret shop-db -o jsonpath='{.data.POSTGRES_USER}' | base64 -d     # shop
```

Remember that base64 is encoding, not encryption; anyone who can `get secrets` in the namespace can read them.

Milestone 3 — Data tier: PostgreSQL and Redis (≈1.5 h)

`k8s/postgres.yaml` (a headless Service for stable DNS, a StatefulSet with a volume claim):

```yaml
apiVersion: v1
kind: Service
metadata: { name: postgres, namespace: shop }
spec:
  clusterIP: None
  selector: { app: postgres }
  ports: [{ name: pg, port: 5432 }]
---
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: postgres, namespace: shop }
spec:
  serviceName: postgres
  replicas: 1
  selector: { matchLabels: { app: postgres } }
```

```
  template:
    metadata: { labels: { app: postgres } }
    spec:
      securityContext: { runAsUser: 70, runAsGroup: 70, fsGroup: 70 }
      containers:
        - name: postgres
          image: postgres:17-alpine
          ports: [{ containerPort: 5432, name: pg }]
          env:
            - { name: POSTGRES_DB, value: shop }
            - { name: PGDATA, value: /var/lib/postgresql/data/pgdata }
          envFrom: [{ secretRef: { name: shop-db } }]
          readinessProbe:
            exec: { command: ["pg_isready", "-U", "shop", "-d", "shop"] }
            periodSeconds: 5
          resources:
            requests: { cpu: 100m, memory: 256Mi }
            limits: { memory: 512Mi }
          volumeMounts: [{ name: data, mountPath: /var/lib/postgresql/data }]
  volumeClaimTemplates:
    - metadata: { name: data }
      spec:
        accessModes: ["ReadWriteOnce"]
        resources: { requests: { storage: 1Gi } }
```

`k8s/redis.yaml` (cache only: no persistence, so a Deployment is enough):

YAML

```yaml
apiVersion: v1
kind: Service
metadata: { name: redis, namespace: shop }
spec:
  selector: { app: redis }
  ports: [{ port: 6379 }]
---
apiVersion: apps/v1
kind: Deployment
metadata: { name: redis, namespace: shop }
spec:
  replicas: 1
  selector: { matchLabels: { app: redis } }
  template:
    metadata: { labels: { app: redis } }
    spec:
      containers:
        - name: redis
          image: redis:7.4-alpine
          args: ["--save", "", "--appendonly", "no", "--maxmemory", "64mb", "--maxmemory-
  policy", "allkeys-lru"]
          ports: [{ containerPort: 6379 }]
          readinessProbe: { exec: { command: ["redis-cli", "ping"] }, periodSeconds: 5 }
```

```
          resources:
            requests: { cpu: 50m, memory: 64Mi }
            limits: { memory: 128Mi }
```

Seed the schema with a one-off Job (the `db/init.sql` from Project 2 in a ConfigMap), and make it idempotent:

```bash
kubectl -n shop create configmap shop-init-sql --from-file=init.sql=db/init.sql
```

```yaml
apiVersion: batch/v1
kind: Job
metadata: { name: shop-seed, namespace: shop }
spec:
  backoffLimit: 4
  template:
    spec:
      restartPolicy: OnFailure
      containers:
        - name: psql
          image: postgres:17-alpine
          command: ["sh", "-c", "until pg_isready -h postgres -U shop; do sleep 2; done;
  psql \"$DATABASE_URL\" -f /sql/init.sql"]
          envFrom: [{ secretRef: { name: shop-db } }]
          volumeMounts: [{ name: sql, mountPath: /sql }]
      volumes:
        - { name: sql, configMap: { name: shop-init-sql } }
```

`init.sql` uses `CREATE TABLE IF NOT EXISTS`, but `INSERT` would duplicate rows on re-run; guard it with `ON CONFLICT DO NOTHING` and a unique key on `name`, or seed only when the table is empty.

Milestone 4 — The API: Deployment and Service (≈1.5 h)

`k8s/api.yaml`:

```yaml
apiVersion: v1
kind: Service
metadata: { name: shop-api, namespace: shop }
spec:
  selector: { app: shop-api }
  ports: [{ name: http, port: 3000, targetPort: http }]
---
apiVersion: apps/v1
kind: Deployment
```

```
metadata: { name: shop-api, namespace: shop }
spec:
  replicas: 2
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
  selector: { matchLabels: { app: shop-api } }
  template:
    metadata: { labels: { app: shop-api } }
    spec:
      terminationGracePeriodSeconds: 30
      securityContext: { runAsNonRoot: true, seccompProfile: { type: RuntimeDefault } }
      containers:
        - name: api
          image: ghcr.io/you/shop-api:sha-REPLACED_BY_PIPELINE
          ports: [{ name: http, containerPort: 3000 }]
          envFrom:
            - configMapRef: { name: shop-api-config }
            - secretRef: { name: shop-db }
          startupProbe:   { httpGet: { path: /healthz, port: http }, periodSeconds: 2,
  failureThreshold: 30 }
          livenessProbe:  { httpGet: { path: /healthz, port: http }, periodSeconds: 10,
  failureThreshold: 3 }
          readinessProbe: { httpGet: { path: /readyz,  port: http }, periodSeconds: 5,
  failureThreshold: 2 }
          resources:
            requests: { cpu: 100m, memory: 128Mi }
            limits: { memory: 256Mi }
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: { drop: ["ALL"] }
```

Two details matter. `/readyz` checks PostgreSQL, so a Pod whose database is down is removed from the Service without being restarted, whereas `/healthz` has no dependency checks, so a database outage does not cause a restart storm (probes §8.15). And the container exits cleanly on `SIGTERM`, which is what lets `maxUnavailable: 0` mean zero failed requests.

Milestone 5 — Ingress and Kustomize (≈45 min)

`k8s/ingress.yaml`:

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: shop-api, namespace: shop }
spec:
  ingressClassName: nginx
  rules:
    - host: shop.example.com
      http:
        paths:
          - { path: /, pathType: Prefix, backend: { service: { name: shop-api, port: {
  name: http } } } }
```

`k8s/kustomization.yaml` (so the pipeline can set the image tag without editing YAML by hand):

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: shop
resources: [api.yaml, ingress.yaml, postgres.yaml, redis.yaml]
images:
  - name: ghcr.io/you/shop-api
    newTag: sha-REPLACED_BY_PIPELINE
```

```bash
cd k8s && kubectl kustomize . >/dev/null && echo "renders"
kustomize edit set image ghcr.io/you/shop-api=ghcr.io/you/shop-api:sha-<sha>      # or:
  kubectl kustomize edit set image
kubectl apply -k .
kubectl -n shop rollout status deploy/shop-api --timeout=120s
curl -si -H 'Host: shop.example.com' http://localhost:8080/products | grep -iE 'HTTP/|x-
  cache'
```

Milestone 6 — Replace `deploy.sh` in the pipeline (≈1.5 h)

In the `deploy` job of Project 3, replace the SSH step with a cluster step. For a local kind cluster, run a **self-hosted runner** on your machine or deploy manually; for a cloud cluster use OIDC to get short-lived credentials (EKS access entries):

```yaml
      - name: Deploy to Kubernetes
        run: |
          cd k8s
          kustomize edit set image ghcr.io/${{ github.repository }}=ghcr.io/${{
  github.repository }}:sha-${{ github.sha }}
          kubectl apply -k .
          kubectl -n shop rollout status deploy/shop-api --timeout=180s || {
            kubectl -n shop rollout undo deploy/shop-api; exit 1; }
      - name: Smoke test
        run: BASE=https://shop.example.com ./smoke.sh
```

A running cluster reachable by a hosted runner needs credentials and network access; do not expose a lab API server to the internet for this. This is the reason Project 6 moves to a pull-based model (Argo CD): the cluster pulls changes, and CI needs no cluster credentials.

5 Tasks

- [ ] **T1 — Create the cluster and inspect it** *(≈45 min, beginner)*
  - **Goal:** know what is running before you add anything.
  - **Steps:**
    1. Create the kind cluster and install the Ingress controller.
    2. Run `kubectl get pods -A`, `kubectl get nodes -o wide`, and find the API server, etcd, scheduler and controller-manager Pods in `kube-system`.
    3. Sketch the control plane and node components from memory and compare with cluster architecture §8.1.
  - **Done when:** all nodes are `Ready` and you can name which component created each Pod you see.
- [ ] **T2 — Deploy the data tier** *(≈1 h, intermediate)*
  - **Goal:** PostgreSQL with durable storage and Redis for caching.
  - **Steps:**
    1. Apply the namespace, ConfigMap, Secret, `postgres.yaml` and `redis.yaml`.
    2. Run the seed Job and read its logs.
    3. Delete the Pod `postgres-0` and wait for it to return; query the table again.

- **Done when:** the three products are still present after the Pod is recreated, and `kubectl -n shop get pvc` shows `Bound`.

- [ ] **T3 — Run the API with probes and limits** *(≈1 h, intermediate)*
  - **Goal:** two healthy replicas behind a Service.
  - **Steps:**
    1. Apply `api.yaml` with your Project 3 image tag.
    2. `kubectl -n shop get endpoints shop-api` and `kubectl -n shop describe pod -l app=shop-api`.
    3. `kubectl -n shop port-forward svc/shop-api 3000:3000` and curl `/readyz`.
  - **Done when:** two Pods are `Ready`, the Service lists two endpoint addresses, and `/readyz` returns 200.

- [ ] **T4 — Expose it through Ingress** *(≈30 min, intermediate)*
  - **Goal:** the same URL path you had with Nginx, now through an Ingress.
  - **Steps:**
    1. Apply `ingress.yaml`.
    2. `curl -si -H 'Host: shop.example.com' http://localhost:8080/products` twice.
    3. Run `kubectl -n shop logs deploy/shop-api` and `kubectl -n ingress-nginx logs deploy/ingress-nginx-controller` to follow one request.
  - **Done when:** the first call has `X-Cache: MISS`, the second `HIT`, and you found your request in both logs.

- [ ] **T5 — Roll out a change with zero failed requests** *(≈45 min, intermediate)*
  - **Goal:** prove `maxUnavailable: 0` works.
  - **Steps:**
    1. In one terminal run `while true; do curl -s -o /dev/null -w '%{http_code}\n' -H 'Host: shop.example.com' http://localhost:8080/readyz; sleep 0.2; done | sort | uniq -c`.
    2. In another, change the image tag and `kubectl apply -k .`.
    3. Stop the loop and read the counts. Then repeat after changing the strategy to `maxUnavailable: 1` and `terminationGracePeriodSeconds: 0`.
  - **Done when:** the first rollout shows only `200`, and you can explain any non-200 in the second.

- [ ] **T6 — Break it and recover with rollback** *(≈45 min, intermediate)*
  - **Goal:** a bad release does not take the site down.

- **Steps:**
  1. Set the image to a tag that does not exist and apply. Watch `kubectl -n shop get pods -w` (`ImagePullBackOff`).
  2. Set `readinessProbe.path` to `/nope` and apply (the new Pods never become Ready).
  3. Use `kubectl -n shop rollout undo deploy/shop-api` and `kubectl -n shop rollout history deploy/shop-api`.
- **Done when:** in both cases the old Pods kept serving traffic during the failed rollout, and `rollout undo` restored the working version.

- [ ] **T7 — Wire the pipeline to the cluster** *(≈1.5 h, advanced)*
  - **Goal:** merge to `main` deploys to Kubernetes.
  - **Steps:**
    1. Install a self-hosted runner on the machine that runs kind, or script the deploy locally with `kustomize edit set image` as above.
    2. Replace the SSH deploy step with the Kubernetes step and add the automatic `rollout undo`.
    3. Merge a visible change and watch the run.
  - **Done when:** the commit's `sha-...` image is running in the cluster (`kubectl -n shop get deploy shop-api -o jsonpath='{.spec.template.spec.containers[0].image}'`) and the smoke test passed.

- [ ] **T8 — Prove it: explain a request and a failure** *(≈1 h, advanced)*
  - **Goal:** show the cluster is understood, not just running.
  - **Steps:**
    1. Without the guide, write down every object and DNS name touched by `curl -H 'Host: shop.example.com' http://localhost:8080/products`.
    2. Delete a node's Pod, then scale Redis to 0, and describe what each user sees and why.
    3. Compare Project 4 (EC2/ALB) and Project 5 on updates, health, secrets and scaling in a table of at least five rows.
  - **Done when:** your trace and table are in `docs/project-05.md`, and both failure behaviors match your predictions.

6 Definition of done

- [ ] `kubectl -n shop get pods` shows API (2), Redis (1) and PostgreSQL (1) Pods `Running` and `Ready`.

- [ ] `curl -H 'Host: shop.example.com' http://localhost:8080/products` works through the Ingress, with MISS then HIT.
- [ ] The API Deployment has requests and limits, startup, liveness and readiness probes, non-root user, read-only root file system and dropped capabilities.
- [ ] A rolling update shows no failed requests, and `kubectl rollout undo` recovers a bad release.
- [ ] The database survives `kubectl delete pod postgres-0`.
- [ ] No secret value is committed; manifests contain only references.
- [ ] The pipeline (or a documented command) deploys by image tag, never `latest`.

7 Break it on purpose

1. **Limit too low.** Set the API memory limit to `32Mi`. Watch `OOMKilled` and `CrashLoopBackOff` in `kubectl describe pod`, find the last state and exit code 137, and raise the limit. *(Lesson: requests schedule Pods, limits kill them.)*
2. **Selector mismatch.** Change the Service selector to `app: shop-apii`. The Service has no endpoints and the Ingress returns 503. Find it with `kubectl get endpoints`. *(Lesson: Services and Pods are linked only by labels.)*
3. **Break DNS naming.** Recreate the Secret with a `DATABASE_URL` whose host is `postgres.default.svc.cluster.local` (the wrong namespace) and restart the Deployment. Use a debug Pod (`kubectl run -it --rm debug --image=busybox:1.37 -n shop -- nslookup postgres`) to diagnose why `/readyz` fails. *(Lesson: Service DNS names include the namespace.)*
4. **Lose the database Pod's volume claim.** Delete the PVC while the Pod is running and see why it stays `Terminating` until the Pod is gone (protection finalizer); then recreate the lab and restore from `pg_dump`. *(Lesson: Kubernetes protects data objects, but not from every mistake.)*

8 How this evolves into the next project

This cluster works, but it is a **demo-grade** setup: no TLS, plaintext secrets, any Pod can talk to any Pod, nobody is limited by RBAC, the replicas are fixed, manifests are copy-edited per environment, and CI holds credentials for the cluster. Project 6 — Production Kubernetes closes those gaps: TLS on the Ingress, an HPA, RBAC and NetworkPolicies, a proper secrets workflow, a Helm chart replacing the raw YAML, and Argo CD pulling from Git so that CI no longer needs cluster credentials.

LAB NOTES What I ran, what I saw, what it means

THE SUMMIT PROJECTS

P6 Production Kubernetes

Project 6 of 9 | Uses levels: Levels 8, 9, 16, 17 | ~16 hours

1 Goal and outcome

Turn the demo cluster from Project 5 into something you would let a team depend on. You add what a basic cluster lacks: **TLS** on the Ingress, **autoscaling**, **least-privilege RBAC**, **secrets that can live in Git safely**, **default-deny NetworkPolicies**, **enforced Pod security**, a **Helm chart** replacing raw YAML, and **GitOps with Argo CD** so that the cluster pulls its desired state from Git and CI needs no cluster credentials.

The depth for each part is in the level capstones: Level 9 (Helm) Capstone 9, Level 16 (hardening) Capstone 16 and Level 17 (GitOps) Capstone 17. This project integrates them around **one application and one delivery path**, so you see how they interact.

You finish with:

- a `shop-gitops` repository holding the Helm chart and per-environment values (`dev`, `prod`), the single source of truth for the cluster;
- Argo CD syncing both environments automatically, with prune and self-heal, and promotion by pull request;
- HTTPS on `https://shop.example.com` (a self-signed issuer in the lab; Let's Encrypt on a real domain);
- an HPA scaling the API from 2 to 6 replicas under load;
- RBAC giving a `shop-devs` group read-only access (no Secrets), enforced and tested with `kubectl auth can-i`;
- a sealed secret in Git instead of a plaintext Secret, and Pod Security `restricted` enforced in the `shop` namespace;
- NetworkPolicies proving only the intended flows work.

2 Architecture

What changed from Project 5: delivery moves from "CI runs `kubectl apply`" to "CI updates Git and Argo CD reconciles" (NEW), and the cluster gains security and scaling controls

(NEW).

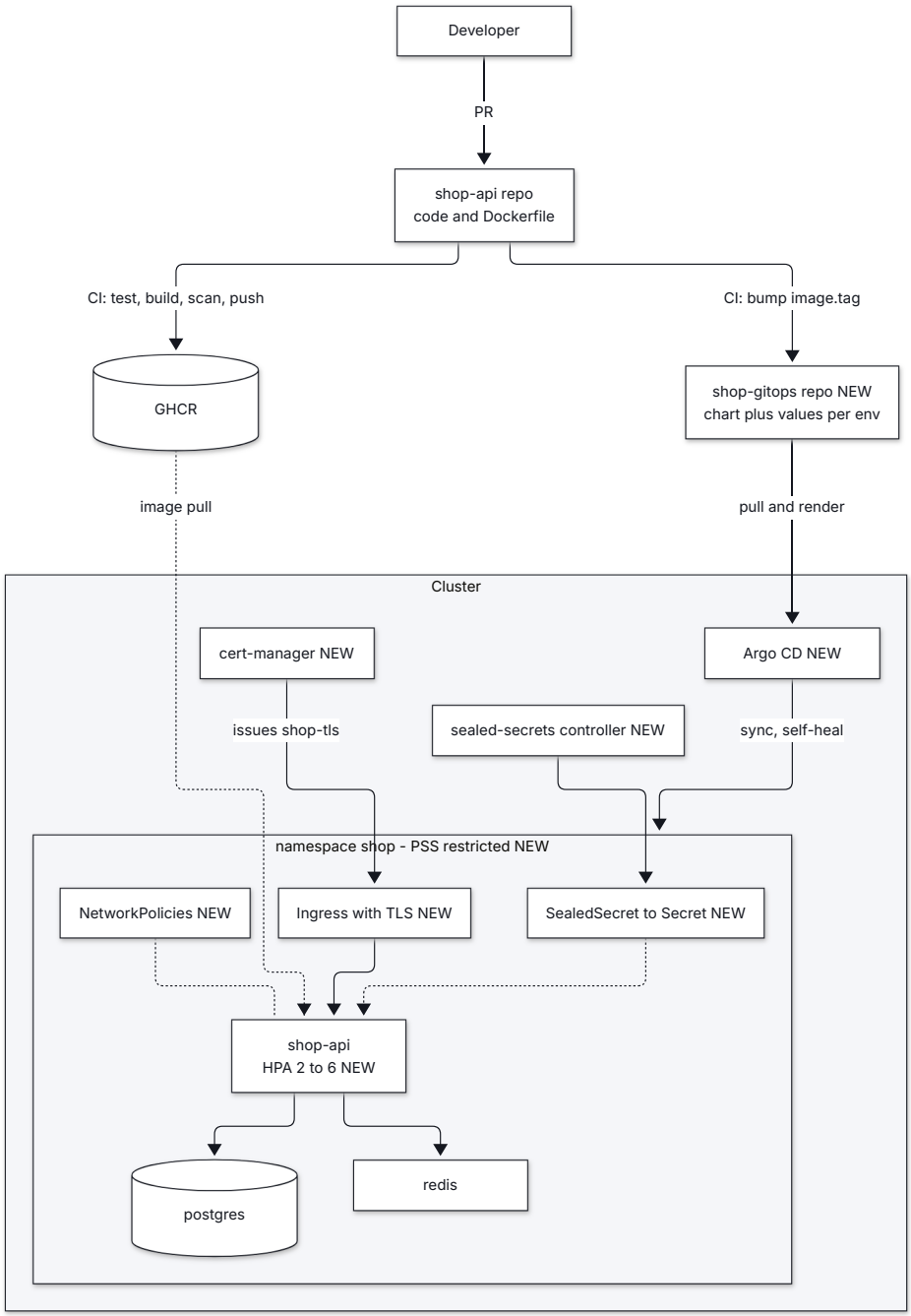


Figure 6-1

Network policy intent (anything not drawn is denied):

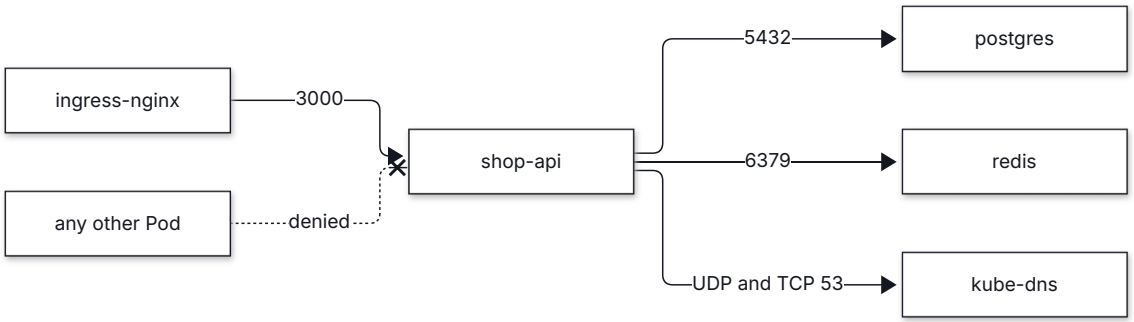


Figure 6-2

Project 5	Project 6
Raw YAML applied by CI	Helm chart in Git, reconciled by Argo CD
CI holds cluster credentials	CI only pushes to Git; the cluster pulls
HTTP only	TLS from cert-manager
Fixed `replicas: 2`	HPA on CPU, plus a PodDisruptionBudget
Secret created by hand	SealedSecret committed to Git
Any Pod talks to any Pod	Default-deny plus explicit allows
`baseline` Pod security	`restricted` enforced
Everyone is admin	Namespaced Roles and a group binding

3 Prerequisites

- A working Project 5 cluster and image (kind is fine, 8 GB RAM; about 4 CPUs). Tools: `kubectl`, `helm`, `kubeseal`, `git`, `gh`, optionally the `argocd` CLI. Cost: nothing locally. On EKS, add the cost of nodes and the control plane, and destroy when done.
- Kubernetes core: RBAC §8.18, NetworkPolicies §8.19, HPA §8.16, Ingress §8.5, Secrets §8.7.
- Helm: charts §9.1, values §9.2, templates §9.3, releases §9.4, packaging applications §9.6.
- Advanced Kubernetes: RBAC in depth §16.1, NetworkPolicies in depth §16.2, Pod security §16.3, HPA, VPA and cluster autoscaler §16.7.
- GitOps: desired state §17.1, Argo CD and Flux §17.2, environment promotion §17.4, CI to GitOps flow §17.7, rollback through Git §17.6.
- Secrets: secrets management §11.4.

4 Build it step by step

Milestone 1 — The Helm chart and the GitOps repository (≈3 h)

Create the `shop-gitops` repository. Turn the Project 5 manifests into a chart, as in the Level 9 capstone [Capstone 9]:

```
shop-gitops/
  charts/shop-api/
    Chart.yaml            # name: shop-api, version: 0.1.0, appVersion from CI
    values.yaml           # safe defaults, every key commented
    values.schema.json    # rejects typos and bad types
    templates/            # deployment, service, ingress, hpa, pdb, networkpolicy,
  serviceaccount, _helpers.tpl
  envs/
    dev/values.yaml
    prod/values.yaml
  apps/                   # Argo CD Application manifests (Milestone 5)
```

`charts/shop-api/values.yaml` (excerpt):

```yaml
image:
  repository: ghcr.io/you/shop-api
  tag: ""                       # set per environment by CI, never "latest"
replicaCount: 2
service: { port: 3000 }
ingress:
  enabled: true
  className: nginx
  host: shop.example.com
  tls: { enabled: true, issuer: lab-selfsigned }
autoscaling: { enabled: true, minReplicas: 2, maxReplicas: 6,
  targetCPUUtilizationPercentage: 70 }
resources:
  requests: { cpu: 100m, memory: 128Mi }
  limits: { memory: 256Mi }
existingSecret: shop-db         # created by the SealedSecret
```

`envs/dev/values.yaml` sets a lower `maxReplicas` and `image.tag`; `envs/prod/values.yaml` sets bigger limits and `replicaCount: 3`. Check rendering and quality before anything touches the cluster:

```bash
helm lint charts/shop-api -f envs/dev/values.yaml
helm template shop charts/shop-api -f envs/dev/values.yaml | kubectl apply --dry-
  run=server -f -
```

Make the Deployment restart when its ConfigMap changes, using a checksum annotation, and put `securityContext` (non-root, read-only root file system, `drop: [ALL]`, `seccompProfile: RuntimeDefault`) in the chart so it is not optional.

Milestone 2 — TLS with cert-manager (≈1 h)

```bash
helm repo add jetstack https://charts.jetstack.io && helm repo update
helm install cert-manager jetstack/cert-manager -n cert-manager --create-namespace --set
  crds.enabled=true
```

```yaml
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata: { name: lab-selfsigned }
spec: { selfSigned: {} }
```

The chart's Ingress adds `cert-manager.io/cluster-issuer: {{ .Values.ingress.tls.issuer }}` and a `tls:` block with `secretName: shop-tls`.

```bash
kubectl -n shop get certificate shop-tls                        # READY True
curl -ks --resolve shop.example.com:8443:127.0.0.1 https://shop.example.com:8443/readyz
```

`-k` is needed only because the lab issuer is self-signed. On a real domain switch the issuer to an ACME `ClusterIssuer` for Let's Encrypt (HTTP-01 or DNS-01) and remove `-k` (TLS §12.4).

Milestone 3 — Autoscaling (≈45 min)

Install metrics-server (on kind add `--kubelet-insecure-tls`, lab only). The chart's `hpa.yaml`:

```yaml
{{- if .Values.autoscaling.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: {{ include "shop-api.fullname" . }} }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: {{ include "shop-
  api.fullname" . }} }
  minReplicas: {{ .Values.autoscaling.minReplicas }}
  maxReplicas: {{ .Values.autoscaling.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: {{
  .Values.autoscaling.targetCPUUtilizationPercentage }} }
{{- end }}
```

When the HPA is enabled, remove `replicas:` from the Deployment template; otherwise every sync would reset the count Argo CD sees. Add a PodDisruptionBudget with `minAvailable: 1` so a node drain cannot take every replica at once.

Milestone 4 — RBAC, Pod security, secrets and NetworkPolicies (≈3 h)

RBAC. A read-only role for developers, without Secrets:

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: shop-viewer, namespace: shop }
rules:
  - apiGroups: [""]
    resources: [pods, pods/log, services, endpoints, configmaps, events]
    verbs: [get, list, watch]
  - apiGroups: [apps]
    resources: [deployments, replicasets, statefulsets]
    verbs: [get, list, watch]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: shop-devs-view, namespace: shop }
subjects: [{ kind: Group, name: shop-devs, apiGroup: rbac.authorization.k8s.io }]
roleRef: { kind: Role, name: shop-viewer, apiGroup: rbac.authorization.k8s.io }
```

```bash
kubectl auth can-i list pods    -n shop --as=jane --as-group=shop-devs     # yes
kubectl auth can-i get secrets  -n shop --as=jane --as-group=shop-devs     # no
kubectl auth can-i delete pods  -n shop --as=jane --as-group=shop-devs     # no
```

Pod security. Label the namespace `pod-security.kubernetes.io/enforce=restricted` and fix what it rejects (PostgreSQL and Redis need `runAsNonRoot`, an explicit `runAsUser`, `allowPrivilegeEscalation: false`, `drop: [ALL]`, and `seccompProfile`). Read the warnings; they tell you exactly which field is missing (Pod security §16.3).

Sealed secrets. Install the Sealed Secrets controller, then seal the existing secret so that the encrypted form is safe to commit:

```bash
kubectl create secret generic shop-db -n shop --dry-run=client -o yaml \
  --from-literal=POSTGRES_USER=shop --from-literal=POSTGRES_PASSWORD="$DB_PASS" \
  --from-literal=DATABASE_URL="postgres://shop:${DB_PASS}@postgres:5432/shop" \
  | kubeseal --format yaml > envs/dev/sealed-shop-db.yaml
git add envs/dev/sealed-shop-db.yaml
```

Only the controller in that cluster can decrypt it. Back up the controller's key; losing it means re-sealing everything. In a cloud setup, External Secrets Operator reading AWS Secrets Manager (the secret from Project 4) is the usual alternative.

NetworkPolicies. Start with deny-all, then allow only the drawn flows (the chart renders these):

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny, namespace: shop }
spec: { podSelector: {}, policyTypes: [Ingress, Egress] }
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: api, namespace: shop }
spec:
  podSelector: { matchLabels: { app: shop-api } }
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
        - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: ingress-nginx }
  }
      ports: [{ port: 3000 }]
  egress:
    - to: [{ podSelector: { matchLabels: { app: postgres } } }]
      ports: [{ port: 5432 }]
```

```
    - to: [{ podSelector: { matchLabels: { app: redis } } }]
      ports: [{ port: 6379 }]
    - to:
        - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: kube-system } }
          podSelector: { matchLabels: { k8s-app: kube-dns } }
      ports: [{ port: 53, protocol: UDP }, { port: 53, protocol: TCP }]
```

Add matching ingress policies for `postgres` and `redis` that accept traffic only from `app: shop-api`. NetworkPolicies need a CNI that enforces them (Calico or Cilium, as in Project 5); with a CNI that ignores them they silently do nothing.

Milestone 5 — Argo CD and the delivery flow (≈3 h)

BASH

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server --timeout=300s
```

`apps/shop-api-dev.yaml`:

YAML

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: shop-api-dev, namespace: argocd }
spec:
  project: default
  source:
    repoURL: https://github.com/you/shop-gitops.git
    targetRevision: main
    path: charts/shop-api
    helm: { valueFiles: [../../envs/dev/values.yaml] }
  destination: { server: https://kubernetes.default.svc, namespace: shop }
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions: [CreateNamespace=true]
```

Define the sealed secret and policies as part of the same repository path (a second source or an `envs/dev` Kustomize layer) so one Application owns the full desired state. For `prod`, remove `automated` (manual sync) or protect the `prod` values with CODEOWNERS (environment promotion §17.4).

Change the application pipeline from Project 5: instead of `kubectl apply`, the `deploy` job commits the new tag to the GitOps repository.

```yaml
      - name: Bump dev image tag in GitOps repo
        env: { GH_TOKEN: "${{ secrets.GITOPS_PAT }}" }
        run: |
          git clone https://x-access-token:${GH_TOKEN}@github.com/you/shop-gitops.git
          cd shop-gitops
          yq -i '.image.tag = "sha-${{ github.sha }}"' envs/dev/values.yaml
          git -c user.name=ci -c user.email=ci@example.com commit -am "shop-api: dev →
  sha-${{ github.sha }}"
          git push
```

Use a fine-grained token that can write only to that one repository. Promotion to `prod` is a pull request that copies the tag from `envs/dev` to `envs/prod`; rollback is `git revert` (roll-back through Git §17.6).

```bash
kubectl -n argocd get applications                          # shop-api-dev  Synced  Healthy
kubectl -n shop delete service shop-api                     # drift on purpose: self-heal
  recreates it within a minute
kubectl -n shop get service shop-api
```

5 Tasks

- [ ] **T1 — Package the app as a chart** *(≈2 h, intermediate)*
  - **Goal:** the Project 5 YAML becomes a parameterised, tested chart.
  - **Steps:**
    1. Create `charts/shop-api` with deployment, service, ingress, hpa, pdb templates and a `values.schema.json`.
    2. Run `helm lint` and `helm template` for dev and prod values; diff the two renders.
    3. Install it manually into a scratch namespace and run `helm test` or the smoke test.
  - **Done when:** `helm lint` is clean, the dev and prod diffs show only intended differences, and a bad value (`replicaCount: "two"`) is rejected by the schema.
- [ ] **T2 — Turn on HTTPS** *(≈1 h, intermediate)*
  - **Goal:** the Ingress serves a certificate issued by cert-manager.
  - **Steps:**
    1. Install cert-manager and the `lab-selfsigned` issuer.
    2. Enable `ingress.tls` and sync.

3. Run `openssl s_client -connect localhost:8443 -servername shop.example.com </dev/null | openssl x509 -noout -subject -dates`.

- **Done when:** `kubectl get certificate` shows `READY True` and the command prints a subject for `shop.example.com`.

- [ ] **T3 — Scale under load** *(≈1 h, intermediate)*
  - **Goal:** watch the HPA add and remove replicas.
  - **Steps:**
    1. Install metrics-server and confirm `kubectl top pods -n shop` works.
    2. Generate load (`kubectl run load --rm -it --image=busybox:1.37 -n shop -- sh -c 'while true; do wget -q -O- http://shop-api:3000/products >/dev/null; done'`, adjusting your NetworkPolicy to allow it temporarily).
    3. Watch `kubectl -n shop get hpa -w` and then stop the load.
  - **Done when:** replicas rise above 2 and later fall back to 2, and you captured the HPA events with `kubectl describe hpa`.

- [ ] **T4 — Enforce least privilege with RBAC** *(≈45 min, intermediate)*
  - **Goal:** developers can look but not touch secrets or delete.
  - **Steps:**
    1. Apply the Role and RoleBinding.
    2. Run the three `kubectl auth can-i` checks with `--as` and `--as-group`.
    3. Try `kubectl auth can-i --list -n shop --as=jane --as-group=shop-devs` and explain every line.
  - **Done when:** the checks print `yes`, `no`, `no` as shown, and you can name why `pods/log` is a separate resource.

- [ ] **T5 — Default-deny and prove the policy** *(≈1.5 h, advanced)*
  - **Goal:** only the intended flows work.
  - **Steps:**
    1. Apply the default-deny and allow policies.
    2. From a debug Pod in the `shop` namespace, try `nc -zv postgres 5432` (must fail) and from the API Pod verify the request through the Ingress still works.
    3. Remove the DNS egress rule and observe what breaks, then restore it.
  - **Done when:** direct Pod-to-database from an unlabelled Pod times out, the app still serves `/products`, and you can explain why removing DNS egress broke the API.

- [ ] **T6 — Put the secret in Git safely** *(≈1 h, advanced)*

- **Goal:** no plaintext secret anywhere in the repository history.
- **Steps:**
  1. Install Sealed Secrets, seal `shop-db`, delete the plaintext Secret and let Argo CD recreate it.
  2. Run a secret scanner over the repo (`gitleaks detect`).
  3. Copy the sealed file to a different cluster (or namespace) and observe that it will not decrypt.
- **Done when:** the API runs with a Secret created by the controller, the scanner finds nothing, and the cross-cluster attempt fails.

☐ **T7 — Deliver a change through Git only** *(≈1.5 h, advanced)*

- **Goal:** a merge to the app repo ends up running in the cluster without anyone running `kubectl`.
- **Steps:**
  1. Install the Argo CD Application with automated sync.
  2. Merge an application change; watch the pipeline bump the dev tag and Argo CD sync it.
  3. Open a PR promoting the tag to `prod`; after merge, sync and verify.
- **Done when:** the running image tag in each environment matches the Git values file, and the audit trail is the Git history.

☐ **T8 — Prove it: recover from drift and a bad release** *(≈1.5 h, advanced)*

- **Goal:** demonstrate GitOps self-healing and rollback.
- **Steps:**
  1. Delete the Service and edit the Deployment's image by hand; watch Argo CD revert both.
  2. Merge a release that breaks `/readyz`; observe the rollout stall while the old Pods keep serving.
  3. `git revert` the bad commit and watch the environment recover. Explain who could do each action and why.
- **Done when:** all three recoveries happen without a manual `kubectl apply`, and you can describe the blast radius of a stolen CI token versus a stolen cluster admin credential.

6 Definition of done

- [ ] `argocd` shows both environments `Synced` and `Healthy`; every change since setup came through a Git commit.
- [ ] `https://shop.example.com` works with a certificate managed by cert-manager; HTTP redirects to HTTPS.
- [ ] The HPA scales between 2 and 6 replicas; a PodDisruptionBudget exists; requests and limits are set.
- [ ] `kubectl auth can-i get secrets --as=jane --as-group=shop-devs -n shop` returns `no`; CI holds no cluster credentials.
- [ ] No plaintext secret in Git history; the sealed secret decrypts only in its cluster.
- [ ] Default-deny NetworkPolicies are in place with exactly the intended allows, and the namespace enforces Pod Security `restricted`.
- [ ] A rollback by `git revert` has been rehearsed.

7 Break it on purpose

1. **Forget DNS in the egress policy.** Apply default-deny with no DNS rule; the API cannot resolve `postgres`. Find it with `kubectl logs` and a debug Pod. *(Lesson: deny-all breaks name resolution first.)*
2. **Hand-edit a managed resource.** `kubectl -n shop edit deploy shop-api` and change the image. Argo CD reports `OutOfSync` and, with self-heal, reverts it. Turn self-heal off and see the difference. *(Lesson: Git, not kubectl, is the source of truth.)*
3. **Violate Pod Security.** Add `privileged: true` to a container in a branch. The Pod is rejected at admission with a clear message, and Argo CD shows the sync failed. *(Lesson: policy at admission blocks bad config before it runs.)*
4. **Lose the sealing key.** Delete the Sealed Secrets controller's key Secret in a throwaway cluster and reinstall; existing SealedSecrets stop decrypting. Restore from your backup. *(Lesson: the key is a recovery dependency; back it up.)*

8 How this evolves into the next project

The platform now delivers and protects itself, but you can only guess how it behaves. When the p95 latency doubles, you have `kubectl top` and `kubectl logs`. Project 7 — Observability adds the missing senses: Prometheus metrics and alerts, Grafana dashboards, Loki logs,

OpenTelemetry traces and Sentry error tracking, all deployed through the same GitOps repository so that observability is itself declarative. The NetworkPolicies you wrote will need new allows for scraping, which is the first lesson of the next project.

THE SUMMIT PROJECTS

P7 Observability

Project 7 of 9 | Uses levels: Level 10 | ~14 hours

1 Goal and outcome

Make the platform from Project 6 **explain itself**. You add the three pillars plus error tracking to the same GitOps repository: **metrics** (Prometheus and Grafana), **logs** (Loki), **traces** (OpenTelemetry into Tempo) and **errors** (Sentry). The goal is not pretty dashboards; it is that when something breaks you can go from an alert to the failing request to the line of code in a few minutes, using evidence.

The tools and concepts are taught in Level 10: three pillars §10.1, Prometheus and PromQL §10.2, Grafana dashboards §10.3, Loki §10.6, OpenTelemetry §10.9 and Sentry §10.10. This project wires them into **one running system** and ties them to a reliability target.

You finish with:

- the API exposing `/metrics` (request rate, errors, latency histogram, cache hit ratio) and writing structured JSON logs that carry a request ID and trace ID;
- a Prometheus scraping the API through a `ServiceMonitor`, Grafana with a "shop-api" dashboard (the four golden signals), and alert rules that page on SLO burn rather than on CPU;
- logs searchable in Grafana by `app`, `level` and `trace_id`, and a click from a log line to its trace;
- distributed traces showing one request across Nginx (Ingress), the API, Redis and PostgreSQL;
- unhandled exceptions arriving in Sentry with a release tag and a stack trace;
- an SLO document: "99.5 percent of `/products` requests succeed over 30 days, and 95 percent complete in under 300 ms".

2 Architecture

What changed from Project 6: a `monitoring` namespace with the telemetry backends (NEW), instrumentation in the API (NEW), new NetworkPolicies to allow telemetry, and all of it installed by Argo CD from Git.

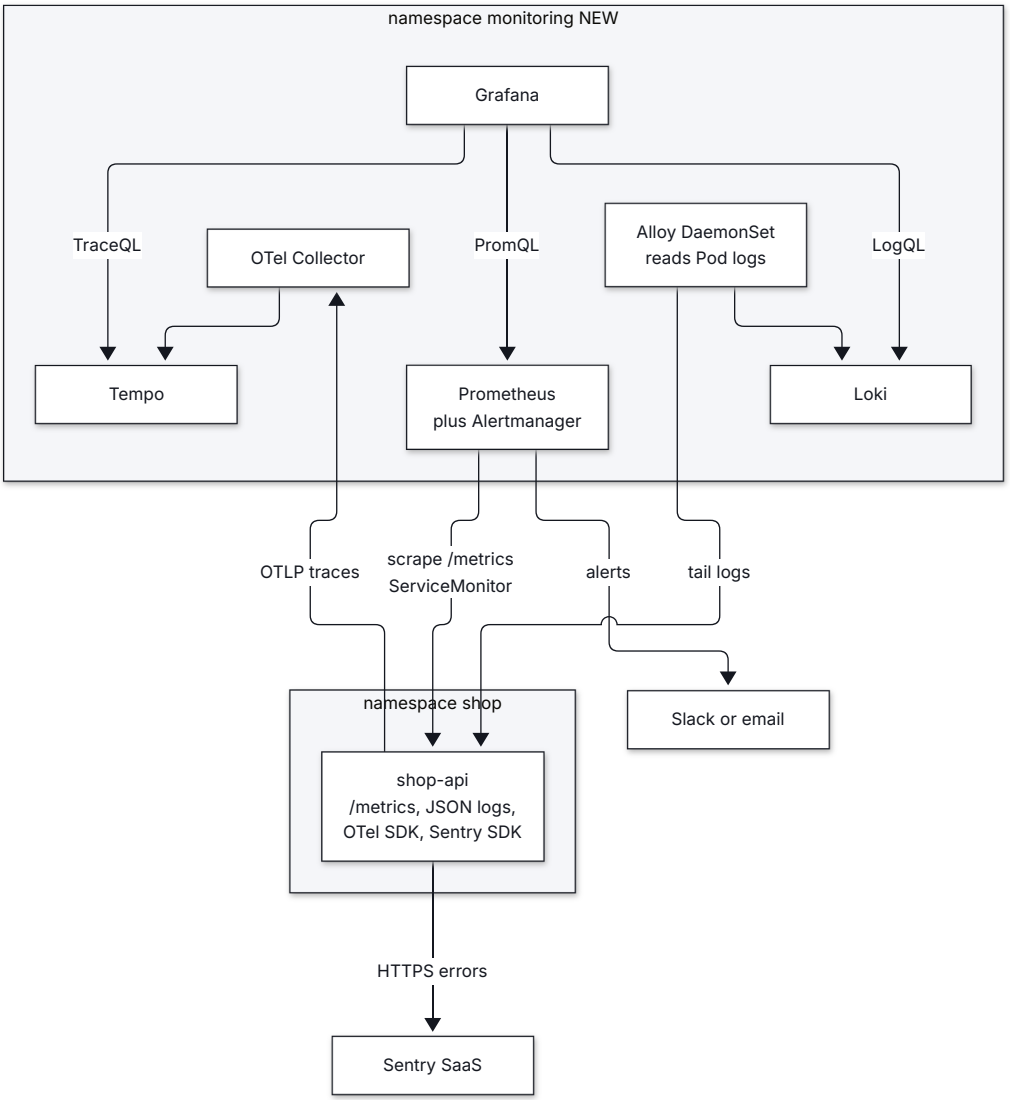


Figure 7-1

One request, three kinds of evidence, joined by a trace ID:

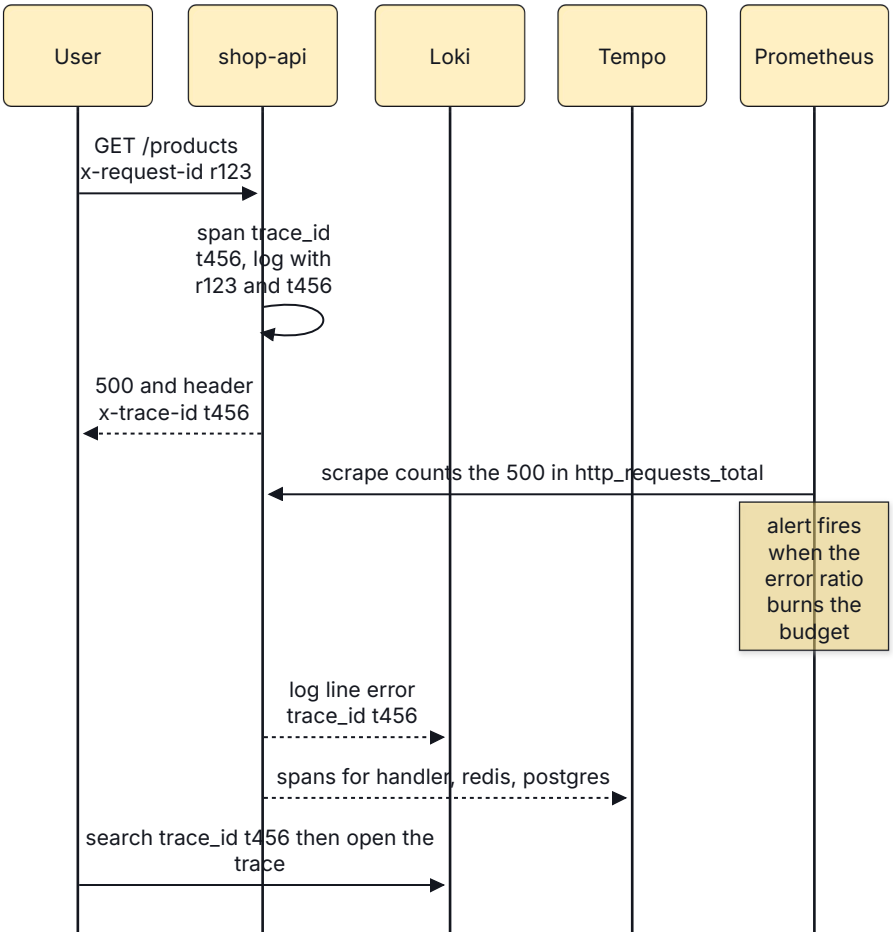


Figure 7-2

Question during an incident	Pillar	Where
Is it broken, how badly, since when?	metrics	Grafana dashboard, alerts
Which request and which error message?	logs	Loki, query by `trace_id`
Where did the time go across services?	traces	Tempo
Which code path threw, in which release?	errors	Sentry

3 Prerequisites

- The Project 6 cluster with Argo CD, and the `shop-gitops` repository. Resources: roughly 12 GB RAM free for a kind cluster with the full stack, or run on a small cloud cluster (cost applies; destroy it afterwards). Tools: `kubectl`, `helm`, `curl`, a free Sentry account (SaaS free tier) for the DSN.
- Level 10: metric types §10.4, structured logging §10.5, correlation IDs §10.7, distributed tracing §10.8, what to monitor §10.11.
- Reliability targets: SLIs §14.1 and SLOs §14.2 (read these; this project defines one SLO) and alert fatigue §14.8.
- Secrets: the Sentry DSN is not highly sensitive but should still travel as a sealed secret §11.4.

4 Build it step by step

Milestone 1 — Instrument the application (≈3 h)

```bash
npm install prom-client @sentry/node @opentelemetry/api @opentelemetry/auto-
  instrumentations-node @opentelemetry/sdk-node --no-audit --no-fund
```

Metrics. Use **route templates**, never raw URLs, as labels, or the label cardinality will explode (metric types §10.4):

```js
const client = require('prom-client');
client.collectDefaultMetrics();
const httpTotal = new client.Counter({ name: 'http_requests_total', help: 'Requests',
  labelNames: ['method', 'route', 'status'] });
const httpDuration = new client.Histogram({ name: 'http_request_duration_seconds', help:
  'Latency',
  labelNames: ['method', 'route'], buckets: [0.01, 0.05, 0.1, 0.3, 0.5, 1, 2.5] });
const cacheTotal = new client.Counter({ name: 'cache_requests_total', help: 'Cache
  lookups', labelNames: ['result'] });
// in the response 'finish' handler:  route = '/products' or '/products/:id' or
  'unmatched'
//   httpTotal.inc({ method, route, status: res.statusCode }); end({ method, route }) for
  the timer
// in cachedProducts():                cacheTotal.inc({ result: 'hit' | 'miss' })
// GET /metrics:                       res.end(await client.register.metrics())
```

Logs. One JSON object per line, with `request_id` (from `x-request-id` or generated) and `trace_id` (from the active OpenTelemetry span via `trace.getActiveSpan()?.spanContext().traceId`). Return the request ID in a response head-

er. Never log request bodies, tokens or passwords.

Traces. Start the SDK before the app loads, with environment variables only (no code change in handlers):

```bash
# in the container command (Deployment args)
node --require @opentelemetry/auto-instrumentations-node/register server.js
# environment (chart values)
OTEL_SERVICE_NAME=shop-api
OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector.monitoring.svc:4318
OTEL_TRACES_SAMPLER=parentbased_traceidratio
OTEL_TRACES_SAMPLER_ARG=0.2
```

The auto-instrumentation covers HTTP, `pg` and Redis clients, so a request produces spans for the handler and both data stores. Sampling 20 percent keeps cost down; raise it for debugging.

Errors. Initialise Sentry as the first thing the process does and attach the release:

```js
const Sentry = require('@sentry/node');
Sentry.init({ dsn: process.env.SENTRY_DSN, release: process.env.APP_VERSION, environment:
  process.env.APP_ENV,
  tracesSampleRate: 0 });            // traces go through OpenTelemetry; Sentry captures
  errors only
```

Set `APP_VERSION=sha-<commit>` in the chart from `image.tag`, so each Sentry issue names the release that introduced it (Sentry §10.10). Rebuild with the Project 3 pipeline and verify locally:

```bash
curl -s localhost:3000/metrics | grep -E
  '^(http_requests_total|http_request_duration_seconds_count|cache_requests_total)'
```

Milestone 2 — Install the backends through GitOps (≈3 h)

Add one Argo CD `Application` per component to `shop-gitops/apps/`, each pointing to a community Helm chart with **pinned chart versions** (find them with `helm search repo <chart> --versions` and pin in `targetRevision`; never track `latest`).

Component	Chart (repo)	Role
kube-prometheus-stack	`prometheus-community`	Prometheus Operator, Prometheus, Alertmanager, Grafana, node and kube-state exporters
loki	`grafana`	log storage (single binary mode, filesystem, lab only)

Component	Chart (repo)	Role
alloy	`grafana`	DaemonSet that reads Pod logs and pushes them to Loki
tempo	`grafana`	trace storage (single binary, lab only)
opentelemetry-collector	`open-telemetry`	gateway: receives OTLP, forwards to Tempo

Example for the metrics stack (the pattern is the same for the others; keep the values in your repo):

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: kube-prometheus-stack, namespace: argocd }
spec:
  project: default
  sources:
    - repoURL: https://prometheus-community.github.io/helm-charts
      chart: kube-prometheus-stack
      targetRevision: <PINNED_CHART_VERSION>
      helm: { valueFiles: [$values/monitoring/kube-prometheus-stack.values.yaml] }
    - repoURL: https://github.com/you/shop-gitops.git
      targetRevision: main
      ref: values
  destination: { server: https://kubernetes.default.svc, namespace: monitoring }
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions: [CreateNamespace=true, ServerSideApply=true]
```

In the stack values, enable discovery of ServiceMonitors from all namespaces (`serviceMonitorSelectorNilUsesHelmValues: false`), add Loki and Tempo as Grafana data sources, and give Grafana an admin password from a sealed secret. The OTel Collector configuration (receivers, a batch processor, one exporter):

```yaml
receivers:
  otlp:
    protocols:
      http: { endpoint: 0.0.0.0:4318 }
processors:
  batch: {}
exporters:
  otlp/tempo:
    endpoint: tempo.monitoring.svc:4317
    tls: { insecure: true }          # lab only, in-cluster
service:
  pipelines:
    traces: { receivers: [otlp], processors: [batch], exporters: [otlp/tempo] }
```

Milestone 3 — Scrape, dashboard and alert (≈3 h)

Add to the chart a `ServiceMonitor` (the API Service needs a named port `http`):

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata: { name: shop-api, namespace: shop, labels: { release: kube-prometheus-stack } }
spec:
  selector: { matchLabels: { app: shop-api } }
  endpoints: [{ port: http, path: /metrics, interval: 15s }]
```

Because Project 6 denies all traffic by default, scraping needs a new NetworkPolicy allowing the `monitoring` namespace to reach port 3000, and the API needs egress to the OTel Collector (4318) and to Sentry on 443 (an `ipBlock` for `0.0.0.0/0` that excludes the private ranges). Do this deliberately: a silent, empty target in Prometheus is the classic symptom of a missing allow.

Dashboard panels (PromQL). Put them in a ConfigMap labelled `grafana_dashboard: "1"` so the Grafana sidecar loads them from Git:

```promql
# traffic
sum(rate(http_requests_total{job="shop-api"}[5m])) by (route)
# errors (ratio)
sum(rate(http_requests_total{job="shop-api",status=~"5.."}[5m])) /
  sum(rate(http_requests_total{job="shop-api"}[5m]))
# latency p95
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket{job="shop-api"}
  [5m])) by (le, route))
# cache hit ratio
sum(rate(cache_requests_total{result="hit"}[5m])) / sum(rate(cache_requests_total[5m]))
# saturation
sum(rate(container_cpu_usage_seconds_total{namespace="shop"}[5m])) by (pod)
```

Alerts on the SLO (99.5 percent availability, so the error budget is 0.5 percent), using a fast multi-window burn rate that pages when the budget is being consumed about 14 times faster than allowed:

```yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata: { name: shop-api-slo, namespace: shop, labels: { release: kube-prometheus-stack
  } }
spec:
  groups:
    - name: shop-api-slo
      rules:
        - record: shop:http_error_ratio:rate5m
          expr: sum(rate(http_requests_total{job="shop-api",status=~"5.."}[5m])) /
  sum(rate(http_requests_total{job="shop-api"}[5m]))
        - record: shop:http_error_ratio:rate1h
          expr: sum(rate(http_requests_total{job="shop-api",status=~"5.."}[1h])) /
  sum(rate(http_requests_total{job="shop-api"}[1h]))
        - alert: ShopApiErrorBudgetBurnFast
          expr: shop:http_error_ratio:rate5m > (14.4 * 0.005) and
  shop:http_error_ratio:rate1h > (14.4 * 0.005)
          for: 2m
          labels: { severity: page }
          annotations:
            summary: "shop-api is burning its 30 day error budget about 14 times too fast"
            runbook: "docs/runbooks/shop-api-5xx.md"
```

Write the runbook the alert points to: first checks, which dashboards, which rollback.

Milestone 4 — Prove the pipeline from alert to root cause (≈2 h)

Create the failure, then investigate using only the tools you built:

```bash
kubectl -n shop scale statefulset/postgres --replicas=0           # break the dependency
for i in $(seq 1 300); do curl -sk -o /dev/null --resolve shop.example.com:8443:127.0.0.1
  https://shop.example.com:8443/products; done
kubectl -n shop scale statefulset/postgres --replicas=1           # restore
```

Follow this sequence and write it down: alert fires, dashboard shows 5xx and `readyz` failing, Loki query `{namespace="shop", app="shop-api"} | json | level="error"` shows `database unavailable` with a `trace_id`, Tempo trace shows the `pg` span failing, and Sentry holds the exception with the release. This is the output to keep for the Prove-it task.

5 Tasks

- [ ] **T1 — Instrument and verify locally** *(≈2 h, intermediate)*
  - **Goal:** metrics, logs and traces come out of the API before the cluster is involved.
  - **Steps:**
    1. Add the counters, histogram and `/metrics` endpoint; run the stack with Compose from Project 2.
    2. Make 20 requests and read `/metrics` for `http_requests_total` and `http_request_duration_seconds_bucket`.
    3. Confirm one log line per request with `request_id` and `trace_id`.
  - **Done when:** `/metrics` shows route-labelled counters (no raw URLs with IDs) and a log line contains both IDs.

- [ ] **T2 — Install the monitoring stack with Argo CD** *(≈2 h, intermediate)*
  - **Goal:** the whole stack is declared in Git.
  - **Steps:**
    1. Create the Applications for the five components with pinned chart versions.
    2. Wait for `Synced` and `Healthy`; port-forward Grafana and Prometheus.
    3. In Prometheus, open Status, Targets.
  - **Done when:** `kubectl -n monitoring get pods` shows everything running, and the Kubernetes targets are `UP`.

- [ ] **T3 — Get the API scraped** *(≈1 h, intermediate)*
  - **Goal:** `shop-api` appears as an `UP` target.

- **Steps:**
  1. Add the ServiceMonitor and the NetworkPolicy that lets the `monitoring` namespace scrape port 3000.
  2. First apply only the ServiceMonitor and look at the target error; then add the policy.
  3. Run `up{job="shop-api"}` in Prometheus.
- **Done when:** `up{job="shop-api"}` returns 1 for each replica, and you saw the target fail before the policy.

☐ **T4 — Build the golden-signals dashboard** *(≈1.5 h, intermediate)*

- **Goal:** traffic, errors, latency and saturation for the API on one screen, as code.
- **Steps:**
  1. Create the panels from the PromQL above in Grafana.
  2. Export the dashboard JSON into a labelled ConfigMap in the GitOps repo.
  3. Delete the dashboard in the UI and let Argo CD and the sidecar restore it.
- **Done when:** the dashboard comes back from Git after deletion, and each panel moves when you run a load loop.

☐ **T5 — Centralise and correlate logs** *(≈1.5 h, intermediate)*

- **Goal:** find any request's logs from its trace ID.
- **Steps:**
  1. Install Loki and Alloy; confirm `{namespace="shop"}` returns lines in Grafana Explore.
  2. Add a derived field on the Loki data source linking `trace_id` to Tempo.
  3. Pick a slow request and jump from its log line to its trace.
- **Done when:** clicking a `trace_id` in a log line opens the trace in Tempo showing the handler, Redis and PostgreSQL spans.

☐ **T6 — Write and test an SLO alert** *(≈1.5 h, advanced)*

- **Goal:** page on user impact, not on a hunch.
- **Steps:**
  1. Write the SLO sentence and the `PrometheusRule`; load it with Argo CD.
  2. Inject errors by scaling PostgreSQL to 0 (or add a `?fail=1` test route).
  3. Watch the alert go `Pending` then `Firing` in Prometheus and Alertmanager, then restore the service and watch it resolve.

- **Done when:** the alert fires within a few minutes of the outage and clears after recovery, and the rule is in Git.

- [ ] **T7 — Capture errors with release context** *(≈1 h, intermediate)*
  - **Goal:** an exception with a stack trace and the commit that shipped it.
  - **Steps:**
    1. Create a free Sentry project, seal the DSN into the cluster, and set `APP_VERSION`.
    2. Add a `/boom` route that throws (behind a feature flag; remove it afterwards).
    3. Trigger it and open the issue in Sentry.
  - **Done when:** the Sentry issue shows the release `sha-...`, the environment, and the stack trace; no PII appears in the event.

- [ ] **T8 — Prove it: diagnose an unknown fault from telemetry alone** *(≈1.5 h, advanced)*
  - **Goal:** show the telemetry is good enough to debug blind.
  - **Steps:**
    1. Ask a friend (or your future self, using a random pick from the drills in section 7) to break something without telling you what.
    2. Using only Grafana (metrics, Loki, Tempo) and Sentry, find the failing component, the first bad deploy or change time, and the user-visible effect.
    3. Write a five-line incident summary: impact, detection, cause, fix, follow-up.
  - **Done when:** your diagnosis matches the injected fault, and you can say which signal found it first and what additional instrumentation would have found it sooner.

6 Definition of done

- [ ] Prometheus, Grafana, Loki, Alloy, Tempo and the OTel Collector are installed by Argo CD from pinned chart versions.
- [ ] `up{job="shop-api"} == 1` for every replica; the dashboard shows the four golden signals and is restored from Git if deleted.
- [ ] A log line, a trace and a metric can be tied together for one request by `trace_id`.
- [ ] An SLO is written down, and an alert on its burn rate fires in a drill and links to a runbook.
- [ ] Sentry receives an unhandled exception tagged with the release; no secrets or PII are logged or sent.
- [ ] NetworkPolicies still default-deny, with explicit allows for scraping, tracing and Sentry.

- [ ] Label cardinality is bounded (no user IDs, raw paths or request IDs as metric labels).

7 Break it on purpose

1. **Cardinality explosion.** Change the metrics middleware to use `req.url` as the route label and send 5,000 requests to distinct URLs. Watch Prometheus memory and `prometheus_tsdb_head_series` climb, then fix it with route templates. *(Lesson: labels multiply series.)*
2. **Silent gap.** Remove the scrape allow from the NetworkPolicy. The dashboard shows "No data", the alert does not fire, and `up` is 0. Add an alert on `absent(up{job="shop-api"})` so a missing target is itself an alert. *(Lesson: monitor the monitoring.)*
3. **Alert on the cause instead of the symptom.** Add an alert on CPU above 70 percent. Generate load and see it fire while users are fine; then break the database while CPU is low and see it stay silent. Compare with the burn-rate alert. *(Lesson: alert fatigue comes from cause-based paging.)*
4. **Lose the trail.** Stop propagating the trace context (disable the Redis or `pg` instrumentation, or send a request with a malformed `traceparent`) and observe broken or orphaned traces. Restore it. *(Lesson: tracing works only if every hop propagates context.)*

8 How this evolves into the next project

You can now see failures, but seeing is not the same as surviving. The database is still a single Pod on one volume, backups do not exist beyond the Project 1 `pg_dump` habit, and nobody has measured how long recovery takes. Project 8 — Disaster Recovery uses the telemetry you just built to run controlled failure drills (a destroyed node, a deleted namespace, a corrupted table, a lost region's worth of infrastructure), measure real recovery time and data loss, and turn the results into RPO and RTO numbers, runbooks and automated restores.

THE SUMMIT PROJECTS

P8 Disaster Recovery

Project 8 of 9 | Uses levels: Levels 13-15 | ~14 hours

1 Goal and outcome

Prove that the platform you built in Projects 4 to 7 can **survive and recover from real failures**, and replace "we have backups" with measured numbers. You break the system on purpose (a database Pod, a bad `DELETE`, a deleted namespace, a lost cluster, a destroyed cloud environment), recover it with written runbooks, and record the **actual RPO** (how much data you lost) and **actual RTO** (how long it took) for each scenario.

The concepts come from Level 13 Capstone 13 (replication, WAL archiving and point-in-time recovery), Level 14 Capstone 14 (incidents, postmortems, SLOs) and Level 15 §15.1 (backups, restores, failover, drills). This project is the integration: one real stack, scripted failure drills, and measured results. A backup that has never been restored is a hope, not a backup.

You finish with:

- a PostgreSQL cluster managed by the CloudNativePG operator: one primary plus one replica, continuous WAL archiving and scheduled base backups to an **object store outside the cluster**;
- a **data-class table** with RPO and RTO targets, chosen before you test;
- four rehearsed drills: instance failure (automatic failover), human error (point-in-time recovery), cluster loss (rebuild from Git plus restore), and a cloud-environment rebuild with Terraform and a database snapshot;
- a `heartbeat` writer and a `drill.sh` timer that turn each drill into numbers;
- runbooks and a blameless postmortem for each drill, and one "game day" where someone else injects the failure.

2 Architecture

What changed from Project 7: the single PostgreSQL StatefulSet is replaced by a replicated, operator-managed cluster (NEW), backups leave the cluster (NEW), and the observability

stack from Project 7 becomes the way you **detect** each failure and **verify** each recovery.

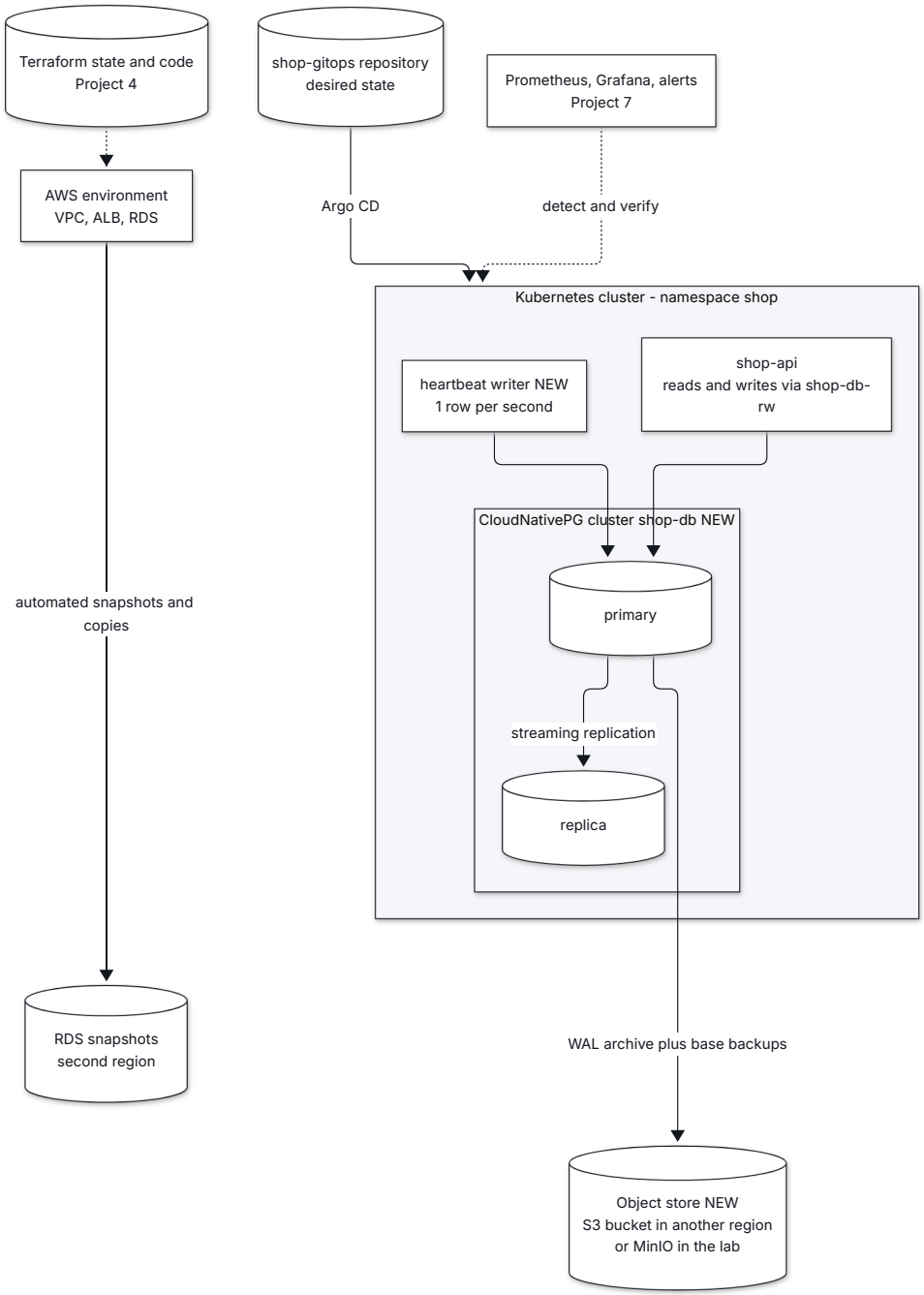


Figure 8-1

The point-in-time recovery timeline for the "bad DELETE" drill:

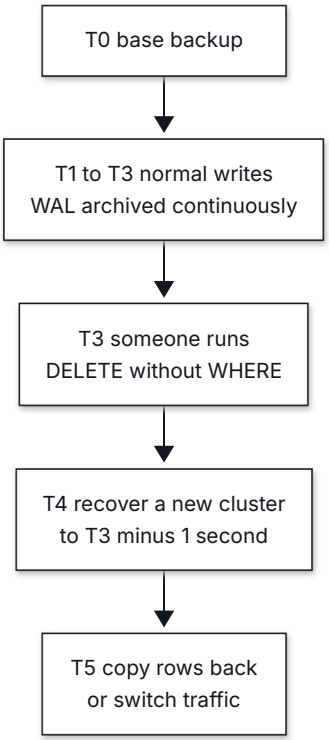


Figure 8-2

Failure	Blast radius	Recovery path	You measure
Primary Pod or node dies	one instance	operator promotes the replica	failover time, lost transactions (RPO near 0)
Bad `DELETE` or deploy corrupts data	all data, replicated instantly	PITR from base backup plus WAL	data-loss window, restore time
Cluster or namespace deleted	everything in Kubernetes	rebuild cluster, Argo CD syncs Git, restore database	end-to-end RTO
AWS environment destroyed	everything on AWS	`terraform apply`, restore database from snapshot	RTO, snapshot age (RPO)

3 Prerequisites

- The Project 7 cluster (or just Project 6 if you skip telemetry), `kubectl`, `helm`, `git`, and for the cloud drill the Project 4 Terraform code and an AWS sandbox account. Cost: the Kubernetes part is free on kind; an S3 bucket in a second region costs cents; the AWS drill

costs the same as Project 4 for the hours it runs. **Destroy everything at the end.**

- Databases: PostgreSQL replication §13.4, backups and PITR §13.5.
- Disaster recovery: backups §15.1, replication §15.2, snapshots §15.3, restore procedures §15.4, failover §15.5, RPO §15.6, RTO §15.7, offsite backups §15.8, testing backup restoration §15.9, failure scenario drills §15.10.
- SRE practice: incident management §14.5 and postmortems §14.7. Operators: operators §16.4 and stateful workloads §16.8.

4 Build it step by step

Milestone 1 — Decide objectives before you test (≈1 h)

Write `docs/dr/objectives.md`. The numbers are yours to choose; the discipline is choosing them first, because otherwise any result looks acceptable.

Data class	Example	RPO target	RTO target	Protection
Orders and customers	PostgreSQL	5 minutes	30 minutes	replica, WAL archiving, offsite backup
Product catalog	PostgreSQL	1 hour	1 hour	same database
Cache	Redis	not applicable	5 minutes	rebuilt on demand, never backed up
Infrastructure and config	Git, Terraform state	0 (every commit)	1 hour	Git remote plus mirrored copy, versioned state bucket

Milestone 2 — A replicated, backed-up PostgreSQL (≈3 h)

Install the operator (operators §16.4) and create an object store the cluster does not live in:

```bash
helm repo add cnpg https://cloudnative-pg.github.io/charts && helm repo update
helm upgrade --install cnpg cnpg/cloudnative-pg -n cnpg-system --create-namespace
kubectl -n cnpg-system rollout status deploy -l app.kubernetes.io/name=cloudnative-pg
```

Backup target: for a real exercise use an S3 bucket in a different region from your cluster, with an IAM user limited to that bucket (versioning on, no public access). In a pure lab use MinIO with a Deployment and PVC in a separate namespace `backup` (use the current MinIO image tag from its documentation) and a bucket `shop-backups`. Store the access keys as a sealed secret `backup-creds` with keys `ACCESS_KEY_ID` and `ACCESS_SECRET_KEY`.

`shop-db.yaml` (commit it to `shop-gitops`; the backup section of CloudNativePG is moving from the built-in `barmanObjectStore` to the Barman Cloud plugin, so check the current CloudNativePG documentation for the exact fields in your operator version):

```yaml
apiVersion: postgresql.cnpg.io/v1
kind: Cluster
metadata: { name: shop-db, namespace: shop }
spec:
  instances: 2
  storage: { size: 2Gi }
  bootstrap:
    initdb: { database: shop, owner: shop }
  backup:
    retentionPolicy: "7d"
    barmanObjectStore:
      destinationPath: s3://shop-backups/shop-db
      endpointURL: http://minio.backup.svc:9000        # omit for AWS S3
      s3Credentials:
        accessKeyId:     { name: backup-creds, key: ACCESS_KEY_ID }
        secretAccessKey: { name: backup-creds, key: ACCESS_SECRET_KEY }
      wal: { compression: gzip }
---
apiVersion: postgresql.cnpg.io/v1
kind: ScheduledBackup
metadata: { name: shop-db-nightly, namespace: shop }
spec:
  schedule: "0 0 2 * * *"            # six fields: seconds first
  cluster: { name: shop-db }
  backupOwnerReference: self
```

The operator creates the Services `shop-db-rw` (primary) and `shop-db-ro` (replicas) and a generated application Secret `shop-db-app` containing a ready-to-use `uri`. Point the API at it by changing `DATABASE_URL` to read from that secret key (`secretKeyRef: {name: shop-db-app, key: uri}`) and update the NetworkPolicy for the new Pod labels (`cnpg.io/cluster: shop-db`). Seed the schema with the Project 5 Job, then take an on-demand backup and check it:

```bash
kubectl -n shop get cluster shop-db                       # Cluster in healthy state, 2
  ready instances
kubectl cnpg status shop-db -n shop 2>/dev/null || kubectl -n shop get pods -l
  cnpg.io/cluster=shop-db
cat <<'EOF' | kubectl apply -f -
apiVersion: postgresql.cnpg.io/v1
kind: Backup
metadata: { name: first-backup, namespace: shop }
spec: { cluster: { name: shop-db } }
EOF
kubectl -n shop get backup first-backup -w                # phase: completed
```

Milestone 3 — The measuring tools (≈1 h)

A heartbeat writer that records one row per second gives you an objective way to measure data loss. Run it as a Pod using the `postgres` image against `shop-db-rw`:

```bash
kubectl -n shop exec -it shop-db-1 -- psql -U postgres -d shop -c \
  "CREATE TABLE IF NOT EXISTS heartbeat (n bigserial PRIMARY KEY, ts timestamptz NOT NULL
  DEFAULT now());
   GRANT INSERT, SELECT ON heartbeat TO shop; GRANT USAGE ON SEQUENCE heartbeat_n_seq TO
  shop;"
kubectl -n shop run heartbeat --image=postgres:17-alpine --restart=Never --env
  "PGHOST=shop-db-rw" \
  --env "PGUSER=shop" --env "PGDATABASE=shop" --env "PGPASSWORD=$(kubectl -n shop get
  secret shop-db-app -o jsonpath='{.data.password}' | base64 -d)" \
  --command -- sh -c 'while true; do psql -qc "INSERT INTO heartbeat DEFAULT VALUES" ||
  true; sleep 1; done'
```

`drill.sh` stamps the stages of a drill so RTO is a subtraction, not a memory:

```bash
#!/usr/bin/env bash
# usage: ./drill.sh <name>   then press Enter at each stage
name="${1:?drill name}"; log="docs/dr/drills/$(date -u +%F)-$name.log"; mkdir -p
  "$(dirname "$log")"
stamp() { printf '%s  %s\n' "$(date -u +%FT%TZ)" "$1" | tee -a "$log"; }
stamp "START $name"
read -rp "Press Enter when the failure is injected... " ; stamp "FAILURE_INJECTED"
read -rp "Press Enter when you (or an alert) DETECTED it... " ; stamp "DETECTED"
read -rp "Press Enter when service is RESTORED for users... " ; stamp "SERVICE_RESTORED"
read -rp "Press Enter when data is VERIFIED... " ; stamp "DATA_VERIFIED"
echo "Record RTO = SERVICE_RESTORED - FAILURE_INJECTED; RPO = failure time - newest
  heartbeat row" | tee -a "$log"
```

Milestone 4 — Drill A: lose the primary (≈1 h)

```bash
./drill.sh primary-loss
kubectl -n shop delete pod -l cnpg.io/instanceRole=primary --wait=false     # inject
kubectl -n shop get cluster shop-db -w                                      # watch
  promotion
curl -sk --resolve shop.example.com:8443:127.0.0.1 https://shop.example.com:8443/readyz
kubectl -n shop exec "$(kubectl -n shop get pod -l cnpg.io/instanceRole=primary -o name |
  head -1)" -- psql -U postgres -d shop -tAc "SELECT max(ts), count(*) FROM heartbeat"
```

Expected outcome: the operator promotes the replica within seconds to a minute (your measurement decides), the API's `/readyz` flaps and recovers, and the heartbeat shows a gap of seconds, not minutes.

Milestone 5 — Drill B: human error, then point-in-time recovery (≈2 h)

```bash
./drill.sh bad-delete
date -u +%FT%TZ                                                  # note GOOD_TIME, just
  before the mistake
kubectl -n shop exec -it shop-db-1 -- psql -U postgres -d shop -c "DELETE FROM products;"
  # the mistake
```

Replication copies the mistake to the replica instantly, which is why replicas are **not** backups. Recover into a **new** cluster at a time just before the delete, leaving the damaged one untouched until you have verified:

```yaml
apiVersion: postgresql.cnpg.io/v1
```

```
kind: Cluster
metadata: { name: shop-db-restore, namespace: shop }
spec:
  instances: 1
  storage: { size: 2Gi }
  bootstrap:
    recovery:
      source: shop-db
      database: shop
      owner: shop
      recoveryTarget: { targetTime: "2026-10-09 12:34:56+00" }      # GOOD_TIME, with a
  small safety margin
  externalClusters:
    - name: shop-db
      barmanObjectStore:
        destinationPath: s3://shop-backups/shop-db
        endpointURL: http://minio.backup.svc:9000
        s3Credentials:
          accessKeyId:     { name: backup-creds, key: ACCESS_KEY_ID }
          secretAccessKey: { name: backup-creds, key: ACCESS_SECRET_KEY }
```

BASH

```
kubectl -n shop wait cluster/shop-db-restore --for=condition=Ready --timeout=15m
kubectl -n shop exec shop-db-restore-1 -- psql -U postgres -d shop -tAc "SELECT count(*)
  FROM products"   # 3, not 0
```

Then either dump the missing rows from the restored cluster and insert them into production (`pg_dump --data-only --table=products ... | psql`), or repoint the application at `shop-db-restore-rw` through a Git change. Prefer the first for a small mistake and the second for broad corruption.

Milestone 6 — Drill C: lose the whole cluster (≈2 h)

The claim of GitOps is that the cluster is cattle. Test it:

BASH

```
./drill.sh cluster-loss
kind delete cluster --name shop                                    # inject
# recover: new cluster, then bootstrap from Git
kind create cluster --name shop --config kind-shop.yaml
kubectl create namespace argocd && kubectl apply -n argocd --server-side --force-conflicts
  -f <argocd install manifest>
kubectl apply -f shop-gitops/apps/root.yaml                        # the app-of-apps from
  Git
```

You will find three things that Git alone does not restore: the **Sealed Secrets private key** (restore it from your backup first, or every SealedSecret in Git is garbage), the **database data** (set the new `shop-db` Cluster to `bootstrap.recovery` from the object store instead of `initdb`), and any manual step you forgot to write down. Each one goes into the runbook.

Milestone 7 — Drill D: rebuild the AWS environment (≈3 h, optional but recommended)

With the Project 4 stack running and RDS snapshots enabled (and a copy in a second region):

```bash
./drill.sh aws-rebuild
cd envs/dev && terraform destroy -var image_tag=sha-<sha>       # inject: environment gone
terraform apply -var image_tag=sha-<sha>                         # rebuild: network, ALB,
  instances, empty RDS
# Replace the empty RDS by restoring the latest snapshot: set snapshot_identifier on
  aws_db_instance, plan, apply
aws rds describe-db-snapshots --query 'DBSnapshots[].
  [DBSnapshotIdentifier,SnapshotCreateTime]' --output table
```

Pay attention to: how old the newest snapshot is (that is your RPO), whether `deletion_protection` and a final snapshot would have prevented the loss, and which secrets (the managed master password) change after a restore. Run Project 2's `smoke.sh` against the new ALB. Destroy everything afterwards.

Milestone 8 — Runbooks, postmortems and a game day (≈2 h)

For each drill write `docs/dr/runbooks/<scenario>.md` (detection, decision, exact commands, verification, rollback) and a short blameless postmortem following postmortems §14.7: impact, timeline from `drill.sh`, what worked, what surprised you, action items with owners. Then ask a colleague or friend to pick a scenario and inject it without telling you, while you follow only the runbooks.

5 Tasks

- [ ] **T1 — Write RPO and RTO targets first** *(≈45 min, beginner)*
  - **Goal:** objectives defined before testing.
  - **Steps:**
    1. List every data store and configuration source in the platform.
    2. Assign an RPO and RTO to each, and the protection that should meet them.
    3. Commit `docs/dr/objectives.md`.

- **Done when:** the file exists in Git before the first drill, and every row has numbers and a mechanism.

- [ ] **T2 — Stand up the replicated, backed-up database** *(≈2 h, advanced)*
  - **Goal:** two instances, WAL archiving and a completed base backup in the object store.
  - **Steps:**
    1. Install the operator, create the object store and credentials, apply `shop-db.yaml`.
    2. Take an on-demand `Backup` and wait for `completed`.
    3. List the bucket and find `base` and `wals` prefixes.
  - **Done when:** `kubectl get backup` shows `completed`, the bucket contains both base backup and WAL files, and the API serves `/products` from the new cluster.

- [ ] **T3 — Fail over the primary and measure** *(≈1 h, intermediate)*
  - **Goal:** a measured failover time and data-loss figure.
  - **Steps:**
    1. Start the heartbeat writer and `./drill.sh primary-loss`.
    2. Delete the primary Pod and note detection and restoration times.
    3. Query the heartbeat for gaps (`SELECT max(gap) FROM (SELECT ts - lag(ts) OVER (ORDER BY ts) AS gap FROM heartbeat) g`).
  - **Done when:** your log has RTO and a measured gap, and you compare both with your Milestone 1 targets.

- [ ] **T4 — Recover from a destructive `DELETE` with PITR** *(≈2 h, advanced)*
  - **Goal:** a restored copy of the table as of before the mistake.
  - **Steps:**
    1. Run the `bad-delete` drill and record `GOOD_TIME`.
    2. Create `shop-db-restore` with `recoveryTarget.targetTime`.
    3. Copy the rows back into the live cluster and re-run the smoke test.
  - **Done when:** `/products` returns the original rows, and your log gives the restore duration and the number of seconds of data lost.

- [ ] **T5 — Rebuild the cluster from Git and a backup** *(≈2.5 h, advanced)*
  - **Goal:** the cluster is disposable; the data is not.
  - **Steps:**
    1. Back up the Sealed Secrets key and confirm it is **not** in the same cluster you will destroy.

2. Delete the cluster, recreate it, bootstrap Argo CD, and restore the database from the object store.
3. Record each manual step you needed and fold it into the runbook.

- **Done when:** `smoke.sh` passes against the rebuilt cluster, the database contains data from before the deletion, and the runbook now lists every manual step.

☐ **T6 — Restore an AWS environment from a snapshot** *(≈3 h, advanced; costs money)*

- **Goal:** cloud rebuild with a measured RPO from snapshot age.
- **Steps:**
  1. Enable automated RDS backups and copy a snapshot to a second region.
  2. Destroy and recreate the Project 4 environment; restore the database from the snapshot via Terraform.
  3. Run the smoke test and destroy everything again.
- **Done when:** the restored database contains the pre-destruction rows, the log has RTO and snapshot age, and the CLI shows no resources remaining afterwards.

☐ **T7 — Write runbooks and postmortems** *(≈2 h, intermediate)*

- **Goal:** another person could recover without you.
- **Steps:**
  1. Write one runbook per scenario with exact commands and verification.
  2. Write a blameless postmortem for the worst drill.
  3. Give the runbooks to a friend and watch (silently) where they get stuck.
- **Done when:** the friend completed one recovery from the runbook alone, and every point where they hesitated has been fixed in the text.

☐ **T8 — Prove it: run a game day** *(≈2 h, advanced)*

- **Goal:** recovery works under mild pressure with an unknown failure.
- **Steps:**
  1. Have someone inject one of the four failures without telling you which.
  2. Detect it with the Project 7 alerts and dashboards, then recover using only the runbooks.
  3. Fill in the final table of targets versus measured RPO and RTO, and state which targets you missed and what change would meet them.
- **Done when:** service and data are restored, the table is committed, and each missed target has an action item with a cost estimate.

6 Definition of done

- [ ] Objectives (RPO and RTO per data class) are written and committed before the drills.
- [ ] PostgreSQL has a replica, continuous WAL archiving and scheduled base backups to a store **outside** the cluster; retention is configured.
- [ ] A restore from backup has been performed within the last week and timed; PITR to a chosen timestamp succeeded.
- [ ] Losing the primary causes automatic failover; the measured downtime and data loss are recorded.
- [ ] The whole cluster can be rebuilt from Git plus the object store plus the backed-up sealing key.
- [ ] Every drill has a log from `drill.sh`, a runbook and a postmortem; the targets-versus-measured table exists.
- [ ] Backup credentials are least privilege (write-only for the writer where possible), and the offsite copy cannot be deleted by the same credentials that produce it.

7 Break it on purpose

1. **Corrupt the backups, not the database.** Delete the newest WAL segments from the object store and try a PITR to a time after them. See what the restore can and cannot reach. Fix your monitoring so a gap in the archive alerts. *(Lesson: backups need monitoring too.)*
2. **Lose the key.** Restore the cluster without the Sealed Secrets key and watch every SealedSecret fail. Restore the key and re-sync. *(Lesson: recovery dependencies need their own backup.)*
3. **Fill the archive volume.** Make the object store unreachable for a while (scale MinIO to 0). WAL accumulates on the primary's disk. Watch the volume fill, learn the alert you need, and recover before the database stops. *(Lesson: a failed backup target can become an outage.)*
4. **Restore into production by mistake.** Point a restore at the live cluster name in a throw-away lab and see why you always restore to a **new** name and switch over after verification. *(Lesson: restores need a blast-radius plan too.)*

8 How this evolves into the next project

You now have each piece: infrastructure as code, containers, Kubernetes, GitOps, observability and recovery. They were built one at a time, and each had its own repository, its own decisions and its own trade-offs. Project 9 — Senior Architecture asks the senior-level ques-

tion: put it together as one production-style platform on AWS, justify every decision in writing, review it against a production-readiness checklist (security, observability, scale, availability, deployability, recoverability, cost), and be able to defend the trade-offs to someone who disagrees.

THE SUMMIT PROJECTS

P9 Senior Architecture

Project 9 of 9 | Uses levels: Levels 6-20 | ~40 hours

1 Goal and outcome

Build one complete, production-style platform for `shop-api` on **AWS** with **Terraform**, **Docker**, **Kubernetes (EKS)**, **Helm**, **GitOps** and **observability**, and then review it the way a senior engineer would: against seven qualities (secure, observable, scalable, highly available, automatically deployable, recoverable, cost-efficient), with every significant decision written down and defended.

This is not a new tool. Everything here you have built before in a smaller form: the network and database from Project 4, the cluster and delivery from Project 5 and Project 6, the telemetry from Project 7, the recovery drills from Project 8. The senior-level skill is integrating them under constraints (money, time, risk) and explaining the trade-offs. Read the Senior-Level Mindset guide alongside it.

You finish with:

- a Git repository set (`shop-api`, `shop-infra`, `shop-gitops`) from which an empty AWS account becomes the platform with a documented, scripted bootstrap;
- EKS across three Availability Zones, managed node groups (with a spot group for stateless work), RDS PostgreSQL Multi-AZ, ElastiCache, ECR, an ALB, and encryption with KMS;
- no long-lived credentials anywhere: GitHub OIDC for CI, EKS Pod Identity for workloads, External Secrets reading AWS Secrets Manager;
- Argo CD managing platform add-ons and the application; policy-as-code (Kyverno or ValidatingAdmissionPolicy) blocking unsafe workloads;
- SLOs, burn-rate alerts, dashboards, a load test and a chaos test with recorded results;
- a **production-readiness review**, a cost model with a measured bill, and **five architecture decision records**;
- a clean teardown, verified by CLI.

2 Architecture

What changed from Project 8: the Kubernetes cluster is no longer a kind lab; it runs on EKS inside the Terraform-managed VPC (the Project 4 network grows a third AZ and EKS-aware subnet tags), the database is managed by AWS in Multi-AZ mode with encryption, identities are federated instead of stored, and policy and cost controls are added (all NEW unless noted).

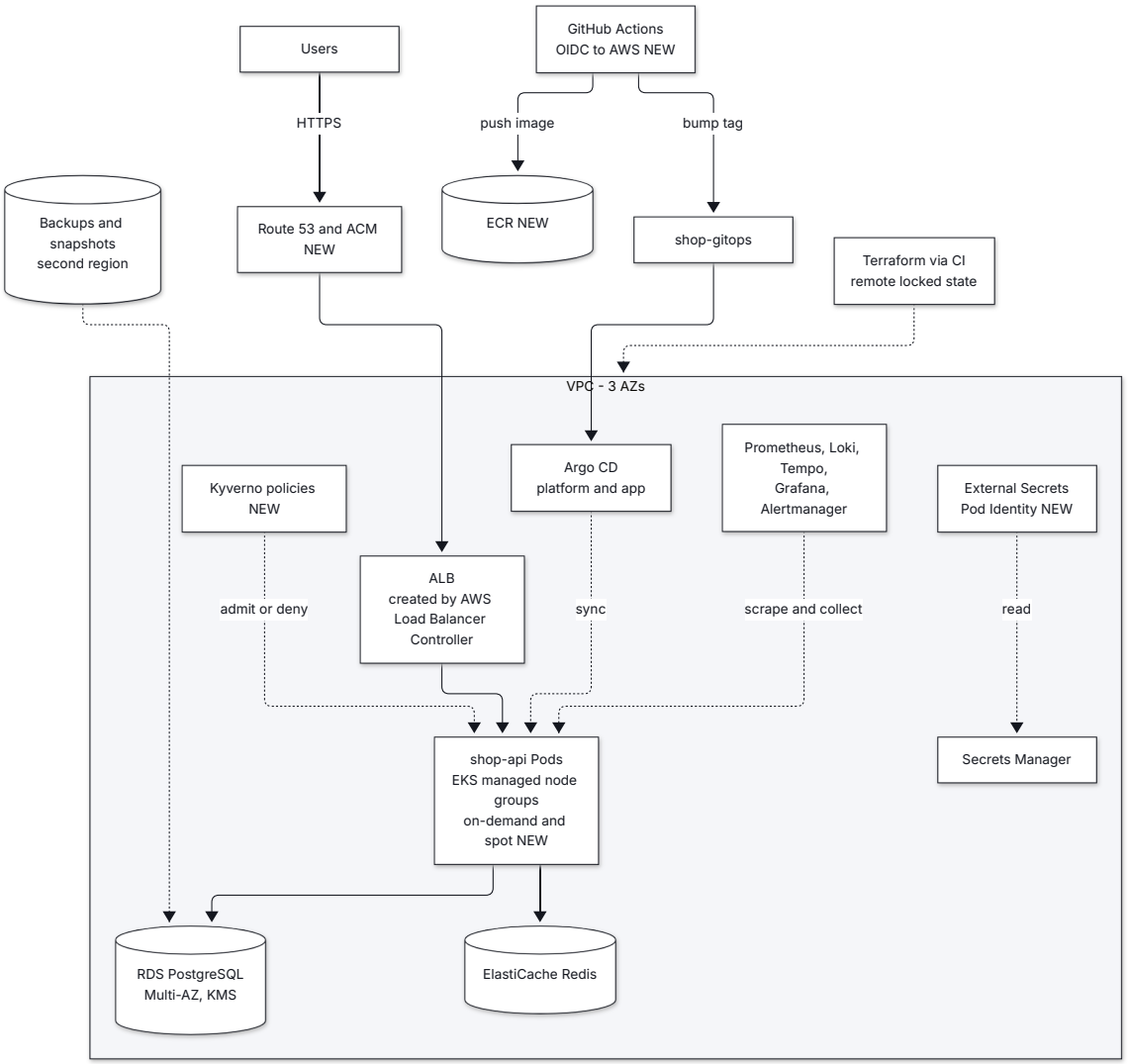


Figure 9-1

The Terraform and GitOps layers, applied in order, so that a failure in one does not take the others with it:

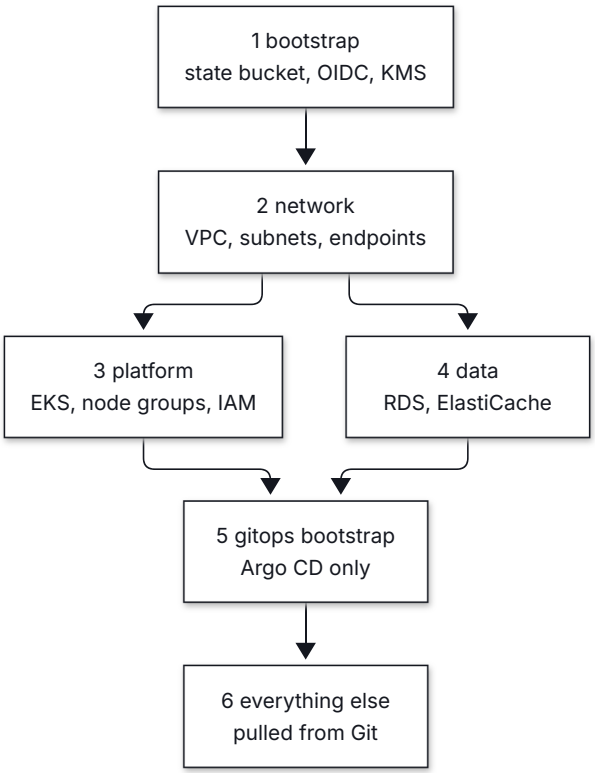


Figure 9-2

Project	What it added	Where it lives now
1 Basic server	the application contract, hardening habits	Dockerfile, health endpoints, SSH-less access (Session Manager)
2 Docker	image, Compose, non-root	the production image and local dev loop
3 CI/CD	tests, scan, registry, rollback by tag	GitHub Actions to ECR, tags by commit
4 Terraform	VPC, ALB, RDS, state	`shop-infra` layers
5 Kubernetes	Deployments, probes, Ingress	Helm chart
6 Production Kubernetes	TLS, HPA, RBAC, policies, Argo CD	platform add-ons in Git
7 Observability	metrics, logs, traces, SLO alerts	`monitoring` namespace
8 Disaster recovery	measured RPO and RTO	runbooks and drills, now on RDS and EKS

3 Prerequisites

- Projects 1 to 8 done, or at least read; a dedicated **AWS sandbox account** with MFA, a **budget and billing alarm**, and no other workloads in it. A domain you own is needed for real HTTPS with ACM and Route 53; without one, use the ALB's DNS name over HTTP for the lab and record the gap as a known risk.
- **Cost warning, read before you start:** EKS control plane, NAT gateways, an ALB, Multi-AZ RDS, ElastiCache and several nodes all bill per hour. Plan on several to a few tens of US dollars per day; compute the number with the AWS Pricing Calculator before you apply, write it into your cost model (milestone 8), and **tear down every session** you can.
- Cloud: VPC §6.4, IAM §6.7, RDS §6.10, ALB §6.14, KMS §6.15, Secrets Manager §6.16, CloudTrail §6.17, CloudWatch §6.18.
- IaC and delivery: Terraform modules §7.3, Terraform CI/CD §7.7, Helm packaging §9.6, GitOps CI flow §17.7, canary deployments §5.12.
- Security and reliability: least-privilege IAM §11.2, secrets management §11.4, Pod security §16.3, admission control §16.6, SLOs §14.2, error budgets §14.4, capacity planning §14.9.
- Platform and architecture: golden paths §18.4, policy as code §18.7, reference architecture walkthrough §19.10, architecture decision records §19.11, FinOps §20.1, Kubernetes infrastructure costs §20.9.

4 Build it step by step

Each milestone ends with evidence you will cite in the final review. Keep a `docs/evidence/` folder (command output, screenshots, links).

Milestone 1 — Requirements, SLOs and decisions before code (≈3 h)

Write `docs/architecture.md` with the questions a senior engineer asks first: who are the users, what is the peak load, what does an hour of downtime cost, what data is sensitive, what is the team's size and on-call capacity? Then write the targets, for example:

Quality	Target for `shop-api`
Availability	99.9 percent of `/products` requests succeed over 30 days (error budget about 43 minutes)
Latency	95 percent of requests under 300 ms
Recoverability	RPO 5 minutes, RTO 1 hour for the database; RTO 4 hours for the whole region's loss (documented, not automated)
Security	no public data stores, no static credentials, all data encrypted at rest and in transit
Cost	stays below an agreed monthly figure; every resource tagged `project`, `env`, `owner`

Create `docs/adr/` and write the first five records now (template in section 5, T1): ADR-1 EKS versus ECS or plain EC2 (be honest about operational complexity and team size), ADR-2 managed RDS versus a database on Kubernetes, ADR-3 GitOps pull model versus CI push, ADR-4 managed node groups versus Karpenter, ADR-5 single region with cross-region backups versus active-active. See operational complexity §19.6.

Milestone 2 — Bootstrap and network (≈4 h)

Layer 1 (`bootstrap/`): a versioned, encrypted S3 state bucket with native locking, the GitHub OIDC provider, separate `plan` (read-only) and `apply` (main branch only) roles, a KMS key for state and secrets, and a budget. Layer 2 (`network/`): reuse the Project 4 network module with three AZs, public, private and data subnets, and the tags EKS and the load balancer controller need:

```hcl
# subnet tags consumed by the AWS Load Balancer Controller and the cluster
public_subnet_tags  = { "kubernetes.io/role/elb" = "1" }
private_subnet_tags = { "kubernetes.io/role/internal-elb" = "1" }
```

Add VPC endpoints for S3 (gateway) and, to cut NAT traffic and exposure, for ECR, Secrets Manager and STS (interface endpoints cost per hour, so decide per your cost model, network egress costs §20.7). Evidence: `terraform plan` output, and a diagram that shows no database route to the internet gateway.

Milestone 3 — Platform layer: EKS (≈5 h)

Build the cluster with Terraform, using the well-maintained community module or your own, pinned to an exact version (**argument names change between major versions, so follow the README of the version you pin**). The design choices to make and record:

```hcl
module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "<PINNED_MAJOR_MINOR>"                  # pin; read its upgrade notes
  # name / cluster_name, Kubernetes version (n or n-1 of the newest supported), VPC and
  private subnets
  endpoint_public_access  = false                   # or restrict to your CIDR; prefer
  private + SSM or VPN
  encryption_config       = { resources = ["secrets"], provider_key_arn = var.kms_key_arn
  }
  enabled_log_types       = ["api", "audit", "authenticator"]
  eks_managed_node_groups = {
    ondemand = { instance_types = ["m6i.large"], min_size = 2, max_size = 4, desired_size
  = 2 }
    spot     = { instance_types = ["m6i.large", "m5.large", "m6a.large"], capacity_type =
  "SPOT", min_size = 0, max_size = 4, labels = { workload = "stateless" } }
  }
}
```

Install the add-ons that EKS expects (VPC CNI with NetworkPolicy support enabled or Cilium, CoreDNS, kube-proxy, EBS CSI driver, metrics-server) and use **EKS Pod Identity** or IRSA for every controller that talks to AWS, each with its own least-privilege role. Never attach broad policies to the node role. Evidence: `aws eks describe-cluster` with logging and encryption shown, `kubectl get nodes -L topology.kubernetes.io/zone` showing three zones.

Milestone 4 — Data layer (≈3 h)

RDS PostgreSQL Multi-AZ with `storage_encrypted`, KMS key, `manage_master_user_password = true`, automated backups with PITR, a cross-region snapshot copy, deletion protection on, and parameter groups for connection logging. ElastiCache Redis (replication group with automatic failover in non-dev). Security groups reference each other and allow only the node or pod security group. Because RDS requires TLS, ship the RDS CA bundle in the image and enable certificate verification in the pool settings rather than the lab shortcut from Project 4 (TLS §12.4). Consider a connection pooler (RDS Proxy or PgBouncer) once replica counts grow (connection pooling §13.6).

Milestone 5 — GitOps bootstrap and platform add-ons (≈5 h)

Terraform installs **only** Argo CD (a Helm release) and one root Application; everything else comes from `shop-gitops`. The app-of-apps lists, each with pinned chart versions and sync waves:

Wave	Component	Why this order
0	namespaces, Pod Security labels, Kyverno (or ValidatingAdmissionPolicy)	policy before workloads
1	AWS Load Balancer Controller, External Secrets Operator, cert-manager or ACM integration, external-dns	needed by apps
2	kube-prometheus-stack, Loki, Alloy, Tempo, OTel Collector	observability before traffic
3	`shop-api` chart (dev, then prod)	the application

External Secrets maps an AWS Secrets Manager secret to a Kubernetes Secret:

```yaml
apiVersion: external-secrets.io/v1          # v1beta1 in older operator releases
kind: ExternalSecret
metadata: { name: shop-db, namespace: shop }
spec:
  refreshInterval: 1h
  secretStoreRef: { kind: ClusterSecretStore, name: aws-secrets-manager }
  target: { name: shop-db }
  dataFrom: [{ extract: { key: shop/prod/db } }]
```

The Ingress uses the ALB controller (`ingressClassName: alb`, annotations `alb.ingress.kubernetes.io/scheme: internet-facing`, `target-type: ip`, `healthcheck-path: /readyz`, and `certificate-arn` for ACM). Add policies that deny `:latest` images, require resource limits, non-root and read-only root file systems, and allow images only from your ECR registry (policy as code §18.7). Evidence: Argo CD showing all applications `Synced` and `Healthy`, and a rejected `kubectl run --image=nginx:latest`.

Milestone 6 — Delivery pipeline (≈4 h)

Extend the Project 3 pipeline: test, build, scan (Trivy), push to **ECR using OIDC** (no stored keys), sign or attest the image if you want supply-chain evidence, then commit the new tag to `shop-gitops/envs/dev`. Promotion to `prod` is a reviewed pull request. Add Terraform pipelines per layer (`fmt`, `validate`, `tflint`, a security scanner, `plan` on PR, `apply` on merge to `main` by the `apply` role in a protected environment). Add a progressive delivery step for prod (Argo Rollouts canary with an analysis on the Project 7 error-rate metric) or at least a

rolling update with `maxUnavailable: 0` and a PodDisruptionBudget (canary deployments §5.12). Evidence: a pipeline run from commit to production with timings; a rollback by `git revert`.

Milestone 7 — Reliability: observe, load, break (≈5 h)

Reuse the Project 7 dashboards and the SLO burn-rate alerts, and tie alerts to a runbook. Then test claims instead of assuming them:

```bash
# load test (k6 in a container), ramp to 3x expected peak, watch HPA, p95 latency and
  error ratio
docker run --rm -i grafana/k6 run - <<'EOF'
import http from 'k6/http'; import { check } from 'k6';
export const options = { stages: [{ duration: '2m', target: 50 }, { duration: '5m',
  target: 150 }, { duration: '1m', target: 0 }],
  thresholds: { http_req_failed: ['rate<0.01'], http_req_duration: ['p(95)<300'] } };
export default function () { check(http.get(`${__ENV.BASE}/products`), { ok: (r) ⇒
  r.status ⩵ 200 }); }
EOF
# chaos: lose a node, then lose an AZ's capacity (cordon and drain every node in one zone)
kubectl drain <node-in-az-a> --ignore-daemonsets --delete-emptydir-data
aws rds reboot-db-instance --db-instance-identifier shop-prod --force-failover       #
  Multi-AZ failover
```

Use topology spread constraints and PodDisruptionBudgets so zone loss does not drop below capacity (availability §19.2). Re-run two Project 8 drills against this stack: an RDS point-in-time restore to a new instance, and rebuilding the cluster from Terraform plus Git. Record RPO and RTO against the milestone 1 targets.

Milestone 8 — Cost model and optimisation (≈3 h)

Create `docs/cost.md`: a table of each resource, unit price (from the pricing page), quantity, hours and monthly cost, plus a "what scales with traffic" column. After a day, compare it with Cost Explorer, grouped by the `project` tag. Then try three optimisations and record the saving and the risk: right-size requests from Prometheus data (rightsizing §20.5), move stateless Pods to the spot group with PDBs and graceful shutdown (spot instances §20.3), and add S3 lifecycle rules plus log retention limits (storage lifecycle §20.6). Shut down or scale to zero non-production resources out of hours.

Milestone 9 — The production-readiness review and teardown (≈4 h)

Complete the review below with **evidence links** in every row. Where the answer is "no", write the risk, the owner and the date, rather than hiding it. Then destroy in reverse layer order (Argo CD applications first so load balancers are removed; then Terraform layers), and

verify with the CLI that no ALB, NAT gateway, EKS cluster, RDS instance, ElastiCache cluster or unattached volume remains.

Quality	Question	Evidence (link)	Status
Secure	Any static credentials? Public data stores? Unencrypted data? Policies enforced?	IAM Access Analyzer, `kubectl` policy denial, scan reports	
Observable	Can you go from an alert to a trace and the failing release in minutes?	Project 7 runbook drill	
Scalable	What is the first bottleneck at 3x load?	k6 report, HPA events	
Highly available	What happens when one node, one AZ, one database instance fails?	chaos results	
Automatically deployable	Commit to production without a human running a command? Rollback time?	pipeline runs	
Recoverable	Measured RPO and RTO versus targets? When was the last restore?	drill logs	
Cost-efficient	Bill versus model? Largest line item and why?	`docs/cost.md`	

5 Tasks

- [ ] **T1 — Write requirements, SLOs and five ADRs** *(≈3 h, intermediate)*
  - **Goal:** decisions recorded before they are built.
  - **Steps:**
    1. Write `docs/architecture.md` with users, load, data sensitivity, SLOs and a budget.
    2. For each of the five decisions, create an ADR with Context, Options considered, Decision, Consequences (what gets harder) and a review trigger.
    3. Ask someone to challenge the weakest ADR and amend it.
  - **Done when:** five ADRs are committed, each lists at least two alternatives and one negative consequence, and one was revised after the challenge.

- [ ] **T2 — Bootstrap the account and network with Terraform** *(≈4 h, advanced; costs money)*
  - **Goal:** repeatable foundations with no static credentials.
  - **Steps:**
    1. Apply the bootstrap layer (state, OIDC, KMS, budget) and the network layer.
    2. Run `plan` in CI on a PR through OIDC; merge to apply.
    3. Prove the data subnets have no route to an internet gateway.
  - **Done when:** a PR plan appears as a comment, apply runs from CI, and `aws ec2 describe-route-tables` shows no `igw-` route for data subnets.

- [ ] **T3 — Provision EKS and verify the controls** *(≈5 h, advanced; costs money)*
  - **Goal:** a private, encrypted, three-zone cluster.
  - **Steps:**
    1. Apply the platform layer with pinned module versions.
    2. Verify envelope encryption, audit logs, node zones and that no workload uses the node role for AWS access.
    3. Open the control plane only via the private path you chose.
  - **Done when:** `kubectl get nodes -L topology.kubernetes.io/zone` shows three zones and `aws eks describe-cluster` shows encryption and logging enabled.
- [ ] **T4 — Stand up the data tier and prove the boundaries** *(≈3 h, advanced; costs money)*
  - **Goal:** encrypted, Multi-AZ data stores reachable only from the application.
  - **Steps:**
    1. Apply the data layer; read the plan for every security-group rule.
    2. From a debug Pod without the app label, try to connect to RDS and Redis.
    3. Trigger `reboot-db-instance --force-failover` and measure application impact.
  - **Done when:** the unlabelled Pod cannot connect, the app reconnects after failover, and you recorded the downtime.
- [ ] **T5 — Bootstrap GitOps and enforce policy** *(≈5 h, advanced)*
  - **Goal:** the cluster configures itself from Git, and unsafe workloads are denied.
  - **Steps:**
    1. Install Argo CD with Terraform and the root app; let sync waves install the platform add-ons.
    2. Deploy `shop-api` with External Secrets; confirm no secret value exists in Git.
    3. Try to deploy a Pod with `:latest`, as root, and from a foreign registry.
  - **Done when:** all Applications are `Synced` and `Healthy`, and all three bad Pods are rejected with policy messages.
- [ ] **T6 — Ship a change through the whole pipeline** *(≈3 h, advanced)*
  - **Goal:** commit to production without manual commands.
  - **Steps:**
    1. Make a visible change in `shop-api`; open a PR; observe tests, scan and plan checks.
    2. Merge; follow the image to ECR, the tag bump in `shop-gitops`, and the Argo CD sync.
    3. Promote to prod by PR; then `git revert` it and time the rollback.

- **Done when:** the change is live in prod, the rollback restored the old behavior, and both timings are in `docs/evidence/`.

- [ ] **T7 — Load test and chaos test against the SLO** *(≈4 h, advanced)*
  - **Goal:** evidence that the platform meets its targets, or a list of why not.
  - **Steps:**
    1. Run the k6 profile and watch the dashboard, HPA and error ratio.
    2. Drain every node in one AZ and force an RDS failover; keep the load running.
    3. Calculate how much error budget the experiment consumed.
  - **Done when:** you have a report with p95 latency, error rate, scaling behavior and budget consumed, and an action item for the first bottleneck.

- [ ] **T8 — Cost model, one optimisation and teardown** *(≈3 h, intermediate; saves money)*
  - **Goal:** know the bill, reduce it safely, and prove cleanup.
  - **Steps:**
    1. Compare `docs/cost.md` against Cost Explorer by tag after a day.
    2. Implement one optimisation (spot for stateless Pods, right-sized requests, or scheduled dev shutdown) and measure.
    3. Tear down in reverse layer order; run the CLI checks for leftover billable resources.
  - **Done when:** the model and actuals differ by an explained amount, the optimisation has a before-and-after number, and the checks return empty lists.

- [ ] **T9 — Prove it: defend the design in a review** *(≈2 h, advanced)*
  - **Goal:** demonstrate senior-level reasoning, not just a working system.
  - **Steps:**
    1. Present the architecture, the readiness table and the ADRs to a reviewer (a peer, a mentor, or a recorded 20-minute walkthrough).
    2. For every "no" or "partial" in the table, state the risk, who is exposed, and what you would do with another week and another month.
    3. Answer: what would you remove if the budget were cut by half, what would break first at 10x traffic, and what is the most likely cause of your next incident?
  - **Done when:** the reviewer signs off or lists concrete objections, you recorded them, and your answers to the three questions are written in `docs/review.md`.

6 Definition of done

- [ ] An empty sandbox account becomes the platform by following the documented bootstrap; teardown leaves nothing billing.
- [ ] No static credentials: CI uses OIDC, workloads use Pod Identity or IRSA, secrets come from Secrets Manager via External Secrets; Git holds no secret values.
- [ ] Three-AZ EKS, Multi-AZ RDS, encrypted storage, private data tier, restricted control-plane access, audit logging enabled.
- [ ] Every deploy (infrastructure and application) is a reviewed Git change applied by automation; rollback is a revert.
- [ ] Policies block `latest`, root, missing limits and untrusted registries; NetworkPolicies default-deny.
- [ ] SLOs defined, burn-rate alerts firing in a drill, load and chaos results recorded, and RPO and RTO measured against targets.
- [ ] A cost model compared with actuals; resources tagged; at least one optimisation validated.
- [ ] The production-readiness table is complete with evidence, and five ADRs exist with consequences and review triggers.

7 Break it on purpose

1. **Delete the NAT gateway while a rollout runs.** New nodes and Pods cannot pull images or reach AWS APIs. Observe how the readiness gates, PDBs and the previous ReplicaSet protect users, then decide whether VPC endpoints or one NAT per AZ is justified. *(Lesson: shared dependencies hide in the network.)*
2. **Revoke the External Secrets role.** Remove the policy and wait for the refresh interval. Existing Pods keep running, but a rotated secret would not arrive. Add an alert on `ExternalSecret` sync failures. *(Lesson: control-plane dependencies fail quietly.)*
3. **Roll a bad cluster upgrade.** In a throwaway copy, upgrade the control plane one minor version and then the nodes, with a deprecated API still in your manifests. Find the failure with the upgrade checks and fix the manifests first. *(Lesson: upgrades are planned, tested events.)*
4. **Tag drift and the surprise bill.** Create an untagged large instance by hand and see it appear in Cost Explorer as "no tag". Add a policy or Terraform check that fails untagged resources. *(Lesson: cost allocation depends on enforcement.)*

8 How this evolves into the next project

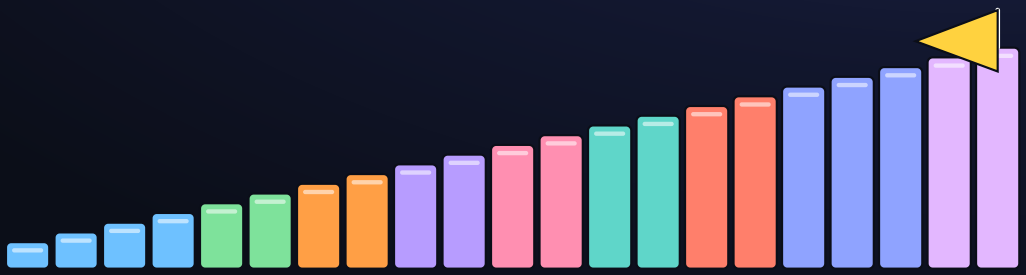
There is no Project 10, by design. The next step is not another tool but another system: take an application you did not write, run the production-readiness review from section 4 on it in a day, rank the gaps by risk, and close the most important ones. Repeat that on a team, with real users and an on-call rota, for a year or two, and the judgment this project rehearses becomes experience. Keep the platform as your reference implementation, and keep extending it: add a developer portal and golden paths (Level 18), multi-region failover, a service mesh (Level 16), or a deeper cost program (Level 20). Revisit the learning order and the timeline to decide what to deepen next.

LAB NOTES What I ran, what I saw, what it means

APPENDICES

The Back of the Pack

The template behind every chapter, the kit for sharing the roadmap, and the licence that lets you reuse all of it.



IN THIS PART

A	Topic Template	3959
B	Captions for the DevOps Roadmap posts	3963
C	DevOps Roadmap carousel	3967
D	MIT License	3971

LAB NOTES What I ran, what I saw, what it means

APPENDIX A

A Topic Template

Every topic in this roadmap is one Markdown file with the same shape, so you always know where to find the explanation, the diagram, the hands-on guide and the tasks.

File naming: `NN-topic-slug.md` inside `level-NN-level-slug/`. The last file in each level is `99-capstone-project.md`, which ties the topics of that level together.

Required sections (in this order)

MARKDOWN

```
# L{level}.{topic} — {Topic Title}

> **Level {level}: {Level Title}** · Topic {topic} of {total} · ⏱ ~{N} hours ·
  Difficulty: {Beginner|Intermediate|Advanced}

## 1. What it is
Plain-English explanation in 1-2 short paragraphs. Define every acronym on first use.

## 2. Why it matters in DevOps
What breaks, costs money or wakes someone up at 3 a.m. when this is misunderstood.

## 3. Real-world system example
A named, real, publicly documented system or company (e.g. Netflix, GitHub, Stripe,
  Cloudflare, Shopify,
Airbnb, Spotify, Google, AWS itself, Kubernetes, Linux kernel) and how this topic shows up
  there.
Only state facts you are confident are publicly documented. Do not invent numbers, dates
  or internal details.
If a precise claim is uncertain, describe the pattern generically ("large streaming
  platforms typically…").

## 4. Diagram
At least one Mermaid diagram showing the real mechanism (flow, sequence, architecture or
  state machine).
Add a second diagram when it helps (e.g. "before vs after", "failure path").

## 5. How it works
Key concepts, a comparison table or a short reference of the important commands/options.

## 6. Step-by-step guide
```

```
Numbered steps. Every command is copy-pasteable, in a fenced block with a language tag,
  with the
expected output or what to look for. State prerequisites first (OS, tools, accounts,
  cost).
Use a safe sandbox (a local VM/container, Docker, kind, LocalStack, free tier) wherever
  possible.

## 7. Real-world scenario
A short story (an outage, a migration, a scaling event) showing the topic applied under
  pressure:
symptom → investigation → fix → lesson.

## 8. Common mistakes and best practices
Two short lists: "Don't" and "Do".

## 9. Tasks
A checklist of hands-on tasks, easiest first. Each task has an ID, a goal, steps and a
  "Done when" check.

## 10. Self-check questions
4-6 questions that a senior engineer would expect you to answer without notes.
Put answers inside a `<details>` block.

## 11. Further reading
Official documentation and well-known books only. Never guess a deep URL; link to a
  documentation root
or name the document/book instead.
```

Task format

MARKDOWN

```
- [ ] **T1 — Short imperative title** *(≈20 min, beginner)*
  - **Goal:** what you will have achieved.
  - **Steps:**
    1. …
    2. …
  - **Done when:** an observable, verifiable result (a command's output, a URL that
  responds, a metric that appears).
```

Rules for tasks:

- 4-6 tasks per topic, progressing from "follow along" → "modify" → "break it and fix it" → "explain / automate it".
- The last task is always a **Prove it** task: do it without the guide, then explain it out loud or in writing.

- Every task is independently verifiable. "Understand X" is not a task; "Show X with this command and explain the output" is.
- Anything that may cost money says so up front and ends with a cleanup step.

Diagram rules (Mermaid)

- Use fenced ```` ```mermaid ```` blocks; they render natively on GitHub.
- Allowed types: `flowchart`, `sequenceDiagram`, `stateDiagram-v2`, `classDiagram`, `erDiagram`, `gantt`, `timeline`. Prefer `flowchart` and `sequenceDiagram`.
- Wrap any label that contains spaces plus punctuation such as `( ) / : , . | #` or quotes in double quotes: `A["Load balancer (ALB)"]`. Never put raw parentheses or slashes in an unquoted label.
- Use `<br/>` for line breaks inside labels, not `\n`.
- Do not use the words `end`, `graph` or `subgraph` as node ids. Node ids are short ASCII (`A`, `lb`, `db1`).
- Keep diagrams readable: ≤ 15 nodes each.
- Edge labels: `A ⟶|label| B` (no quotes needed for simple words; quote if punctuation).

LAB NOTES What I ran, what I saw, what it means

APPENDIX B

B Captions for the DevOps Roadmap posts

Post under the matching images in `carousel/` (or the single images in this folder). The link in every caption is `github.com/mascot200/devops-roadmap`; it shows the full roadmap once the branch is merged into `main`.

Platform	Attach	Link behaviour
LinkedIn	`carousel/devops-roadmap-carousel.pdf` (upload as a document)	Clickable in the post
X	4 images: slides 1, 3, 7 and 10 (X allows 4 per post)	Clickable
Facebook	Slides 1 to 10 as a photo post	Clickable
Instagram	Slides 1 to 10 as a carousel	**Not clickable**: use "link in bio"
TikTok	Slides 1 to 10 as a photo post	**Not clickable**: use "link in bio"

LinkedIn

Most DevOps advice is a list of tools. This is a path.

I published a free, open-source DevOps roadmap that takes you from your first Linux command to senior-level architecture, in the order that makes sense.

What is inside:

- 21 levels: Linux, networking, Git, Docker, CI/CD, AWS, Terraform, Kubernetes, observability, security, SRE, GitOps and more
- 248 topics, each with a real-world example, diagrams, a step-by-step guide and hands-on tasks
- 1,700+ tasks with a way to check each one, so you know when you actually got it
- 9 projects that grow into one production-style system

The goal is not to know 50 tools. It is to take one app to production and make it secure, observable, scalable, highly available, automatically deployable, recoverable and cost-efficient.

Plan for 8 to 12 months at 2 to 3 hours a day to reach a strong practical level. Senior judgment takes longer, and it comes from real production work.

It is MIT licensed. Fork it, use it with your team, send fixes.

Roadmap: github.com/mascot200/devops-roadmap

Swipe through the document for the full path.

#DevOps #Kubernetes #Terraform #AWS #Linux #OpenSource #PlatformEngineering #SRE

X

A DevOps roadmap, not a tool list.

Free and open source: 21 levels from Linux to senior architecture, 248 topics, 1,700+ hands-on tasks, 9 projects.

Every topic has a real-world example, diagrams and a step-by-step guide.

github.com/mascot200/devops-roadmap

#DevOps

Facebook

New and free: a DevOps roadmap from beginner to senior.

If you want to work in cloud or DevOps but do not know what to learn first, this lays it out step by step: Linux, networking, Git, Docker, CI/CD, AWS, Terraform, Kubernetes, monitoring, security and more.

Every topic has a real-world example, diagrams, a guide you can follow and tasks to check your work. There are 9 projects that build on each other, so you finish with something you can show.

It is free and open source. Save it, share it with someone learning DevOps, or start today: github.com/mascot200/devops-roadmap

#DevOps #CloudComputing #LearnTech

Instagram

Don't learn 50 tools. Learn to take one app to production.

Swipe for the full path: 21 levels from Linux to senior-level architecture, 248 topics, 1,700+ hands-on tasks and 9 projects. All free and open source.

Save this post so you can come back to it, and send it to someone who is starting out.

Link in bio: github.com/mascot200/devops-roadmap

#devops #cloudcomputing #linux #docker #kubernetes #terraform #aws #sre #platformengineering #learntocode #techcareers #opensource #softwareengineering

---

TikTok

Don't learn 50 DevOps tools. Learn the path. 21 levels, 248 topics, 1,700+ tasks, 9 projects. Free. Link in bio.

#devops #cloudcomputing #learntocode #techtok #kubernetes #linux #aws

LAB NOTES What I ran, what I saw, what it means

APPENDIX C

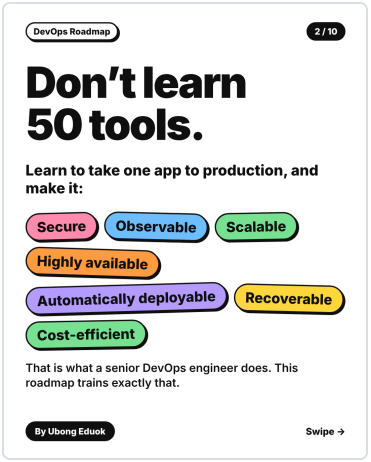
C DevOps Roadmap carousel

Ten slides, **1080 x 1350 px** (4:5). Every slide is its own PNG, so you can download and post them one by one.

Where you post	Use
Instagram, Facebook, X	The PNG slides below, in order 1 to 10
LinkedIn	`devops-roadmap-carousel.pdf` (upload it as a document)



1. Cover
slide-01-cover.png



2. Don't learn 50 tools
slide-02-dont-learn-50-tools.png



3. Every topic comes with
slide-03-every-topic-comes-with.png



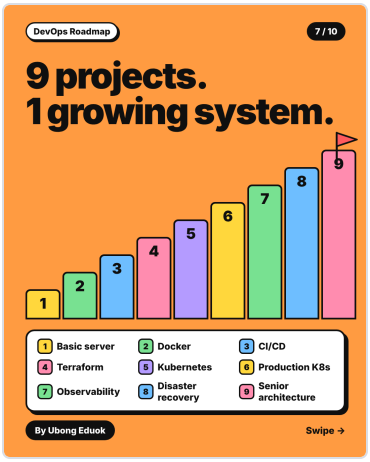
4. The path 1/3: Foundations
slide-04-path-1-foundations.png



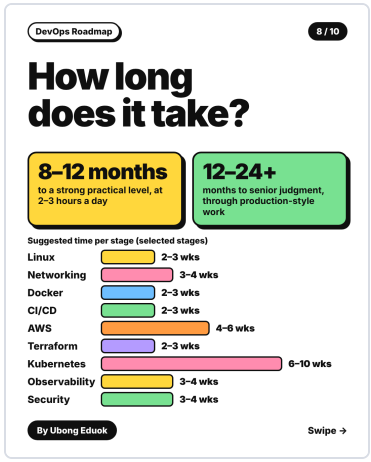
5. The path 2/3: Cloud and Kubernetes
slide-05-path-2-cloud-and-kubernetes.png



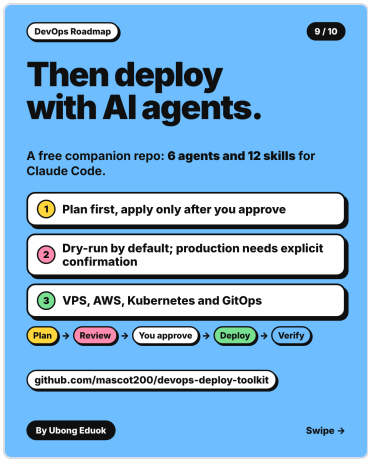
6. The path 3/3: Senior level
slide-06-path-3-senior-level.png



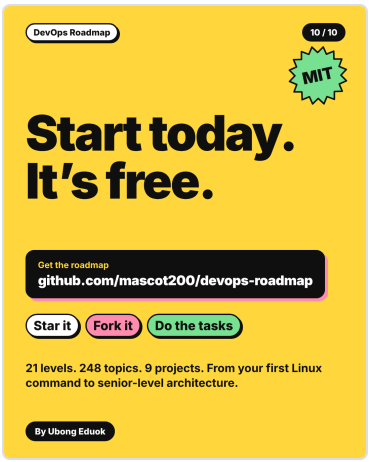
7. 9 projects, 1 growing system
slide-07-nine-projects.png



8. How long does it take?
slide-08-how-long-does-it-take.png



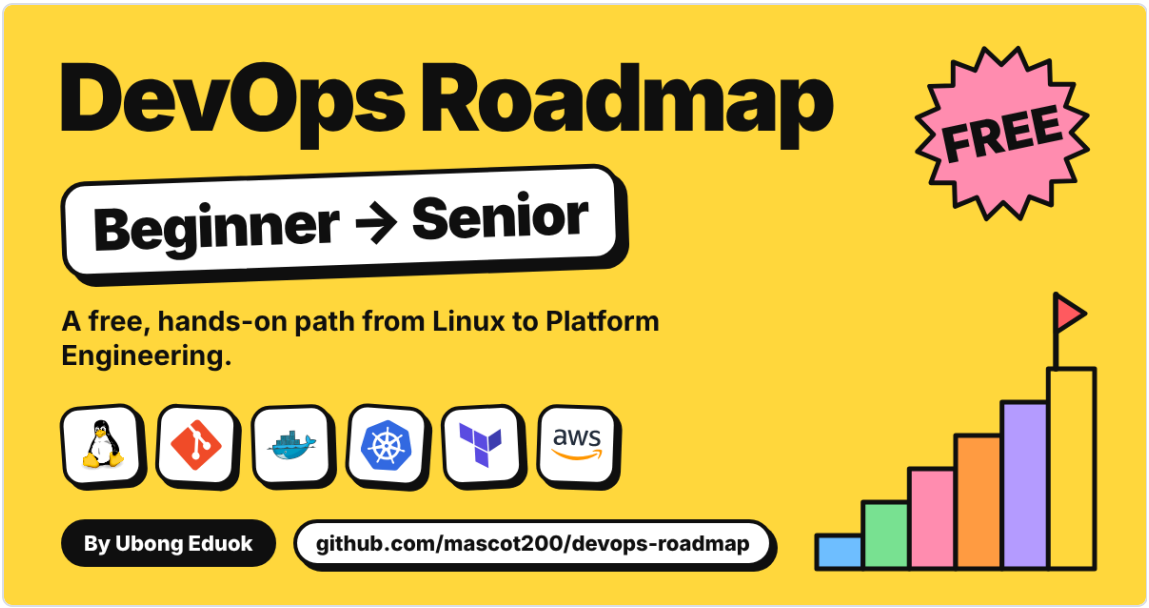
9. Then deploy with AI agents
slide-09-deploy-with-ai-agents.png



10. Start today. It's free.
slide-10-start-today.png

How to download one slide: click *Download PNG* under it, or open the image and use the download button on GitHub. Slide files are named `slide-NN-topic.png`, so they stay in order when you save them.

Single-image versions: landscape 1200x630 and square 1080x1080.



Landscape share image, 1200x630 (devops-roadmap-share-1200x630.png)



Square share image, 1080x1080 (devops-roadmap-square-1080x1080.png)

APPENDIX D

D MIT License

MIT License

Copyright (c) 2026 Ubong Eduok

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

LAB NOTES What I ran, what I saw, what it means

Index

Page numbers point to where a term is defined or discussed in depth. Numbers run continuously through all six volumes; the print-set page at the front of each volume shows which pages it holds.

A

A10: SSRF 2178
ABAC 2191
abort conditions 2941
absolute path 313
accept queue 466
Access control 1235
access logs 1316
Access modes 1657
access points 1251
account 3073
achieved RTO 3059
ACID 2593
ACID "C 2743
acknowledge 2704
ACM 462, 1302, 1311, 2366, 2480
ACME 2365
Acquire / wait timeout 2467
action 677, 1311
Actionable 2907
actionable alerts 2907
Actionable ratio 2884, 2913
Actions 52, 933, 945
Actions, rollback, Run workflow 1028
Active 493, 645, 2285
Active, L4 2404
Active, L7 2404
active scanner 2285
Active series 3639
actual recovery point 3056
AD 2349
Additive 1785, 3153
additive and deny-by-default 3139
address 1409
Addressing layers (TCP/IP model) 402
admission 1765
admission control 3205
Admission Controllers and Webhooks 3205
ADOT 2036
ADR 3677
Advanced 1344
Advanced Kubernetes 3137
Advanced Networking 2321
agent 2035
Agent forwarding 363
aggregated ClusterRoles 3139
Aggregation 1975
Agile 41
AKS 1766, 1780
Alarms 1369
ALB 503, 1311, 2480, 3664
Alembic 1049
Alert fatigue 2907
Alert Fatigue Reduction 2907
alert review 2907
Alert rule 2055
Alert rules 1950
alerting rules 1933
Alertmanager 1933, 2880
Algorithms 2399
Alias 1285
alias records 450
aliases 1325
Allocation 3803
allow 1203
allow and deny 1203
allow/deny with a reason 3475
Alpine 877
Alpine Linux 878
ALPN 2368
also hangs 2601
alternate domain name (CNAME) 1300
Alternating (multi-user) 1343
Amazon 1020, 2940
Amazon Aurora 1261, 3792
Amazon CloudFront 1297
Amazon CloudWatch 1369
Amazon Data Lifecycle Manager 3767
Amazon DynamoDB 2728, 2740
Amazon ECR 3768
Amazon ECS 1067, 1340
Amazon ElastiCache 1273, 2669, 3792
Amazon MSK 2686
Amazon RDS / Aurora 2716
Amazon RDS for PostgreSQL 2640
Amazon RDS Proxy 2464, 2655
Amazon Route 53 450, 1285, 2350
Amazon S3 storage classes 3768
Amazon SQS 2764, 2778
Ambient / sidecarless 2445
ambient mode 2445, 3251
AMI 1141
AMI (Amazon Machine Image) 1141
AMQP 0-9-1 2701
Analysis 1095
Anatomy of a policy 1222, 1784
Anatomy of a service 819
Anatomy of an image 754
Annotation 1951
Annotations 1602
another security group 1207
Ansible 75, 3183
Anti-amplification 2387
anti-entropy 2752
anycast 532, 2335
Anycast IP 2400
AOF 2669
Apache Kafka 2685, 2764
Apache ZooKeeper 2778
API 2035, 2208
API gateways 2218
API pods 3663
API scan 2285
API Security 2217
app 3375
App of Apps 3351, 3522
app repo 3377
AppArmor 3167
application 531, 3317
application layer 503
Application Load Balancer 1311, 1602
Application Load Balancer (ALB) 1311, 2480
application repo 3309
Application Synchronization 3351
ApplicationSet 3522
ApplicationSets 3351
Apply 64
apply role 1489
Approximate sizes 881
appVersion 1891
ARC 706
Architecture 3551
Architecture Decision Record (ADR) 3677
ARG vs ENV 808
Argo CD 1826, 1837, 1850, 3317, 3521

Argo CD and Flux 3317
Argo CD ApplicationSets 3440
Argo Rollouts 1096, 3266
arguments 1409
ARN 1217
ARP 399
Artifact 705
artifact attestations 1004
Artifact Hub 1826, 1880
Artifacts 677, 705, 1003
Artifacts and Runners 705
AS 2335
AS Number (ASN) 2335
AS17557 2336
ASG 1155
ASN 2335
AS_PATH 2335
associative 2756
asynchronous 2623
at least two AZs 1263
at rest 1276
At runtime 889
at the same time 1245
Atlantis 1486
Atlassian 2868, 2880, 2894
At-least-once 2690, 2763, 2782
at-least-once delivery 2701, 2763
At-most-once 2690, 2767
atomic 1865, 2514
Atomic with the effect 2768
Atomicity 2593
Attestation 2500
attestations 971
attributes 1410, 2021
audit 3139
audit policy 3139
audit trail 2125
Auditability 51, 97, 934, 1339, 1485, 3305
audit/warn 3480
Aurora Capacity Units (ACUs) 3792
Aurora Global Database 3048
Aurora I/O-Optimized 1264, 3792
Aurora Serverless v2 1264, 3791
Aurora specifics 1264
Aurora Standard 3792
AUTH token 1276
authentication 466, 777, 1765, 2365, 2497
Authentication and Authorization 2191
Authorisation and audit 3430
authoritative server 449
Authoritative servers 2349
authorization 1765, 2498
authorization policy 3265
authorization server 2203
Auto Scaling 1155, 1369
Auto Scaling group (ASG) 1155
Auto-discovery 3454
automatable 2942
automated gates 3627
Automated Testing in Pipelines 959
Automated update PRs 3443
automatic 2035, 3567
automatic rollback 934
automatic rotation 1330
automatic security patching 2125
Automatic vs manual 3036
Automation 215, 225, 462, 1141, 1203, 1353
Automation tools 664
Automerge 2752
Autonomous System (AS) 2335
Autoscaling for Cost 3743
Autovacuum 2608
Availability 165, 489, 1155, 1179, 1191, 3565
Availability (A) 2739
availability zone (AZ) 3565
availability zones (AZs) 2997, 3779
available 323
awk 249
awk building blocks 252
AWS 110, 412, 518, 1141, 1155, 1167
AWS API Gateway 2218, 2514
AWS Application Load Balancers 476, 2006
AWS Budgets 1374, 3706, 3718
AWS Certificate Manager (ACM) 462, 1311, 2480
AWS CLI v2 1146
AWS Client VPN 2431
AWS CloudFormation 63
AWS CloudFront 2382
AWS CloudTrail 1353
AWS CodeDeploy 1082
AWS Compute Optimizer 3756
AWS Config 1513
AWS Control Tower 2152
AWS EC2 350
AWS Elastic Beanstalk 1082
AWS Elastic Load Balancing 490, 504
AWS Encryption SDK 1326
AWS Global Accelerator 2400
AWS IAM 2192
AWS Lambda 1167, 1340, 2764, 3744
AWS Load Balancer Controller 1602
AWS load balancers 2464
AWS managed keys 1329
AWS Node Termination Handler 3730
AWS owned keys 1329
AWS sandbox account 3510
AWS Secrets Manager 1339
AWS Security Hub 2152
AWS Site-to-Site VPN 2431
AZ 1179, 1261
AZs, Availability Zones 1155
Azure Cosmos DB 2752

B

backend 489
Backends 2025
backend's own 2488
backport 587
Backstage 186, 3406, 3415, 3491, 3521
Backstage Software Templates 3428
Backstage's Software Catalog 3452
Backtrack 1264
backup 2639, 2985
backup account 3073
backup and restore 136, 3060, 3650
Backup Vault Lock 3073
Backups 135, 2985
Backups and Point-in-Time Recovery 2639
Backups and recovery 1264
backwards 599
Bad alerting destroys teams 3635
Bake 1084
bake time 934, 1024, 3624
balancing algorithm 489
Barman 2640
base 3309
base backup 2639
base image 877
base units 1966
Basecamp (37signals) 3612
baseline 1096
baseline scan 2285
Basic Constraints 2369
Basic Server 3837
bastion host 363
batch work 1691
Before runtime 889
before touching sshd 2130
behavior 1741, 3225
BestEffort 1717
BFD 2339
BGP Fundamentals 2335
billing alarm 1369
billing alarm already created 1302
Bin packing 3803
binary search 629
binaryData 1618
BIND 2353
Bind mount 785

bind mounts 851
Binding 1654
bindings 2701
binds 1653
BIRD 2 2340
bisect 629
Bitmap / HyperLogLog 2673
Blackhole 1195
Blameless post-mortems 162
blamelessness 2893
blast radius 109
Blast-radius control 3623
bloat 2607
Block Public Access 1231, 1447, 2152
Block storage 3767
blocks 1405
blue 1081
Blue/green 109, 1023
blue/green deployment 1081
Blue/Green Deployments 1081, 1264
body 475
BOLA 2221
Bootstrap the state bucket 1448
Boring technology 3611
both data and traffic 2731
Bottlerocket 88
branch 565, 975
branch by abstraction 655
Branch per environment 3343
Branch protection rules 639
Branching and Merging 565
branching strategy 651
Breadcrumbs, tags, context 2055
break-even point 3720
bridge 795
BRIN 2579
broadcast 400
Broker 2685
browser 531
Browsers 532
brute-force blocker 2125
B-tree 2579
bucket 1231
bucket policies 1231
buckets 1961
Budget consumed 2897
Budgets 2534
Build 933, 1003
build artifact 1003
Build Artifacts 1003
build cache 829
build context 805
build metadata 663
build once, deploy many 1003
build time 2232
Build vs buy 3611
Building blocks 1951
BuildKit / buildx 971
built and tested 51
built but not pushed 979
Bulkhead 2939
Bulkheads 2533
Burn rate 162, 2851, 3635
Burp Suite 2286
Burstable 1717

C

CA 461, 2365, 3221
Cache 677, 706, 1007, 2669
Cache and CDN 3604
Cache behavior 1298
cache behaviors 1298
cache import/export 971
cache key 1300, 2415
cache miss 2415
cache poisoning 2416
Cache policy 1300
Cache rules (simplified) 832
cache stampede 2669, 3557
Cache-aside 2671
Cache-Control 2415
caches 449
Caching 678, 945
Caching and compression 493
Caching patterns 1276
Caching, redirects and security 476
Caddy 2382
calculated 1288
calendar 2832
Canary 109, 1023, 1095
canary deployment 1095
Canary Deployments 1095
canarying releases 1096
Cancelled 948
cannot be a step 949
Cap telemetry 3604
CAP theorem 2739
capabilities 851
Capacity 1067, 1179
capacity during failover 2405
Capacity Planning 165, 2923
Capacity (step) 2926
Capital One (2019) 3578
Capsule 1640
Capture 2059
cardinality 151, 1961, 1990
Cassandra 2728
Catalog 3451, 3491
catalog-info.yaml 3451
cattle 87
CD 51, 933
CDN 1297, 2415
CDN + WAF 3664
CDNs 2415
CDNs and edges 532
Cedar 2195
CEL 3205, 3209, 3476
CEL validation rules 3195
Celery (Python task queue) 2702
cells 3581, 3623
Centralised logs and metrics 90
certificate 461, 1311
Certificate authority 2449
Certificate Authority (CA) 461, 2365
Certificate Transparency (CT) 462
CertificateRequest 2497
cert-manager 1602, 1826, 2366, 2480, 3179
cgroups 739, 1715
chain 2365, 2497
change 3568, 3755
change reads 1050
change sets 64
change writes 1050
changelog 663
Changelog and migration notes 3467
Changelog styles 667
Chaos engineering 3103
Chaos Mesh 2940
Chaos Monkey 122
Chargeback 3706
chart 1825
chart repository 1879
Chart version 1893
chart-releaser 1880
Charts 1825
chart-testing 1892
checkov 1485
checksum 3089
Chef 75
Cherry-picking 587
chgrp 273
child 297
child SAs 2435
chmod 273
Choose 3755
Choosing 655, 2073, 2675
chosen, small percentage 1095
chown 273
Chrome and Firefox 462
chunks 1989
CI 51, 933
CI (continuous integration) 3375
CI/CD 51, 931, 3443, 3863
CI/CD system 945

CIDR 423, 1779
CIDR block 1179
CIDR rules in AWS 1181
Cilium 2445
circuit breaker 2533, 2939
Circuit Breakers 2513, 2533
circuit breaking 3265
CIS Benchmarks 2126
Citus 2728
Claim 3505
Classic HTTP chart repositories 1879
Classic queues 2702
clean 2374
clean working directory 609
CLI 1238
client 1274, 2203
client certificate 2497
Client IP visibility 503
Client-generated 2767
client's 3267
Client-side 2399
Client-side / app-level 2405
closed 2533
closed-model vs open-model 3592
Closing 411
Cloud (AWS) 1137
Cloud Foundry 1082
Cloud images 76
Cloud Security Groups and Policies 2151
Cloudflare 449, 462, 504, 518, 1934, 2006
CloudFront 1297, 3779
CloudFront → S3 1298
CloudFront Functions 1302
cloud-init 310, 1143
CloudNativePG 2624, 3180
CloudTrail 1353, 1513
CloudTrail data events 1447
CloudTrail Lake 1353
CloudWatch 1369
CloudWatch agent 1371
CloudWatch alarm 1288
cluster 1551
Cluster Autoscaler 3744, 3803
cluster endpoint 1263
Cluster, Nodes and Control Plane 1551
Cluster per tenant 1644
cluster volume 1263
ClusterIP 1590
ClusterRole 1765
ClusterRoleBinding 1765
CM 75
CMD only 808
CN 2369
CNCF 148, 1826, 1922, 2035, 3476
CNI plugin 1555
CNI support 1785
code 3623
code scanning 174
Codecov (2021) 2246
CODEOWNERS 640
Cognitive load 3611
cold cache 3557
Collector 2035
Collector tier 3640
command text 832
Commands 415
commit 565
commit log 2611, 2685
Commit SHA 974, 3378
commit status/check run name 643
Commit the lock file 1410
commitment 3717
commits 2056
commits block 2627
committed 602
Committed offset 2687
committing 2685
Communications Lead 2868
Communications Lead (Comms) 2871
commutative 2756
compactor 1991
Compatibility 945
Complexity compounds 3611
Compliance 285, 889, 1353, 1485, 1513, 1779
Compliance and audits 2125
Compliance and contracts 136
Compliance and customer reviews 3577
Components 1314
Compose 788
Compose Specification 817
Composite actions 945
Composite alarms 1373
Composite (multi-column) index 2583
Composite Resource (XR) 3505
CompositeResourceDefinition (XRD) 3505
composition 1891, 3505, 3521
Compute Savings Plans 3717
Compute the latency SLI 2825
Concurrency 945, 1489
concurrently 1034
conditional 1849
conditional writes 1500
Confidence 3341
Confidentiality 466, 2365
ConfigMap 1615
ConfigMaps 1615
configuration 215, 987, 1513, 1936
Configuration Management 75
Configuration management (CM) 75
Configuration packages 3464, 3506
Configuration translation 2449
Configure credentials 3509
conflict 565
Conflicts 591, 613, 2751
Confluent Cloud 2686
Conftest 3475
congestion window 415
connect timeout 2538
connection 2655
connection draining 126, 494, 1033, 2399
Connection handling 492
Connection IDs 2386
Connection leak 2468
connection migration 2386
Connection Pooling 2463, 2655
connection refused 466
connector 2038
conntrack 435
consequences 2841
Consistency 138, 1231, 1431, 1485, 2593, 2989
Consistency, Availability and the CAP Theorem 2739
Consistency (C) 2739
Consistent hashing 2399, 2731
consolidates 3804
constant 1975
Consul 2445
Consumer ack 2705
Consumer group 2685
Consumer lag 2075
Consumers 2685
Consuming a Secret 1628
container 761
container image 751
Container Image Scanning (Trivy) 2231
container or service name 795
container port 799
Container registry 709, 773
container runtime 1553
Container Storage Interface (CSI) 1653
containerd 739
Containers 215, 297
Containers and Kubernetes 88
Containers and Their Lifecycle 761
Content Delivery Network (CDN) 1297, 2415
Content delivery networks 3589
Context 2055
context propagation 2021
Continuous 3329

Continuous / PITR 139
Continuous Delivery (CD) 51, 933
Continuous Deployment 51
Continuous Deployment (CD) 933
Continuous Integration (CI) 51, 933
Continuous Reconciliation 3329
continuously at runtime 2232
Continuously reconciled 3305
Contributing factors 2897
Control 3341
Control groups (cgroups) 739
control plane 1551, 2445, 3251, 3521
controlled gate 639
controller 98, 1096
controller-runtime 3183
controllers 3179, 3505
Conventional Commits 663
converge 2751
Convergence 75
conversion webhook 3195
Convertible 3717
coordination 2669
copies it up 743
copy 225
Copy between registries/accounts 1007
Core building blocks 1300
Core concepts 1234
Core instructions 807
CoreDNS 450, 1590, 1753
Corefile 1754
correctness 165
correlated 3568
Correlation 1975
correlation ID 2005
Correlation IDs and Request IDs 2005
cosign 889, 1884
Cost 3601
cost allocation tags 3706
Cost and Usage Report 3706
Cost Anomaly Detection 3706
Cost architecture 3601
Cost Categories 3706
Cost controls 3430
Cost Explorer 3706
Cost: none 67, 80
Cost Optimization 3703
Costs 2387, 2479, 3078
Counter 1933, 1961
Counters, Gauges, Histograms and Summaries 1961
Coverage 3717
Covering index 2583
CP 2740
CP-leaning 2743
CPU 323, 2073
CPU cores 326
CPU request 1718
crash-consistent 1253
CRD 3179, 3191
CRDTs 2751
create 225, 3491
Create a ClusterIP Service 1593
Create a key pair 1147
Create an execution role 1172
Create sample logs 242
Create the ConfigMap 1619
Create the dev root 1478
Create the distribution 1303
Create the launch template 1159
Create the postmortem template 2899
Create the prod root 1478
Create the sample service 2824
Create the six subnets 1184
Created 1500, 2610
creates, updates or deletes 3205
creating 75
Credentials 1485
Credibility 2907
CRI 1555
CronJob 1691
CronJob fields 1695
Cross-account design checklist 3077
Crossplane 3428, 3505, 3521
Crossplane compositions 3440, 3463
Cross-shard queries 2731
Cross-shard transactions 2732
Cross-stack reads 1458
CSI 1653
CSI driver 3237
CSP 2290
CT 462, 2366
cumulative 1965
curl 261
current working directory 225
custom actions 3495
Custom Resource Definitions (CRDs) 3191
custom resources 3191
customer files a claim 2842
Customer managed keys 1329
Customers 185, 2339, 3405
CustomResourceDefinition (CRD) 3191
CVE 889
CVEs 2231
CVSS 2231
CycloneDX 2231

D

daemon 309
Daemons and systemd 309
DaemonSet 1679, 1990
DaemonSets 1679
Dapper 1922
dark launches 655
dashboard 1949
Dashboards 1369
dashboards as code 1949
DAST 173
Data events 1356
Data Exports 3706
data key 1326
data loss 3103
data plane 2445
data source 1457, 1949
data sources 1949
Data Sources and Dependencies 1457
data tier 3554
Data transfer 3791
data with validation 3191
Database 1023, 3103
Database changes 3623
Database Connection Pooling 2655
Database Costs 3791
database migration 1049
Database Migrations in Pipelines 1049
database problem 1081
Databases & Distributed Systems 2577
Database/schema 1085
datagrams 411
datapoints to alarm 1373
Dates 2497
DB cluster 1264
dead-letter queue 2080
dead-letter queue (DLQ) 2701
dead-letter topic 2690
dead-lettered 2706
deadline 2538
deadlock 2593
Debezium 2686, 2778
debug IDs 2058
Debuggability 461, 705
Debugging 489, 739, 1217
Debugging connectivity 1203
decapsulation 2323
Decide 2628
Declarative 63, 3305
Declarative where possible 101
Decommission or reuse 1085
Dedup table (inbox) 2767
deduplication 2763, 2907
deduplication and grouping 2907
Deep 126
Default backend 1602
Default branch 645
default gateway 400, 435, 537
Defaulting 3194

defaults 1419
defence in depth 2125
defense in depth 3577
Define the request format 3431
Define your API 3509
delayed replication 3001
delegated authorization 2203
Delegation 449, 1285
delete 225, 1656
deleted or replaced 2610
DeleteOnTermination 1250
Deletion 1330, 1344
Delivery 3665
Delivery automation 189
Dependabot 174, 2245
Dependencies 3459, 3565
dependency 1457
dependency graph 1457, 3451
Dependency scanning 2245
Dependency Scanning (Snyk, Dependabot) 2245
deploy 933, 2055
deploy annotations 1953
Deploy health gate 1037
Deploy two backends 1606
Deployable 1007
deployable artifact 971
deploying 109
Deployment 88, 1577
Deployment architecture 3623
Deployment branches 994
Deployment branches and tags 695
deployment circuit breaker 1068
Deployment Strategies 109
Deployment Strategy 109, 3623
Deployments 489, 1155
Deployments and ReplicaSets 1577
Depth 2074
deregistration delay 1034, 1315
derive 3452
derived data last 3023
derived field 2008
Design reviews 3682
designing 933
Designing a CI/CD Pipeline End to End 933
desired 1155, 3320
Desired State Management 3305
destroyed 1405
detect 963, 2893
detection time 3103
Deterministic 633
developer portal 186, 3415, 3491
Developer Portals 3415
Developers create claims 3509
DevOps 41
DevSecOps 173
DHCP 399
DHCP option set 399
diagnostic context 2912
diff ID 754
different endpoint 1267
digest 177, 889, 973, 1003, 3341
digest files 1355
digests 897
dimensions 1371
direct server return/encapsulation 506
directories 328
Directories + Kustomize overlays 3343
Directory 273
Directory / lookup table 2731
Disabling 1330
disappears on reboot 340
Disaster Recovery 135, 2983, 3649, 3929
Disaster recovery (DR) 135, 3649
disaster recovery drill 2944
Discoverability 3491
disk 323
Disk I/O 2073
Disk space 2075
disposable 1949, 2129
Distance 2997
distributed lock 2777
Distributed Locks and Queues 2777
distributed trace 2021
Distributed Tracing 2021
distribution 1297
Distributions 2040
distroless 841, 877, 2271
diversification 3729
DLM 1250, 3008
DNAT 435
DNS 411, 449, 531, 2349, 2751
DNS Architecture 2349
DNS (Domain Name System) 1285
DNS Resolution and Domain Architecture 449
DNS (Route 53 records) 1085
DNS validation 1321
DNS-01 2368
DNS-based (GSLB) 2404
DNSSEC 2355
Docker 297, 424, 435, 737, 3849
Docker Compose 817
Docker Hub 773
Docker Image Builds and Registries in CI 971
Docker Networks 795
Docker rootless mode 2272
Docker Scout 889
Docker Swarm 1071
Dockerfile 805
dockerignore 805
Docker's default seccomp profile 2272
Docs 3440, 3491
dollar amount per hour 3717
Domain Name System (DNS) 449, 2349
DORA metrics 190
downtime 109
DR 135, 3649
DR is expensive 3649
Drift 63, 1513, 3329, 3365
Drift Detection 1513
drop 435, 609
Dropbox 3602
dropped silently 2705
Dropping capabilities 851
Dry-run / plan mode 101
DS 2349
DSN 2055, 2065
DSR (direct server return) 2404
dual write 1050
Dual-stack and IP families 1592
due date 2895
durability 165, 2593
Duration 2073
Dynamic admission webhooks 3205
dynamic forward 363
dynamic provisioning 1653
Dynamic Testing with OWASP ZAP 2285
dynamic/ephemeral 439
DynamoDB 2728
DynamoDB table 1499

E

each statement 2611
eBGP 2335
EBS and EFS 1245
EBS encryption by default 1250
EBS snapshots 3770
EBS volume 1653
EBS volume types 1249
EC 466
EC2 1141, 1155, 3761
EC2 instance profiles 1218
EC2 Instance Savings Plans 3717
ECDHE 466, 2370
ECR lifecycle policies 3767
ECS 1976
Ed25519 353
edge 2479
edge locations 1297
edge servers 2415
EFS 1653

EFS essentials 1251
egress 1779, 2155, 3153
egress control 3153
egress-only internet gateway 1195
EKS 1766, 1780, 3807
Elastic 1245
Elastic Beanstalk CNAME swap 1085
Elastic Common Schema (ECS) 1976
Elastic IP 1145, 1193
ElastiCache 1273
ElastiCache Serverless 1273
Elasticsearch/OpenSearch 1990
ELB 1158
embedded metric format (EMF) 1370
emergency procedure 3365
Empty 1239
Empty podSelector 3157
empty rule list 3157
encapsulated 2431
encapsulation 2323
encoded, not encrypted 2203
Encrypted files in Git 991
Encryption 138, 1250, 1264, 1443, 2165, 2986
Encryption at rest 1235, 1625
encryption in transit 1276
encryption keys 1325
endpoint 1285
EndpointSlice 1589
End-to-end (e2e) tests 959
Enforce the commit format 671
enforce/deny 3480
Engines 1275
ENI 2324
entity 3493
entity providers/discovery 3495
Entropy and context heuristics 2257
ENTRYPOINT + CMD 807
env file 820
envelope encryption 1325, 2165
env_file: / environment 820
Environment 691, 987, 2055, 3623
Environment pointer in Git 1007
Environment Promotion 3341
Environment protection rules 695
environment variable 987
environment variables 1615
Environments 691, 1471
Envoy 489, 504, 1601, 2218, 2382, 2445
ephemeral 706, 785, 1563
Ephemeral containers 1568
ephemeral copies 1475
ephemeral state 2669
Ephemeral storage 1718
ephemeral values 1423
Equifax (2017) 2246
error budget 161, 2851, 3635
Error Budgets 2851
error tracking 2055
Errors 2073
escalation 2879
escalation policy 2883
ESO 3375
Estimating 3677
ETag 2419
etcd 1551, 1667, 2778
Etsy, Netflix and Google 42
Evaluation rules to memorise 1222
event 945, 1167, 1353, 2055
Event and issue 2055
Event history 1353
Event types 1356
Events 1555, 2023
events/errors 1925
Eventual Consistency 2751
eventually consistent 2751
Everyday commands 821
everything 2597
eviction 2669
Evidence 1007, 2842, 3089
exactly one 2689
Exactly-once 2690
Exactly-once (delivery) 2767
Exactly-once (processing) 2767
EXAMPLES 3709, 3721, 3733, 3744, 3759, 3771
except 1683
exceptions 3480
exchange 2701
exec() 297
exec form 808
execution environment 1167
execution role 1167
exemplars 1925
exempted 3168
exit code 865
expand and contract 1049
expand/contract 113
Expansion 1658
expiration 2354, 3767
Expire eventually 2768
expired certificate 461
Explained 2912
explicit 1457
Explicit deny always wins 1223
Explore 1952
exponential backoff and jitter 2768
Exporters 1935
Exposing apps 1589
expression index 2579
Expressions 1409
Extended Support 1264
external access findings 2142
External pooler 2655
External secrets 1626
External Secrets Operator 1626, 3375
External Secrets Operator (ESO) 3375
Externalised state and config 90
ExternalName 1592
externalTrafficPolicy: Local 1592
extraneous 3614
Extras 3791

F

Facebook 3624
Facebook, 4 October 2021 2350
Facebook/Meta 652
Failback 139, 3033
fail-back 3655
fail-open or fail-closed 2518
Failover 135, 1273, 3033
fails open 1315
failure scenario drill 3103
Failure Scenario Drills 3103
fake 2263
fall 2402
Fallbacks 2539
Fast 629
Fast Snapshot Restore 1250
fast-forward 567
Fastly 2382
fault injection game day 2944
Fault Injection Service (FIS) 2940
Feature flag 109, 1022
feature flags 109, 651, 1099
Feature-flag platforms 652
Fence 2628
Fencing 3036
fencing token 2777
fetch again or retry 1343
FHS 215
fields 249, 1975
FIFO queues 2764
fight other controllers 3329
File ownership 851
File Permissions: chmod and chown 273
file storage 1245
filesystem 323
filter 3770
Finalizers 3179
Find the idle colour 1085
Findability 3415
fingerprint 2055
FinOps 3705
Firecracker 740

Firewalls 435
first value read wins 2129
FIS 2940, 3104
Five Whys 2897
Fix Git immediately 3368
fix or delete 963
Fix what you find 2943
Fixed logical slots 2731
fixed numeric UID/GID 855
fixed version 894
fixed window 2513
Flagger 1096, 3266
flaky test 959
floor 3743
flow control 411, 2386
Fluent Bit 1680
Fluentd 1680
Flux 1826, 1850, 3317
Flyway 1049
Folder, tags, links 1952
followers 2688
follow-the-sun 2880
for: duration 2912
for static secrets 2166
forecast 2923
forecasting 2923
fork() 297
forward fix 3369
forward proxy 489
Forward secrecy 466, 2366
Four Golden Signals 1922, 2073, 2923
FQDN 3154
frames 2381
free 1194
Free Tier 1265
freezes 2611
Frequency 138, 3094
freshness 165
from forks 695
Fulfilment 3430
full commit SHA 951
Full jitter 2538
full or API scan 2286
full scan 2285
Fully automatic 667
Function 3505
function URL 1169
functions 3505

G

GA 3205
Gains 2482
game day 2939, 3103
gated 1007
Gatekeeper 3205, 3475
gateway 2036, 3779
Gateway API 1601, 3252
Gateway endpoint 1192
gateway endpoints 3780
Gauge 1961
gawk 253
G-Counter 2756
Generate load 1747
germane 3614
GHCR 972
GID 285
GIN 2579
GiST 2579
GiST / SP-GiST 2582
Git 3521
Git & GitHub 563
Git itself 610
git reset 599
git revert 599, 3365
GitFlow 651
GitHub 52, 174, 364, 663, 2192, 2257
GitHub Action 2232
GitHub Actions 52, 677, 945, 1486
GitHub Actions and Ansible 98
GitHub Actions for CI/CD 945
GitHub Actions to AWS 2138
GitHub Actions: Workflows, Jobs and Triggers 677
GitHub Dependabot alerts 2245
GitHub encrypted secrets 991
GitHub Environments and Secrets 691
GitHub flow 651
GitHub itself 578, 640
GitHub Release 663
GitHub Release asset 709
GitHub Releases 620
GitHub-hosted runners 705
GitHub's GLB Director 2400
GitHub's OIDC documentation 692
GitHub's REST API 476, 2218, 2514
GitLab 1050, 2594, 2624, 2656
GitLab (January 2017) 3650
GitLab's Container Scanning 2232
gitleaks 2257
GitOps 1023, 3303, 3305
GitOps controllers 3317
GitOps delivery 3521
GitOps repo 3375
Git's own project 630
GKE 1766, 1780
GKE Autopilot 1716
GlitchTip 2056
global 2514
Global Database 1264
global load balancer 2399
Global uniqueness 2732
Globs 229
glue 451
Go 877
Go modules 620
Go proposals 3678
Go template language 1849
golden image 87
golden path 185, 3439
Golden Paths 3439
Golden paths / templates 188
golden signals 3265
Goldilocks 1716, 3756, 3804
Google 161, 490, 652, 960, 2382, 2831
Google Chubby 2778
Google Cloud Load Balancing 2382
Google Cloud Monitoring 2820, 2832
Google Cloud OS Login 2126
Google Cloud's 2400
Google Maglev 2400
Google Public DNS 450
Google Spanner 2740
Google SRE 2880, 2908
Google's BeyondCorp 2432
Google's distroless images 2272
Google's Maglev 504
Google's SRE book 2534, 3624
Governance 1431
gp3 1245
graceful degradation 3565
Graceful degradation / fallback 2943
Graceful shutdown 1033
Graceful termination 1568
graduated 3306
Grafana 1949
Grafana Alloy 1989, 2036
Grafana Dashboards 1949
Grafana Labs 1990
Grafana Loki 1989, 3768
Grafana OnCall 2880
Grafana stack 1922
Grafana Tempo 2021
Grants 1329
Granularity 2539
gratuitous ARP 400
green 1081
Gremlin 2940
gross-margin-style target 3604
group 273, 285
Group coordinator / rebalance 2689
Grouping 2056, 2907
groups 710, 1217, 1766, 2055, 3140
gRPC 2381, 2534
Grype 889
GSLB 2404

Guaranteed 1715
Guardrails 186
gVisor 740

H

HA 121
Habit 229
half the RPO 3050
half-open 2533
halves 426
Handler 79, 1167
hand-made 1925
handoff 2879
hangs 3003
HAProxy 489, 504, 2382, 2514, 2534
HAProxy and nginx 2481
HAProxy Ingress 1602
Harbor 890, 2232
Harden the session cookie 2197
Hardening 2125
Hash 2579, 2669
hash tags 2675
HashiCorp Consul 2778
HashiCorp Packer 88
HashiCorp Vault 991, 2166
HCL 1405, 3525
HCP Terraform 1444, 1486, 1500
Head sampling 2024
header 2323
Header rewriting 492
headers 475
Headless 1592
headless Service 1667, 1753
head-of-line (HOL) blocking 2381
headroom 2923
Health assessment 3351
health check 121, 865, 933
health check grace period 1158
Health Checks 489, 865, 1285, 1311
Health checks and self-healing 2943
Health gates 1033
health signal 1933
Healthy 3353
Helm 1823, 1825, 3180
Helm 3 1825
Helm and Kustomize 1616
Helm charts 3440, 3463
Helm release 1023
Helm values per environment 3343
helm-controller 1866
helm-unittest 1895
helpers 1849
Heroku 2006
Heroku Postgres 2656
Hidden coupling 1086
Hidden files 229
Hierarchical Namespaces 1640
High availability 135, 3640
High Availability and Scalability 121
High availability (HA) 121
high cardinality 2731
highest wins 1422
high-resolution 1372
HikariCP 2463
hinted handoff 2752
Histogram 1961
hit ratio 3557
Homebrew 946
hooks 1865, 3522
Horizontal and Vertical Scaling 2715
Horizontal Pod Autoscaler (HPA) 1741
Horizontal Pod Autoscaling 1741
Horizontal scaling 2715, 3553
horizontally 121
host firewall 2125
host part 423
Host rule 1605
Hosted Prometheus services 1962
hosted zone 1285
HOT 2580
hot standby 2623
HOT update 2611
hotspot 2731
Hotspots 2731
How Docker Works Internally 739
HPA 1741, 3221
HPA, VPA and Cluster Autoscaler 3221
HPACK 2381
HSTS 2370
HTTP 2324, 3154
HTTP (HyperText Transfer Protocol) 475
HTTP keep-alive 2463
HTTP Methods and Status Codes 475
HTTP request 531
HTTP-01 2368
HTTP/1.0 2466
HTTP/1.1 2381, 2463
HTTP/2, HTTP/3 and QUIC 2381
HTTP/2 (RFC 9113) 2381
HTTP/3 412, 462
HTTP/3 (RFC 9114) 2381
Hubble 3154
hygiene 2262

I

IaC scan 175
IAM 1217
IAM Access Analyzer 1223, 2142
IAM Identity Center user 1221
IAM role 1218
IAM user 1221
iBGP 2335
IC 2868
ICMP 411
ICS 2867
ID token 2203
Idempotency 69, 75, 475, 2763
idempotency key 2763
idempotency keys 2218
idempotent 97, 476, 1691, 2685, 2756, 2763
Identity 1667, 2497, 3661
Identity types 1221
Idle 706, 3803
idle timeout 2464
IDOR 2181
IDP 3405
ignored 612
IGW 1191
illustrative example prices 3601
Image Caching and Layer Optimization 829
image index 752
Image scan 174
image scanner 889
image scanning 972, 2231
Image Scanning and Container Security 889
Images and Layers 751
IMDS 1141
IMDSv2 1145
Immediately fail readiness 1037
Immutability 3073
Immutability and access 138
immutable 752, 951, 3073
Immutable Infrastructure 87
immutable tags 971
Imperative 63
impersonation 3139
Implement cache-aside in Node.js 1278
implicit 1457
in parallel 677, 937, 3568
in place 253
In series 3568
in your infrastructure 1085
In-app pool 2655
inception 2354
inception and expiration times 2354
incident 2867
Incident Command System (ICS) 2867
Incident Commander (IC) 2868
Incident Management 2867
Incident response and on-call 165
include 1849
incomplete multipart uploads 3770

Indentation 1849
index 599, 2579
index-only scan 2583
In-flight work and queues 1086
Inform 3705
Informational headers 2517
Informer / cache 3182
Infrastructure 3666
Infrastructure as Code 63
Infrastructure as Code (IaC) 63
Infrastructure Automation 97
infrastructure template 3463
Infrastructure Templates 3463
ingestion (per GB) 1373
Ingress 1601, 1779, 1837, 3153
Ingress controller 1601
IngressClass 1601
ingress-nginx 1601
Inhibition 2908
init 1170
Init containers 1565
init system 309
Initialise 1412
Initialize containers 966
inject 2058
inodes 327, 2074
input 3475
Input variables 1419
inside the container 820
inside your VPC 1273
Insights events 1357
Instagram 2716, 2728
Install and verify 1411
Install Calico 1786
Install Crossplane 3508
Install kubectl and kind 1556
Install providers and functions 3509
Install the controller 1606
Instance 3791
instance metadata service 2138
Instance refresh 1159
Instance Scheduler on AWS 3744
Instance store 1250
instance type 1141
Instance type naming 1144
Instance warmup 1158
instances 1141
Instrumentation 2035
Integrating Argo CD, Terraform and Kubernetes into a Platform 3521
Integration 3468
Integration tests 959
Integrity 466, 1354, 2365, 3089
Interactive rebase 577
Interactive rebase to-do verbs 580
intercepting proxy 2285
Interface 3427
Interface endpoint 1192
interface endpoints 3780
Interface (PrivateLink) 3782
intermediate certificates 2365
internal 1320
Internal Developer Platform (IDP) 3405
Internal Developer Platforms 3405
Internet Gateway (IGW) 1191
internet-facing 1314
interpolation 820
Interruption notice 3729
interval, timeout and threshold 126
Interviews and audits 2177
intrinsic 3614
Introduce drift by hand 1521
Intuit 3318
Invalidation 1298
Inventory 77, 2942
Invocation models 1171
I/O and throughput 3791
io2 1246
IOPS 1245
IP 399, 2324
IP address 399
IP, MAC, ARP and DHCP 399
IPsec 2431
iptables 435
IPv4 address 423
IPv4, IPv6, CIDR and Subnets 423
IPv6 1195
IPv6 address 423
IPv6 basics 427
irreversible 3682
Isolation 565, 2593, 3073
isolation level 2593
ISR 2688
ISRG Root X1 2366
issue 2055
Issue a leaf certificate 2372
Issuer 2369
Istio 2445, 2534, 3251

J

Jaeger 1922, 2021
Java/Node/Python 882
Jepsen 2740
Jitter 2533
Job 677, 945, 1691
jobs 945
Jobs and CronJobs 1691
journal 335
journalctl 335
JSON 1975
JSON document 1949
JSON logs 1373
Juice Shop 2286
JVM 2940
JVM/Node.js runtimes 1718
JWT and OAuth 2203
JWT-SVID 2501

K

Kafka 2685, 2728
Karpenter 3221, 3730, 3744, 3804
Kata Containers 740
Kayenta 934, 1096
KEDA 1742, 3222, 3743
keep 609
keepalived 399
key 680, 1231, 2165
key administrators 1330
Key commands 2234
Key concepts 1145
Key pair 1141, 2435
key policy 1325
Key RDS building blocks 1263
Key rules 843
Key rules of behaviour 1769
Key types 1329
Key usage 2500
Key Usage / EKU 2369
key users 1330
Key workflow syntax 681
Keycloak 2204
keypair 349
kind 1552, 2023
kinds 3452
KMS 1325
KMS key 1325
KMS provider 1626
Knative 3743
knee 2925
Kong 2218
KRaft 2686
KRaft controller 2689
KSK 2352
kube-apiserver 1553
kubebuilder 3183
kubeconform 1892
kube-controller-manager 1553
Kubecost 3804
kubectl get endpoints 1592
kubelet 310, 1552, 1703, 1727
kube-prometheus-stack 1826
kube-proxy 1553, 1589
Kubernetes 52, 63, 122, 297, 1547, 3889
Kubernetes and load balancers 262
Kubernetes Deployment 1020, 1067

Kubernetes Deployments 110
Kubernetes Infrastructure Costs 3803
Kubernetes itself 1640, 2498
Kubernetes kube-proxy 504
Kubernetes manifests 67
Kubernetes on EKS 1246
Kubernetes operators 2629
Kubernetes' own controllers 3330
Kubernetes Secrets 991
Kubernetes Service selector 1081
Kubernetes-native 1952
kube-scheduler 1553, 1703
Kyverno 3205, 3475

L

L2 402, 2323
L3 402, 2323
L4 402, 503, 538, 2323, 2399
L4 refused 2329
L4 versus L7 Load Balancing 503
L7 503, 538, 2323, 2403
label selector 1589
labels 705, 1933, 1961, 1989
Labels and selectors 1555
Lag 2685
Lambda 1167
Lambda@Edge 1302
largest 1744
last accessed information 1223
last mile 3309
last stage 843
Last-Modified 2419
Latency 162, 461, 2073, 3589
latency budget 3589
latest 971
latest record per key 2691
launch template 1155
Launch the instance 1148
Launch-day and sale-day outages 3553
launches 1155
Layer 7 1311
Layer cache in CI 971
layer digest 754
Layer optimization 829
layers 2323
Layout 1432
lazy loading (cache-aside) 1274
LDS 2453
leader 2687
Leader election 3183
leading indicator 3444
leaf + intermediates 2369
leaf certificate 2365
Leak detection 2467
leaks 2659
Leaky bucket 2513
leaky bucket / queue 2517
Learning 2867
leases 2777
Least connections 493, 2399
Least privilege 273, 2137
least request 2403
least-privilege 3139
Least-privilege IAM 2137
Let's Encrypt 462, 1602, 2366
Level 1: Linux Fundamentals 215, 225, 237, 249, 261, 273
Level 12: Advanced Networking 2323, 2335, 2349, 2365, 2381, 2399
Level 14: SRE 2819, 2831, 2841, 2851, 2867, 2879
Level 15: Disaster Recovery 2985, 2997, 3007, 3019, 3033, 3047
Level 16: Advanced Kubernetes 3139, 3153, 3167, 3179, 3191, 3205
Level 17: GitOps 3305, 3317, 3329, 3341, 3351, 3365
Level 18: Platform Engineering 3405, 3415, 3427, 3439, 3451, 3463
Level 19: Architecture 3553, 3565, 3577, 3589, 3601, 3611
Level 2: Networking 399, 411, 423, 435, 449, 461
Level 20: Cost Optimization 3705, 3717, 3729, 3743, 3755, 3767
Level 4: Docker 739, 751, 761, 773, 785, 795
Level 5: CI/CD 933, 945, 959, 971, 987, 1003
Level 6: Cloud (AWS) 1141, 1155, 1167, 1179, 1191, 1203
Level 8: Kubernetes 1551, 1563, 1577, 1589, 1601, 1615
Level 9: Helm 1825, 1837, 1849, 1865, 1879, 1891
Levels 1976
Levels of rigor 3094
Level-triggered 3182
Leverage 1431
Library in the app 2539
Lifecycle hook on termination 1162
Lifecycle hooks 1157
lifecycle policies 774, 972
lifecycle policy 3767
Lifecycle rule 1447
Lifecycle rules 1231
limit 1715
LimitRange 1639, 1718
limits 1715
Limits to know 1171
linearizability 2739
linearizable 2742
LinkedIn 2686
Linkerd 2445, 3251
links 2021
Lint 1055
Lint / static analysis 936
Lint a postmortem 2900
Linux capabilities 2271
Linux Filesystem Hierarchy 215
Linux Fundamentals 213
Linux kernel networking 518
Linux Security and SSH Hardening 2125
Liquibase 1049
List 2669
List components and data 3652
List the user journeys 2823
Listener 1311
listeners 1311
Lists 3195
lists are replaced whole 1840
LiteSpeed 2382
LitmusChaos 2940
Little's Law 3557
live 1081, 3319
Liveness 122, 865, 1037
Liveness and Readiness Probes 1727
Liveness probe 1727
Liveness vs readiness 2404
Load 2923, 3589
load average 323
load averages 326
Load balancer 1311, 3661
load balancer (LB) 489, 2399
Load Balancing 532, 2399
load test 2923
load testing 2923
LoadBalancer 1589
Load-balancing algorithms 1315
local 1191, 2514
local backend 1443
local forward 370
local lab only 1746
local link 399
local network 531
local only 609
local values 1419
local-path-provisioner 1654
LocalStack 1172, 1183, 1208, 1223, 1235, 1291
Location 138
lock 1499, 2593
Lockfiles 2245
locking 1443
log 1975
log agent 1989

Log aggregation 1989
Log Aggregation with Loki 1989
Log classes 1373
log file validation 1357
log group 1369
log groups 1369
Log retention 3771
log rotation 335
log store 1989
log stream 1369
log streams 1369
Logical 139
Logical backup 2639
Logical replication 2629
log-only mode first 2528
LogQL 1989
Logs 147, 761, 1369, 1921
Logs and registries 3767
Logs Insights 1369
Long queries on replicas 2720
long window 2854
longest matching prefix 2336
longest prefix 439
longest-prefix match 1191, 2347
Longhorn 1654
look 225
lost an acknowledged write 2747
low cardinality 1992
lower latency 2415
LSM 3172
Lua health script 3356
Lua script 2781
LWW 2755
LWW-Register 2756
Lynis 2126

M

MAC 399
MAC address 399
MADR 3678
Maglev hashing 2403
main process 761
Maintenance 2584
MAJOR 663, 1896
MAJOR.MINOR.PATCH 663
Managed by Terraform 1952
Managed clouds 3140
Managed Resource (MR) 3505
Managed services 2629
Management events 1354
mandatory in practice 3454
manifest digest 974
MANRS 2340
manual 2035, 2939
Manual runbook 2629
Manual tag 667
Matrix 677
Matrix builds 945
mawk 253
Max lifetime 2467
Max size 2467
max surge 1067
max unavailable 1067
maximum 1155, 1222
Maximum Tolerable Downtime (MTD) 3059
Meaning of the bits 277
Measure 934, 2942, 3418, 3755
Memcached 1273
memory 323, 2073
memory limits 1715
memory request/limit 1721
memory-bound 3761, 3794
Merge 639, 1485
merge commit 565, 591
Merge methods 643
Merge when ready 644
Merging 565
Mesh: mTLS, Routing, Retries, Timeouts, Circuit Breaking and Observability 3265
message broker 2701
Message queue 3661
Metadata 3442, 3451
MetalLB in layer-2 mode 400
Meta's Katran 504, 2400
method 475
metric 1961
Metric filters 1373
Metric math 1372
metric name 1372
Metric types 1936, 1961
Metrics 147, 1095, 1369, 1921, 2021, 3408
Metrics to watch 1316
metrics-server 1741
MFA 2196
MFA (Multi-Factor Authentication) 2191
migration tool 1049
Migrations and failover 449
Migrations and one-off tasks 1865
millicores 1715
Min idle 2467
minimal base image 877
Minimal Base Images 877
Minimal image options 2276
minimum 1158, 3746
MINOR 663, 1896
Misconfigured HPAs cause flapping 1741
missing intermediate 461
Mitigate 2897
mitigate first, understand later 2867
mixed instances policy 3730
modes 3168
Module 67, 76, 1431
Modules 1431, 1471
Money 829, 1369
Monitoring 147, 1921, 3635
Monitoring architecture 3635
Monotonic reads 2755
Monotonic writes 2755
mount 309
mount target 1248
Mounts 1653
move around 225
move or rename 225
MR 3505
MSS 2327
MTD 3059
mTLS 2445, 2497
mTLS everywhere 2450
MTTR 1019
MTU 2432
MTU (Maximum Transmission Unit) 2327
multi-architecture 972
Multi-AZ 1250, 2624
Multi-AZ design principles 1182
Multi-AZ with automatic failover 1276
multipart upload 1231
multi-platform 971
multiplexed 2381
multi-region replication 1342
multi-region trail 1354
multi-site active/active 145, 3060, 3650
multi-stage build 841
Multi-stage Builds 841
Multi-tenancy 1766
multi-window, multi-burn-rate 2855
musl 877
must declare 1643
mutable 752
mutable infrastructure 87
mutate 3205
mutating 3205
mutating admission webhook 3206
MutatingAdmissionPolicy 3205
Mutual TLS (mTLS) 2497
MVCC 2607
MySQL-compatible 1261

N

N+1 2925
nack / reject 2705
name 1839, 2021, 2498
Name resolution tools 525

name servers 450, 1285
named restore point 2644
named templates 1849
Named volume 785
namespace 1370, 1639, 1865, 3167
Namespace per tenant 1644
Namespaced 1618
Namespaces 739, 1555, 1639
Naming and tagging 776
Naming conventions 1966
NAT 435, 531
NAT Gateway 1191, 3779
NAT traversal (NAT-T) 2435
NAT types 439
native histograms 1938, 1966
native S3 lock file 1499
Native S3 locking 1446
native sidecar 1564
Naturally idempotent write 2767
Navigating and Managing Files 225
NDP 400
Negative caching 453
neighbour table 401
netfilter 435
Netflix 52, 88, 122, 148, 186, 1141
Netflix Hystrix 2534
Netflix Open Connect 2416
Netflix's "paved road 3440
Network 323, 2073, 3665
Network ACL 1203
Network ACLs are stateless 436
Network activity events 1357
network aliases 799
Network and Transfer Commands 261
Network Debugging Tools 517
Network drivers 798
Network Egress Costs 3779
Network Load Balancer (NLB) 2480
network model 2323
network namespace 795
network namespaces 2340
Networking 397
NetworkPolicies 1779
NetworkPolicies in Depth 3153
NetworkPolicy 1641, 1779, 3153
new 289, 350, 1780, 2640, 3007
new commit 587
new endpoint 3008
new hashes 581
new instance 1264
new ReplicaSet 1577
new revision 1865
new, separate TLS connection 2479
new version 2607
NFS 1251
nftables 435
Nginx 298, 309, 335, 489, 504, 1034
NLB 503, 1311, 2480, 3664
node 3103
node affinity 1703
Node scaling 3221
node-based clusters 1273
NodeLocal DNSCache 1754
NodePort 1589
nodes 3803
nodeSelector 1703
Noisy or flaky alerts 2883
noncurrent versions 3770
non-root container 851
Non-root Containers 851
nothing is enforced 1779
Notify 933
Notion 2728
Novel 2912
NSEC / NSEC3 2354
Nuclei 2286

O

OAC 1297
OAI 1301
OAuth 2.0 2203
Object 1231, 2192
Object Lock 3073
Object Lock modes 3077
object schema validation 3208
object storage 1231, 3767
objects 1231, 1554
Observability 147, 1919, 1921, 2539, 3265, 3917
Observability and cost 187
Observability wiring 2449
Observable 101, 937
OCI 739
OCI image manifest 751
OCI registries 1879
OCSP stapling 2370
off 654, 1356
Off-hours shutdown 3743
offset 2685
off-site backup 3073
Off-site Backups 3073
OIDC 972, 987, 1218, 1485, 2138, 2203
OLM 3183
On a Linux host 454
on your own machine 2177, 2290
Onboarding 3415
Onboarding and incident response 3677
On-call 2879
On-call Practices 2879
once, at first boot 1145
on-demand pool 3807
one at a time 1667
one Availability Zone 1245
One consistent way 2036
One front door 3491
one node 1657
One pipeline per stack 1489
one Pod 1658
one point in time 3011
One question per dashboard 1953
one runner 677
one server 2727
one step tighter 2845
one time 1003
One traffic switch 1081
One volume for everything 3011
One Zone 1251
OOM killer 328
OOM-killed 1717
OOMKilled and CrashLoopBackOff 1715
OPA 3475
OPA / conftest 1492
OPA Gatekeeper 3205
OPA (Open Policy Agent) 2195
Open 2533
Open a pull request 1494
Open Container Initiative (OCI) 739
Open incidents and tickets 2883
Open Policy Agent (OPA) 3475
OpenBao 2166
OpenCost 3706, 3804
OpenID Connect (OIDC) 987, 2138, 2203
open-loop 2927
opens 2533
Open-source reference 532
OpenStack 2702
OpenTelemetry 148, 2022, 2035, 3636
OpenTelemetry log data model 1976
OpenTelemetry Operator 2036
OpenTelemetry (OTel) 1922
OpenTelemetry Protocol 2035
OpenTofu 64, 1405
OpenVPN 2431
Operability 3439
Operate 3705
Operation 1499
Operational Complexity 3611
Operations 987, 1298, 2386, 2727
Operations Lead 2868
Operations Lead (Ops) 2871
Operator 3179, 3237
Operator Lifecycle Manager (OLM) 3183
Operator SDK 3183
OperatorHub.io 3183
Operators 3179

Opsgenie 2880, 2908
Optimize 3705
optimized 3756
options 354
or withdraw 2338
Order does not matter 1409
order of restore 3019
ordering 2685
O(requests) 2517
organization 691
organization trail 1357
origin 489, 1297, 2415
Origin Access Control (OAC) 1297
Origin Access Identity (OAI) 1301
origin group 1298
Origin request policy 1301
origin shield 2416
origins 1297
orphan 301
OR-Set 2756
OSI and TCP/IP Models 2323
OSI model 2323
OTel 2035
other account 3073
Other EBS facts 1250
Other levers 2276
Other signals 1925
OTLP 2035
Outages 461, 1780
outbox event router 2778
outbox pattern 2777
outlier detection 2399, 2534
out-of-band 357
out-of-hours pages per shift 2884
output is smooth 2517
outputs 677, 1419
outside 1170
overlap 993, 2502
overlay 796, 3309
over-provisioned 3756
overwritten 3143
OWASP 2177
OWASP DevSlop 2286
OWASP Top 10 2177
Owned 2912
owner 273, 2893
ownerReferences 3182
Ownership 273

P

P2C 2403
PACELC 2739, 2743
Package repo promote 1007
Packaging Applications for Kubernetes 1891
page 2879
page cache 328
PagerDuty 2868, 2880, 2908
Pages per shift 2884
Panel 1949
panels 1949
panic threshold 2404
parallel 125
parallelism 2688
parameters 3209, 3495
parent 297
parent layer 829
parent span 2021
parent span id 2005
partial index 2579
partial indexes 2580
Partial Upfront 3717
Partition 2685
partition pruning 2727
Partition tolerance (P) 2739
partitioning 2715, 2727
Partitioning and Sharding 2727
partitions 2685
Passive 493, 2285
Passive / outlier detection 2404
passive scanner 2285
Passthrough 2479
PATCH 663, 1896
path 225
Path rule 1603
path-vector 2335
Patroni 2624
paved road 3439
Peer selectors 1784
peers 2339, 3157
Penetration 2673
people 1221, 2867
per AZ 1195
per component 3649
per connection 1592
per data class 3047
per group 951
per stream 2381
percentiles 3589
Performance 1753, 1980, 3589
Performance Insights 1264
Performance is cost 3589
period 1371
permission boundaries 2137
Permission boundary 2140
permission policies 1223
Permissions 785, 1167, 1343
permissions policy 2149
per-node agents 1679
Persistence 340, 2669
PersistentVolume (PV) 1653
PersistentVolumeClaim (PVC) 1653
personal access token 777
pets 87
pg_auto_failover 2629
pgBackRest 2640, 2986
PgBouncer 2463, 2655
Physical / snapshot 139
Physical backup 2639
physical streaming replication 2623
PID 297
pilot light 136, 3060, 3650
Pinning 682
pipe 237
pipeline 51, 933, 1989
Pipeline hand-off 1007
pipelines 2039
Pitfall 619, 640, 652, 664, 692, 706
Pitfalls 2009
PITR 2639
PKI (Public Key Infrastructure) 2365
Placement 1143
plaintext HTTP 2479
Plan 64, 1405
plan role 1489
Planning a VPC 426
Planning checklist 1182
Planning Lead 2868
Platform Engineering 185, 3403
platform team 185
Playbook 77
plugin system 3491
PN-Counter 2756
Pod 1551, 1563, 3103
pod affinity and anti-affinity 1703
Pod scaling 3221
Pod Security 3167
Pod Security Admission 1644
Pod Security Standards 852, 2272
Pod Security Standards (PSS) 3167
Podman 2271
Pods 1563, 3167
point-in-time recovery (PITR) 1264, 2639
points of presence, PoPs 2415
policies 1217, 2137, 2705
Policy 877, 2335, 2445
Policy / admission 177
Policy anatomy 2141
Policy as Code 3475
policy checks 1485
Policy engines 3205, 3521
policy generation 2142
policy simulator 1223
Policy types 1222
policy validation 2142

Policy-based 2434
policy's own namespace 1784
policyTypes 1784
pool sizes explicit configuration 2468
Pool too big 2468
pools 3733
Port forwarding 363
Portability 249
Ports 411, 435, 1779
Possession 2497
post-deployment migrations 1050
PostgreSQL 136, 298, 310, 335, 3590, 3661
PostgreSQL Indexes 2579
PostgreSQL operators 3180
PostgreSQL-compatible 1261
postmortem 2893
Postmortems 2893
Powertools for AWS Lambda 2764
PPID 297
PR 1049
Prebuilt binaries 881
Precedence 1605
pre-commit hook 175
Predictability 1779
Prefetch 2706
Prepare the status-page templates 2874
prepared transaction 2612
pre-release 663
present in most queries 2731
Presigned URL 1232
Prevent 2897
preview before applying 1013
Price class 1301
Pricing 1171
Pricing models (introduction) 1145
primary 2623
primary group 289
primary store 2669
Principles 2195
priority 1703
priority order 1315
Prisma Migrate 1049
Privacy 2060
private 349, 1232, 1285
private hosted zones 2350
Private ranges (RFC 1918) 426
privilege escalation 1769
privilege escalations in disguise 3139
probe 1727
Probe mechanisms 1731
Probes 1565
problem 3418
process 297, 2655, 2894
Processes, PIDs and Signals 297
producer 2685
product 185
Production 1050
Production Kubernetes 3903
Profiles 820, 1925
Progressive delivery 3627
progressive traffic shifting 1095
project 817
projected token 1770
Prometheus 1922, 1933, 2035, 2820
Prometheus Alertmanager 2908
Prometheus and PromQL 1933
Prometheus Operator 3179
prometheus-adapter 1742
promoted 3033
promotes 1095
promotion 1003, 3341
PromQL (Prometheus Query Language) 1933
Promtail 1989
protected branch 639
Protected Branches and Pull Requests 639
Protection 492
protection rules 691, 991
Protocol evolution 532
Provenance 1003
Provenance attestations 889
Provider 64, 1405, 3505
ProviderConfig 3505
providers 2339
Providers and Resources 1405
Provision a dashboard provider 1954
Provision the data source 1953
provisioned concurrency 1171
provisioning 1949
Prow 52
Prowler 2152
proxy 3251
PROXY protocol 507, 2484
ProxyJump 363
Prune 3329
Pruning 3193, 3521
PSA 3167
PSK 2437
PSP 3167
PSS 3167
public 644, 1179, 1191, 1285, 1606
Public IPv4 1145
Public key 2365
public subnet 1192
publish a port 795
Publisher confirms 2705
Pub/Sub 2669
pull model 1933
pull request (PR) 639
Pulled automatically 3305
pulls 773
pull-through cache or mirror 774
Puppet 75
push 773, 933
Push protection 2257
Push to Amazon ECR 779
PV 1653, 3237
PVC 1653, 3237
Pyrra 2820, 2832, 2852
Python PEPs 3678

Q

QoS classes 1716
quality gates 1891
quantiles 1961
quarantine 963
queries 1950
queries over data 3479
Query the machine 1943
Queues 2073, 2701
QUIC 2326
QUIC (RFC 9000) 2381
Quirks 2883
quorum 2675
quorum or lease 3037
quorum queues 2702
Quorums 2744
Quotas 1207, 2513
Quoting 229

R

RabbitMQ 2701
Random 2399
Random with two choices 493
range 1993
Range sharding 2731
Ransomware and account compromise 136
Rate 2073
Rate Limiting 2513
Rate limiting algorithms 2221
Rate limits and back-pressure 476
rate of budget consumption 165
raw objects and actions 2217
RBAC 1639, 1765
RBAC in Depth 3139
RBAC (Role-Based Access Control) 2191
RBAC versus ABAC 2195
RDB 2669
RDS and Aurora 1261
RDS and other services 3718
RDS Proxy 1262, 3791
Reachability and path tools 525

read 1356
READ COMMITTED 2596
read repair 2752
read replica 3791
Read replicas 2624
read scale 2997
Read scaling 1263, 2623
read throughput 2719
reader endpoint 1263
Readiness 122, 865, 1033
Readiness probe 1727
Reading the error codes 1316
README/wiki page 3409
read-only 851, 1513
Read-only mode 3568
Read-only root filesystem 851
readOnlyRootFilesystem: true 3171
Read-through 2673
ready 1033, 1741
read-your-writes 2751
Real 2907
real commit 609
Real infrastructure 1517
realism 959
real-user monitoring 1925
reap 765
rebalance 2685
Rebalance recommendation 3730
Rebase and merge 578
Rebasing 577
reclaim 3729
Reclaim policy 1653
reconcile loop 3179
Reconciliation 3329
Reconciling 1513
record key 2689
Record the deployed version 1024
recording rules 1933
records 2685
Recovery 75, 613, 3238
recovery conflicts 2628
Recovery Point Objective (RPO) 3047
Recovery skills 599
recovery time 3103
Recovery Time Objective (RTO) 3059
recreate 109
recursive resolver 449, 2349
recursive resolver's 2354
RED 2073, 2074
Red Hat OpenShift GitOps 3318
redaction 1980
Redirect clients 2628
redirection 237
Redirection and pipes 241
Redis 2513, 2669, 3661
Redis Cluster 2669, 2728
Redis Enterprise Active-Active 2752
Redis OSS 1273
reduce the load itself 3791
redundancy 122
Redundancy and failover 2943
Re-enable automation 3368
Reference Production Architecture Walkthrough 3661
referenced by name 1894
reflog 599
refresh 3333
region 3073, 3103
regional edge caches 1297
regions 2997
registered 439
registrar 450
Registries 773
registry 971
Rego 3475
regression 2056
Regressions ship silently 3589
Regulation and customers 3649
Rehearse 1024
rejected 46, 3169
rejected in 0 ms 2544
relay 403
release 1825, 1865, 2055
Release 3, CONTRACT 1058
release branches 651
Release branches in TBD 655
release management 663
Release PR bot 667
Release strategy 3623
Releases 619, 1865, 2055
releasing 109
Reliability 1141, 1231
Reliability Engineering 2939
reliable, ordered byte stream 411
reload config 302
remote forward 363
remove the old 1050
removed from Service endpoints 1727
Rename 1052
Renovate 2245
Reorg-proof 3454
repeat 1861
Repeat pages 2884
REPEATABLE READ 2596
repetitive 2939
replace 1411
Replace templates/_helpers.tpl 1856
Replace values.yaml 1856
replaced 1426
replaced immediately 1558
replaces 1155
replay 2382, 2623
replayed 2382
replicas 2623
ReplicaSet 1577
Replication 2623, 2669, 2719, 2997
Replication factor 2685
replication slot 2607, 2623, 3001
repmgr 2629
Repositories 1879
repository 691
Reproducibility 619, 678, 1405
reproducible 1009, 1949
Reproducible builds 805
request 475, 1715, 1741
Request collapsing 2420
request ID 2005
Request limits 2467
request-based 2820
requests 1715
Requests inflation 3803
Required checks 677
Required reviewers 691, 991
requires 2498, 3224
Rescuing a commit 587
reserved concurrency 1171
Reserved Instances and Savings Plans 3717
Reserved Instances (RIs) 3717
reset and revert 599
reset (changes lost) 601
resets 1840
Resilience 1298, 2415, 3265
Resilience libraries and proxies 2940
resource 64, 1405, 2035
resource and account level 2151
Resource hooks 3351
resource owner 2203
resource policy 1172, 1217, 1342
Resource Requests and Limits 1715
resource server 2203
resource types 3191
ResourceQuota 1639, 1718
Resources 1567, 3241
response 531
Response codes 453
Response headers policy 1301
Resque 2670
Restart policies 765
restartPolicy 1564
restarts the container 1727
restore drill 2639
restore procedure 3019
Restore Procedures 3019
Restore tested 139

Restore the original NACL 1211
restricted API keys 2218
Retain 1658
Retention 136, 1003, 1989, 2685, 2986
retention period 705
retries 3265
Retrieving secrets 1343
Retrofitting is expensive 3577
retry 2763
Retry storm 2468
Retry-After 2514
reusable environment 1471
Reusable Environments 1471
Reusable workflows 945
reused 2616
reuses 2463
revalidated 2418
Reverse Proxies and Load Balancers 489
reverse proxy 489
Reverse proxy upstream 1084
reversible 3682
revision 1 1865
Revocation 467, 2501
rewrite addresses 506
Riak 2752
Rightsizing 3755
rise 2402
Risk 587, 3179, 3415, 3475
risk of data loss 2673
Risky changes 2882
ROA 2340
roaming 2435
Role 1765
Role-Based Access Control (RBAC) 1765
RoleBinding 1765
roles 1217, 2137
Roles in practice 1223
roles over heroics 2867
Roll back 1162
Roll the cluster back 3368
Rollback 933, 1019, 1577, 3365
Rollback Through Git 3365
Rollback triggers 1019
roll-forward 1019
rolling 109, 2831
rolling deployment 1067
Rolling Deployments 1067
rolling update 1577
rolling updates 110
Rook/Ceph 1654
root 285
root CAs 2365
root cause 2897
root module 1409, 1431, 1472
root on that host 367
Root user 1221
Root volume 1143
Rootful vs rootless 2275
Rootless and Minimal Containers 2271
Rootless mode 856
rotate 1339
rotates 1339
Rotation 1329, 1339, 1625, 1980, 2165, 2879
rotation is the fix 2262
rotation Lambda function 1343
Rotation strategies 2169
Round robin 491, 2400
Route 53 1285, 2400
route leak 2336
route table 1191
Route table anatomy 1194
Route Tables, Internet Gateway and NAT Gateway 1191
route templates 2077
Route-based 2434
Routing 435, 492, 2701, 2720
routing key 2701
routing, NAT and firewalls 531
Routing, NAT, Ports and Firewalls 435
Routing policies 1288
routing rules 3269
routing table 435
Row 1950
row-level locks 2593
RPO 2627, 2639, 3047
RPO - Recovery Point Objective 3047
RPO: 17 orders lost 3113
RPO (Recovery Point Objective) 3649
rsync 261
RTO 2639, 3047, 3059
RTO - Recovery Time Objective 3059
RTO about 14 s 3113
RTO (Recovery Time Objective) 3649
Rule 1311, 1601
rules 1311, 1765, 3475
Rules of the model 1785
Rulesets 639
runbook 2879, 3019
runbooks 2879
Runner 677
Runner types compared 709
Runners 705, 945
running 2285
Runtime hardening 889
Runtime hardening checklist 895
Rust 880
Rust RFC 3678

S

S3 1231, 3661
S3 Block Public Access 2152
S3 Bucket Keys 1326
S3 Intelligent-Tiering 3767
SA 2435
Safe 475
Safe by default 101
Safe deploys 2943
SaltStack 75
same backend PID 2663
same namespace 1757
same node 1563
Same settings in Kubernetes 2279
samples 1933
sampling 2021
SAN 467, 2367, 2501
sandbox account 1223
SAST 173
Saturation 2073, 2923
Savings Plans (SPs) 3717
SBOM 889, 1007
sc1 1250
SCA 173
scaffolding templates 3463
Scalability 121, 489, 3553
scales 1155
scales linearly 2942
Scale-to-zero 3743
Scaling 1245
scaling axes 3553
Scan 933
Scan each image 885
Scanner comparison 2248
Scanners 2245
Scanning 889
Schedule tests 3655
scheduled 3743
Scheduled scaling 3743
Scheduling 1679, 1703
scope 2057, 2177
Scope and naming 1643
Scoped 2767
scoping and policy boundary 1639
scorecards 3451
scp 261, 1222
SCP (Service Control Policy) 2142
scrapes 148
scratch 877
Scribe 2868
Scryer 1156
SDK 2035
SDLC 41
Sealed Secrets 1626, 3375

Search 3415
Searching contents: grep 241
Searching files: find 241
Seasonality 2928
seccomp 889, 2271, 3167
secondary CIDR blocks 1181
secret 987, 1625, 2165
Secret hygiene rules 695
secret scanning 174
Secrets 691, 987, 1625, 3375, 3666
Secrets and Environment Variables 987
Secrets management 2165
Secrets Management and Encryption 2165
Secrets Manager 1339
Secrets Scanning 2257
Secrets Store CSI Driver 1626
Security 87, 215, 309, 450, 489, 678
Security / DevSecOps 2123
Security Architecture 3577
Security defaults 3430
security group 1147, 1203
security group referencing 1203
Security Group (SG) 1203
Security Groups and Network ACLs 1203
Security Groups are stateful 436
Security groups (SGs) 2151
sed 249
sed building blocks 253
segments 2691
Self-contained 633
Self-heal 3329
Self-healing 1161
Self-hosted runner security rules 710
Self-hosted runners 705
Self-service Infrastructure 188, 3427
self-service route 3439
SELinux 3170
semantic conventions 2024, 2035
Semantic version 975
Semantic versioning 3467
Semantic Versioning and Release Management 663
Semantic Versioning (SemVer) 663
SemVer rules worth memorising 667
Send requests 2013
Senior Architecture 3943
sensitivity check 3682
Sentinel 2669
Sentry 2055
Separate bucket/account 1447
separate failure domains 3640
Sequence CRDTs 2756
sequential scan 2579
Serializability 2743
SERIALIZABLE 2596
series 125
Serve stale data 3568
server 2497, 3103
Server Name Indication (SNI) 461
Server push 2386
Serverless 1273
service 309, 1589
service account 285
service catalog 3451
Service Catalogs 3451
service container 959
Service Control Policies (SCPs) 2151
service credit 2842
Service Discovery 450, 1753, 2446
Service Level Agreement (SLA) 2841
Service Level Agreements (SLAs) 2841
Service Level Indicator (SLI) 2819
Service Level Indicators (SLIs) 2819
Service Level Objective (SLO) 2831, 3635
Service Level Objectives (SLOs) 2831
Service Mesh 2445, 3251
Service Mesh with Istio or Linkerd 3251
Service meshes 2480
ServiceAccount 1765
ServiceAccount + RBAC 1683
ServiceAccounts 1770
Services 1589
Services and Logs: systemctl and journalctl 335
Service-to-service communication 1589
session 2463, 2659
Session affinity 1592
session keys 461
Session resumption 2480
session resumption/tickets 2370
Session security checklist 2195
Session stickiness 493
Sessions 2191
Set 2669
Settings, Actions, General 1010
setuid, setgid and sticky 273
SEV1 2872
SEV2 2872
SEV3 2872
SEV4 2872
severity 889
SG 1203
sh is PID 1 765
sha256 3077
Shallow 126
Shape/size limits 3430
shard key 2727
Sharding 2727
shared 2418
shared CI workflow 3409
shared dependencies 3554
Shared namespace 1644
Shared network 1564
Shared volumes 1564
Shed non-essential features 3568
shield 2416
shift left 173
Shopify 3612
Short TTL + stale-while-revalidate 2420
short window 2854
shorter 2466
short-lived 2498
short-lived credentials 987
short-lived signed identity token 691
Showback 3705
siblings 2755
Sidecar 2445
Sidecar or proxy 2539
Sidekiq 2670
Sign 1004
Signals 297
signature 619, 2365
Signature Version 4 1301
Signing 889
Sigstore 1004
Silencing 2908
silent 2435
Single-user 1343
Site Reliability Engineering 161
Site Reliability Engineering (SRE) 161
Size-reduction toolbox 833
skipped 939
SLA 161, 2841, 3047
SLA (external) 2841
SLA (Service Level Agreement) 161
Slack 2716
SLI 161, 2819, 2841
SLI (Service Level Indicator) 161
Sliding window counter 2517
Sliding window log 2517
slim 877
SLO 161, 2831, 2844, 3635
SLO (internal) 2841
SLO (Service Level Objective) 161
SLO-driven alerting 3635
Sloth 2820, 2832, 2852
slow calls 2538
Slow query or lock 2468
Slow start 1315, 2399
SLSA 1004
small drop-in files 290
small states 1505
smaller 3013

SME 2871
smoke 3592
smoke test 962
snapshot 2607, 3007
Snapshots 1245, 1658, 3007
SnapStart 1171
SNAT 435
SNI 461, 1315, 2374, 2479
SNI (Server Name Indication) 2370
SNS topic 1371
Snyk 2245
Soak 2926, 3592
socket 309
SOCKS proxy 363
SOCKS5 proxy 367
Software Catalog 186, 3415, 3491
Software Development Lifecycle (SDLC) 41
software supply chain security 173
Software Templates 186, 3415, 3491
SolarWinds (2020) 2246
SOPS 1626, 3375
Sorted set 2669
Source IP hash 2403
Source maps 2055
Space 2584
spans 1921, 2021
SPDX 2231
Special bits 273
Speed 751, 959, 1003, 3089, 3341, 3439
SP-GiST 2579
spider 2285
SPIFFE ID 2498
SPIFFE/SPIRE 2497
Spike 2926, 3592
Spinnaker 52, 934
SPIRE Agent 2498
SPIRE Server 2498
split brain 139
split-brain 2623, 3033
Split-horizon DNS 2349
Spot Instances 3729
Spot pool 3807
Spotify 3491
Squash and merge 578
SRE 161, 2817
SRV 1756
SSE-KMS 1235
SSE-S3 1235
ssh 78, 261
SSH keys 622
SSH Keys, known_hosts and authorized_keys 349
SSH (Secure Shell) 349, 2125
SSH server 367
SSH tunneling 363
ssh-agent 363
ssh-agent and Port Forwarding 363
SSRF 2178
st1 1250
Stability 1715
stack 64
stack of layers 751
Stack Overflow 2716, 3612
stage 841, 933
stages 51
staging 1053, 1606, 2366
staging labels 1339
stale-while-revalidate 2416
Standard 1344, 3717
standbys 2623
stanza 2644
startup 865, 1037
Startup probe 1727
stash 609
state 63, 1405, 1513, 3521
State and Remote Backends 1443
State compatibility 1019
state file 1443, 3521
State Locking 1499
stateful 435, 1203, 2715, 3237
stateful, irreplaceable data 3652
Stateful sessions and caches 1085
Stateful systems 1753
Stateful vs stateless filtering 439
Stateful Workloads 3237
StatefulSet 1667
StatefulSets 1667
stateless 475, 1203, 2715, 3237, 3553
statement 2468, 2659
Static 3467
Static website hosting 1235
statistic 1372
status 2021
status checks 639
status code 475
Status subresource 3183
stderr 237
stdin 237
stdout 237
steady floor 3720
steal 328
stickiness 1315
Stop 1956, 1994
Storage 3791
Storage Lifecycle Policies 3767
StorageClass 1653, 3237
Store 1003
stored by GitHub 705
Stored with the outcome 2768
Stream 2669
stream-aligned teams 186
streams 2381, 2705
Stress 2926, 3592
strict 2517
Strict serializability 2743
STRIDE 3577
Strimzi 2686, 3179, 3238
String 2669
Stripe 476, 1050, 2006, 2514, 2764
Stripe's API 2218
structural 3194
Structured Logging 1975
Structured metadata 1992
STS 1217
stub resolver 449
subject 1765, 2501
Subject / SAN 2369
Subject Alternative Name (SAN) 2501
Subject Alternative Names (SAN) 467
Subject matter expert (SME) 2871
subnet 1179, 1203
subnets 1179
Subnetting 423
Subscription filters 1373
subsets 3269
sudo 285
sudo systemctl daemon-reload 313
sudoers 285
Summary 1961
Supabase 2656
supplementary groups 288
supply chain attack 174, 2245
Supply chain attacks 2246
Supply-chain incidents 678
Support 3405
suspect commits 2056
sustainable load 2879
SVID 2500
Switch 1081
Syft 890
symmetric 1325
symptom-based alerts 2907
SYN cookies 466
sync 3329, 3351
Sync loop 3329
Sync policies and options 3351
Sync waves 3351, 3522
sync waves and hooks 3352
Synchronous 2623
Syntax 788, 843
System Monitoring Commands 323
system of record first 3023
systemctl 335
systemd 309

T

tabletop 2944
tactical 2939
tag 619
Tagging strategy 971
Tags 619
tail latency 3589
Tail sampling 2024
Tailscale 2431
taint 1703
taints and tolerations 1703
Take a snapshot 1253
target 309, 475, 1311, 1933, 3762
Target group 1311
target groups 1311
target user's 354
targets 1933
task execution role 1343
TBD 651
TCP 411, 466
TCP Connections and TLS Handshakes 461
TCP keepalive 2463
TCP three-way handshake 461, 531
TCP, UDP and ICMP 411
TCP/IP model 2323
Tear down 1944
TechDocs 3415, 3491
Telemetry costs money 3635
Tell people 2943
template 1849
template repo 3409
Templates 1849
temporary credentials 691
terminates TLS 1311
Terraform 63, 1405, 3875
Terraform / Infrastructure as Code 1403
Terraform CI/CD 1485
terraform destroy 1408
terraform.lock.hcl 1410
Terraform/OpenTofu 3521
Terraform/OpenTofu modules 3463
Terragrunt 1471
Test 933, 1432, 3153, 3649
test hooks 1866
test pyramid 959
Test the restore 3011
Testability 87
tested runbooks 2873
TestGrid 52
testing 3143
Testing and auditing commands 1770
Testing Backup Restoration 3089
text 237
text files 63
Text Processing with awk and sed 249
tflint 1485
The Full Flow: CI to GitOps 3375
The Three Pillars: Metrics, Logs and Traces 1921
Thrash 3226
threat modeling 3577
three pillars 147, 1921
Three ways to automate 667
thresholds and colours 1953
throttled 1715
Throttling repeats 2912
throughput 1249, 3589
Throughput in and out 2080
throwaway 2989
thundering herd 2420
ticket 2895
ticket-level 2912
ticket-ops to API-ops 3427
TIDs 2579
tier 3060
Tier your reliability 3604
time series database (TSDB) 1933
Time to acknowledge 2884
time to decision 3064
Time to detect (TTD) 2897
Time To Live (TTL) 449
time to mitigate 2884
Time to mitigate (TTM) 2897
Time to resolve (TTR) 2897
timed out 466
timeline 2628, 2641, 2894
timeline 2 2633
Timeouts 2533, 2744, 2939, 3265, 3591
timer 309
timing and compatibility 1049
TLS 461, 1297, 2324, 2365
TLS 1.2 2370
TLS 1.3 2366, 2484
TLS handshake 531
TLS Termination 491, 1601, 2479
TLS-only policy 1447
tmpfs mount 785
to completion 1691
toil 97, 161
Token bucket 2513
Tokens 2203
tolerance 1744
toleration 1703
Tool comparison 2262
Tools 1223, 2880
topic 2685
Topologies (node-based) 1276
Topology 2399
topology spread constraints 1704
total/read timeout 2538
trace id 149, 2005, 2021
trace to logs 2009
Traceability 3341
TraceQL 2022
Traces 147, 1921
tracing backend 2021
Trade-off matrices 3677
Trade-offs 1925
Trade-offs and Architecture Decision Records 3677
Trade-offs are explicit 3601
trade-offs honestly 3681
Traefik 1601
Traffic 2073
trail 1353
transaction 2465, 2593, 2656
transaction mode 2468
transactions 2690
Transactions and Locks 2593
Transit engine 2166
transitions 3770
transitive dependencies 2245
Transport 454
transport layer 411, 503
Transport Layer Security (TLS) 2365
treat missing data 1373
Trend 326
Triage 2289
Trigger 936, 2896
Trivy 889, 2231
trivy config 1485
TruffleHog 2257
truncating 241
trunk first 655
Trunk-based Development and GitFlow 651
Trust 959, 2867
trust boundaries 3579
Trust bundle 2498
Trust domain 2501
trust policy 692, 989, 1218, 2138
Try a registry module 1438
Try the merge queue 646
Try the workspace alternative 1479
TTD 2897
TTL 449, 1285, 2349, 2673
TTL jitter 2673
TTL only 2673
TTM 2897
TTR 2897
tunable consistency levels 2740
Tuning 869
Tuning fields 1732

Tunnel mode 2435
tuples 2607
TVP 3405
twelve-factor app 336
two A records 1760
two edge PoPs 2421
Two environments 1081
two kinds of address 399
two or three 2598
two-minute interruption notice 3729
Types 1409, 1419

U

Uber 3636
Uber, Shopify 2686
Uber's engineering blog (2016) 2608
Ubuntu 2126
UDP 411, 2381
UDP 443 2381
UID 285, 1951
umask 273
umbrella chart 1891
Unbound 2353
uncacheable 1298
uncommitted work 609
under ~10 minutes 937
under-provisioned 3756
undo log 2608
Union filesystems (OverlayFS) 739
Unique constraint / upsert 2767
Unique index 2583
unit 309
Unit cost 3601
unit economics 3601, 3706
Unit tests 959
units 1953
Unit-test the alerts 2860
Unstructured 1975
untracked 609
Update strategies 1683
Updates 1670
Upgrades 3237, 3256
upstream 489
Urgent 2907
Usability 3089
USE 2073
Useful commands 403
Useful flags 2235
user 285
User data 1141
user namespaces 2271, 3172
user-defined bridge 795
users 1217, 2137
Users, Groups, sudo and root 285
Using them in YAML 695
UTC 1356
Utilization 2073, 3717

V

VACUUM 2607
validate 3205
Validate locally 953
Validate the YAML locally 58
validated 1419
validating 3205
ValidatingAdmissionPolicy 3205, 3476
ValidatingAdmissionPolicy (VAP) 3205
ValidatingAdmissionPolicyBinding 3209
validation 3019
Validation / keepalive probe 2467
Validation layers 3023
Validity 2365
Valkey 1273, 2669
Values 1837
values schema 1837
VAP 3205
Variable 217, 1949
variables 987
Variables and Outputs 1419
Vary: Accept-Encoding 2419
vcluster 1640
Velero 3242
verification 2257
Verified 622
version 1339
Version the path 3443
version your file names 1302
Version-controlled and reviewed 101
Versioned and immutable 3305
versioned immutable ConfigMaps 1618
Versioned, immutable releases 1019
Versioning 1231, 1433, 1447, 1891, 3074
Vertical Pod Autoscaler 1716
Vertical scaling 2715, 3553
vertically 121
Viewing 237
Viewing and Searching Text 237
Views 2351
Virtual 218
Virtual clusters 1644
Virtual Private Network (VPN) 2431
visibility map 2583
visibility plus gentle pressure 3455
visibility timeout 2778
Vitess 2716, 2728
volumes 1653
Volumes and Bind Mounts 785
volumes kept 821
Volumes, PersistentVolumes and PersistentVolumeClaims 1653
VolumeSnapshot 3241
VPA 3221, 3756
VPC and Subnets 1179
VPC endpoints 1191
VPC origins 1300
VPC (Virtual Private Cloud) 1179
VPN 2431
VPNs 2431
vulnerable 2177

W

W3C Trace Context 2005
wait 1499, 1865
Waiting 693, 2468
waits 1054, 2593
WAL 2623, 2639
WAL-G 2640, 2986
war room 2870
warm standby 136, 3060, 3650
Warm-up 1068, 1085
Waterfall 41
waves 934
waypoint 3251
waypoint proxies 2446
web cache deception 2416
Web pods 3663
website endpoint 1235
Weighted round robin 493, 2403
well-known 439
wget 261
What a runbook contains 3022
What DevOps Actually Is 35
What Happens When You Visit https://example.com 531
what is on them 75
what made it long 2897
What to Monitor: Golden Signals, RED and USE 2073
wheel of misfortune 2944
Where events go 1357
where in your architecture 2479
Where scanners look 2262
where the bytes go 3779
where the feedback arrives 47
where the time went 2021
Where TLS ends 503
Where values can live 694
which thing 1217
who decides 3033
whole cluster 1765
whole-fleet 3734
Wikimedia 1950
window 414
windows-based 2820
Wire it into CI 3471

WireGuard 411, 2431
with a commit 3365
withdrawal 2353
within a partition 2685
without changing application code 3265
without decrypting anything 508
without Kubernetes 2450
Witness/odd number 3037
work queue 2777
Work queue and backoff 3182
worker nodes 1551
Worker pods 3663
workflow 677, 945
Workload API 2498
workspaces 1471
WORM 3074
wraparound 2607
write 1357
Write a Composition 3509
Write a kind config 1556
Write lab.js 1926
Write skew 2596
Write the Alloy pipeline 1996
Write the app 1981
Write the child module 1435
Write the Collector config 2040
Write the Compose file 1941, 2043
Write the ConfigMap 1857
Write the dashboard JSON 1954
Write the declaration message 2874
Write the Deployment 1857
Write the Dockerfile 1940
Write the downstream service 2011, 2027
Write the front service 2012, 2026
Write the function 1172
Write the instrumented app 1939
Write the logger once 1981
Write the Loki config 1995
Write the permission policy 1224
Write the post-mortem 3368
Write the Prometheus config 1940, 2042
Write the reusable module 1477
Write the Tempo config 2042
Write the tracing bootstrap 2026
Write the TypeScript service 2061
Write user data 1147
write-ahead log 2593
Write-behind 2673
writer 1262
Writes 2579
write-through 1274, 2673
written authorisation 2289

X

X509-SVID 2500
xDS 2445, 3253
XIDs 2611
xmin horizon 2607
XR 3505
XRD 3505
XSS 2182

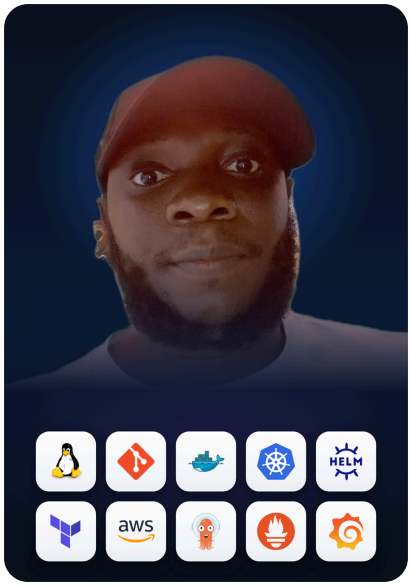
Y

Yjs 2752
YouTube 2716

Z

Zalando 2624
zero endpoints 1592
zero trust 3577
zero-downtime deployment 1033
Zero-downtime Deployments 1033
Zero-downtime deploys 1589
zero-trust 2432, 2497
Zipkin 1922, 2022
zombie 297
zone 449
ZooKeeper 1667
ZSK 2352
ztunnel 3251

About the Author



Ubong Eduok

Author and instructor, DevOps Roadmap: Beginner to Senior

Ubong Eduok created the DevOps Roadmap, the open-source, hands-on curriculum that this book sets in print, along with its companion **devops-deploy-toolkit** for deploying real applications with an AI pair-deployer.

The roadmap is free to read and released under the MIT License, so that anyone can use it, including in their own courses and projects.

GitHub: github.com/mascot200
Roadmap: github.com/mascot200/devops-roadmap
Toolkit: github.com/mascot200/devops-deploy-toolkit

COLOPHON

This book is set in Source Serif 4 for the text, Inter for headings, tables and diagrams, and JetBrains Mono for code, with DejaVu Sans and Noto Emoji supplying a few symbols. All are open-licensed typefaces. Trim size is 7.5 by 9.25 inches.

It was typeset directly from the Markdown of the repository at commit faf55fa, so the book and the repository can be kept in step. The 570 diagrams were drawn from their Mermaid sources as vector art. The staircase motif has one step for each of the twenty-one levels. The typesetting toolchain is in the `book/` folder of the repository, and you can rebuild the whole book from it with a single command.